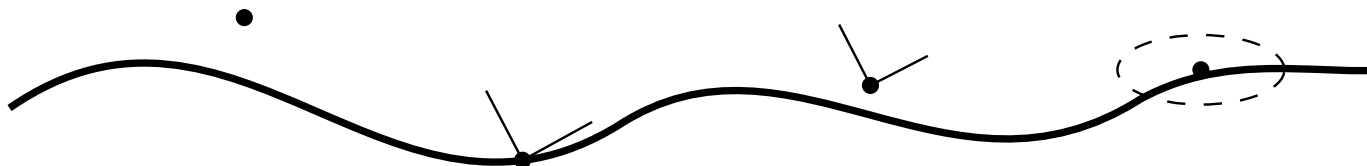


AIM Foundations

**Curvature-Driven Modelling, Classification, Runtime Evaluation, Realization,
and Optimization of Structured Railway Alignments**

Uwe Falz

July 26, 2026



Contents

Preface	xx
Abstract	xxi
Introduction	xxiii
1. The Railway Problem Before the Information Problem	xxiii
2. A Change That Does Not Stay in One Model	xxiii
3. The Problem Is Not Geometry Exchange Alone	xxiv
4. Research Question and Central Thesis	xxv
5. Contributions	xxvi
6. Methodological Orientation	xxvii
7. How to Read the Thesis	xxvii
 I. Why Alignments Matter	 1
Introduction	2
1. Motivation	3
1.1. Why a Railway Transition Is Not a Road-Corridor Detail	3
1.2. The Curvature Edit	3
1.3. Three Powerful Views, Three Necessary Limits	4
1.4. Fragmented Knowledge at the Moment of Action	4
1.5. Why Curvature Exposes the Problem	5
1.6. The Requirement for AIM	5
 2. Problem Statement	 6
2.1. From Motivation to Formalization	6
2.2. Core Deficiency of Existing Approaches	6
2.3. Formal Gap	6
2.4. Missing Coupling of State Variables	7
2.5. Data and Reality Gap	8
2.6. Lack of a Unified Optimization Framework	8
2.7. Target Formulation	8
2.8. Scope of the Problem	9
2.9. Research Questions	10

Contents

2.10.	Working Hypothesis	10
2.11.	Alignment	10
2.12.	Summary	12
3.	The AIM Proposition	13
3.1.	From Consequence to Engineering Environment	13
3.2.	From Curvature to Consequence	13
3.3.	Knowledge at the Moment of Action	14
3.4.	Responsibilities That Do Not Collapse	14
3.5.	Why Alignment-Specific?	15
3.6.	The Threshold to Foundations	15
II.	Mathematical Foundations	17
	Mathematical Foundations	18
4.	Information Modelling	19
4.1.	One Alignment, Several Accounts	19
4.2.	What Representations Contribute	19
4.3.	Why the Alignment Organizes the Relations	20
4.4.	Neighbouring Modelling Paradigms	20
4.5.	The Distinctions the Model Must Preserve	20
5.	Conceptual Structure	22
5.1.	The Ambiguity a Data Model Cannot Resolve Silently	22
5.2.	The Engineering Object and Its States	22
5.3.	Realization and Representation	23
5.4.	Observation and Knowledge	23
5.5.	Evaluation and Decision	23
5.6.	Relations Made Available	24
5.7.	Canonical Interpretation	24
6.	Axioms as Failure Boundaries	25
6.1.	What the formal model must not collapse	25
6.2.	The constructive dependency	26
7.	Principles for Using the Formal Machinery	27
7.1.	Preserve roles through every operation	27
7.2.	Keep context visible	27
7.3.	Limit every computed consequence	27
8.	Notation as a Dependency Language	28
8.1.	Where does the construction apply?	28

Contents

8.2.	What is given, and what is reconstructed?	28
8.3.	Which condition and source does a quantity describe?	28
8.4.	In which space is a comparison computed?	29
8.5.	Which operation produces the next artifact?	29
9.	Core Equations	30
9.1.	Intrinsic State and Controls	30
9.2.	Planar Intrinsic Geometry	30
9.3.	Cant Evolution and Foundational pose3 System	31
9.4.	Railway Dynamic Relations	32
9.5.	Track–World Relations	32
9.6.	Representation and Engineering Realization	33
9.7.	Surface Curvature	34
9.8.	Optimization Relations	34
9.9.	Canonical Role	35
10.	State Transition	36
10.1.	The state that answers the question	36
10.2.	Planar transition driven by curvature	36
10.3.	Spatial transition and controls	37
10.4.	Degrees of freedom during the edit	37
10.5.	What may be compared	38
11.	Typed Operators	39
11.1.	Reconstruct the edited state	39
11.2.	Realize the state metrically	40
11.3.	Relate track and world descriptions	40
11.4.	Produce representations and observations	41
11.5.	Relate target and physical condition	41
11.6.	Evaluate the qualified consequence	41
11.7.	Composition without responsibility transfer	42
12.	Metric Realization	43
12.1.	From intrinsic order to measurable geometry	43
12.2.	Keep the five roles separate	43
12.3.	Making the discrepancy computable	44
12.4.	Why the ellipsoid follows	44
13.	Execution Order and Responsibility	45
13.1.	The curvature-edit chain	45
13.2.	Where physical realization and observation enter	45
13.3.	Responsibility at each boundary	45

Contents

13.4.	Dependencies, not an implementation stack	46
14.	Curvature Realized on the Earth	47
14.1.	What remains and what changes	47
14.2.	Why one tangent is no longer enough	47
14.3.	The curvature decomposition	48
14.4.	Which curvature carries the alignment edit?	48
14.5.	Projection is an account of the result	49
15.	Spatial Pose in the Earth Realization	50
15.1.	From surface trajectory to pose3	50
15.2.	Cant rotates the realized cross-section	50
15.3.	Gravity makes the pose physically interpretable	51
15.4.	Curvature, speed, cant, and effective acceleration	51
15.5.	What this specialization establishes	51
III.	Engineering Concepts	53
	Engineering Concepts	54
16.	From Calculated Transition to Engineering Object	55
16.1.	What is the proposal about?	55
16.2.	What kind of account is present?	55
16.3.	Why identity is required next	56
17.	Identity Through Transition Change	57
17.1.	Object continuity and state succession	57
17.2.	Constructive identity makes the edit intelligible	57
17.3.	When does change create another object?	58
17.4.	Provenance preserves the path through change	58
18.	Applicability from Distributed Qualifications	59
18.1.	Purpose determines the required support	59
18.2.	Realization gives the construction metric meaning	59
18.3.	Other qualifications remain distributed	59
18.4.	The first operational test	60
19.	Representing the Candidate Without Replacing It	61
20.	Evidence About the Transition	62
21.	Knowledge for Evaluating the Candidate	63

22.	From Evaluated Proposal to Authorized State	64
22.1.	Proposal, candidate, and evaluation	64
22.2.	Authority changes engineering status	64
22.3.	Return to the transition	65
IV.	Geometry and Curvature	66
	Geometry	67
23.	Curvature Space	69
23.1.	Curvature makes the element sequence visible	69
23.2.	Reconstruction before geographic placement	70
23.3.	Curvature space and the necessary distinctions	70
23.4.	Sparse Curvature Representation	71
23.5.	Hybrid Curvature Representation	72
23.6.	Normalized Curvature Space	72
23.7.	Curvature Derivatives	73
23.8.	Frequency Interpretation	73
23.9.	Curvature Space and Runtime	73
23.10.	Curvature Space and Realization	74
23.11.	Curvature Space and Optimization	74
23.12.	Canonical Interpretation	74
23.13.	Connection to AIM	75
24.	Curvature Classes	76
24.1.	Basic Curvature Classes	76
24.2.	Continuity Classes	77
24.3.	Boundedness Classes	78
24.4.	Transition Classes	78
24.5.	Analytical Classes	79
24.6.	Sparse Curvature Classes	80
24.7.	Operational Classes	81
24.8.	Frequency Classes	81
24.9.	Admissibility	82
24.10.	Classification versus Quality	83
24.11.	Canonical Interpretation	83
24.12.	Connection to AIM	83
25.	Curvature Transitions	85
25.1.	Transition Principle	85
25.2.	Normalized Transition Space	85
25.3.	Transition Families	86

Contents

25.4.	Clothoid Transition	87
25.5.	Polynomial Transitions	87
25.6.	Trigonometric Transitions	87
25.7.	Half-Wave Interpretation	88
25.8.	Berlinish Composition	88
25.9.	Curvature Anchors	90
25.10.	Transition Continuity	91
25.11.	Sparse Transition Representation	91
25.12.	Lookup-Based Transitions	91
25.13.	Transition Derivatives	92
25.14.	Transition Symmetry	92
25.15.	Transition Frequency Behaviour	93
25.16.	Transition Realization	93
25.17.	Transition Optimization	94
25.18.	Canonical Interpretation	94
25.19.	Connection to AIM	94
26.	Curvature Classification	96
26.1.	Classification Principle	96
26.2.	Continuity Classification	97
26.3.	Curvature Behaviour Classification	97
26.4.	Symmetry Classification	98
26.5.	Derivative Classification	99
26.6.	Spectral Classification	100
26.7.	Realization Classification	100
26.8.	Runtime Classification	101
26.9.	Operational Classification	101
26.10.	Family Classification	102
26.11.	Classification Vectors	102
26.12.	Canonical Interpretation	103
26.13.	Connection to AIM	103
V.	Railway State Model	104
State		105
27.	From Curvature to Planar Pose	107
27.1.	What the Starting Pose Contributes	107
27.2.	The Running Transition	108
27.3.	Persistent Information and Derived State	108
27.4.	Why pose2 Is Not Yet Railway State	108

28.	From Planar Pose to Spatial Railway State	109
28.1.	Additional Laws, Not Additional Identity	109
28.2.	The Moving Railway Frame	110
28.3.	What pose3 Adds	110
28.4.	Target, Realization, Physical State, and Observation	111
28.5.	Cant and Curvature Meet Downstream	111
29.	Moving Frames and Their Limits	112
29.1.	Longitudinal Direction	112
29.2.	Uncanted Cross-Section	112
29.3.	Why a Pure Frenet Frame Is Insufficient	112
29.4.	Degrees of Determination	113
29.5.	World–Track and Vehicle-Space Use	113
30.	Speed-Qualified Consequences	114
30.1.	From Arc Length to Time	114
30.2.	A Reduced Illustrative Relation	114
30.3.	Variation and Operational Meaning	115
30.4.	Evaluation Is a Later Operation	115
31.	Admissibility States	116
31.1.	Admissibility State	116
31.2.	Operational Envelopes	116
31.3.	Layer Position	116
31.4.	Connection to AIM	117
32.	World–Track Coordinate Transformation	118
32.1.	Track Coordinate System	119
32.2.	Track-to-World Transformation	119
32.3.	World-to-Track Transformation	121
32.4.	Coordinate Transformation versus Projection	121
32.5.	Numerical Evaluation	121
32.6.	Ambiguity and Context Dependence	121
32.7.	Topology and Operational References	122
32.8.	Relation to Curvature	122
32.9.	Runtime Dependency	122
32.10.	LOD and Hybrid Evaluation	122
32.11.	Connection to CRS and Realization	123
32.12.	Key Insight	123
32.13.	Connection to AIM	123

VI. Dynamic Railway State	125
Dynamics	126
33. Where the State Model Ends and Dynamics Begins	128
33.1. Inherited Railway State	128
33.2. New Responsibilities	128
33.3. One State, Several Uses	129
34. State Terms as Qualified Uses	130
34.1. Terms Already Established	130
34.2. Terms Introduced by a Use or Model	130
34.3. Dependency Instead of Hierarchy	131
35. Runtime Operators	132
35.1. Operator Overview	132
35.2. Realization Operator	132
35.3. Dynamics Operator	132
35.4. Vehicle Interpretation Operator	133
35.5. Operational Evaluation Operator	133
35.6. Operator Composition	133
35.7. Runtime Abstraction and Implementation	133
35.8. Canonical Interpretation	134
35.9. Connection to AIM	134
36. Vehicle Space Relative to the Railway State	135
36.1. Relative Quantities	135
36.2. Subsystem and Reference-Point Uses	135
37. Traversal Law and Operational Velocity	137
37.1. Space Becomes Time	137
37.2. Which Velocity Is Meant?	137
37.3. Compact Variable-Speed Example	138
38. From Temporal Exposure to Modelled Response	139
38.1. A Result, Not a Second State Definition	139
38.2. Reduced Kinematics	139
38.3. The Explicit Model Boundary	140
38.4. The Running Comparison	140
38.5. AXTRAN and Declared Objectives	141
39. What <i>Schwerpunkt</i> Can Mean	142
39.1. Center of mass and reduced reference point	142

Contents

39.2.	What may be derived	143
39.3.	Schwerpunkt interpretation and realization	143
39.4.	Claim status	144
40.	Dynamic Balance as a Qualified Residual	145
40.1.	Balance relation	145
40.2.	Cant deficiency, excess and trajectory	145
40.3.	Do not collapse the roles	146
41.	Vienna Functions as a Bounded Example	147
41.1.	Separate laws	147
41.2.	Compact alternative-objective example	148
41.3.	Claim-status audit	148
42.	Realization-Aware Prediction and Comparison	149
42.1.	Typed comparison chain	149
42.2.	Realization filtering	149
42.3.	Calibration and uncertainty boundary	150
43.	Optimization Produces Candidates	151
43.1.	Problem contract	151
43.2.	Degrees of freedom	152
43.3.	Objective and scaling	152
43.4.	Candidate contract	153
43.5.	Evaluation and Engineering Decision	153
VII.	Runtime and Evaluation	154
Runtime		155
44.	Runtime Services	157
44.1.	Runtime Interpretation	157
44.2.	Runtime State Evaluation	157
44.3.	Persistent Information versus Derived Runtime State	158
44.4.	Runtime pose3 Evaluation	158
44.5.	Projection Services	158
44.6.	Frame Services	160
44.7.	Derived Dynamic Runtime Quantities	160
44.8.	Hybrid and LOD-Dependent Evaluation	160
44.9.	Caching and Consistency	160
44.10.	Metric-Aware Runtime	161
44.11.	Runtime Pipelines and Service Interaction	161

44.12.	Runtime, Realization, and Representation Boundaries	162
44.13.	Conceptual Runtime Architecture	162
44.14.	Canonical Interpretation	163
44.15.	Connection to AIM	163
VIII Reality and Data		164
Reality		165
45.	Measurement Data and Observation Context	167
45.1.	Role of Data	167
45.2.	Data Characteristics	167
45.3.	Interpretation Layer	167
45.4.	From Data to Structured Model Input	168
45.5.	Geometric and Topological Evidence	168
45.6.	Container and Exchange Context	168
45.7.	Connection to AIM	168
46.	Coordinate Systems and World Embedding	169
46.1.	Coordinate Reference Systems	169
46.2.	Embedding Context	169
46.3.	Transformations and Consistency	170
46.4.	Local Frames and Numerical Stability	170
46.5.	Stationing versus CRS Coordinates	170
46.6.	Relation to Metric Realization	170
46.7.	Connection to AIM	170
47.	Kilometre Line and Stationing	171
47.1.	Stationing Role	171
47.2.	Relation to Geometry	171
47.3.	Layer Separation	171
47.4.	Topology and Operational Continuity	172
47.5.	Connection to AIM	172
48.	Measurement and Imperfect Data	173
48.1.	Observation Context	173
48.2.	Imperfect Data Characteristics	173
48.3.	Inference from Observation	173
48.4.	Observation and Runtime Reconstruction	174
48.5.	Engineering Use	174
48.6.	Connection to AIM	174

49.	Topology and Special Elements	175
49.1.	Topology Context	175
49.2.	Special Elements	175
49.3.	Geometry–Topology Separation	175
49.4.	Data Ambiguity and Inference	175
49.5.	Connection to AIM	176
50.	Construction and Physical Realization	177
50.1.	Physical Realization versus Metric Realization	177
50.2.	Realization Operator View	178
50.3.	Construction and Adjustment Process	178
50.4.	Sampling, Locality, and Filtering	178
50.5.	Runtime and Optimization Implications	179
50.6.	Connection to AIM	179
51.	Realized pose3 and Target–Realized Comparison	181
51.1.	Target, Realized, and Measured pose3	181
51.2.	Realized State Components	182
51.3.	Runtime Reconstruction of Realized pose3	182
51.4.	Observation Operators and Sensor Integration	182
51.5.	Dynamic Comparison Quantities	183
51.6.	Realization Operator on pose3	183
51.7.	Connection to AIM	183
IX.	Alignment Modelling	185
	Modelling	186
52.	Sparse Alignment Model	189
52.1.	General Idea	190
52.2.	Element Types	190
52.3.	Minimal Parameterization	191
52.4.	Initial Data and Pose	191
52.5.	Sparse Longitudinal Band Structure	191
52.6.	Concatenation	193
52.7.	Normal Form and Alternation	194
52.8.	Normalized Representation of Elements	194
52.9.	Structural Role of the Sparse Model	194
52.10.	Relation to Imported and Standardized Data	195
52.11.	Relation to BIM and BIMalignment	195
52.12.	Summary	195

53.	Transition Functions	196
53.1.	Definition by Curvature Evolution	196
53.2.	Normalized Representation	196
53.3.	Scaling	197
53.4.	Regularity and Smoothness	198
53.5.	Transition Families	198
53.6.	Decomposition Approaches	199
53.7.	Operator View	199
53.8.	Implementation Perspective	199
53.9.	Relation to Railway Engineering	200
53.10.	Summary	200
54.	Transition Families and Classification	201
54.1.	Curvature-Based Representation	201
54.2.	Classification by Functional Form	201
54.3.	Classification by Higher-Order Behaviour	202
54.4.	Comparison of Families	203
54.5.	Interpretation as Control Functions	203
54.6.	Connection to Railway Engineering	203
54.7.	Main Insight	203
54.8.	Conclusion	203
55.	Schwerpunkt-Trassierung as a Qualified Model Idea	204
55.1.	Historical proposition, present boundary	204
55.2.	A deliberately reduced reference-point model	205
55.3.	From model output to an optimization problem	206
55.4.	Degrees of freedom and candidate status	206
55.5.	Scientific potential without canonization	207
X.	System and Pipeline	208
Analysis		209
55.6.	AIM Processing Pipeline	211
55.7.	AIM Pipeline Overview	214
55.8.	AIM Master Architecture	214
55.9.	Pipeline Story: From Measurement to Alignment	215
55.10.	Failure Modes and Limitations of the AIM Pipeline	217
55.11.	Design Principles of Alignment-Based Information Modelling	221

XI. Optimization	226
Optimization	227
56. Numerical Reconstruction for a Reviewable Candidate	230
56.1. Why a curvature edit needs numerics	230
56.2. From a continuous model to computable samples	230
56.3. Residuals answer declared questions	231
56.4. Accuracy belongs to the result evidence	231
56.5. Why derivatives and sparsity appear next	232
57. From an Alignment Problem to a Mathematical Candidate	233
57.1. General Optimization Problem	234
57.2. Parameterization	234
57.3. Constraints	235
57.4. Relation to Numerical Reconstruction	236
57.5. Sequential Quadratic Programming	236
57.6. Alignment Optimization as Control Problem	236
57.7. Network-Level Optimization	236
57.8. Robustness and Uncertainty	236
57.9. Implementation Perspective	236
57.10. Historical Context	237
57.11. Formal Optimization in Curvature Space	237
57.12. Coupled Optimization of Curvature and Cant	239
57.13. Summary	241
57.14. Discrete Formulation of Alignment Optimization	242
57.15. Sequential Quadratic Programming (SQP)	245
57.16. Construction of Initial Guesses	247
57.17. Analytical Structure of the Optimization Problem	249
57.18. Parametric Representation instead of Sampling	251
57.19. Hybrid Parametric–Discrete Model	254
57.20. Hybrid Representation and Block Structure	256
58. A Solver Architecture for Reviewable Candidates	260
58.1. Architecture begins at a responsibility boundary	260
58.2. The problem package	260
58.3. Assembly makes mathematical structure explicit	261
58.4. Why SQP, derivatives, and sparsity belong behind interfaces	261
58.5. Termination, convergence, and verification	261
58.6. The candidate package and review boundary	262
58.7. Current AXTRAN evidence and the general architecture	262
58.8. What the engineer receives	262

59.	Qualified Realization-Aware Optimization	264
59.1.	Five objects that must remain separate	264
59.2.	Problem contract before the minimum	265
59.3.	Curvature and cant: coupled question, separate ownership	265
59.4.	Objectives, units and scaling	266
59.5.	Constraints and applicability	266
59.6.	Uncertainty and calibration boundary	266
59.7.	Inverse and frequency interpretations	267
59.8.	Repeated optimization is a workflow hypothesis	267
59.9.	Candidate, evaluation and decision	267
59.10.	Vehicle, Earth and implementation boundaries	267
59.11.	Claim-status summary	268
XII.	Engineering and Standards	269
Engineering		270
60.	Standards and Data Models	272
60.1.	General Perspective	272
60.2.	LandXML	272
60.3.	IFC	273
60.4.	railML	274
60.5.	Legacy and Proprietary Formats	275
60.6.	Canonical Internal Model	276
60.7.	Transformation Pipeline	276
60.8.	Relation to BIM	276
60.9.	Interoperability	277
60.10.	Discussion	277
60.11.	Summary	277
61.	Constraint Code Architecture	278
61.1.	General Principle	278
61.2.	Constraint Registry	278
61.3.	Constraint Object Model	279
61.4.	Context Object	279
61.5.	Geometric Constraints	280
61.6.	Dynamic Constraints	280
61.7.	Regulatory Constraints	281
61.8.	Topological Constraints	281
61.9.	Constraint Assembly	282
61.10.	Factory Functions	282

Contents

61.11.	Constraint Presets	283
61.12.	Integration into the Solver	283
61.13.	Discussion	284
61.14.	Summary	284
62.	Railway Design Rules	285
62.1.	Categories of Rules	285
62.2.	Geometric Constraints	286
62.3.	Dynamic Constraints	287
62.4.	Coupling of Curvature and Cant	288
62.5.	Speed-Dependent Constraints	288
62.6.	Constraint Representation in AIM	288
62.7.	Integration into Optimization	289
62.8.	Relation to Standards	289
62.9.	Discussion	289
62.10.	Summary	289
XIII	Applications	290
	Applications	291
63.	Railway Alignment Applications	293
63.1.	Application Context	293
63.2.	Use of Canonical State and Runtime Concepts	293
63.3.	Transition and Coupled Behaviour in Practice	293
63.4.	Operational and Maintenance Consequences	293
63.5.	Interoperability Context	293
63.6.	Connection to AIM	294
64.	Vehicle Space and Platform Interaction	295
64.1.	Application Context	295
64.2.	Derived Vehicle-Space Interpretation	295
64.3.	Engineering Use Cases	295
64.4.	Representation Boundary	295
64.5.	Connection to AIM	295
65.	Railway Dynamics versus Aircraft Dynamics	296
65.1.	Comparative Context	296
65.2.	Railway Consequence	296
65.3.	AIM Interpretation	296
65.4.	Connection to AIM	296

66.	Application Examples	297
66.1.	Example Pattern A: Transition Comparison	297
66.2.	Example Pattern B: Cant-Coupled Interpretation	297
66.3.	Example Pattern C: Realized-State Comparison	297
66.4.	Connection to AIM	297
67.	Use Cases	298
67.1.	Use-Case Structure	298
67.2.	Representative Use Cases	298
67.3.	Workflow Consequences	298
67.4.	Connection to AIM	298
68.	BIM Alignment Modelling	299
68.1.	Application Context	299
68.2.	AIM-to-BIM Integration Role	299
68.3.	Pipeline Interpretation	299
68.4.	Boundary Clarification	300
68.5.	Connection to AIM	300
69.	Contribution	301
69.1.	Contribution Scope	301
69.2.	Application-Level Contributions	301
69.3.	Consequence for Engineering Practice	301
69.4.	Connection to AIM	301
70.	Discussion	302
70.1.	Positioning	302
70.2.	Strengths Observed in Applications	302
70.3.	Limits and Practical Conditions	302
70.4.	Interoperability Consequence	302
70.5.	Connection to AIM	302
XIV	Discussion	304
	Discussion	305
71.	axtranNew: From Transition Grammar to Alignment Calculation	307
71.1.	Original Intent	307
71.2.	Mathematical Capability	307
71.3.	Observed Current Implementation	308
71.4.	Current Boundary and Future Development	308
71.5.	From Geometry Optimization to Runtime-State Optimization . .	308

Contents

71.6.	Connection to AIM	309
XV.	Outlook	310
	Outlook	311
72.	BIM and Alignment Modelling	314
72.1.	BIM as a Data Integration Paradigm	314
72.2.	Alignment in BIM Standards	314
72.3.	Limitations of Current BIM Alignment Modelling	315
72.4.	AIM as a Computational Layer for BIM	316
72.5.	Alignment-Centred BIM	317
72.6.	Integration with BIM Workflows	317
72.7.	Relation to BIMinfra	318
72.8.	Discussion	318
72.9.	Summary	319
73.	Future Work	320
73.1.	Purpose of this Chapter	320
73.2.	Extension of the Dynamic Model	320
73.3.	Schwerpunkt-Trassierung	321
73.4.	Advanced Optimization Methods	321
73.5.	Network-Level Modelling	322
73.6.	Constraint and Objective Libraries	322
73.7.	Integration with BIM and Data Standards	323
73.8.	Interactive and Real-Time Systems	323
73.9.	AI and Assisted Design	324
73.10.	AXTRAN Revival and Beyond	324
73.11.	Software Architecture and Deployment	324
73.12.	Long-Term Vision	325
73.13.	Summary	325

Preface

This thesis grew from a recurring engineering experience: an alignment may be geometrically precise and still be difficult to understand as the same engineering object across design systems, measurements, construction records, and decisions. Alignment-Based Information Modelling (AIM) is an attempt to make that continuity explicit and computable.

The argument begins with engineering practice, develops the conceptual and mathematical foundations that the problem requires, and returns to implementation, evaluation, and application. The reader is not expected to know the AIM terminology in advance.

Abstract

An engineer changes the curvature of a railway alignment. The design system immediately draws a new curve, but the engineering consequences do not remain inside that drawing. Measurements refer to a physical track; GIS data place it in a wider world; CAD and exchange models describe selected constructions; rules and decisions govern whether the change is admissible. Across these representations, the alignment must remain identifiable, while intended, realized, and observed states must remain distinguishable. The central problem is therefore not the exchange of geometry alone. It is how to make alignment-specific engineering knowledge available at the moment a change is interpreted, evaluated, or decided.

The point of departure was more specific. Railway transition design must shape the passage between curvature states while a vehicle is physically constrained to follow its rail pair. Curvature, cant, speed, and their rates of change act through that constraint on vehicle, track, comfort, safety, and wear. The originating contribution called *Berlinish* treats known transition forms through a composable grammar of entering half-wave, optional clothoid core, and exiting half-wave. *axtranNew* arose as the intended calculation core for composing, solving, comparing, and optimizing such structures. These names denote an originating research contribution and an implementation lineage; they are not introduced here as approved Knowledge Kernel concepts.

This thesis develops *Alignment-Based Information Modelling (AIM)* as a knowledge-based, representation-aware, and implementation-capable response. Its central claim is that an alignment should be modelled through its intrinsic constructive structure—in particular its longitudinal ordering and curvature evolution—and then interpreted through explicit state, realization, observation, and evaluation relations. A curve is a necessary geometric realization of that structure, but it is not the whole engineering object.

The work contributes a conceptual separation of object identity, constructive definition, metric and physical realization, representation, observation, knowledge, and decision; a curvature-driven state and operator formulation; an alignment-specific engineering treatment of transitions, cant, constraints, and realization; a reference-oriented computational pipeline including hybrid reconstruction and structured optimization; and an explanatory sequence that connects these contributions from engineering problem to application.

The wider uAIM environment follows from the need to preserve and operate this transition knowledge: a transition registry structures functions and families; TransEd makes them inspectable; SPOT carries the identity and state of concrete engineering objects; alignmentOS makes calculated alignment knowledge usable; the Knowledge Kernel

Contents

stabilizes terminology and responsibility; Research tests and extends claims; and the Thesis explains their relation. The resulting framework is broader than its origin, but that origin supplies its first engineering necessity.

The mathematical and numerical developments show how geometry can be derived from curvature, how relevant states and constraints can be evaluated, and how alignment problems can be posed for computation. They do not imply that a solver owns engineering knowledge or decision authority. AIM instead provides a disciplined way to preserve meaning while representations change and to apply engineering knowledge where consequences arise.

Introduction

1. The Railway Problem Before the Information Problem

A railway vehicle cannot choose a nearby path when the geometry becomes unfavourable. Its wheelsets are constrained to follow the rail pair. A transition between a straight and a circular arc therefore changes more than a drawn route: the curvature law and the development of cant, together with speed, govern how lateral acceleration, roll, force development, comfort, wear, and admissibility evolve along the vehicle trajectory.

Compare a clothoid with a transition whose curvature derivative also settles smoothly at its ends. Both can connect the same boundary curvatures. The clothoid gives constant $\kappa'(s)$, whereas a smoothed law can suppress the endpoint changes of $\kappa'(s)$ and control $\kappa''(s)$. The latter may offer preferable physical behaviour, yet usually spends length on that smoothing. In the originating engineering comparison, the clothoid's decisive advantage was therefore not assumed to be universal dynamic superiority; it was often its compact, transparent construction. That observation opened the question that precedes AIM: how can different known transition forms be expressed, calculated, and compared within one functional language?

The originating answer was *Berlinish*, a generalized transition grammar, and *axtranNew*, the intended calculation core that makes this grammar applicable. The broader ufAIM environment developed later because formulas and solver results alone could not keep the resulting knowledge durable, inspectable, editable, attributable, and usable across engineering workflows. This history is a Thesis interpretation of the development lineage, not a claim that *Berlinish* or *axtranNew* is an approved Kernel concept.

2. A Change That Does Not Stay in One Model

Consider an alignment received in local engineering coordinates. An engineer changes the curvature of one transition. The geometric effect is immediate: the downstream course moves, derived positions change, and a new curve appears on the screen. Yet this apparently local edit opens a larger set of questions. Does the edited model still describe the same alignment? Which observations refer to the existing track, and which values express the proposed design? Which rules apply in this project context? Who is entitled to accept the consequence?

No single view answers all of these questions. GIS can place the alignment among terrain, parcels, networks, and environmental data, but relevance must be selected from an

abundance of available context. CAD gives the engineer exact construction and editing power, while usually presenting only the scope needed for the immediate task. BIM and exchange models can connect objects and attributes across disciplines, but delivery requirements often determine what is represented and at what depth. Measurements, calculation sheets, standards, issue records, and experienced engineers hold further parts of the meaning.

This fragmentation is not evidence that the tools have failed. Each is useful within its purpose. The difficulty appears between their purposes: engineering knowledge is distributed across representations, documents, workflows, and people, while the consequence of an alignment change must be understood at one particular moment of action.

Engineering software can describe almost every visible consequence of an alignment change, yet it often cannot state what must remain the same while those descriptions change.

That tension motivates this thesis.

3. The Problem Is Not Geometry Exchange Alone

Geometric interoperability matters, but successful transfer of coordinates or curve elements does not settle the engineering problem. Six distinctions must survive the transfer.

Identity

The engineering object must remain distinguishable across files, views, coordinate realizations, and workflow changes. Numerical equality alone does not establish that two descriptions concern the same object, and geometric change alone does not decide that one object has become another.

Constructive intent

The ordered longitudinal construction and its curvature evolution explain how the alignment is defined. Sampled points or rendered curves may expose this construction, but do not own it.

Reality and observation

The intended alignment, the physically realized track, and an observation of that track are related but different. A measurement is evidence obtained under particular conditions; it is not automatically the physical state or accepted engineering knowledge.

Change and consequence

A curvature edit propagates into position, orientation, cant interaction, vehicle response, clearances, construction, and compliance. The dependencies that carry those consequences must be explicit enough to evaluate.

Knowledge ownership

A format, viewer, solver, or database may carry and use engineering knowledge without acquiring authority over its meaning. Scope, provenance, applicability, and accountable ownership must remain visible.

Decision

Calculation and optimization can produce evidence and alternatives. They do not by themselves accept an engineering change. Evaluation and decision authority must remain distinct.

The unresolved problem can therefore be stated concisely: how can an alignment retain coherent engineering meaning across representations and changes, while making the right knowledge applicable when an engineer acts?

4. Research Question and Central Thesis

The guiding research question is:

How can alignment-specific engineering identity, construction, state, reality, observation, knowledge, and evaluation be organized so that changes remain traceable and their consequences become computable across representations?

This question generalizes the originating transition problem. The initial technical question asked how curvature functions, boundary conditions, component lengths, and degrees of freedom could describe and solve railway transitions without declaring one curve family universally best. Once those calculated transitions had to survive editing, exchange, observation, comparison, and accountable use, the information-modelling question became unavoidable.

The thesis answers with Alignment-Based Information Modelling (AIM). It is knowledge-based: engineering use depends on admitted meaning and applicability. It is alignment-specific: longitudinal construction and curvature evolution organize the object. It is representation-aware: no file, view, or coordinate realization is mistaken for the object itself. It is implementation-capable: the relations are developed into states, operators, constraints, pipelines, and numerical methods.

Proposition 0.1 (Central thesis). *An alignment can be treated coherently across design, realization, observation, evaluation, and change when its intrinsic constructive identity is kept distinct from its representations and is connected to applicable engineering knowledge through explicit state and operator relations.*

Curvature has a privileged role in this argument. For an alignment, ordered curvature evolution belongs to the intrinsic constructive structure, while position and pose arise through metric interpretation and integration. This is why the thesis moves from the familiar curve to a curvature-driven generative model. It does not deny geometry; it explains where geometry comes from and what geometry alone cannot own.

5. Contributions

The work makes five kinds of contribution.

Conceptual contribution

It establishes a coherent alignment-centred account of engineering object identity, constructive identity, representation, metric and physical realization, observation, engineering knowledge, evaluation, and decision.

Formal and methodological contribution

It develops curvature-driven pose and state formulations, explicit spatial and runtime operators, realization-aware dependencies, constraint evaluation, and structured optimization formulations.

It also makes the originating Berlinish composition explicit as a mathematical framework for relating half-wave, clothoid, asymmetric, and compound transition forms without promoting that research contribution to Kernel authority.

Engineering contribution

It connects alignment transitions, cant, vehicle-related quantities, stationing, coordinate context, observations, construction effects, and railway rules without collapsing their different meanings into one geometric model.

Implementation and reference contribution

It specifies computational pipelines and sparse or hybrid representations that support reconstruction, evaluation, and optimization while preserving the boundary between canonical engineering meaning and derived products.

Within that contribution, *axtranNew* denotes the calculation lineage from connection and boundary solving to editable alignment structures. The current reference implementation supports registry-based transition evaluation, sparse reconstruction, selected native edits, comparisons, and bounded optimization problems; a general production-ready optimizer remains future work.

Explanatory contribution

It provides a dependency-ordered account through which a technically qualified reader can move from an ordinary alignment edit to the conceptual and computational machinery needed to assess its consequences.

These are foundation contributions. The thesis demonstrates formulations, relations, and implementation pathways; it does not claim to replace every domain tool, provide a complete multibody railway model, or transfer engineering decision authority to computation.

6. Methodological Orientation

The investigation alternates between conceptual separation and constructive demonstration. It first asks what kind of thing each item is—object, representation, state, observation, knowledge, or decision—and which dependencies are legitimate. It then develops mathematical models and operators that make selected dependencies computable. Railway examples, standards-related constraints, data reconstruction, and optimization test whether the distinctions remain useful under engineering pressure.

The argument proceeds from examples to formalization. Equations are introduced when they expose a dependency or enable evaluation, not as substitutes for the engineering question. Numerical solutions are treated as candidates or evidence within an accountable process, not as self-authorizing answers.

7. How to Read the Thesis

The parts are ordered around questions the reader can carry forward.

1. *Why does an alignment require more than a curve?* The opening part sharpens the practical problem and the AIM proposition.
2. *What must remain distinct before mathematics can help?* Mathematical Foundations and Engineering Concepts establish identity, knowledge, representation, realization, observation, and decision boundaries.
3. *How does intrinsic construction acquire geometric and operational meaning?* Geometry, Railway State, and Dynamic Railway State derive pose, cant, vehicle-related quantities, and admissibility from longitudinal state.
4. *How are consequences evaluated while a system is being used?* Runtime and Evaluation makes the application of operators and knowledge explicit.
5. *How do design, the physical track, and evidence about it remain related without becoming identical?* Reality and Data treats coordinates, measurement, construction, and realized state.
6. *How can the account be implemented and changed deliberately?* Alignment Modelling, System and Pipeline, and Optimization develop sparse models, computational dependencies, and solution methods. They also show how Berlinish, axtranNew, transitionDB, TransEd, SPOT, and alignmentOS occupy different roles in the development response.
7. *What survives contact with rules, exchange systems, and engineering use?* Engineering and Standards, Applications, Discussion, and Outlook test the framework's reach and state its limits.

Introduction

The curvature edit introduced at the start will recur as a test: what remains the same, what changes, what is observed, what is inferred, and who may decide? The next part begins by returning to that edit before any formal ontology is required.

Part I.

Why Alignments Matter

Why Alignments Matter

An alignment edit is never only an edit to a curve. It changes a constructive definition, produces geometric consequences, may be compared with observations, and may require an accountable engineering decision. This part examines that chain before introducing the formal machinery used later in the thesis.

Three questions organize the chapters: why does current practice fragment the meaning of an alignment; which deficiency must a new account resolve; and what kind of framework could make engineering knowledge applicable without confusing the object with any one of its descriptions? The answers establish the need for AIM and prepare the mathematical foundations that follow.

1. Motivation

1.1. Why a Railway Transition Is Not a Road-Corridor Detail

A road corridor influences a driver’s available path, but a railway vehicle is kinematically constrained by its rail pair. The track therefore prescribes the curvature history experienced by the vehicle. A change in $\kappa(s)$, its derivatives, cant $u(s)$, or speed is transmitted into wheel–rail interaction, vehicle response, passenger comfort, track loading, wear, and safety-related assessment. Transition design is consequently a central railway problem, not merely a convenient interpolation between map elements.

Consider two laws with the same length and boundary curvatures. A clothoid has constant curvature derivative. A polynomial or trigonometric half-wave can instead arrange selected endpoint conditions on $\kappa'(s)$ and $\kappa''(s)$. Similar-looking centerlines can therefore carry different dynamic implications. Conversely, demanding smoother endpoint behaviour may require more length or a different allocation of degrees of freedom. The clothoid is important, simple, and frequently compact; these advantages do not make it the universal physical optimum [4, 29].

This comparison produced the originating research direction of the work. Berlinish sought a shared functional grammar for such alternatives, while axtranNew sought to calculate their admissible compositions. AIM later widened the question from *which transition can be calculated?* to *how can the resulting engineering knowledge retain meaning and become usable?*

1.2. The Curvature Edit

The curvature edit introduced in the opening moves the downstream positions and changes the tangent direction. If cant and speed remain fixed, vehicle-related quantities change as well. Constraints satisfied before the edit may no longer be satisfied afterwards; standard railway design couples curvature, cant, speed, and their rates of change [7, 26].

The visible curve is therefore the beginning of the consequence, not its end. The edit must be interpreted against terrain and adjacent assets, compared with the existing or measured track, checked against applicable rules, and presented as an alternative for review. At every step, a different representation may be appropriate. What must not disappear is the relation among the alignment, the edit, the resulting states, the evidence, and the authority to decide.

1.3. Three Powerful Views, Three Necessary Limits

GIS, CAD, and BIM approach this situation from different strengths. GIS places the alignment in extensive spatial context and supports interoperable access, processing, visualization, and discovery of geospatial information [39]. This breadth is indispensable for land, environment, networks, and spatial analysis, but the engineer must still determine which available context is applicable to the alignment change.

CAD concentrates geometric construction, communication, and direct manipulation [20]. It makes the curvature edit tangible and produces geometry needed for design and delivery. Its geometric model, however, does not by itself establish the provenance of an observation, the authority behind a rule, or the durable identity shared by another representation.

BIM organizes built-asset information for exchange among software applications and project participants. IFC makes this role explicit through schemas and workflow-specific model views [5]. These structures can carry alignment information without thereby owning its intrinsic construction or all knowledge required to evaluate a change.

The issue is not to choose a winner among the three. The issue is that the alignment must pass between their views without becoming merely a GIS feature, a CAD curve, or a BIM container. A representation is useful precisely because it selects and expresses something for a purpose. That selection also means no single representation should silently become the whole engineering object.

1.4. Fragmented Knowledge at the Moment of Action

Some relevant knowledge is encoded in software. Some resides in standards, project requirements, measurement reports, calculation procedures, and design decisions. Some remains with engineers who know which rule is applicable and which apparent discrepancy is consequential. Fragmentation becomes costly when an action demands these pieces together. Construction-informatics research relates this persistent fragmentation to specialization, heterogeneous tools, and only partially interoperable collaboration [51].

For the curvature edit, the engineer needs more than data access. The software must know which object is being changed, which state is intended, which observations describe another state, which context selects the rules, and which quantities depend on the edit. It must also preserve the boundary between calculation and authorization.

This is the practical motivation for making engineering knowledge applicable. Applicable knowledge is not simply text stored beside geometry. It has scope, provenance, conditions, and an accountable meaning; it must be connected to the state and operation for which it matters.

1.5. Why Curvature Exposes the Problem

A railway alignment is often introduced as a sequence of straights, circular arcs, and transition curves. That description is useful, but curvature reveals the generative relation beneath it. Along intrinsic arc length (s), curvature governs the change of heading,

$$\theta'(s) = \kappa(s),$$

and integration produces planar pose and position. Changes in curvature and cant also enter engineering assessments of vehicle response and comfort [32, 4].

Curvature thus connects constructive intent with geometric and dynamic consequence. Yet the resulting coordinates still do not own the alignment's identity, and a measured curve still does not automatically reveal the intended curvature law. The distinction between intrinsic construction, metric realization, physical realization, and observation is not philosophical decoration. It determines what can safely be inferred after the edit.

1.6. The Requirement for AIM

A useful response must satisfy four demands at once:

- preserve alignment identity across purposeful representations;
- expose the constructive and state dependencies through which changes acquire consequences;
- keep intended, realized, and observed information distinct and traceable;
- apply engineering knowledge without assigning decision authority to a file format, software component, or numerical solver.

Alignment-Based Information Modelling is developed to meet these demands. Its starting point is not a larger file format. It is an alignment-specific account of what the engineering object is, how it acquires measurable and operational state, how evidence relates to it, and how knowledge participates in evaluation.

Summary 1.1. The difficulty exposed by a curvature edit is not that software cannot draw the new curve. It is that the engineering meaning of the change crosses tools, states, evidence, rules, and authority. AIM begins by preserving those distinctions and making their dependencies explicit.

2. Problem Statement

2.1. From Motivation to Formalization

The previous chapter followed one curvature edit across geometry, context, evidence, rules, and authority. We now identify the formal gaps that prevent those relations from being treated coherently.

2.2. Core Deficiency of Existing Approaches

Existing approaches offer strong geometric, spatial, object-oriented, and numerical capabilities. The core deficiency is not the absence of these capabilities but the absence of one alignment-specific account of their relations. A transferred curve does not establish durable object identity. A measured position does not state whether it is intended, physically realized, or observed. A successful constraint check does not identify the owner of the rule or authorize a change.

Within the geometric and numerical domain, a related gap remains: curvature, cant, dynamic quantities, realization effects, and optimization are frequently handled as successive tasks rather than as explicitly coupled dependencies. The thesis therefore addresses both semantic continuity and computational coupling. Solving only one would leave the curvature edit incompletely understood.

The originating deficiency was already visible inside transition theory. Named curves were commonly treated as separate constructions even when their curvature functions could be compared through common boundary values and derivative behaviour. That made it difficult to distinguish four different things: the identity of a function family, its parameterization, the boundary conditions of a particular task, and the instantiated transition geometry obtained after solving that task.

2.3. Formal Gap

Let

$$\gamma : [0, L] \rightarrow \mathbb{R}^2$$

denote a planar alignment.

In classical formulations, the curve γ is the primary object, while curvature is derived as

$$\kappa(s) = \theta'(s).$$

2. Problem Statement

However, from a dynamic perspective, the situation is reversed:

- curvature determines acceleration,
- curvature variation determines jerk,
- and therefore curvature governs physical behaviour.

This reveals a fundamental mismatch between:

- geometric modelling (curve-first),
- and physical interpretation (curvature-first).

2.4. Missing Coupling of State Variables

Let

$$\kappa(s), \quad \phi(s)$$

denote curvature and cant.

In practice, these variables are strongly coupled through

$$a_{\text{eff}}(s) = v^2 \kappa(s) - g \sin \phi(s).$$

Despite this coupling:

- κ is typically designed geometrically,
- ϕ is designed in a separate process,
- and their interaction is only checked afterwards.

This separation prevents a unified optimization of the system.

Operational coupling does not require identical functional identity. Curvature and cant may be described by separate laws,

$$\kappa(s) \quad \text{and} \quad u(s),$$

with distinct parameters and boundary conditions. Choosing a common law, coupled parameters, or deliberately separated developments is a modelling and optimization decision. ViennaCurve examples motivate this freedom by showing how curvature and cant development can be shaped separately while their combined operational consequence is still evaluated. This Thesis treatment is an originating research interpretation; it does not assert a new unified Kernel concept.

2.5. Data and Reality Gap

Modern railway projects rely heavily on real-world data, including:

- measurement polylines,
- legacy alignment descriptions,
- and heterogeneous coordinate systems.

Existing models struggle to:

- integrate noisy and incomplete data,
- reconcile different coordinate reference systems,
- and maintain a consistent representation across scales.

This leads to a disconnect between:

- theoretical alignment models,
- and real-world engineering data.

2.6. Lack of a Unified Optimization Framework

Alignment design problems naturally take the form:

$$\min J(\kappa, \phi)$$

subject to geometric and engineering constraints.

However, in current practice:

- the objective function is implicit or fragmented,
- constraints are applied in separate stages,
- and optimization is not formulated as a single problem.

This prevents the use of systematic numerical optimization methods.

2.7. Target Formulation

We aim to formulate alignment design as a unified problem:

$$\text{find } (\kappa, \phi)$$

such that:

2. Problem Statement

- geometric consistency is satisfied,
- engineering constraints are fulfilled,
- and a global objective functional is minimized.

This requires:

- a curvature-based representation,
- a coupled state model,
- and a consistent treatment of data and constraints.

For transitions, the target includes the compositional form

$$T(s) = H_{\text{in}}(s) + C(s) + H_{\text{out}}(s),$$

where an entering half-wave, an optional clothoid section, and an exiting half-wave occupy consecutive subdomains and each component may have zero length. This is the Berlinish mathematical framework developed later in section 25.8; it is presented as an originating research contribution rather than as Kernel authority.

2.8. Scope of the Problem

The problem addressed in this work includes:

- modelling of alignments via curvature functions,
- integration of cant as a state variable,
- reconstruction of geometry from curvature,
- incorporation of engineering rules as constraints,
- and formulation of optimization problems.

The problem explicitly excludes:

- detailed multibody vehicle dynamics,
- exhaustive modelling of every infrastructure asset,
- and project-specific construction planning.

2.9. Research Questions

The central question stated in the introduction is developed through five operational questions:

- Which intrinsic construction distinguishes an alignment independently of its coordinates and representations?
- How can that construction be realized as geometric, dynamic, and operational state through explicit operators?
- How can intended state, physical realization, observations, and accepted engineering knowledge remain related without becoming identical?
- How can applicable rules and engineering consequences be expressed for computation while their ownership and decision authority remain explicit?
- Which representations and numerical methods support reconstruction, evaluation, and optimization without redefining the engineering object?

2.10. Working Hypothesis

The working hypothesis is that a curvature-driven constructive account, combined with explicit state, realization, observation, knowledge, and evaluation relations, provides a coherent foundation for alignment engineering across representations. Within that foundation, coupled curvature and cant models, hybrid reconstruction, engineering constraints, and structured optimization can make selected consequences computable without treating a numerical result as an engineering decision.

2.11. Alignment

2.11.1. Motivation

In classical geometry, a railway track is often represented as a curve $\gamma(s)$ in space.

This representation is insufficient for engineering purposes, as it does not capture the intrinsic quantities governing construction, dynamics, and realization.

In particular, curvature and cant are not derived properties, but primary design variables.

2.11.2. Definition

Definition 2.1 (Alignment). An **alignment** is a structured object defined over arc-length $s \in [0, L]$, consisting of the following components:

- a curvature function

$$\kappa : [0, L] \rightarrow \mathbb{R},$$

2. Problem Statement

- a cant function

$$\phi : [0, L] \rightarrow \mathbb{R},$$

- an initial pose

$$(\gamma_0, \theta_0),$$

together with the induced state variables

$$\theta(s) = \theta_0 + \int_0^s \kappa(\sigma) d\sigma,$$

$$\gamma(s) = \gamma_0 + \int_0^s (\cos \theta(\sigma), \sin \theta(\sigma)) d\sigma.$$

2.11.3. Interpretation

An alignment is not defined by its position $\gamma(s)$, but by its curvature $\kappa(s)$.

The curve $\gamma(s)$ is a derived quantity obtained by integration.

Thus, curvature is the primary design variable, while position is a consequence.

2.11.4. State Structure

Definition 2.2 (Alignment state). The **state** of an alignment at position s is given by

$$(\gamma(s), \theta(s), \kappa(s), \phi(s)).$$

This state couples geometry, orientation, and dynamics in a unified representation.

2.11.5. Structural Nature

Proposition 2.3. *An alignment is a low-dimensional structured object, even though its realization may involve high-dimensional data.*

This structure enables:

- efficient storage,
- robust optimization,
- meaningful interpretation.

2.11.6. Relation to Realization

The alignment represents the intended geometry.

The physically constructed track is given by the realization operator:

$$\gamma_{\text{real}} = \mathcal{R}(\kappa, \phi).$$

Thus, the alignment exists in design space, while the track exists in physical space.

2.11.7. Relation to Data

Measured data typically provide sampled positions or polylines.

An alignment must be reconstructed from such data via inverse problems.

2.11.8. Key Insight

Proposition 2.4. *An alignment is not a curve, but a curvature-driven generative model of a curve.*

2.11.9. Summary

Summary 2.5. The alignment is the central object of railway geometry.

It is defined by curvature and cant, from which all geometric and dynamic quantities are derived.

This perspective shifts the focus from describing curves to defining generative structures, forming the foundation of Alignment-Based Information Modelling.

2.12. Summary

Summary 2.6. Current alignment design methods suffer from fragmentation between geometry, dynamics, and data.

This work addresses the problem of developing a unified, curvature-driven framework that integrates these aspects into a single coherent model.

The goal is to enable consistent modelling, analysis, and optimization of railway alignments.

3. The AIM Proposition

3.1. From Consequence to Engineering Environment

The preceding chapters did not reveal a shortage of representations. They revealed a shortage of continuity between them. The curvature edit can be drawn, located, exchanged, measured, checked, and discussed, yet each operation sees a different part of its meaning. AIM responds by organizing an engineering environment around the alignment and the dependencies through which an action acquires consequences.

In that environment, the engineer can ask one connected sequence of questions: Which alignment is being changed? Which constructive law changes? Which states follow from it? Which context and observations are relevant? Which knowledge and constraints apply? What evidence supports acceptance or rejection? The system does not answer all of these questions in the same way. It preserves their relations so that each responsible tool or person can answer the appropriate one.

Proposition 3.1 (AIM proposition). *Alignment-Based Information Modelling is an alignment-specific engineering environment in which durable object identity, constructive definition, realization, representation, observation, applicable knowledge, evaluation, and decision remain distinct but explicitly related at the moment of action.*

3.2. From Curvature to Consequence

The running edit illustrates the required chain. Ordered curvature evolution belongs to the alignment's intrinsic construction. Under a metric interpretation it generates plan geometry; with further state and context it supports spatial, dynamic, and rule-based evaluation. Different representations expose different parts of that chain. None owns the entire chain.

3. The AIM Proposition

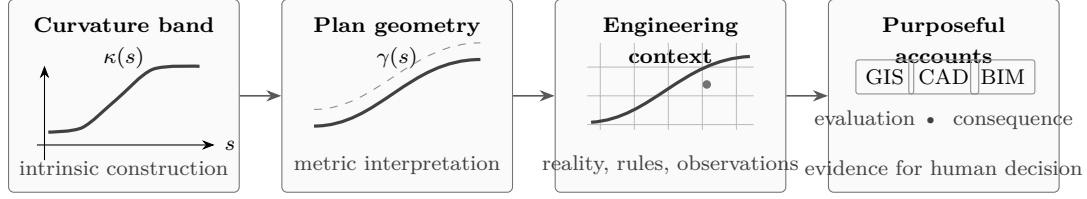


Figure 3.1.: One curvature edit, several necessary interpretations. The curvature band generates plan geometry; engineering context relates that geometry to reality; purpose-specific representations expose selected information; and evaluations reveal consequences. The arrows express dependency, not ownership or automatic decision authority.

Figure 3.1 is deliberately not a software pipeline. It is a reasoning figure. Reading from left to right follows the consequence of an edit; reading from right to left asks what provenance and construction are needed to justify a displayed result. A GIS view may supply context, a CAD view may expose exact geometry, a BIM view may carry exchange objects, and an observation may report the track. Their claims become comparable only when their relation to the same engineering object and to the relevant state is known.

For planar arc-length parameter (s), the first constructive dependency is compact:

$$\theta'(s) = \kappa(s), \quad \gamma'(s) = (\cos \theta(s), \sin \theta(s)).$$

The curvature law determines heading evolution, and heading determines the planar path. Railway transition-curve literature and design standards connect the same curvature evolution, together with cant and speed, to acceleration, comfort, and admissibility rather than to visible shape alone [33, 32, 7, 4].

3.3. Knowledge at the Moment of Action

Making knowledge applicable does not mean copying every rule into every model. It means that a change can be related to knowledge whose scope, provenance, applicability, and authority are explicit. When the curvature band changes, the environment can identify affected states and invoke the relevant evaluations. It can show which observation supports a comparison and which constraint is violated. It can preserve why the result is applicable here.

This is more demanding than attaching attributes to a curve. Attributes can store values; applicability requires relations among object, state, context, knowledge, and operation. Those relations are the reason formal foundations are needed.

3.4. Responsibilities That Do Not Collapse

AIM does not turn one model or program into a universal authority.

3. The AIM Proposition

Representations

GIS, CAD, BIM, tables, plots, and exchange files select information for a purpose. They communicate and support action but do not create engineering identity or authority merely by storing a value.

Models and operators

Constructive and state models make dependencies explicit. Operators derive, transform, compare, and evaluate states under stated assumptions.

Computation

Numerical methods can reconstruct states, propagate consequences, test constraints, and produce candidate solutions. A solver result is evidence, not acceptance.

Engineers and accountable authorities

People establish applicability, interpret evidence, own engineering knowledge, and decide whether a proposed change is accepted within their authority.

The reference application is a realization horizon for this separation. It can make a curvature band editable, derive a geometric view, attach context, and surface evaluation results in one interaction. Its value to the thesis is not as a product demonstration but as a test: can the concepts remain distinct when an engineer actually changes something?

3.5. Why Alignment-Specific?

Generality is useful only after the governing dependency is visible. Railway location, direction, cant, vehicle-related quantities, stationing, adjacent objects, and many engineering rules are interpreted along or relative to an alignment. Curvature evolution and ordered longitudinal construction therefore provide a domain-specific organizing spine that a generic asset container or geospatial feature does not supply on its own.

Alignment-specific does not mean alignment-exclusive. Structures, signals, terrain, parcels, vehicles, and project organizations remain engineering objects with their own identities and knowledge. AIM states how they can be related to an alignment without absorbing them into it.

3.6. The Threshold to Foundations

The proposition now creates a formal obligation. If identity must survive a change of representation, identity and representation require different formal roles. If an observation is evidence rather than reality, observation and physical state require different relations. If curvature generates geometry, the constructive law and its metric realization require an explicit operator. If evaluation informs but does not decide, candidate, result, and decision cannot share one undifferentiated status field.

3. *The AIM Proposition*

The next part develops the minimum conceptual and mathematical language needed to preserve these distinctions and make selected dependencies computable. It does not restart the argument. It gives the argument a form that can be tested.

Summary 3.2. AIM connects an alignment edit to its consequences by preserving explicit relations among construction, state, context, evidence, knowledge, evaluation, and authority. The formal work that follows is necessary because these relations cannot be recovered reliably from geometry or data containers alone.

Part II.

Mathematical Foundations

Mathematical Foundations

The curvature edit has carried the argument to a precise threshold. The thesis must now describe what remains the same, what changes, which quantities follow, and under which context an evaluation is valid. Natural-language distinctions are necessary, but they are not sufficient for derivation, comparison, or implementation.

This part therefore gives selected engineering dependencies a mathematical form. It begins with information modelling and conceptual structure because a formula is only useful when the roles of its inputs and outputs are clear. It then states the governing axioms and notation, develops state and operator relations, and explains how intrinsic construction acquires measurable spatial meaning through metric realization.

Mathematics does not decide the identity of an engineering object, admit knowledge, or exercise engineering authority. It can make dependencies explicit, derive states, propagate a curvature change, compare an observation with a target, and evaluate a constraint. The distinctions established in Part I must survive every such operation.

The orienting question is:

Which formal dependencies make consequences computable without confusing the alignment, its states, its representations, the evidence about it, and the decision that may follow?

The first chapter crosses this threshold through the representations already encountered in practice.

4. Information Modelling

4.1. One Alignment, Several Accounts

The edited alignment now appears in several places. A GIS view locates it among terrain and networks. A CAD model exposes exact construction geometry. A BIM or exchange model connects it with project objects and delivery workflows. A survey records observations of the physical track. A calculation evaluates a consequence, and a decision record states what an accountable engineer accepts.

Simply combining these records does not produce a coherent information model. The same coordinate may express a design target, a physically realized state, or a measured estimate. The same identifier may be copied between files without proving that the records refer to the same engineering object. A rule may be stored beside a model without being applicable in the current context.

Information modelling begins here: not with a larger container, but with an account of what the information is about, what role each record plays, and which relations justify its use. Interoperability research similarly distinguishes the exchange of data from the semantic, organizational, and human conditions needed for collaboration [51].

4.2. What Representations Contribute

Representations are indispensable because engineering work requires selection. Geospatial standards organize access, processing, visualization, and discovery of spatial information [39]. CAD geometry supports precise construction and technical communication [20]. IFC provides standardized descriptions for exchanging built-asset information among applications and participants, with model views tailored to recognized workflows [5].

Each representation answers a purpose. That strength also marks its boundary. No encoding, model view, drawing, or database row establishes by itself:

- whether two records concern the same durable engineering object;
- whether a value is intended, realized, or observed;
- which constructive law produced a displayed curve;
- which knowledge is applicable and who owns its meaning;
- whether an evaluation result has been accepted as a decision.

These are not missing attributes with universally obvious values. They are engineering distinctions and relations that an information model must preserve.

4.3. Why the Alignment Organizes the Relations

Every information model chooses an organizing reference, explicitly or implicitly. In railway engineering, many questions are posed along or relative to the alignment: location, direction, curvature, cant, speed, vehicle-related quantities, stationing, clearances, and the placement of adjacent objects. The alignment supplies an intrinsic longitudinal order and a constructive curvature evolution from which selected states can be derived [33, 32].

AIM therefore places the alignment at the centre of these relations. This does not reduce a railway to one object. Bridges, tunnels, switches, signals, stations, terrain, vehicles, observations, and decisions retain their own roles. Alignment-centred means that information whose engineering meaning depends on longitudinal construction or track-relative state has an explicit reference through which that dependence can be interpreted.

Definition 4.1 (Alignment-Based Information Modelling). Alignment-Based Information Modelling (AIM) is an alignment-oriented information-modelling approach in which alignment identity and intrinsic construction provide the reference for relating geometric, spatial, operational, observational, and evaluative information without identifying the engineering object with any one representation.

4.4. Neighbouring Modelling Paradigms

BIM and AIM need not compete. BIM standards provide object-oriented structures and exchange mechanisms for the built environment. AIM supplies the alignment-specific constructive and state dependencies needed to interpret alignment information. A BIM workflow may carry an AIM-derived representation; it does not thereby become the owner of alignment identity or engineering knowledge.

This thesis also uses *Process Information Modelling* (PIM) as a proposed umbrella for approaches organized around engineering processes, state evolution, operational behaviour, and lifecycle relations rather than exclusively around physical assets. PIM is a thesis-level framing, not an asserted external standard and not a prerequisite for the AIM argument.

4.5. The Distinctions the Model Must Preserve

The running example now requires seven answers:

Object identity

What makes the edited and represented records concern one alignment?

4. Information Modelling

Constructive definition

Which ordered curvature law defines the alignment independently of sampled coordinates?

Realization

How does that intrinsic definition acquire metric meaning, and how does intended construction relate to physical infrastructure?

Representation

Which information has been selected for this GIS, CAD, BIM, numerical, or visual purpose?

Observation

Which source-grounded evidence reports something about the alignment or its state, under which conditions and uncertainty?

Knowledge

Which information may be relied upon within an explicit scope, provenance, applicability, and authority?

Evaluation and decision

What consequence was computed, and what accountable act accepts, rejects, or selects an alternative?

The list is not introduced as an ontology to memorize. Each distinction repairs a failure already visible in the curvature edit. The next chapter makes their roles explicit before later chapters formalize the relations that can safely be computed.

Summary 4.2. Representations make engineering information usable for particular purposes, but exchange alone does not establish identity, state, applicability, or authority. AIM organizes the missing relations around the alignment so that one engineering action can be traced across its different accounts.

5. Conceptual Structure

5.1. The Ambiguity a Data Model Cannot Resolve Silently

Suppose the edited CAD curve and a survey polyline differ by several centimetres. A data model can store both coordinate sets, attach identifiers, and calculate a residual. It cannot infer from those operations alone what the difference means. The survey may observe the physical track; the CAD curve may express an intended state; both may represent the same alignment; and the residual may or may not be acceptable under the applicable engineering knowledge.

Treating every record as another version of “the alignment” hides precisely the distinctions needed to reason about the discrepancy. Treating every different geometry as a different object loses continuity. Treating the measured record as reality confuses evidence with its subject. Treating a computed pass or fail as a decision transfers authority to an operation that does not own it.

The minimum remedy is to assign different formal roles to object, state, realization, representation, observation, knowledge, evaluation, and decision. Once separated, they can be related explicitly: an observation can concern a physical state; a representation can expose an intended state; an evaluation can compare them under applicable knowledge; a decision can use the resulting evidence without becoming identical to it.

5.2. The Engineering Object and Its States

An engineering object supplies durable distinguishability across changes of representation, realization, workflow, and project context. An alignment is therefore not identical to a CAD curve, GIS feature, exchange entity, database row, or rendered line. Multiple descriptions may concern the same alignment, and a changed description does not by itself decide whether the engineering object was modified or replaced.

A state describes that object under a specified engineering context. Intended, metric, physical, operational, and evaluated states answer different questions about one object. State change can therefore be discussed without making object identity a side effect of coordinate equality.

For the alignment, intrinsic longitudinal order and curvature evolution provide the constructive basis from which selected geometric states are derived. Later chapters formalize those derivations; the conceptual requirement here is that the derived state does not become the owner of identity.

5.3. Realization and Representation

Realization and representation answer different questions.

Metric realization asks how an intrinsic construction acquires measurable spatial meaning under an explicit metric space, context, and applicable rules. Physical realization asks how engineering intent becomes physical infrastructure. Neither is merely a change of file format.

A representation selects information so that it can be communicated, stored, exchanged, visualized, or processed. CAD geometry, GIS datasets, IFC entities, sampled polylines, plots, and runtime structures are representations. A representation may expose a realized state, but it cannot create missing realization semantics or establish identity by itself.

This separation makes a useful question possible: does a difference arise from the object's intended construction, from the chosen metric context, from physical realization, from observation, or only from representation?

5.4. Observation and Knowledge

An observation is source-grounded information about an object, state, or context, together with the provenance, method, time, conditions, and uncertainty needed to interpret it as evidence. A survey polyline is therefore not simply “the real alignment.” It is an observation whose relation to a physical state must be established.

Engineering knowledge is information admitted for engineering use within an explicit scope, provenance, applicability, and authority. Observations may support knowledge; storage or repetition does not admit them automatically. Standards, engineering principles, derived results, and accountable decisions may also support knowledge without becoming interchangeable sources.

The distinction allows conflicting observations to remain recorded while an engineer determines what may be relied upon for the present task.

5.5. Evaluation and Decision

Evaluation applies knowledge, constraints, preferences, and comparison methods to states or alternatives. It can derive a residual, identify a violation, rank candidates, or show how the curvature edit propagates. These results are evidence within an engineering process.

Decision is the accountable act that accepts, rejects, or selects within a defined authority and context. A solver, viewer, or rule engine may support that act but does not exercise it merely by producing output. This boundary is what allows computation to become more capable without making responsibility less clear.

5.6. Relations Made Available

With the roles separated, the thesis can state dependencies without category errors:

- one engineering object may have several states and representations;
- intrinsic construction may be metrically realized without transferring identity to coordinates;
- a physical state may be observed without the observation becoming that state;
- an observation may support knowledge through explicit interpretation and admission;
- an evaluation may use applicable knowledge and produce evidence for a decision without becoming the decision.

These relations turn the earlier discrepancy into a tractable engineering question. The intended and observed records can be related to one alignment, their states and provenance can be identified, and a comparison can be made under stated knowledge and context.

5.7. Canonical Interpretation

Proposition 5.1. *Engineering objects, states, realizations, representations, observations, engineering knowledge, evaluations, and decisions have distinct roles. Their explicit relations preserve continuity while making selected consequences computable.*

This conceptual structure now licenses the formal development. The following chapters state the axioms, notation, state models, and operators required to express these relations precisely. They may formalize a dependency; they may not erase the responsibility boundary that made the dependency meaningful.

6. Axioms as Failure Boundaries

Conceptual Structure established why AIM must distinguish an engineering object from its states, observations, evaluations, and representations. The formal task is now to make those distinctions usable. Consider the recurring case of a design alignment available as a CAD curve and a surveyed condition available as a sampled polyline. The two geometries can be compared computationally, but that comparison becomes meaningful only after five common failure modes have been excluded.

6.1. What the formal model must not collapse

Axiom 6.1 (Object continuity). *An engineering object's identity is not replaced by a change of state, observation, evaluation, or representation.*

The CAD curve and survey polyline may concern one alignment, different states of it, or even different objects. Geometric similarity cannot settle that identity question; identity must already be established by the engineering context in which the comparison is made.

Axiom 6.2 (Construction–representation separation). *Constructive engineering knowledge and a representation derived from it occupy different formal roles.*

A sampled polyline can support rendering, exchange, or computation. Its vertices do not thereby become the constructive definition of the alignment, and resampling does not create a new authoritative object.

Axiom 6.3 (Reality–observation separation). *A physical state and an observation of that state are distinct inputs to engineering reasoning.*

Survey points are source-grounded evidence. They do not constitute the physical railway itself, and uncertainty or incompleteness in the observation must not be transferred silently to physical reality.

Axiom 6.4 (Evaluation–decision separation). *An evaluation produces a qualified result; it does not constitute an engineering decision.*

A computed deviation may support acceptance, maintenance planning, or redesign. No threshold, residual, or solver output acquires decision authority merely by being computed.

Axiom 6.5 (Intrinsic–derived separation). *Derived geometry does not replace the intrinsic construction from which it is obtained.*

Coordinates, tangents, offsets, and sampled vertices are useful consequences. The formal construction must therefore expose the dependencies that produce them.

6.2. The constructive dependency

Let $I \subseteq \mathbb{R}$ be an interval parameterized by arc-length s . Let $\kappa : I \rightarrow \mathbb{R}$ be curvature, and let $\theta : I \rightarrow \mathbb{R}$ and $\gamma : I \rightarrow \mathbb{R}^2$ denote heading and planar position. The mathematical axioms below state the dependency that turns intrinsic construction into derived geometry.

Axiom 6.6 (Arc-length parameterization).

$$\|\gamma'(s)\| = 1 \quad \text{for all } s \in I.$$

Axiom 6.7 (Heading evolution).

$$\theta'(s) = \kappa(s).$$

Axiom 6.8 (Tangent reconstruction).

$$\gamma'(s) = \begin{pmatrix} \cos \theta(s) \\ \sin \theta(s) \end{pmatrix}.$$

Proposition 6.9 (Curvature-driven reconstruction). *Given an initial position $\gamma(s_0)$, an initial heading $\theta(s_0)$, and a sufficiently regular curvature function κ , the planar trajectory is uniquely determined on I .*

Proof. Integration of theorem 6.7 gives

$$\theta(s) = \theta(s_0) + \int_{s_0}^s \kappa(\sigma) d\sigma.$$

Substitution into theorem 6.8 and a second integration give

$$\gamma(s) = \gamma(s_0) + \int_{s_0}^s \begin{pmatrix} \cos \theta(\sigma) \\ \sin \theta(\sigma) \end{pmatrix} d\sigma.$$

The initial conditions fix the remaining integration constants. □

This result is deliberately limited. It reconstructs a planar trajectory from stated inputs and assumptions. It does not establish object identity, prove that a survey observes the same state, confer authority on a representation, or decide whether a discrepancy is acceptable.

7. Principles for Using the Formal Machinery

The axioms fixed the boundaries and the first constructive dependency. The principles below do not introduce the distinctions again. They state how subsequent mathematics must preserve them when states are derived, realized, compared, or represented.

7.1. Preserve roles through every operation

Principle 7.1 (Typed dependency). Every operation shall identify what it receives, what it produces, and the context in which the result is meaningful.

This rule prevents a change of mathematical form from becoming an unannounced change of engineering role. Sampling a CAD curve produces a representation; interpreting survey points produces an observation; an evaluation consumes qualified inputs and produces an evaluation result. None of those operations transfers identity or authority to its output.

7.2. Keep context visible

Principle 7.2 (Explicit applicability). A metric, mapping, residual, or constraint is meaningful only with its applicable state, realization context, and assumptions made explicit.

Before a CAD curve and survey polyline can be compared, they must be placed in a compatible metric realization and related to the applicable alignment state. A coordinate transformation can make coordinates compatible; it cannot establish that the sources concern the same object or condition.

7.3. Limit every computed consequence

Principle 7.3 (Derived-result discipline). A computed result shall retain the provenance and limitations of the inputs and operation from which it was derived.

Thus a deviation field can report distance under a stated mapping and a stated uncertainty model. It cannot determine whether the observed condition is acceptable, which source is authoritative, or what action shall follow. Those questions require engineering concepts whose roles the mathematics can use but cannot manufacture.

Together these principles establish a practical test for every symbol and equation that follows: its dependencies must be visible, its output must have one declared role, and its non-consequences must remain explicit.

8. Notation as a Dependency Language

Notation is introduced here in the order in which the CAD-curve and survey-polyline problem requires it. A symbol earns a place by naming an input, an output, a context, or an operation. It does not assign engineering identity or authority.

8.1. Where does the construction apply?

The interval $I \subseteq \mathbb{R}$ is the intrinsic longitudinal domain, and $s \in I$ is arc-length. A prime denotes differentiation with respect to s , unless another independent variable is stated:

$$f'(s) = \frac{df}{ds}(s).$$

The symbol $\mathcal{K}$ denotes the admissible space of curvature functions. Writing $\kappa \in \mathcal{K}$ therefore states both the constructive input and the regularity conditions required by the applicable model.

8.2. What is given, and what is reconstructed?

The intrinsic input is curvature $\kappa(s)$. Cant rotation $\phi(s)$ and its rate $\omega(s) = \phi'(s)$ are added when the spatial railway state requires them. Heading $\theta(s)$, planar position $\gamma(s)$, and local directions $\mathbf{t}(s), \mathbf{n}(s), \mathbf{b}(s)$ are derived through the stated evolution and realization relations. The local frame is

$$F(s) = (\mathbf{t}(s), \mathbf{n}(s), \mathbf{b}(s)).$$

A generic state is $x(s) \in \mathcal{X}$, a control is $u(s)$, and an evaluated or derived quantity lies in $\mathcal{Y}$. The names $\text{pose2}(s)$ and $\text{pose3}(s)$ refer to the state concepts defined in chapter 10; they are not synonyms for a stored curve or polyline.

8.3. Which condition and source does a quantity describe?

Subscripts preserve the role of a quantity:

$$(\cdot)_{\text{target}}, \quad (\cdot)_{\text{real}}, \quad (\cdot)_{\text{meas}}.$$

8. Notation as a Dependency Language

They distinguish an intended condition, a physically realized condition, and a measured description used as observation. Further subscripts may name a source, frame, coordinate domain, or realization context. Qualification prevents accidental substitution; it does not prove that two qualified quantities concern the same engineering object.

8.4. In which space is a comparison computed?

Track-related coordinates (s, d, h) belong to Ω_{track} , while a realized point x_w belongs to Ω_{world} . The track-to-world and world-to-track operations are

$$\mathcal{P}_{\text{tw}} : \Omega_{\text{track}} \rightarrow \Omega_{\text{world}}, \quad \mathcal{P}_{\text{wt}} : \Omega_{\text{world}} \rightarrow \Omega_{\text{track}}.$$

A coordinate transformation $\mathcal{C}$ changes one world-space description into another. It is not a world-to-track interpretation and does not establish object identity.

8.5. Which operation produces the next artifact?

The recurring operators are sampling $\mathcal{S}$, interpolation $\mathcal{I}$, evaluation $\mathcal{E}$, frame construction $\mathcal{F}$, physical response $\mathcal{M}$, local adjustment $\mathcal{A}$, and realization $\mathcal{R}$. Each later use must state a domain, codomain, and applicable assumptions. In particular, $\mathcal{E}$ produces an evaluation result, never a decision.

Symbols needed only under an Earth-surface realization are deferred until that context is established: κ_g and κ_n denote geodesic and normal curvature, and $\mathbf{n}_E$ the ellipsoid surface normal. Their meaning depends on the applicable surface and metric connection.

Scalars are italic, vectors bold, sets and spaces capital or calligraphic, and operators calligraphic where this makes their role visible. These typographic conventions support the dependency language; they do not replace explicit typing.

9. Core Equations

The equations below are executable knowledge: each relation names the inputs it receives, the output it produces, the assumptions under which that output is meaningful, and the questions it cannot decide. The sequence follows the recurring CAD-curve and survey-polyline case from construction, through realization and representation, to comparison.

9.1. Intrinsic State and Controls

The foundational spatial railway state is written as

$$x(s) = (\gamma(s), \theta(s), \phi(s)), \quad (9.1)$$

where $\gamma(s)$ denotes planar position, $\theta(s)$ heading, and $\phi(s)$ cant rotation.

The associated control pair is

$$u(s) = (\kappa(s), \omega(s)), \quad \omega(s) = \phi'(s). \quad (9.2)$$

The distinction between state and control follows the state model of chapter 10: the state records the condition of the alignment, while the controls govern its evolution along arc-length.

Execution contract. The equations receive a longitudinal domain, an initial state, and admissible controls. They produce a state trajectory. They assume the chosen state model and regularity conditions apply. They do not identify the engineering object or determine the authority of any data source.

9.2. Planar Intrinsic Geometry

Arc-length parameterization gives

$$\gamma'(s) = \mathbf{t}(s), \quad (9.3)$$

and directional evolution is governed by

$$\theta'(s) = \kappa(s). \quad (9.4)$$

9. Core Equations

In the planar case, the unit tangent is

$$\mathbf{t}(s) = \begin{pmatrix} \cos \theta(s) \\ \sin \theta(s) \end{pmatrix}. \quad (9.5)$$

The canonical planar reconstruction system is therefore

$$\begin{aligned} \gamma'(s) &= \begin{pmatrix} \cos \theta(s) \\ \sin \theta(s) \end{pmatrix}, \\ \theta'(s) &= \kappa(s). \end{aligned} \quad (9.6)$$

Given an initial position and heading, the curvature function determines the planar alignment trajectory.

Execution contract. The reconstruction receives $\kappa(s)$, $\gamma(s_0)$, and $\theta(s_0)$; it produces $\theta(s)$, $\mathbf{t}(s)$, and $\gamma(s)$. It assumes arc-length parameterization and sufficient regularity. A CAD curve sampled from γ is therefore a derived representation. The reconstruction cannot decide whether a surveyed polyline observes the same alignment or whether any discrepancy is acceptable.

9.3. Cant Evolution and Foundational pose3 System

Cant rotation evolves according to

$$\phi'(s) = \omega(s). \quad (9.7)$$

Together with the planar reconstruction equations, this gives the foundational pose3 system

$$\begin{aligned} \gamma'(s) &= \begin{pmatrix} \cos \theta(s) \\ \sin \theta(s) \end{pmatrix}, \\ \theta'(s) &= \kappa(s), \\ \phi'(s) &= \omega(s). \end{aligned} \quad (9.8)$$

Abstractly, the state evolution may be written as

$$x'(s) = f(x(s), u(s)). \quad (9.9)$$

This equation expresses the state-system form only. Profile, metric realization, spatial frame construction, and physical interpretation are introduced separately.

Execution contract. The system receives curvature and cant-rate controls plus an initial state; it produces a pose trajectory. It assumes that the selected pose model is applicable. It does not produce a physical realization or an observation of one.

9.4. Railway Dynamic Relations

For the classical local engineering approximation, the effective lateral acceleration is

$$a_{\text{eff}} = v^2 \kappa - g \sin \phi. \quad (9.10)$$

Ideal cant balance satisfies

$$v^2 \kappa = g \sin \phi. \quad (9.11)$$

For constant speed, differentiation with respect to time yields the corresponding jerk relation

$$j = v^3 \kappa' - g v \cos \phi \phi'. \quad (9.12)$$

These equations provide the foundational coupling between curvature, cant, and operational velocity. Their railway interpretation and limits are developed in the state and dynamics chapters.

Execution contract. These relations receive realized metric quantities and an applicable local engineering approximation; they produce acceleration, equilibrium, or jerk quantities. They assume the stated speed and differentiation conditions. They evaluate physical consequences but cannot determine operational admissibility or authorize a decision.

9.5. Track–World Relations

In a local planar realization, a track-related point with longitudinal coordinate s and lateral offset d is mapped to world space by

$$x_w(s, d) = \gamma(s) + d \mathbf{n}(s). \quad (9.13)$$

Definition 9.1 (Track-to-world mapping). The mapping defined by eq. (9.13) relates track-related coordinates to a world-space realization.

The corresponding inverse problem is expressed as

$$(s^*, d^*) = \arg \min_{s, d} \|x_w - (\gamma(s) + d \mathbf{n}(s))\|. \quad (9.14)$$

Definition 9.2 (World-to-track mapping). The world-to-track mapping determines an intrinsic track-related interpretation of a world-space point within an applicable realization context.

A transformation between two world-space coordinate descriptions is written abstractly as

$$x_2 = \mathcal{C}(x_1). \quad (9.15)$$

9. Core Equations

Coordinate transformation and world-to-track mapping are distinct operations: the former changes world-space representation, while the latter relates world space to an intrinsic track-related domain.

Execution contract. Track-to-world receives a realized alignment frame and track-related coordinates; it produces world coordinates. World-to-track receives a world point and the same applicable realization; it produces a candidate intrinsic interpretation. Uniqueness requires a suitable local domain. Neither operation establishes that two sources refer to the same object.

9.6. Representation and Engineering Realization

Sampling a continuous function at positions s_i is written as

$$\mathcal{S}(f) = \{f(s_i)\}, \quad (9.16)$$

and reconstruction from sampled values as

$$\mathcal{I}(\{f(s_i)\}) = \tilde{f}(s). \quad (9.17)$$

An engineering realization operator relates an intended state to a physically realized state:

$$\mathcal{R} : \text{pose3}_{\text{target}} \longrightarrow \text{pose3}_{\text{real}}. \quad (9.18)$$

A schematic decomposition used in later realization chapters is

$$\mathcal{R} = \mathcal{M} \circ \mathcal{I} \circ \mathcal{S}, \quad (9.19)$$

where sampling, reconstruction, and physical response remain distinct operator responsibilities.

At curvature level, the corresponding relation is

$$\mathcal{R}(\kappa_{\text{target}}) = \kappa_{\text{real}}. \quad (9.20)$$

Definition 9.3 (Curvature realization). Curvature realization denotes the relation between intended and physically realized curvature under an applicable engineering realization process.

The associated inverse problem seeks an intended curvature satisfying

$$\mathcal{R}(\kappa_{\text{target}}) \approx \kappa_{\text{desired}}. \quad (9.21)$$

Execution contract. Sampling receives a continuous description and sample locations; interpolation receives sampled values and an interpolation rule; realization receives a target

9. Core Equations

condition and a physical-response model. Their outputs retain those dependencies. In the test case, a survey polyline can support an observed description of realized geometry, but neither sampling nor interpolation turns the observation into physical reality or gives it constructive authority.

9.7. Surface Curvature

For a curve constrained to a surface, total curvature decomposes into geodesic and normal components:

$$\kappa^2 = \kappa_g^2 + \kappa_n^2. \quad (9.22)$$

The geodesic curvature is expressed intrinsically by

$$\kappa_g = \|\nabla_s \mathbf{t}\|. \quad (9.23)$$

The applicable surface, metric, connection, and engineering interpretation are established in the ellipsoid and spatial-realization chapters.

Execution contract. These equations receive a curve, surface, metric, and connection; they produce curvature components in that realization. Without those inputs the symbols are not executable. The components cannot decide which surface model is authoritative for a project.

9.8. Optimization Relations

A schematic realization-aware optimization problem is

$$\min_u J(\mathcal{R}(x(u))). \quad (9.24)$$

For nonlinear least-squares problems, the objective takes the form

$$\min_x \frac{1}{2} \|r(x)\|^2, \quad (9.25)$$

with the Gauss–Newton approximation

$$H \approx J^T J. \quad (9.26)$$

These equations specify common numerical structures only. Engineering observations, residuals, constraints, admissibility, and decisions retain their solver-independent meaning.

Execution contract. Optimization receives a declared objective, variables, residuals, constraints, and any realization operator included in the model; it produces a numerical

9. *Core Equations*

candidate. Its validity depends on those choices and on solver assumptions. A minimum is an evaluation result, not an engineering decision.

9.9. Canonical Role

The equations now provide a dependency chain rather than a formula catalogue. They can reconstruct a design trajectory, realize it in a metric context, interpret observations, and compute qualified differences. The next chapters provide the state and operator machinery needed to execute that chain consistently. Engineering Concepts then name the responsibilities that computation must preserve.

10. State Transition

Suppose an engineer edits curvature on an interval $J \subseteq I$, replacing $\kappa(s)$ by $\tilde{\kappa}(s) = \kappa(s) + \Delta\kappa(s)$. Curvature changes directly. Downstream heading and position change through integration; cant changes only if its own control is edited or a coupling constraint requires it. The alignment identity, the initial condition chosen for reconstruction, and the applicable realization context remain fixed unless the engineering task explicitly changes them.

This single edit establishes the purpose of a state model: to answer which condition follows from declared inputs under a stated context. It does not describe a storage snapshot and is not a generic container for every property associated with the alignment.

10.1. The state that answers the question

Definition 10.1 (Engineering state). An engineering state is a structured description of an engineering object under a specified condition and within a specified context.

For the present question, a state trajectory is

$$x : I \rightarrow \mathcal{X}, \quad s \mapsto x(s).$$

The mapping records the quantities needed to propagate the edit. A file record, cached polyline, or database row may represent values of this trajectory, but storage does not determine which condition or context those values describe.

10.2. Planar transition driven by curvature

Definition 10.2 (pose2). The planar alignment state is

$$\text{pose2}(s) = (\gamma(s), \theta(s)), \tag{10.1}$$

where $\gamma(s)$ is planar position and $\theta(s)$ heading.

Its evolution is

$$\begin{aligned} \gamma'(s) &= \begin{pmatrix} \cos \theta(s) \\ \sin \theta(s) \end{pmatrix}, \\ \theta'(s) &= \kappa(s). \end{aligned} \tag{10.2}$$

10. State Transition

With $\gamma(s_0)$ and $\theta(s_0)$ fixed, applying $\tilde{\kappa}$ produces a new state trajectory $\widetilde{\text{pose2}}$. Inside and downstream of the edited interval, heading must be reintegrated and position reconstructed. An existing CAD curve or sampled polyline derived from the former trajectory is consequently stale until its representation is regenerated.

Execution contract. The transition receives the curvature edit, reconstruction interval, and initial pose; it produces a planar state trajectory. It assumes arc-length parameterization and the stated boundary conditions. It preserves alignment identity. Its result inherits the provenance of the edit and boundary conditions. It cannot infer whether the edit is permitted, whether a survey concerns the same condition, or whether the result should be accepted.

10.3. Spatial transition and controls

Definition 10.3 (Foundational pose3 state). The foundational spatial railway state is

$$\text{pose3}(s) = (\gamma(s), \theta(s), \phi(s)), \quad (10.3)$$

where $\phi(s)$ is cant rotation about the local tangent.

This is the intrinsic input to a later spatial construction, not a world-space pose. Profile, metric context, and frame construction are still required before it becomes spatially measurable.

Definition 10.4 (Foundational controls). The foundational control pair is

$$u(s) = (\kappa(s), \omega(s)), \quad \omega(s) = \phi'(s). \quad (10.4)$$

The controls propagate the state through

$$\begin{aligned} \gamma'(s) &= \begin{pmatrix} \cos \theta(s) \\ \sin \theta(s) \end{pmatrix}, \\ \theta'(s) &= \kappa(s), \\ \phi'(s) &= \omega(s). \end{aligned} \quad (10.5)$$

10.4. Degrees of freedom during the edit

The edit is interpretable only when the status of each quantity is explicit.

Fixed: alignment identity, applicable context, reconstruction origin, and any boundary values declared invariant by the task.

Edited: κ on J , with source, author, and scope as provenance.

Derived: heading, tangent, planar position, and every representation sampled from the changed trajectory.

Coupled: cant, profile, or adjoining transition quantities when an applicable engineering relation links them to the curvature edit.

Constrained: continuity, regularity, boundary, admissibility, and project-specific requirements supplied by the applicable knowledge.

Free: quantities not fixed, derived, coupled, or constrained by the declared problem. They remain candidate degrees of freedom; a solver may explore them but cannot assign them engineering authority.

This classification is the bridge from formal state propagation to alignment-transition design: it identifies the variables available to a later construction or optimization method without prescribing software or a particular solver.

10.5. What may be compared

Metric realization can make the revised state spatially measurable. A survey polyline may then provide observational evidence in a compatible context, and an evaluation operator may compute residuals. The comparison relates qualified states and sources; it does not turn the polyline into physical reality, replace the alignment's constructive definition, or convert an evaluation into a decision.

The next chapter makes the actions in this chain explicit as typed operators.

11. Typed Operators

The revised curvature has produced a revised intrinsic state. The next question is not which concepts exist, but which action produces each subsequent consequence. AIM represents such actions as typed operators.

Definition 11.1 (Operator). An operator is a mapping

$$\mathcal{O}_c : \mathcal{A} \rightarrow \mathcal{B}$$

whose input type, output type, applicable context c , assumptions, and result provenance are declared.

The subscript is shown when context would otherwise be ambiguous. An algorithm may implement an operator, but the engineering contract is not defined by implementation alone.

Principle 11.2 (State–operator relation). Operators evaluate, transform, or relate engineering states without becoming constituents of the engineering object itself.

Proposition 11.3. *Engineering dependencies that change the meaning, domain, or status of a state shall be represented explicitly through operators or relations rather than hidden inside an undifferentiated state description.*

11.1. Reconstruct the edited state

The geometric reconstruction operator can be written schematically as

$$\mathcal{D}_{I,s_0} : (\text{pose2}(s_0), \kappa) \longrightarrow \text{pose2}|_I.$$

Execution contract. Input: an initial pose, curvature function, and interval. Output: the derived state trajectory. Required context: longitudinal domain and boundary conditions. Applicability: the reconstruction axioms and regularity assumptions must hold. Preserved identity: the alignment to which the edited state is assigned. Provenance: the edit, initial state, and reconstruction method. It cannot decide admissibility or whether the edited state should replace an approved state.

11.2. Realize the state metrically

Definition 11.4 (Metric realization operator).

$$\mathcal{G}_c : \mathcal{X} \rightarrow \mathcal{X}_{\text{metric}}.$$

Execution contract. Input: the intrinsic state. Output: its metric interpretation. Required context: metric model, embedding data, and applicable conventions. Applicability: the context must cover the state domain and requested quantities. Preserved identity: the engineering object and state role. Provenance: intrinsic source plus realization context. It cannot perform physical construction, create an observation, or select a decision.

Definition 11.5 (Frame operator). The frame operator constructs the local frame required by the applicable realization:

$$\mathcal{F}_c : \mathcal{X}_{\text{metric}} \rightarrow \mathcal{F}.$$

Its output is derived orientation data. It preserves identity and inherits the state and realization provenance; it cannot determine which frame convention a project ought to adopt.

11.3. Relate track and world descriptions

Definition 11.6 (Track-to-world operator).

$$\mathcal{P}_{\text{tw},c} : \Omega_{\text{track}} \rightarrow \Omega_{\text{world}}.$$

It receives track coordinates and a realized alignment frame and produces a world-space point. Its applicability requires a valid local frame over the requested domain. The result preserves the referenced object and carries the realization provenance; it cannot establish a physical or observed condition.

Definition 11.7 (World-to-track operator).

$$\mathcal{P}_{\text{wt},c} : \Omega_{\text{world}} \rightarrow \Omega_{\text{track}}.$$

It receives a world-space point and applicable realized alignment and produces a candidate track-related interpretation. Existence and uniqueness depend on the search domain and geometry. Its provenance must retain the world source, realization, and selection rule. It cannot prove that a survey point observes the alignment or choose among ambiguous projections without an applicable rule.

Definition 11.8 (Coordinate transformation operator).

$$\mathcal{C}_c : \Omega_{\text{world}} \rightarrow \Omega_{\text{world}}.$$

11. Typed Operators

It changes coordinate representation under a declared transformation context. It preserves the represented object's identity and retains source/target CRS provenance. It cannot replace world-to-track interpretation or metric realization.

11.4. Produce representations and observations

Sampling the revised metric trajectory produces a CAD curve or polyline representation. The representation operator receives qualified state knowledge and representation parameters; it produces a derived artifact, preserves object identity, and retains source-state and method provenance. It cannot become the constructive definition merely because it is stored, exchanged, or rendered.

An observation operation instead receives a physical condition through a measurement process and produces source-grounded evidence with time, method, conditions, and uncertainty. A survey polyline is usable in the comparison only through that provenance. Observation cannot turn its output into physical reality or admitted engineering knowledge by itself.

11.5. Relate target and physical condition

Definition 11.9 (Realization operator).

$$\mathcal{R} : \mathcal{X}_{\text{target}} \rightarrow \mathcal{X}_{\text{real}}.$$

The operator receives a target state plus an applicable construction, maintenance, or physical-response model and produces a realized-state description. It preserves the identity of the engineering object while changing the condition described. Its provenance includes the target, process model, and relevant physical inputs. It cannot manufacture an observation of the result or decide that the realized condition is acceptable.

11.6. Evaluate the qualified consequence

Definition 11.10 (Evaluation operator).

$$\mathcal{E}_c : \mathcal{X} \times \Omega_{\text{track}} \rightarrow \mathcal{Y}.$$

For the recurring discrepancy, evaluation receives the metrically realized target state, compatible survey evidence, comparison locations, uncertainty, and applicable rules. It produces residuals or other qualified consequences. It preserves the identity and roles of its inputs, and its provenance records those inputs, context, operator version, and assumptions. It cannot admit evidence as knowledge, accept a proposal, or issue an engineering decision.

11. Typed Operators

Principle 11.11 (Solver independence). Engineering observations, constraints, residuals, candidate solutions, and decisions are independent of the numerical solver used to evaluate them.

11.7. Composition without responsibility transfer

For compatible types,

$$\mathcal{O}_1 : \mathcal{A} \rightarrow \mathcal{B}, \quad \mathcal{O}_2 : \mathcal{B} \rightarrow \mathcal{C}, \quad \mathcal{O}_2 \circ \mathcal{O}_1 : \mathcal{A} \rightarrow \mathcal{C}.$$

Principle 11.12 (Operator separation). Operators with different engineering responsibilities shall remain conceptually distinguishable even when they participate in the same workflow.

The curvature-edit chain can therefore be read as reconstruction, metric realization, representation or observation interpretation, and evaluation. Composition transmits values and provenance. It does not transfer identity, change truth mode silently, or confer decision authority on the final number.

12. Metric Realization

After the curvature edit has produced a revised intrinsic state, the trajectory is mathematically determined but not yet spatially measurable. Distances to survey points, world positions, and frame orientations require a metric context. Metric realization is the action that supplies that context without changing the alignment's identity or constructive history.

12.1. From intrinsic order to measurable geometry

Intrinsic construction supplies the longitudinal domain, curvature evolution, ordered states, and applicable boundary conditions. A metric realization context supplies the metric space, reference surface where applicable, embedding conventions, and other information required to interpret length, direction, curvature, and position.

Definition 12.1 (Metric realization). Metric realization is the interpretation of an intrinsic engineering description within a specified metric space.

The action is written

$$\mathcal{G}_c : \mathcal{X} \rightarrow \mathcal{X}_{\text{metric}},$$

where the subscript c makes the realization context explicit. Two contexts may produce different coordinates or metric consequences from the same intrinsic state while preserving the same engineering object.

Execution contract. The operator receives an intrinsic state and realization context; it produces a metric state in which spatial quantities can be evaluated. It requires an applicable metric model and sufficient embedding data. It preserves engineering identity and carries the provenance of both the intrinsic state and context. It cannot create a physical railway, observe its condition, select an authoritative coordinate source, or decide whether a discrepancy is acceptable.

12.2. Keep the five roles separate

In the curvature-edit example, the roles are now operationally distinct:

Intrinsic construction determines how the edited curvature propagates into heading and trajectory.

12. Metric Realization

Metric context determines how the trajectory obtains measurable distance, direction, curvature, and spatial position.

Coordinate representation records the realized result in a selected coordinate description; $\mathcal{C}$ may change that description without repeating the intrinsic construction.

Physical realization concerns the constructed or maintained railway and is related to the target state by $\mathcal{R}$, not by $\mathcal{G}_c$.

Observation provides source-grounded evidence about a condition, for example survey samples with provenance and uncertainty.

These roles may share numerical coordinates after processing. Numerical agreement does not make them interchangeable.

12.3. Making the discrepancy computable

Let x_{target} be the revised intrinsic target state. The metric state

$$x_{\text{target}}^c = \mathcal{G}_c(x_{\text{target}})$$

can be represented as a CAD curve or sampled polyline. Survey evidence must be interpreted in a compatible context before comparison. A coordinate transformation can reconcile two world-space descriptions; a world-to-track mapping can supply a track-related interpretation. Neither operation proves common identity, common condition, or source authority.

Once compatibility and applicability have been established, evaluation may produce a qualified residual. Changing c may change that residual because projection, surface model, or metric interpretation has changed. That sensitivity is a property of the evaluation context, not evidence that the intrinsic alignment acquired a new identity.

12.4. Why the ellipsoid follows

A locally Euclidean realization may be sufficient for a bounded engineering task. Where projection distortion or Earth-surface geometry matters, the realization context must instead expose the reference surface and its metric structure. The ellipsoid chapters therefore do not begin a new geometry story. They instantiate the metric-realization contract for a surface on which straightness, curvature, normal direction, and frame transport require surface-aware definitions.

13. Execution Order and Responsibility

The preceding chapters already supply the concepts, state, operators, and realization contract. The purpose of this chapter is only to show their execution order and to identify where responsibility changes. It does not introduce another architecture catalogue.

13.1. The curvature-edit chain

Starting from an identified alignment and an authorized curvature edit, the formal machinery proceeds as follows:

$$\begin{aligned} (\text{pose2}(s_0), \tilde{\kappa}) &\xrightarrow{\mathcal{D}} \tilde{x}_{\text{intrinsic}} \xrightarrow{\mathcal{G}_c} \tilde{x}_{\text{metric}} \\ &\xrightarrow{\mathcal{P}, \mathcal{C}} \tilde{x}_{\text{comparable}} \xrightarrow{\mathcal{E}_c} y_{\text{qualified}}. \end{aligned}$$

Reconstruction propagates the curvature edit. Metric realization makes the state measurable. Spatial mappings and coordinate transformations place qualified sources in a compatible comparison domain. Evaluation produces the consequence. Each arrow has its own applicability and provenance; no arrow changes alignment identity.

The chain uses the curvature relation eq. (9.4), reconstruction relation eq. (9.3), and cant evolution relation eq. (9.7). Their execution precedes metric comparison; they do not wait for a CAD representation to define the alignment.

13.2. Where physical realization and observation enter

Physical realization is not another step between intrinsic and metric description. It relates conditions:

$$x_{\text{target}} \xrightarrow{\mathcal{R}} x_{\text{real}}.$$

Observation supplies evidence about a condition through a measurement process. Thus a survey polyline does not enter the main chain as “reality.” It enters as qualified evidence that may be mapped into a compatible metric domain and compared with a qualified target or realized state.

13.3. Responsibility at each boundary

The useful layer distinctions are consequences of the operator chain:

13. Execution Order and Responsibility

Intrinsic responsibility owns the constructive controls, boundary conditions, and state propagation.

Metric responsibility owns the metric space, reference surface, embedding, and validity of measurable spatial consequences.

Representation responsibility owns sampling, coordinate form, precision, and exchange provenance.

Physical responsibility owns the relation between target and realized conditions.

Observation responsibility owns source, method, time, uncertainty, and evidential limitations.

Evaluation responsibility owns the comparison method, applicability, assumptions, and qualified result—not the decision.

Principle 13.1 (Metric realization principle). Semantic alignment logic is separated from metric realization behaviour.

Principle 13.2 (Canonical separation principle). Intrinsic geometry, coordinate embedding, metric realization, physical realization, observation, and evaluation shall not be merged into one implicit operation.

13.4. Dependencies, not an implementation stack

Definition 13.3 (Canonical transformation structure). The formal transformation structure is an ordered composition of typed operators whose intermediate results preserve identity, context, and provenance.

The ordering constrains meaning, not software deployment. Operations may be cached, recomputed, or implemented by different tools provided their contracts remain intact.

Proposition 13.4. *A coordinate transformation may change metric properties of a representation under projection; it does not change intrinsic alignment identity.*

Proposition 13.5. *An alignment consequence is trustworthy only when the constructive, metric, representational, physical, observational, and evaluative responsibilities on which it depends remain explicit.*

Proposition 13.6. *Operational consequences arise from declared compositions of state, context, operators, and applicable engineering knowledge.*

The result of the chain is therefore modest but useful: the theory can say exactly what changed, which state followed, which action produced each artifact, where it is measurable, which freedoms remain, and what evidence may be compared. It still cannot turn a computed result into an engineering decision.

The ellipsoid chapters now specialize this chain by supplying an Earth-surface metric context.

14. Curvature Realized on the Earth

A planar engineering realization treats the applicable region as a Euclidean plane. For many design, construction, and maintenance tasks this is useful and sufficiently accurate. Its limitation appears when the engineering consequence depends materially on projection distortion or on the curvature of the Earth itself. Then a projected CAD curve is still a valuable coordinate representation, but it cannot own the intrinsic construction or determine the surface metric from which its coordinates were derived.

The question for this chapter is therefore precise: what changes when the same identified alignment, including the curvature edit introduced in chapter 10, is realized on the Earth rather than in a planar engineering space?

14.1. What remains and what changes

The alignment identity, intrinsic longitudinal order, edit provenance, and engineering role of curvature remain invariant. The realization context changes. Instead of asking $\mathcal{G}_c$ for a planar metric trajectory, we ask $\mathcal{G}_\mathcal{E}$ to interpret the state on a reference surface

$$\mathcal{E} \subset \mathbb{R}^3.$$

The result is a surface curve

$$\gamma_\mathcal{E} : [0, L] \rightarrow \mathcal{E}, \quad \|\gamma'_\mathcal{E}(s)\| = 1.$$

Thus s retains longitudinal meaning, while position, direction, and curvature acquire an Earth-surface interpretation.

14.2. Why one tangent is no longer enough

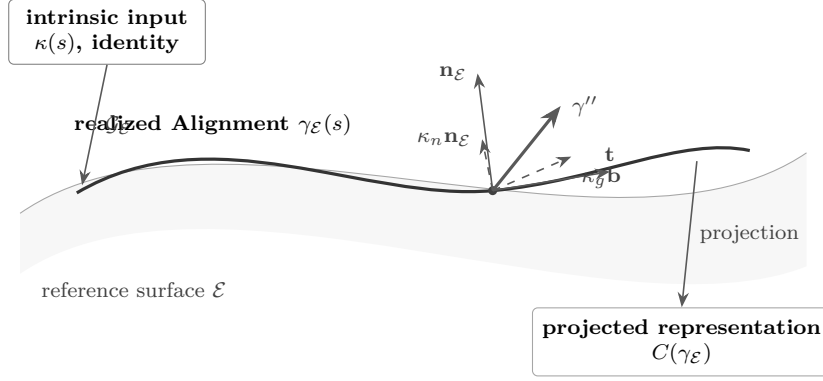
The realized tangent

$$\mathbf{t}(s) = \gamma'_\mathcal{E}(s)$$

states the direction of travel. On a plane, a fixed normal to that plane is usually left implicit. On the ellipsoid the surface normal $\mathbf{n}_\mathcal{E}(s)$ changes with position, so it must enter the realization explicitly. The oriented lateral direction in the tangent plane is

$$\mathbf{b}(s) = \mathbf{n}_\mathcal{E}(s) \times \mathbf{t}(s).$$

14. Curvature Realized on the Earth



Projection changes the coordinate account; the surface decomposition supplies the metric consequence.

Figure 14.1.: Earth realization preserves the intrinsic alignment and separates its surface-turning curvature from curvature induced by the reference surface. Projection produces another representation of the realized curve, not a new constructive definition.

These directions are introduced because the curvature vector now has two distinguishable causes.

14.3. The curvature decomposition

The change of tangent in ambient space decomposes as

$$\gamma_E''(s) = \underbrace{\kappa_g(s) \mathbf{b}(s)}_{\text{turning within the surface}} + \underbrace{\kappa_n(s) \mathbf{n}_E(s)}_{\text{curvature imposed by the surface}},$$

where, for the chosen orientation,

$$\kappa_g(s) = \langle \nabla_s \mathbf{t}(s), \mathbf{b}(s) \rangle, \quad \kappa_n(s) = \langle \gamma_E''(s), \mathbf{n}_E(s) \rangle.$$

Consequently,

$$\|\gamma_E''(s)\|^2 = \kappa_g^2(s) + \kappa_n^2(s).$$

Geodesic curvature κ_g is needed because it measures signed direction change within the reference surface. Normal curvature κ_n is needed because even a surface-straight path bends in three-dimensional space as the Earth curves beneath it. Total ambient curvature alone would mix these two engineering meanings.

14.4. Which curvature carries the alignment edit?

For the surface realization, the railway-relevant horizontal turning curvature is

$$\kappa_{\text{rail}}(s) = \kappa_g(s).$$

14. Curvature Realized on the Earth

The edited intrinsic curvature law is therefore realized as the required geodesic-curvature evolution, subject to the chosen surface context and boundary data. Normal curvature is a consequence of that context; it is not an additional horizontal alignment edit.

In the planar realization the surface normal is constant and $\kappa_n = 0$. The relation reduces locally to

$$\kappa_g(s) = \kappa(s) = \theta'(s).$$

Planar engineering is therefore not invalidated. It is the applicable local realization when surface and projection effects lie below the accuracy demanded by the task. Ellipsoidal realization matters when extent, required precision, projection distortion, or Earth-referenced spatial consequences make those effects material.

14.5. Projection is an account of the result

A projected curve $C(\gamma_{\mathcal{E}})$ supports CAD, GIS, exchange, and computation. Curvature computed directly from that representation may differ from κ_g because the projection changes metric properties. This does not make projected work unusable; it makes its applicability and accuracy part of the execution contract.

The Earth realization receives the intrinsic state, reference surface, metric, orientation, and boundary data. It produces a surface trajectory and its geodesic/normal decomposition. It preserves alignment identity and edit provenance. It cannot select a project CRS, prove that an ellipsoidal treatment is necessary for every task, or decide whether the result is acceptable.

The next chapter uses precisely these outputs to construct spatial pose and to connect curvature, cant, gravity, and speed.

15. Spatial Pose in the Earth Realization

The preceding chapter produced the realized surface trajectory, $\mathbf{t}$, $\mathbf{n}_{\mathcal{E}}$, $\mathbf{b}$, and κ_g . These are not a second alignment definition. They are the metric outputs required to ask a spatial engineering question: how do cant, gravity, and speed act at each position of the same alignment?

15.1. From surface trajectory to pose3

Applying the frame operator gives the uncanted surface frame

$$F_{\mathcal{E}}(s) = (\mathbf{t}(s), \mathbf{b}(s), \mathbf{n}_{\mathcal{E}}(s)).$$

The surface-based pose is consequently

$$\text{pose3}_{\mathcal{E}}(s) = (\gamma_{\mathcal{E}}(s), \mathbf{t}(s), \mathbf{n}_{\mathcal{E}}(s), \kappa_g(s), \phi(s)).$$

Planar heading θ has not been discarded arbitrarily: its role is now carried by a tangent vector field whose comparison from point to point depends on the surface connection.

The curvature edit changes κ_g directly in this realization and therefore changes the propagated tangent, surface trajectory, and local frame. The reference ellipsoid and gravity model remain fixed context; cant remains fixed unless edited or coupled by an applicable constraint.

15.2. Cant rotates the realized cross-section

Cant rotation $\phi(s)$ acts about the tangent:

$$\mathbf{b}_{\phi}(s) = R_{\mathbf{t}}(\phi(s)) \mathbf{b}(s), \quad \mathbf{n}_{\phi}(s) = R_{\mathbf{t}}(\phi(s)) \mathbf{n}_{\mathcal{E}}(s).$$

This operation receives the realized surface frame and cant state and produces the canted cross-section. It preserves alignment identity and retains both state and realization provenance. It does not infer cant from a rendered rail model or decide that the resulting balance is acceptable.

15.3. Gravity makes the pose physically interpretable

Let $\mathbf{g}(s)$ be the applicable local gravity vector at $\gamma_{\mathcal{E}}(s)$. A simplified realization may take $\mathbf{g} \parallel -\mathbf{n}_{\mathcal{E}}$; a higher-fidelity realization may distinguish ellipsoid normal, geoid normal, and local gravity. The choice belongs to the metric and physical evaluation context, not to alignment identity.

Gravity is introduced here because cant has an engineering consequence only relative to a force direction. The surface normal constructs the frame; the gravity vector supplies the physical direction used by the evaluation.

15.4. Curvature, speed, cant, and effective acceleration

For speed v , the curvature-induced lateral term is

$$a_{\kappa}(s) = v^2 \kappa_g(s).$$

Projecting the curvature and gravity contributions into the canted lateral direction gives, under the adopted sign convention,

$$a_{\text{eff}}(s) = v^2 \kappa_g(s) - \mathbf{g}(s) \cdot \mathbf{b}_{\phi}(s).$$

Under the local planar approximation this reduces to

$$a_{\text{eff}}(s) = v^2 \kappa(s) - g \sin \phi(s).$$

The dependency is now explicit: the curvature edit changes κ_g and the realized frame; speed supplies the operational condition; cant rotates the cross-section; gravity supplies the physical direction; evaluation produces effective acceleration. The result depends on all five inputs. It cannot decide an admissible speed, approve the edit, or replace the engineering rule under which it is assessed.

15.5. What this specialization establishes

Usable now: AIM can preserve identity and provenance while reconstructing an intrinsic state, applying an explicit metric context, producing planar representations, and evaluating qualified consequences. A local planar realization remains sufficient whenever its declared accuracy is sufficient for the task.

Formal specialization: These chapters state how the same operator contracts specialize to a reference ellipsoid, including the geodesic/normal decomposition, surface pose, canted frame, and gravity-aware dependency. This is a mathematical specialization, not a claim of completed production software.

15. Spatial Pose in the Earth Realization

Research: Validated applicability criteria, error bounds between planar/projected and ellipsoidal realizations, robust numerical realization over project-scale data, and the required fidelity of gravity models remain research and validation work.

Practical scope: AIM does not require every edit to run on an ellipsoid, replace established project CRS workflows, or use a geodesic solver when a qualified planar realization answers the engineering question.

Earth curvature has therefore earned a limited and exact place in the formal machinery. It becomes necessary when the realization context makes surface effects material; otherwise it remains an available specialization. In both cases the alignment identity, constructive curvature history, and responsibility boundaries remain intact.

Part III.

Engineering Concepts

Engineering Concepts

The Berlinish grammar has made a family of transition constructions expressible. Suppose that axtranNew, or another calculation service, now selects an entry half-wave, a clothoid core, and an exit half-wave and returns parameters that satisfy the stated geometric boundary conditions. The result is mathematically valid within that calculation. It is not yet the accepted transition of a railway alignment.

To become usable engineering information, the result must acquire meaning that the formula and solver cannot supply. It must refer to an identifiable alignment and a particular state of it. Its ordered construction must remain inspectable. Its metric realization, coordinate reference system, stationing interpretation, source, derivation, and temporal qualification must be known. It must then be evaluated for a stated purpose, such as operation at a given speed under applicable cant constraints. Finally, an entitled authority—not the calculation—must decide whether it may govern engineering work.

Figure 15.1 is the route through this part. The chapters do not introduce a row of independent containers. They follow one proposed transition as its mathematical possibility becomes an assertion about an Engineering Object, then evidence for an applicable and potentially accepted state.



Figure 15.1.: One transition proposal acquires engineering meaning through explicit identity, construction, realization, provenance, evaluation, and authority. No arrow silently grants the status of the next stage.

The governing distinction is therefore

$$\text{calculated} \neq \text{accepted} \neq \text{applicable} \neq \text{authoritative}. \quad (15.1)$$

A calculated proposal can fail the intended-speed or cant conditions, depend on an unsuitable realization, rest on incomplete evidence, or remain merely one alternative. Conversely, a historically accepted transition can be authoritative evidence for an existing asset without being the mathematical optimum. Engineering Concepts are needed to preserve these differences while keeping the underlying alignment recognizable through change.

16. From Calculated Transition to Engineering Object

The calculation returns a parameterized Berlinish composition $H_{\text{in}} + C + H_{\text{out}}$. At this point it is a proposal in a space of expressible transition functions. To say that it modifies the transition between two identified neighbouring elements is a different kind of statement: the proposal is now about a particular alignment.

16.1. What is the proposal about?

Definition 16.1 (Engineering object). An Engineering Object is a durable engineering entity whose identity is independent of its representations, observations, realizations, and computational models.

The alignment is the durable subject. The proposed transition is an element within its ordered construction and contributes to a candidate state of that alignment. A SPOT-capable engineering object model may carry the subject, state, and their relations, but a software record does not create their engineering identity. The solver output, a CAD curve generated from it, and a row in an exchange file are accounts of the proposal, not three new alignments.

The proposal must therefore name the alignment it concerns, the predecessor state from which it was derived, and the interval and neighbouring elements whose construction it would change. Without those references, the numerical result remains a reusable transition description rather than an instantiated Alignment element.

16.2. What kind of account is present?

Table 16.1 keeps together distinctions that will be used throughout this part. They are relations to one engineering subject, not competing names for the same thing.

This makes a derived editable representation of an imported transition possible without destroying the import. The imported record remains source-grounded evidence; reconstruction creates a traceable derived state; editing creates a proposal. Each account retains its own provenance and truth mode while referring, if the identity claim is justified, to the same alignment.

16. From Calculated Transition to Engineering Object

Table 16.1.: Roles in the running transition example

Role	In the running example	What the role does not establish
Object	the enduring alignment	no particular geometry, file, or approval
Representation	CAD curve, sampled polyline, or exchange record	neither object identity nor physical truth
State	intended state or curvature-edited candidate state	no authority merely because it is computable
Evidence	import history, calculation record, or survey observation	no automatic acceptance or applicability
Evaluation	test against speed, cant, realization, and applicable rules	no engineering decision
Authority	accountable acceptance, rejection, or selection	no transfer of ownership to the tool

16.3. Why identity is required next

The object distinction prevents a file, curve, or solver result from becoming the alignment. It does not yet tell us whether replacing the clothoid core, changing only a half-wave parameter, or rebuilding an imported curve preserves the same object. That question requires identity through change.

17. Identity Through Transition Change

The proposed transition now refers to one alignment. The engineer changes the length of the entry half-wave and recomputes the coupled parameters. Geometry and derived pose change. The subject need not.

17.1. Object continuity and state succession

Definition 17.1 (Engineering object identity). Engineering Object Identity is the basis by which an Engineering Object remains distinguishable throughout its engineering lifecycle.

Object identity answers *which enduring alignment?*; state identity answers *which qualified condition or revision of it?* The accepted predecessor and the calculated proposal may therefore refer to the same alignment while remaining different states. A recorded derivation relation connects them without overwriting the earlier evidence.

A parameter edit normally supplies a candidate successor state: the alignment reference, longitudinal order, and traceable predecessor remain present while the parameterized construction changes. This is an explanatory case, not a universal lifecycle rule. Acceptance of the successor still requires the applicable authority.

17.2. Constructive identity makes the edit intelligible

The Berlinish expression is not merely display geometry. Its ordered components, intrinsic longitudinal parameterization, function-family choices, and boundary relations make the construction inspectable. Replacing the clothoid core by another admissible family is therefore a constructive change, even if a sampled polyline happens to remain close to the old one. Resampling or transforming that polyline is only a representational change unless the engineering construction is also changed.

Definition 17.2 (Identity composition). Identity composition is the coherent combination of stable identity aspects and canonical relations required to distinguish an object without reducing identity to a representation-dependent attribute.

For an alignment, Constructive Identity contributes the intrinsic sequence and longitudinal construction by which the elements form the alignment. Alignment Identity draws on that constructive distinction; it is not a CRS identifier, station label, database key, or generated geometry. Operational stationing may address positions along the object, but it does not replace intrinsic longitudinal parameterization as an identity-bearing construction.

17.3. When does change create another object?

Some cases are clear. Coordinate transformation, serialization, rendering, and regeneration of derived geometry do not by themselves replace the alignment. Preserving the source record while deriving an editable model also does not erase the source identity or provenance. At the other end, an explicitly commissioned replacement alignment can be governed as another object even if it reuses much geometry.

Between those cases lies a genuine boundary. Replacing a transition component may be admitted as a successor state or may participate in a replacement object, depending on the object-type-specific sameness conditions and the authorized engineering act. The active Kernel does not prescribe a universal threshold for that judgment. Geometric distance, a software key, or the solver operation cannot decide it.

17.4. Provenance preserves the path through change

The candidate records its predecessor, the calculation service and version, input conditions, chosen function families and parameters, resulting derivations, responsible actor, and time. If the candidate is rejected, this evidence remains. If it is accepted, the decision relates the new accepted state to its predecessor rather than rewriting history.

Identity now tells us which object and state the proposal concerns. It still does not tell us where the transition is metrically meaningful or whether its evidence is sufficient for a particular task. Those dependencies establish applicability.

18. Applicability from Distributed Qualifications

The candidate has an object reference, a constructive account, and a traceable predecessor. The project now asks whether it is sufficient for an intended operating speed. That question cannot be answered by attaching one generic context label.

The active Knowledge Kernel does not establish a unified “Engineering Context” object. This thesis therefore keeps the required qualifications with the concepts that own them.

18.1. Purpose determines the required support

The stated purpose is to assess the candidate for operation at a specified speed, not merely to display it. That purpose calls for applicable curvature and cant limits, vehicle and operational assumptions, tolerances, and evidence quality. A CAD preview can be sufficient for visual review while being insufficient for operational assessment. Purpose changes the required support, not the identity of the alignment.

18.2. Realization gives the construction metric meaning

The intrinsic sequence and $\kappa(s)$ do not depend on a projected drawing. A Metric Realization interprets them as measurable geometry under a Realization Context: metric space, reference system or surface, frames, conventions, precision, and applicability dependencies. The coordinate reference system is one dependency in that account, not its owner and not the alignment’s identity.

Stationing used to address the affected interval must likewise be related explicitly to the intrinsic longitudinal parameter. A chainage convention or station equation may change how a position is addressed without changing the constructive sequence. If an Earth-referenced assessment and a local planar assessment use different valid realizations, their results must retain those contexts rather than be silently combined.

18.3. Other qualifications remain distributed

The calculation and source evidence carry temporal validity and provenance. The intended speed and assessment purpose carry their own scope. Applicable standards, project rules, and cant constraints retain their owners and validity domains. Project phase and workflow

status may qualify which assertion is being considered, but they are not a completed Kernel abstraction and are not promoted here into one.

The candidate is applicable only if these dependencies jointly support the stated consequence. Applicability is an outcome of that qualified relation, not a property conferred by successful computation.

18.4. The first operational test

The project can now ask a precise question: under the selected Metric Realization and stated speed, does the proposed curvature-and-cant state meet the applicable constraints with evidence of the required quality and validity? A negative or indeterminate outcome leaves the mathematical result calculated but inapplicable. A positive outcome makes it usable for this purpose, but still does not make it authoritative.

To perform that test without confusing the source with a generated curve, the next chapters separate representations, observations, and accepted knowledge.

19. Representing the Candidate Without Replacing It

The proposed transition is now generated as a continuous curve for calculation, a sampled polyline for CAD review, and an exchange record for coordination. These forms make the candidate usable; none becomes the alignment or owns its identity.

Definition 19.1 (Representation). A representation is a derived description of engineering information intended for communication, visualization, exchange, computation, or interaction.

Each representation must identify what state it expresses, its purpose, the derivation that produced it, and any approximation. The CAD polyline may be adequate for visual coordination yet omit the function-family identity and derivatives required to reconstruct the Berlinish composition. The continuous calculation model may support curvature evaluation while being unsuitable as an exchange contract. Fitness is purpose-relative.

Transformation, resampling, or regeneration can therefore produce another representation of the same candidate state. Conversely, two geometrically equal files need not refer to the same object. Representation equality is neither object identity nor engineering acceptance.

An imported historical transition illustrates the other direction. Its file can remain authoritative source evidence within its scope even when its sampled geometry is not mathematically optimal. Deriving an editable continuous model preserves the import and records a new derivation; it does not retroactively turn the derived model into the original observation.

The candidate is now communicable. To compare it with the railway as found, the account must next distinguish generated geometry from observed evidence.

20. Evidence About the Transition

A survey polyline crosses the same transition interval. Its coordinates look like the calculated curve, but they answer a different question. The solver derived what follows from stated inputs; the survey records evidence obtained from a physical condition.

Definition 20.1 (Engineering observation). An Engineering Observation is recorded information about an Engineering Object, state, or qualification together with the conditions under which that information was obtained.

For the survey, provenance includes the observed subject, acquisition method and time, coordinate and realization dependencies, processing history, uncertainty, and source responsibility. For the imported historical record, provenance identifies its source system and exchange history. For the axtranNew result, operation provenance identifies inputs, algorithm, version, and derivation. These are different evidence paths and must not be collapsed into a generic “geometry source.”

Several observations may disagree while still concerning one alignment. A survey does not become physical reality, and its coordinates do not overwrite the intended state. Comparison requires compatible realizations and temporal qualification. The resulting residual is derived evidence about the stated comparison, not a new observation and not an identity decision.

Observation preserves what was encountered; provenance makes its later use assessable. The next question is which rules, accepted derivations, and prior decisions may legitimately be relied upon when evaluating the candidate.

21. Knowledge for Evaluating the Candidate

The proposal and survey now have explicit provenance. Neither says by itself which curvature, cant, speed, or realization conditions govern the task. That support comes from applicable Engineering Knowledge.

Definition 21.1 (Engineering knowledge). Engineering Knowledge is accepted engineering information that may be used to interpret, evaluate, constrain, or decide engineering questions within its applicable qualifications.

In the running case, such knowledge can include the accepted predecessor state, applicable railway rules, validated calculation methods, tolerances, and authorized operational assumptions. Each retains its scope, validity, and owner. A rule for one speed regime or realization cannot be borrowed merely because its variables have familiar names.

Definition 21.2 (Knowledge ownership). Knowledge Ownership is the responsibility assigning Engineering Knowledge to the concept or authority entitled to establish, maintain, and revise it.

Ownership prevents a solver default from becoming a project rule and prevents an imported value from becoming accepted intent through repeated use. Observation, calculation, assumption, proposal, acceptance, and decision are different truth modes. Provenance explains where a statement came from; acceptance and ownership explain whether and within which scope it may be relied upon.

Evaluation can now relate the candidate to the applicable knowledge. It may find the geometry mathematically consistent but unsuitable for the stated speed, incompatible with cant constraints, unsupported by sufficiently current evidence, or inferior to another admissible candidate. That finding is decision evidence, not the decision itself.

The final chapter returns to the proposed transition and asks who may change its engineering status.

22. From Evaluated Proposal to Authorized State

The Berlinish composition has survived the required geometric, realization, speed, cant, and evidence checks. It is now an applicable candidate for the stated purpose. It is still not the accepted state of the alignment.

22.1. Proposal, candidate, and evaluation

Definition 22.1 (Proposal). A proposal is a candidate engineering change submitted for evaluation before acceptance or rejection.

Definition 22.2 (Candidate solution). A candidate solution is a proposed engineering state or configuration that may be evaluated against applicable knowledge, constraints, and preferences.

axtranNew searches or solves within the expressible transition space. The engineering object model carries the resulting proposal and its relation to the alignment. Evaluation determines purpose-relative sufficiency. These are separate responsibilities: successful generation does not imply admissibility, and admissibility does not select among all admissible alternatives.

22.2. Authority changes engineering status

The active Kernel fixes the boundary that evaluation does not exercise decision authority; it does not yet establish a complete canonical identity for every Engineering Decision. For this explanatory sequence, the following narrow operational formulation is sufficient:

Definition 22.3 (Engineering decision). An Engineering Decision is an authorized acceptance, rejection, or selection that resolves an engineering proposal or alternative within explicitly stated qualifications.

The entitled authority may accept the candidate as a successor state, reject it, request further evidence, or select another candidate. The decision names its scope, subject, evidence, applicable knowledge, responsible authority, and effect. It does not erase the predecessor, rejected candidates, survey, or calculation record.

If accepted, the new state remains related to the same alignment when the authorized identity judgment preserves object continuity. If the change is governed as a replacement

object, that relation must be stated instead. The Kernel does not supply a universal geometric threshold between these outcomes; the solver therefore cannot manufacture one.

22.3. Return to the transition

The running example has crossed the complete meaning chain:

1. Berlinish expressed an ordered transition construction.
2. axtranNew or an equivalent service calculated a parameterized proposal.
3. the proposal was related to an identifiable alignment and candidate state while preserving its constructive composition;
4. Metric Realization and Realization Context made it spatially measurable;
5. representations communicated it and provenance separated calculation, import, derivation, and observation;
6. applicable knowledge and purpose-relative evaluation tested its sufficiency; and
7. an entitled authority, if it chose to do so, admitted the successor state.

At no point did computation create authority, representation acquire identity, observation become physical reality, or evaluation constitute decision. This is the engineering meaning carried forward into Geometry and Curvature: the mathematics will now operate on identifiable, qualified states rather than on anonymous curves.

Part IV.

Geometry and Curvature

Geometry

The transition arriving from Engineering Concepts is no longer an anonymous curve. It is a proposed state of an identified alignment. Its Berlinish composition is inspectable, its derivation has provenance, and the realization under which it was evaluated is explicit. Calculation, applicability, and authority remain distinct. Geometry must now answer a narrower question:

What structure belongs intrinsically to this alignment, and what acquires meaning only through representation, placement, realization, observation, or operational purpose?

The structure that remains with the alignment

The element order, orientation, intrinsic longitudinal parameterization, and curvature law belong to the constructive account of the alignment state. In this part, *stationing* means that intrinsic parameterization by s , normally chosen as arc length. Operational chainage and displayed station labels may address s ; they do not replace it or become Alignment Identity.

A starting pose and the intrinsic law reconstruct a local Cartesian plan curve. That curve is already meaningful engineering geometry. A coordinate representation can express it, and a later geographic placement can relate it to a projected or Earth-referenced setting, without making the earlier local alignment inferior or fictitious. The required boundary is

$$\begin{aligned}\text{intrinsic geometry} &\neq \text{coordinate representation,} \\ \text{geographic placement} &\neq \text{physical realization.}\end{aligned}$$

Figure 22.1 follows the same straight–transition–arc state through these readings. The object reference and ordered station domains remain stable; plan position and curvature expose different consequences of the same construction.

Why curvature matters specifically for railway

A railway vehicle is constrained by the rail pair. Unlike a road vehicle, it cannot normally choose another lateral path within a corridor to avoid the alignment’s curvature; this contrast does not imply that road-corridor design is geometrically or dynamically simple. For rail, speed converts spatial variation into time-dependent vehicle excitation. Cant can

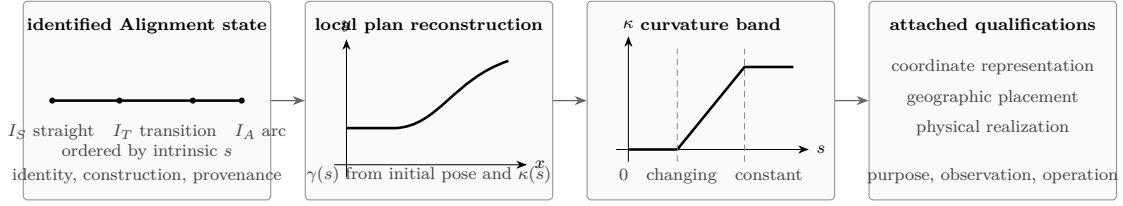


Figure 22.1.: One identified alignment state read as ordered station domains, reconstructed local plan geometry, and a curvature band. Representation, geographic placement, and physical realization remain attached qualifications rather than intrinsic identity.

compensate part of the lateral effect, but it does not erase curvature. Curvature, cant, and their longitudinal rates influence vehicle response, comfort, wheel–rail forces, track demand, and wear.

This is why the curvature band becomes the primary reading in the next chapter. The plan curve shows where the alignment goes. The band shows immediately where it is straight, where curvature is constant, where it changes, and whether element joins preserve the required continuity. The mathematics is therefore not restarting with generic curves; it is exposing the intrinsic consequences of the already identified transition state.

23. Curvature Space

The same alignment, read intrinsically

Consider the identified straight–transition–arc state from Figure 22.1. Let its ordered station domains be

$$I_S = [s_0, s_1], \quad I_T = [s_1, s_2], \quad I_A = [s_2, s_3].$$

Here s is intrinsic longitudinal position, not an x -coordinate and not a displayed kilometre label. Orientation fixes the direction in which the sequence is traversed. Together, order and s tell us which element follows which before any geographic placement is chosen.

Principle 23.1 (Curvature-space principle). Within AIM, the plan geometry of an identified alignment state is read primarily through its curvature law over intrinsic longitudinal position.

23.1. Curvature makes the element sequence visible

Definition 23.2 (Curvature law). A curvature law is a function

$$\kappa : I \rightarrow \mathbb{R}$$

defined on an intrinsic arc-length interval $I \subset \mathbb{R}$.

For the running example,

$$\kappa(s) = \begin{cases} 0, & s \in I_S, \\ \kappa_T(s), & s \in I_T, \\ \kappa_A, & s \in I_A. \end{cases}$$

The curvature band therefore reads the construction directly: zero means straight, constant nonzero curvature means circular arc, and changing curvature means transition. Element boundaries appear at s_1 and s_2 , where value and derivative matching can be inspected. A plan view can make two families look almost identical while hiding different endpoint derivatives.

The App’s CurvatureBand is one implementation of this reading. The argument does not depend on that interface: any evaluation that retains s , element domains, κ , and its relevant derivatives can expose the same structure.

Principle 23.3 (Curvature-first principle). Curvature is primary with respect to reconstructed plan coordinates, but it remains a state of an identified alignment rather than a free-floating curve.

23.2. Reconstruction before geographic placement

Definition 23.4 (Intrinsic reconstruction). Given $\kappa(s)$ and an initial local pose $(\gamma(s_0), \theta(s_0))$, the tangent direction and plan curve satisfy

$$\theta'(s) = \kappa(s), \quad \gamma'(s) = \mathbf{t}(s) = \begin{bmatrix} \cos \theta(s) \\ \sin \theta(s) \end{bmatrix}.$$

Integration reconstructs a local Cartesian curve. Changing the origin or rotating that local coordinate frame changes the coordinate representation, not the intrinsic sequence or curvature law. A later Realization Context may place the same state in a projected CRS or on an Earth reference surface. A physical realization may then instantiate it imperfectly. These dependencies add metric, geographic, and physical meaning without retrospectively creating the alignment's intrinsic geometry.

23.3. Curvature space and the necessary distinctions

Definition 23.5 (Curvature space). Curvature space denotes the space of curvature laws

$$\mathcal{K} = \{\kappa : I \rightarrow \mathbb{R}\}$$

within which stated admissibility, smoothness, realization, and operational requirements select applicable subsets.

The symbol (mathcal K) does not make every mathematically expressible law admissible, applicable, or authoritative. It provides the comparison space in which constant, transition, bounded, differentiable, and hybrid laws can be distinguished for the identified engineering state.

Definition 23.6 (Constant curvature). A constant curvature law satisfies $\kappa(s) = \kappa_0$. The case $\kappa_0 = 0$ is straight; $\kappa_0 \neq 0$ reconstructs a circular arc.

Definition 23.7 (Transition curvature). A transition curvature law connects different curvature states over an explicit longitudinal domain.

Its derivatives make qualities at the joins inspectable. Table 23.1 states only the geometric reading; specific operational limits still require applicable railway knowledge, speed, cant, vehicle assumptions, and purpose.

Definition 23.8 (Curvature continuity). A curvature law is C^0 when κ is continuous, C^1 when κ' exists and is continuous, and C^2 when κ'' exists and is continuous on the stated domain.

Table 23.1.: Continuity readings at an element boundary s_b

Quantity	Boundary question	What the answer reveals
κ	Do left and right values agree?	Whether curvature is continuous; a jump introduces an immediate change of lateral geometric demand.
κ'	Do curvature rates agree?	Whether curvature development enters and leaves without a rate jump; at speed this affects the time development of lateral response.
κ''	Do rate derivatives agree?	Whether higher-order transition development is smooth; relevance to vehicle response or comfort remains speed-, cant-, and model-dependent.

Position continuity alone cannot answer these questions. The next chapter classifies the laws by precisely these visible properties before the Berlinish chapter applies them to the transition domain.

23.4. Sparse Curvature Representation

Definition 23.9 (Sparse curvature representation). A sparse representation describes curvature using:

- fixed curvature elements,
- transition elements,
- and compact parameter sets.

Within AIM, sparse alignments are interpreted as compact curvature programs.

The sparse model stores:

- curvature structure,
- transition families,
- and reconstruction rules,

rather than dense geometric point clouds.

This makes sparse alignment suitable for:

- reconstruction,
- import normalization,
- runtime evaluation,
- realization comparison,
- and optimization.

23.5. Hybrid Curvature Representation

Definition 23.10 (Hybrid curvature model). A hybrid curvature model combines:

- parametric curvature structure,
- sampled correction data,
- and local refinement.

Hybrid representations allow:

- compact storage,
- efficient reconstruction,
- realization-aware correction,
- measured-state integration,
- and scalable runtime evaluation.

Hybrid curvature models are especially relevant when measured or realized geometry cannot be represented adequately by a purely analytical target model.

23.6. Normalized Curvature Space

Definition 23.11 (Normalized curvature law). A normalized curvature law is defined on:

$$w \in [0, 1].$$

Normalized representations separate:

- geometric shape,
- from physical scaling.

This enables:

- reusable transition families,
- lookup-based transitions,
- family interpolation,
- and normalized optimization.

Physical curvature laws are obtained from normalized laws by scaling in length and curvature amplitude.

23.7. Curvature Derivatives

Definition 23.12 (Curvature derivatives). Important higher-order quantities include:

$$\kappa'(s), \quad \kappa''(s), \quad \kappa'''(s).$$

These quantities influence:

- jerk,
- realization behaviour,
- dynamic smoothness,
- vehicle response,
- and transition quality.

Curvature derivatives provide the bridge from intrinsic geometry to runtime dynamics.

23.8. Frequency Interpretation

Curvature laws may also be interpreted spectrally.

Low-frequency curvature components determine global geometric structure, whereas high-frequency components influence:

- realization sensitivity,
- numerical instability,
- measurement sensitivity,
- and dynamic roughness.

Proposition 23.13 (Spectral realization principle). *Realization acts approximately as a low-pass filter on curvature space.*

This spectral view connects curvature space directly to construction, maintenance, and physical realization.

23.9. Curvature Space and Runtime

Runtime evaluation operates primarily on curvature structures rather than directly on dense geometry.

Projection, reconstruction, frame evaluation, pose3 evaluation, and dynamics are derived from runtime evaluation of curvature-based states.

Thus, curvature space is not only a design space. It is also the computational state space underlying runtime reconstruction.

23.10. Curvature Space and Realization

Realization transforms target curvature into physically realized curvature.

This may be written abstractly as:

$$\kappa_{\text{real}} = \mathcal{R}_{\kappa}(\kappa_{\text{target}}).$$

The realization operator may attenuate, deform, or locally modify curvature evolution.

Consequently, the physically relevant design object is not only the target curvature law, but also its expected realized curvature state.

23.11. Curvature Space and Optimization

Optimization within AIM primarily acts in curvature space.

Typical optimization variables include:

- transition parameters,
- curvature amplitudes,
- transition lengths,
- curvature anchors,
- and sparse curvature corrections.

AIM optimization may therefore be interpreted as the search for admissible, realizable, and operationally suitable paths in curvature space.

23.12. Canonical Interpretation

Proposition 23.14. *Within AIM, railway alignment geometry is fundamentally a curvature-space problem.*

Proposition 23.15. *Position-space geometry is interpreted as reconstructed geometry derived from curvature evolution.*

Proposition 23.16. *Transition curves are interpreted as controlled transitions within curvature space.*

Proposition 23.17. *Realization, runtime evaluation, and optimization act on curvature-space structures before they act on visualized position geometry.*

23.13. Connection to AIM

Summary 23.18. Curvature space forms the intrinsic geometric foundation of AIM.

Within this framework:

- curvature acts as primary geometry,
- sparse models become curvature programs,
- transition curves become curvature transitions,
- runtime systems operate on intrinsic curvature structure,
- realization transforms curvature states,
- and optimization searches within curvature-space structures.

This interpretation unifies geometry, realization, runtime evaluation, measurement interpretation, and optimization within one intrinsic mathematical framework.

24. Curvature Classes

Motivation

Not all curvature laws possess the same geometric, dynamic, numerical, or physical properties.

Within AIM, curvature laws are therefore classified according to:

- continuity,
- differentiability,
- boundedness,
- transition behaviour,
- realization behaviour,
- runtime stability,
- and operational suitability.

This classification forms the basis for:

- transition evaluation,
- runtime robustness,
- realization analysis,
- admissibility checks,
- and optimization strategies.

Principle 24.1 (Curvature classification principle). Within AIM, railway alignment elements are classified primarily by their curvature behaviour rather than by their visual position-space shape.

24.1. Basic Curvature Classes

24.1.1. Constant Curvature

Definition 24.2 (Constant curvature). A curvature law is constant if:

$$\kappa(s) = \kappa_0.$$

Special cases are:

- $\kappa_0 = 0$: straight line,
- $\kappa_0 \neq 0$: circular arc.

Constant curvature forms the fixed-element basis of classical railway geometry.

24.1.2. Piecewise Constant Curvature

Definition 24.3 (Piecewise constant curvature). A curvature law is piecewise constant if:

$$\kappa(s) = \kappa_i$$

on subintervals I_i .

This corresponds to classical tangent–arc railway geometry.

Piecewise constant curvature is geometrically simple, but curvature jumps are dynamically and operationally problematic.

24.2. Continuity Classes

24.2.1. C^0 Curvature

Definition 24.4 (C^0 -continuous curvature). A curvature law belongs to C^0 if:

$$\kappa(s)$$

is continuous.

Continuous curvature eliminates curvature jumps.

24.2.2. C^1 Curvature

Definition 24.5 (C^1 -continuous curvature). A curvature law belongs to C^1 if:

$$\kappa'(s)$$

exists and is continuous.

This improves dynamic smoothness, realization behaviour, and numerical robustness.

24.2.3. C^2 Curvature

Definition 24.6 (C^2 -continuous curvature). A curvature law belongs to C^2 if:

$$\kappa''(s)$$

exists and is continuous.

Higher continuity reduces excitation effects, numerical instability, and unwanted high-frequency behaviour.

In AIM, continuity class is not merely a mathematical property. It is an indicator for dynamic quality, runtime stability, and realization robustness.

24.3. Boundedness Classes

24.3.1. Bounded Curvature

Definition 24.7 (Bounded curvature). A curvature law is bounded if:

$$|\kappa(s)| \leq \kappa_{\max}.$$

Bounded curvature corresponds to a minimum admissible radius.

24.3.2. Bounded Curvature Gradient

Definition 24.8 (Bounded curvature gradient). A curvature law has bounded curvature gradient if:

$$|\kappa'(s)| \leq \eta_{\max}.$$

This is closely related to jerk limitations in railway dynamics.

24.3.3. Bounded Higher Derivatives

Definition 24.9 (Bounded higher derivatives). Higher-order boundedness conditions include:

$$|\kappa''(s)| < \infty, \quad |\kappa'''(s)| < \infty.$$

Higher-order boundedness becomes relevant for realization behaviour, vehicle response, and transition-family comparison.

24.4. Transition Classes

24.4.1. Monotone Transitions

Definition 24.10 (Monotone transition). A transition curvature law is monotone if:

$$\kappa'(s)$$

does not change sign.

Monotone transitions represent direct curvature change between two curvature states.

24.4.2. Symmetric Transitions

Definition 24.11 (Symmetric transition). A normalized transition law is symmetric if its curvature evolution is symmetric with respect to the midpoint of the transition interval.

For normalized transition coordinates $w \in [0, 1]$, this may be expressed by a symmetry relation of the transition shape function.

Symmetry is not an absolute requirement, but it provides a useful classification property.

24.4.3. Asymmetric Transitions

Definition 24.12 (Asymmetric transition). A transition law is asymmetric if symmetry is intentionally violated.

Asymmetric transitions may appear in:

- constrained engineering situations,
- realization-aware optimization,
- vehicle-dynamics optimization,
- and topology-constrained alignments.

24.5. Analytical Classes

24.5.1. Polynomial Curvature Laws

Definition 24.13 (Polynomial curvature). A polynomial curvature law satisfies:

$$\kappa(s) = \sum_{i=0}^n a_i s^i.$$

Polynomial curvature laws are useful because derivatives and integrals can often be handled systematically.

24.5.2. Trigonometric Curvature Laws

Definition 24.14 (Trigonometric curvature). A trigonometric curvature law contains sinusoidal or cosine-based components.

Trigonometric curvature laws are useful for smooth transition behaviour and frequency-oriented interpretation.

24.5.3. Lookup-Based Curvature Laws

Definition 24.15 (Lookup-based curvature). A lookup-based curvature law is defined by sampled or tabulated representations together with interpolation rules.

Lookup-based laws allow practical inclusion of transition families or realization effects that are not conveniently represented by a simple closed formula.

24.5.4. Hybrid Curvature Laws

Definition 24.16 (Hybrid curvature law). A hybrid curvature law combines:

- parametric structure,
- sampled corrections,
- measured data,
- and local refinement.

Hybrid laws are especially important for measured and realized railway states.

24.6. Sparse Curvature Classes

Definition 24.17 (Sparse curvature representation). A sparse curvature model represents geometry through:

- fixed curvature elements,
- transition elements,
- and compact parameter sets.

Sparse classes emphasize:

- compactness,
- runtime efficiency,
- reconstruction stability,
- and semantic clarity.

Within AIM, sparse curvature classes are not merely storage formats. They define executable curvature programs.

24.7. Operational Classes

24.7.1. Dynamically Smooth Curvature

Definition 24.18 (Dynamically smooth curvature). A curvature law is dynamically smooth if:

- jerk remains bounded,
- cant-rate remains admissible,
- curvature variation is controlled,
- and high-frequency excitation is limited.

24.7.2. Realization-Stable Curvature

Definition 24.19 (Realization-stable curvature). A curvature law is realization-stable if:

- realization preserves its dominant structure,
- construction processes do not introduce instability,
- and its low-frequency behaviour remains identifiable after physical realization.

Realization-stable curvature is a physical property, not merely an analytical smoothness property.

24.7.3. Runtime-Stable Curvature

Definition 24.20 (Runtime-stable curvature). A curvature law is runtime-stable if:

- projection remains numerically stable,
- frame evaluation remains continuous,
- pose reconstruction behaves robustly,
- and local inverse problems remain well-conditioned.

24.8. Frequency Classes

24.8.1. Low-Frequency Curvature

Definition 24.21 (Low-frequency curvature). Low-frequency curvature components determine large-scale geometric behaviour.

These components are usually preserved by construction and maintenance processes.

24.8.2. High-Frequency Curvature

Definition 24.22 (High-frequency curvature). High-frequency curvature components influence:

- realization sensitivity,
- dynamic roughness,
- measurement sensitivity,
- and numerical instability.

Proposition 24.23 (Spectral realization principle). *High-frequency curvature components are attenuated by realization.*

This frequency-oriented classification connects curvature classes with the realization filter interpretation.

24.9. Admissibility

Definition 24.24 (Admissible curvature law). A curvature law is admissible if it satisfies:

- geometric constraints,
- realization constraints,
- operational constraints,
- dynamic constraints,
- and numerical stability conditions.

Admissibility depends on:

- vehicle dynamics,
- realization processes,
- standards,
- topology,
- runtime requirements,
- and intended operational use.

Thus, admissibility is context-dependent. A curvature law may be analytically valid but operationally unsuitable or physically unstable under realization.

24.10. Classification versus Quality

Classification does not automatically imply quality.

A highly smooth curvature law may still be unsuitable if it is:

- difficult to realize,
- dynamically unfavourable,
- numerically unstable,
- incompatible with topology,
- or unsuitable for the intended operation.

Within AIM, curvature quality is evaluated by the interaction of class, context, realization, and runtime behaviour.

24.11. Canonical Interpretation

Proposition 24.25. *Different curvature classes correspond to fundamentally different geometric, operational, runtime, and realization behaviours.*

Proposition 24.26. *Within AIM, railway alignment quality is primarily characterized in curvature space rather than position space.*

Proposition 24.27. *Transition families are interpreted as structured subsets of curvature classes.*

Proposition 24.28. *Admissibility is not a purely geometric property, but a combined geometric, dynamic, runtime, and realization property.*

24.12. Connection to AIM

Summary 24.29. Curvature classes provide the structural classification framework of AIM geometry.

They connect:

- continuity,
- differentiability,
- boundedness,
- transition behaviour,
- realization behaviour,
- runtime stability,

24. Curvature Classes

- dynamic smoothness,
- and optimization suitability.

Within AIM, transition curves are therefore interpreted not merely as geometric entities, but as members of specific curvature classes with distinct operational properties.

This creates the bridge from intrinsic geometry to classification, realization analysis, runtime evaluation, and optimization.

25. Curvature Transitions

Motivation

The running alignment has reached the boundary s_1 between its straight and transition domains. Its curvature must leave zero and reach the constant arc curvature at s_2 . A coordinate interpolant could connect the same end positions while concealing how curvature and its derivatives behave at those boundaries. The required object is therefore a controlled curvature-transition process belonging to this identified alignment state.

Principle 25.1 (Curvature-transition principle). Within AIM, a transition curve is primarily a law of curvature evolution, not a coordinate-space interpolation object.

25.1. Transition Principle

Definition 25.2 (Curvature transition). A curvature transition is a curvature law:

$$\kappa(s)$$

connecting two curvature states:

$$\kappa_A \rightarrow \kappa_B.$$

The transition determines:

- geometric smoothness,
- dynamic behaviour,
- realization stability,
- runtime robustness,
- and operational comfort.

Principle 25.3 (Transition principle). Within AIM, transition geometry is governed primarily by curvature evolution rather than by coordinate interpolation.

25.2. Normalized Transition Space

Definition 25.4 (Normalized transition parameter). Transitions are commonly normalized to:

$$w \in [0, 1].$$

Definition 25.5 (Normalized curvature law). A normalized transition law satisfies:

$$\hat{\kappa}(w) \in [0, 1].$$

Normalization separates:

- intrinsic transition shape,
- from physical scaling.

This enables:

- reusable transition families,
- family interpolation,
- lookup-based evaluation,
- normalized optimization,
- and compact sparse representation.

A physical transition is obtained by scaling normalized curvature and normalized length to the actual curvature interval and physical arc-length.

25.3. Transition Families

Definition 25.6 (Transition family). A transition family is a class of normalized curvature laws sharing common structural properties.

Examples include:

- clothoids,
- polynomial transitions,
- trigonometric transitions,
- Bloss transitions,
- Vienna transitions,
- lookup-based transitions,
- and hybrid transition families.

Within AIM, transition families are interpreted as structured regions or parameterized paths within normalized curvature space.

25.4. Clothoid Transition

Definition 25.7 (Clothoid). The clothoid is defined by linear curvature evolution:

$$\kappa(s) = as + b.$$

In normalized form, the clothoid corresponds to:

$$\hat{\kappa}(w) = w.$$

The clothoid provides constant curvature gradient.

Historically, the clothoid became dominant because:

- it is simple,
- analytically tractable,
- compatible with classical railway practice,
- and dynamically smoother than curvature jumps.

Within AIM, the clothoid is not treated as the universal transition truth, but as one important member of curvature-transition space.

25.5. Polynomial Transitions

Definition 25.8 (Polynomial transition). A polynomial transition satisfies:

$$\kappa(s) = \sum_{i=0}^n a_i s^i.$$

Polynomial transitions allow:

- higher continuity,
- endpoint constraints,
- derivative constraints,
- and optimized dynamic behaviour.

Polynomial transitions are especially useful when smoothness conditions are imposed directly on curvature derivatives.

25.6. Trigonometric Transitions

Definition 25.9 (Trigonometric transition). A trigonometric transition contains sinusoidal or cosine-based curvature components.

These transitions often provide smoother higher-order behaviour than low-degree polynomial transitions.

Trigonometric transitions are naturally connected to spectral interpretation of curvature space.

25.7. Half-Wave Interpretation

Definition 25.10 (Half-wave transition). A half-wave transition is a normalized curvature segment connecting:

$$0 \rightarrow 1$$

or

$$1 \rightarrow 0.$$

Within AIM, many transition families are interpreted compositionally as:

$$\text{halfWave}_1 \rightarrow \text{core} \rightarrow \text{halfWave}_2.$$

This enables:

- modular transition construction,
- asymmetric transitions,
- family interpolation,
- and transition deformation.

The half-wave interpretation provides a common language for comparing classical transition families and newly constructed transition laws.

25.8. Berlinish Composition

Berlinish is the name used in the originating research for the generalized transition composition

$$T(s) = H_{\text{in}}(s) + C(s) + H_{\text{out}}(s).$$

Here H_{in} is an entering half-wave, C is an optional clothoid section, and H_{out} is an exiting half-wave. The plus sign denotes ordered piecewise composition on consecutive subdomains, not addition of three overlapping curvature values. Each component may have zero length.

For a normalized transition coordinate $w \in [0, 1]$, let the evidenced partition be

$$\lambda = (\lambda_1, \lambda_c, \lambda_2), \quad \lambda_i \geq 0, \quad \lambda_1 + \lambda_c + \lambda_2 = 1.$$

The current executable interpretation places the components on

$$H_{\text{in}} : [0, w_1], \quad C : [w_1, w_2], \quad H_{\text{out}} : [w_2, 1],$$

25. Curvature Transitions

with

$$w_1 = \lambda_1, qquad w_2 = \lambda_1 + \lambda_c, qquad \lambda = (w_1, w_2 - w_1, 1 - w_2). \quad (25.1)$$

Thus w_1, w_2 are cumulative normalized component-length fractions and domain cuts in the present implementation. For a physical transition length $L > 0$, the corresponding component lengths are $L\lambda_i$. Research evidence supports this current meaning, but only moderately supports the claim that the historical Berlinish source used the same symbols. They are editable partition variables today; the current bounded AXTRAN optimization does not establish a general solve for them.

This single grammar includes important limiting cases:

- $L_{H_{in}} = L_{H_{out}} = 0$ gives a pure clothoid;
- $L_C = 0$ gives a purely curved or smoothed half-wave composition;
- one zero-length half-wave gives a one-sided composition;
- unequal component lengths or different half-wave functions give asymmetric transitions;
- nonzero lengths for all three components give compound transitions.

Zero normalized component length removes a component from this composition. It is not the same case as a physical transition element with total length ($L=0$), for which no longitudinal domain remains. Keeping these degeneracies separate prevents a valid pure-clothoid partition from being confused with a zero-length physical element.

Polynomial, trigonometric, and mixed function families can supply the half-waves. Their behaviour is compared not only through $\kappa(s)$, but also through

$$\kappa'(s), \quad \kappa''(s),$$

because endpoint conditions and higher-order behaviour distinguish forms that may generate visually similar centerlines. In this way, established national, historical, or proprietary transition forms can be interpreted within one comparison framework without losing their names or provenance.

Four levels must remain distinct. *Function identity* states which polynomial, trigonometric, or mixed law is used. *Parameterization* states how that law is scaled and partitioned. *Boundary conditions* state the curvatures, derivatives, pose conditions, and available lengths a particular connection must satisfy. The *instantiated transition geometry* is the calculated result after those choices have been solved and integrated. A family is therefore not an alignment element, and a plotted curve is not the family definition.

The lower comparison in fig. 25.1 makes a further freedom explicit. Curvature $\kappa(s)$ describes plan-direction change per unit intrinsic length; cant $u(s)$ describes the transverse rail-level or roll state under its stated convention. Speed makes their combined development operationally consequential, but they are not the same engineering quantity. They may

25. Curvature Transitions

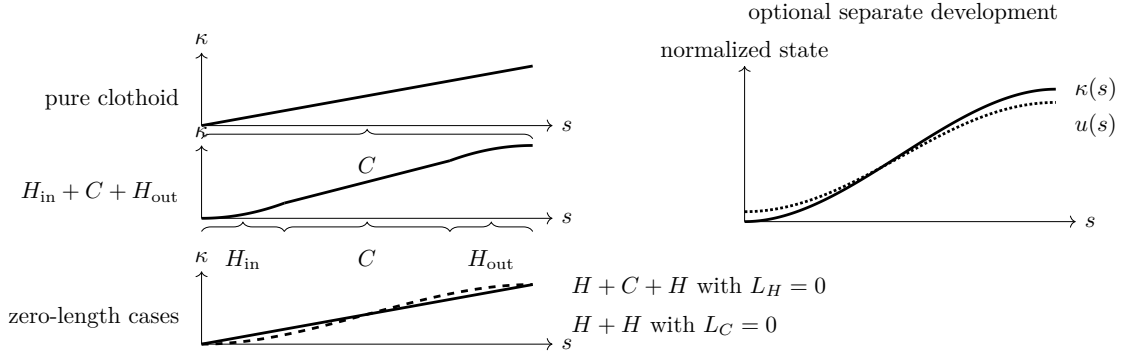


Figure 25.1.: Berlinish comparison: pure clothoid, half-wave–clothoid–half-wave composition, zero-length component cases, and optional separation of curvature and cant development.

use distinct function instances, domains, physical scaling, partitions, boundary conditions, units, and provenance. Optional coupling is a constraint between the laws, not identity of them.

The current transition registry and evaluator implement curvature only; they do not establish a cant-law record or a validated joint curvature–cant optimizer. Heritage Vienna examples show why cant derivatives may enter a coupled dynamic relation, but surviving evidence does not select one authoritative modern coupling equation or objective. Joint optimization and a single canonical coupling therefore remain Research, not established AIM capability.

Remark 25.11 (Authority of the Berlinish term). Berlinish is used in this thesis for an originating research contribution and mathematical framework. Its component interpretation is implemented in the current transition registry, but the name itself is not asserted to be an approved Knowledge Kernel concept.

25.9. Curvature Anchors

Definition 25.12 (Curvature anchors). Normalized transition families may define anchor values:

$$0, \quad c_1, \quad c_2, \quad 1.$$

Anchors describe:

- transition partitioning,
- continuity conditions,
- normalization structure,
- and family composition.

Curvature anchors make it possible to compare transition families by their internal curvature structure rather than by their external position-space appearance.

25.10. Transition Continuity

Definition 25.13 (Transition continuity). Transition quality depends on continuity of:

$$\kappa(s), \quad \kappa'(s), \quad \kappa''(s).$$

Higher continuity improves:

- jerk behaviour,
- realization smoothness,
- runtime stability,
- vehicle response,
- and passenger comfort.

Continuity in position space alone is insufficient for evaluating transition quality.

25.11. Sparse Transition Representation

Definition 25.14 (Sparse transition). A sparse transition representation stores:

- transition family identity,
- normalization parameters,
- curvature anchors,
- scaling parameters,
- and reconstruction rules.

Within AIM, sparse transitions are interpreted as compact curvature operators rather than dense geometric curves.

A sparse transition therefore stores how curvature shall evolve, not a sampled polyline approximation of the resulting geometry.

25.12. Lookup-Based Transitions

Definition 25.15 (Lookup-based transition). A lookup-based transition uses:

- sampled normalized curvature,

- interpolation rules,
- derivative reconstruction,
- and runtime evaluation.

This enables:

- arbitrary transition families,
- experimentally designed transitions,
- realization-adapted transitions,
- vehicle-specific transitions,
- and runtime-efficient evaluation.

Lookup-based transitions are not a fallback for missing theory. They are a practical representation of transition laws in normalized curvature space.

25.13. Transition Derivatives

Definition 25.16 (Transition derivatives). Important quantities include:

$$\kappa'(s), \quad \kappa''(s), \quad \kappa'''(s).$$

These derivatives influence:

- jerk,
- dynamic smoothness,
- realization behaviour,
- vehicle response,
- and optimization stability.

Derivative behaviour is one of the main differences between transition families that may otherwise appear similar in position space.

25.14. Transition Symmetry

Definition 25.17 (Symmetric transition). A normalized transition is symmetric if its curvature evolution is symmetric with respect to the midpoint of the transition interval.

Definition 25.18 (Asymmetric transition). A transition is asymmetric if this symmetry is intentionally violated.

Asymmetric transitions may improve:

- realization quality,
- vehicle dynamics,
- topology fitting,
- or geometric constraint satisfaction.

Symmetry is therefore a classification property, not a universal requirement.

25.15. Transition Frequency Behaviour

Transitions may be interpreted spectrally through their curvature content.

High-frequency curvature behaviour influences:

- realization sensitivity,
- machine smoothing,
- measurement sensitivity,
- and dynamic roughness.

Proposition 25.19 (Spectral transition principle). *Transition quality depends not only on curvature continuity, but also on spectral behaviour within curvature space.*

This connects transition theory directly to realization-stable curvature and curvature bandwidth.

25.16. Transition Realization

The transition that is designed is not necessarily the transition that is physically realized.

Realization may modify transition behaviour through:

- sampling,
- machine response,
- structural filtering,
- ballast mechanics,
- measurement feedback,
- and local correction limits.

Thus, transition quality must be evaluated both in target curvature space and in expected realized curvature space.

25.17. Transition Optimization

Within AIM, optimization acts primarily on:

- transition families,
- normalization structure,
- transition lengths,
- curvature anchors,
- derivative behaviour,
- and curvature evolution.

Thus, optimization is interpreted primarily as optimization in curvature-transition space rather than position space.

A realization-aware optimization may optimize not only the target transition law κ_{target} , but also the expected realized transition law:

$$\kappa_{\text{real}} = \mathcal{R}_{\kappa}(\kappa_{\text{target}}).$$

25.18. Canonical Interpretation

Proposition 25.20. *Transition curves are fundamentally curvature-transition laws.*

Proposition 25.21. *Within AIM, transition geometry is interpreted intrinsically within curvature space.*

Proposition 25.22. *Sparse transition models represent compact transition operators rather than explicit geometric curves.*

Proposition 25.23. *Transition quality depends on curvature evolution, derivative behaviour, spectral content, runtime stability, and realization behaviour.*

25.19. Connection to AIM

Summary 25.24. Curvature transitions form the dynamic geometric core of AIM.

Within this framework:

- transitions are interpreted as curvature evolution processes,
- sparse models become transition operators,
- normalized transition space enables reusable families,
- half-wave decomposition enables modular transition construction,

25. *Curvature Transitions*

- lookup-based laws enable experimental and realization-aware transition design,
- and runtime systems reconstruct geometry dynamically from curvature structure.

This interpretation unifies transition theory, curvature classification, realization behaviour, runtime evaluation, and optimization within one intrinsic curvature framework.

26. Curvature Classification

Motivation

Classical railway alignment theory typically classifies transitions by their analytical formulas or historical engineering usage.

Within AIM, transition and alignment structures are classified intrinsically within curvature space.

This allows classification based on:

- continuity,
- curvature behaviour,
- dynamics,
- realization behaviour,
- spectral structure,
- runtime robustness,
- and operational suitability.

Curvature classification therefore becomes a structural interpretation framework rather than merely a catalogue of curve formulas.

Principle 26.1 (Classification as interpretation). Within AIM, classification is not a naming scheme for curve formulas. It is an interpretation layer for curvature behaviour.

26.1. Classification Principle

Principle 26.2 (Intrinsic classification principle). Within AIM, alignments are classified by intrinsic curvature behaviour rather than by reconstructed position geometry.

Two geometrically similar curves in position space may possess very different curvature behaviour.

Conversely, geometrically different realizations may belong to the same curvature class if their intrinsic curvature evolution behaves equivalently.

Classification is therefore performed in curvature space before it is interpreted in position space.

26.2. Continuity Classification

26.2.1. G^0 Geometry

Definition 26.3 (G^0 -continuous geometry). A geometry is G^0 -continuous if position is continuous.

26.2.2. G^1 Geometry

Definition 26.4 (G^1 -continuous geometry). A geometry is G^1 -continuous if tangent direction is continuous.

26.2.3. G^2 Geometry

Definition 26.5 (G^2 -continuous geometry). A geometry is G^2 -continuous if curvature is continuous.

Within railway alignment modelling, G^2 -continuity is often the minimal requirement for dynamically acceptable transitions.

26.2.4. G^3 Geometry

Definition 26.6 (G^3 -continuous geometry). A geometry is G^3 -continuous if curvature gradient is continuous.

Higher continuity classes improve:

- jerk behaviour,
- realization smoothness,
- runtime stability,
- and vehicle response.

Continuity classification bridges classical geometry notation with curvature-space interpretation.

26.3. Curvature Behaviour Classification

26.3.1. Constant Curvature Class

Definition 26.7 (Constant curvature class). A curvature law belongs to the constant class if:

$$\kappa(s) = \kappa_0.$$

26.3.2. Linear Curvature Class

Definition 26.8 (Linear curvature class). A curvature law belongs to the linear class if:

$$\kappa(s) = as + b.$$

The classical clothoid belongs to this class.

26.3.3. Polynomial Curvature Class

Definition 26.9 (Polynomial curvature class). A curvature law belongs to the polynomial class if:

$$\kappa(s) = \sum_i a_i s^i.$$

26.3.4. Trigonometric Curvature Class

Definition 26.10 (Trigonometric curvature class). A curvature law belongs to the trigonometric class if it contains sinusoidal or cosine-based components.

26.3.5. Lookup-Based Curvature Class

Definition 26.11 (Lookup-based curvature class). A curvature law belongs to the lookup-based class if it is defined by sampled or tabulated curvature values together with interpolation and reconstruction rules.

26.3.6. Hybrid Curvature Class

Definition 26.12 (Hybrid curvature class). A hybrid curvature law combines:

- parametric structure,
- lookup structure,
- measured data,
- and sampled corrections.

26.4. Symmetry Classification

26.4.1. Symmetric Transitions

Definition 26.13 (Symmetric transition). A normalized transition is symmetric if its curvature evolution is symmetric with respect to the midpoint of the normalized interval.

For transition comparison, symmetry should usually be evaluated in normalized transition space rather than in raw physical coordinates.

26.4.2. Antisymmetric Structure

Definition 26.14 (Antisymmetric transition). A transition has antisymmetric structure if its deviation from a reference or midpoint value changes sign under midpoint reflection.

Antisymmetry is especially useful when analysing centred transition shape functions or derivative behaviour.

26.4.3. Asymmetric Transitions

Definition 26.15 (Asymmetric transition). A transition is asymmetric if no symmetry condition is imposed.

Asymmetry may arise intentionally through:

- realization-aware design,
- vehicle-dynamics optimization,
- topology constraints,
- or local engineering constraints.

26.5. Derivative Classification

26.5.1. Bounded Gradient Class

Definition 26.16 (Bounded gradient class). A curvature law belongs to the bounded-gradient class if:

$$|\kappa'(s)| \leq \eta_{\max}.$$

26.5.2. Bounded Higher-Derivative Class

Definition 26.17 (Bounded higher-derivative class). A curvature law belongs to this class if:

$$|\kappa''(s)| < \infty, \quad |\kappa'''(s)| < \infty.$$

These classes influence:

- dynamic smoothness,
- numerical stability,
- realization robustness,
- and optimization conditioning.

26.6. Spectral Classification

26.6.1. Low-Frequency Structures

Definition 26.18 (Low-frequency curvature class). Low-frequency curvature structures are dominated by slowly varying curvature evolution.

Low-frequency structure usually determines the large-scale geometric shape of the alignment.

26.6.2. High-Frequency Structures

Definition 26.19 (High-frequency curvature class). High-frequency curvature structures contain rapidly varying or oscillating components.

High-frequency structures tend to:

- amplify realization sensitivity,
- increase dynamic roughness,
- increase measurement sensitivity,
- and reduce numerical stability.

Proposition 26.20 (Spectral realization principle). *Realization suppresses high-frequency curvature components.*

Spectral classification therefore links transition theory with physical realization behaviour.

26.7. Realization Classification

26.7.1. Realization-Stable Class

Definition 26.21 (Realization-stable class). A transition belongs to the realization-stable class if:

- dominant curvature structure is preserved under realization,
- low-frequency behaviour remains identifiable,
- and realization does not introduce instability.

26.7.2. Realization-Sensitive Class

Definition 26.22 (Realization-sensitive class). A transition belongs to the realization-sensitive class if small realization perturbations strongly alter curvature behaviour.

Realization-sensitive transitions may be mathematically valid but operationally undesirable.

26.8. Runtime Classification

26.8.1. Projection-Stable Class

Definition 26.23 (Projection-stable class). A curvature structure is projection-stable if:

- world-to-track projection remains numerically robust,
- local inverse problems remain well-conditioned,
- and ambiguity remains controllable.

26.8.2. Runtime-Smooth Class

Definition 26.24 (Runtime-smooth class). A curvature structure is runtime-smooth if:

- frame evaluation remains continuous,
- pose reconstruction remains robust,
- visualization remains stable,
- and runtime operators behave predictably.

26.9. Operational Classification

26.9.1. Comfort-Oriented Class

Definition 26.25 (Comfort-oriented class). A transition belongs to the comfort-oriented class if:

- jerk remains low,
- lateral acceleration evolves smoothly,
- and dynamic excitation is minimized.

26.9.2. Realization-Oriented Class

Definition 26.26 (Realization-oriented class). A transition belongs to the realization-oriented class if:

- construction robustness is prioritized,
- machine realization remains stable,
- maintenance sensitivity is reduced,
- and physical filtering preserves the intended behaviour.

26.9.3. Optimization-Oriented Class

Definition 26.27 (Optimization-oriented class). A transition belongs to the optimization-oriented class if:

- parameterization is numerically stable,
- gradients remain well-conditioned,
- constraints are expressible,
- and sparse representation is efficient.

26.10. Family Classification

Definition 26.28 (Transition family classification). Transition families may be classified according to:

- analytical form,
- continuity order,
- derivative behaviour,
- spectral behaviour,
- realization behaviour,
- symmetry,
- runtime stability,
- and operational suitability.

Thus, transition classification becomes multidimensional rather than formula-based.

A single transition family may be favourable in one classification axis and unfavourable in another.

26.11. Classification Vectors

Definition 26.29 (Classification vector). A transition may be assigned a classification vector:

$$\chi(T) = (\chi_{\text{cont}}, \chi_{\text{deriv}}, \chi_{\text{spec}}, \chi_{\text{real}}, \chi_{\text{run}}, \chi_{\text{op}}),$$

where the components describe continuity, derivative behaviour, spectral structure, realization behaviour, runtime stability, and operational suitability.

The classification vector is not intended as a rigid taxonomy. It is a structured way to compare transition families across multiple relevant dimensions.

26.12. Canonical Interpretation

Proposition 26.30. *Within AIM, curvature classification is fundamentally intrinsic rather than coordinate-based.*

Proposition 26.31. *Transition quality cannot be characterized sufficiently by position geometry alone.*

Proposition 26.32. *Realization, runtime stability, dynamics, and optimization require classification directly in curvature space.*

Proposition 26.33. *Transition taxonomy in AIM is multidimensional and context-dependent.*

26.13. Connection to AIM

Summary 26.34. Curvature classification provides the structural interpretation framework of AIM geometry.

Within this framework:

- transitions are classified intrinsically,
- formula names are secondary to curvature behaviour,
- realization behaviour becomes analyzable,
- runtime robustness becomes class-dependent,
- operational suitability becomes comparable,
- and optimization acts directly on curvature structures.

This establishes curvature space as the primary classification domain of railway alignment geometry and provides the basis for a taxonomy of transition families.

Part V.

Railway State Model

State

The straight–transition–arc Alignment from the preceding part is still the engineering object under discussion. Its identity, ordered construction, intrinsic parameter s , curvature law $\kappa(s)$, Berlinish transition partition, provenance, and applicability have already been established. State therefore does not begin with an anonymous coordinate tuple. It begins with a question about this identified object:

At a selected s , what railway state follows from the Alignment, and which additional laws and realization data are required to obtain it?

Curvature first determines change of heading and, with a starting pose, a planar trajectory. Vertical geometry and cant then add independent longitudinal laws. A stated frame convention combines these inputs into a spatial railway interpretation. Only after speed and, where necessary, a vehicle model are supplied do acceleration and other operational quantities follow. Evaluation for a purpose is later still: a computed consequence is not an engineering decision.

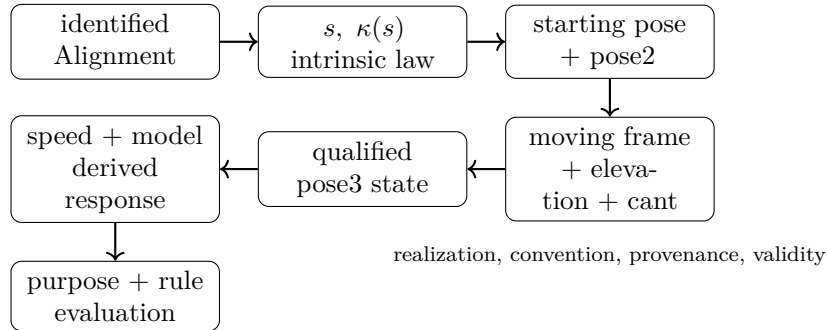


Figure 26.1.: Dependency from the identified Alignment to a qualified engineering consequence.

The arrows in fig. 26.1 denote dependency, not identity or authority. In particular, a realized pose does not become Alignment identity, an observation does not become physical reality, and an evaluation result does not decide whether a proposal is accepted.

A Narrow Explanatory Contract

For this part, a *railway-state record* may be read as the following Thesis-level explanatory structure:

$$\mathcal{S}_{\text{rail}}(s) = (\text{AlignmentRef}, s, \text{pose}, F, \kappa, \text{vertical}, \text{cant}, \text{realization}, \text{provenance}, t_{\text{valid}}, \text{derived}).$$

Speed is included only for a speed-qualified use. Observation provenance is included only for an observed-state estimate. This tuple is not a new canonical Kernel concept and not a universal storage schema; it merely keeps the dependencies and qualifications visible while the argument proceeds.

Kind	Running example	What it does not establish
Intrinsic	s , element order, $\kappa(s)$	coordinates, cant, physical condition, acceptance
Realized	local pose, elevation, frame, represented coordinates	Alignment identity or the constructed asset's actual state
Observed	time-stamped measured rail positions with method and uncertainty	defining geometry or unqualified truth
Derived	$v^2\kappa$, effective lateral action, comparison residual	authority, admissibility, or decision

Table 26.1.: Intrinsic, realized, observed, and derived quantities remain distinct.

The chapters now follow one dependency chain:

Alignment identity $\rightarrow s \rightarrow \kappa(s) \rightarrow \text{pose2} \rightarrow \text{moving frame}$
 $\rightarrow \text{elevation and cant} \rightarrow \text{pose3}$
 $\rightarrow \text{speed-derived quantities} \rightarrow \text{purpose-qualified evaluation}.$

Principle 26.35 (Railway-State Principle). Railway state is derived from an identified Alignment under stated additional laws, realization data, conventions, and use qualifications. No resulting pose, coordinate representation, observation, or evaluation owns the Alignment's identity or authority.

Summary 26.36. Part V is the physical unfolding of the already identified Alignment. It introduces each state quantity only when an engineering consequence requires it.

27. From Curvature to Planar Pose

Choose a position s in the transition of the running straight–transition–arc Alignment. The intrinsic law already answers how strongly the Alignment turns there. It does not yet answer where a drawing, local model, or survey comparison places that point. For that, the same constructive law needs a starting pose in a chosen planar realization frame.

27.1. What the Starting Pose Contributes

Let (x_0, y_0, θ_0) be the position and orientation assigned to $s = 0$ in that frame. The curvature law then determines

$$\theta(s) = \theta_0 + \int_0^s \kappa(\sigma) \, d\sigma, \quad (27.1)$$

and the tangent determines position:

$$x(s) = x_0 + \int_0^s \cos \theta(\sigma) \, d\sigma, \quad (27.2)$$

$$y(s) = y_0 + \int_0^s \sin \theta(\sigma) \, d\sigma. \quad (27.3)$$

These integral equations are the executable reading of the differential system in eq. (9.6). They receive the ordered intrinsic law and an initial condition; they produce one realized planar trajectory. They require coherent orientation, sign, parameter-direction, and metric conventions. They cannot determine an authoritative CRS, physical condition, or acceptance decision.

Definition 27.1 (pose2). At intrinsic position s , the planar pose produced by this reconstruction is

$$\text{pose2}(s) = (\gamma(s), \theta(s)), \quad \gamma(s) = (x(s), y(s)).$$

$\kappa(s)$, its constructive order, and the intrinsic direction belong to the Alignment geometry. The numerical values $x(s)$, $y(s)$, and $\theta(s)$ belong to the selected realization and representation convention. A rigid change of local frame changes those values while leaving the underlying ordered curvature law unchanged; whether a particular edit changes durable object identity remains an engineering identity question, not a consequence of pose coordinates.

Definition 27.2 (Initial pose2 state). An initial pose2 state is the realization datum $(\gamma(0), \theta(0))$ supplied together with the intrinsic curvature law and its orientation convention.

27.2. The Running Transition

On the initial straight, $\kappa = 0$, so heading remains constant. Across the Berlinish transition, the already established partition supplies the piecewise curvature evolution; integrating it accumulates heading without discarding constructive order. On the circular arc, constant κ produces linear heading growth. No sampled polyline is required to define this logic. A CAD curve or survey polyline may approximate or observe the resulting trajectory, but neither becomes the law that generated it.

27.3. Persistent Information and Derived State

Proposition 27.3. *The Alignment’s intrinsic curvature construction is persistent engineering information; pose2 is its planar state under specified initial and realization conditions.*

This replaces the misleading idea that pose2 itself is a persistent “kernel” or owns Alignment identity. Sparse cGeom elements can be read as a compact reconstruction program: they retain the ordered laws from which pose2 can be evaluated. Persisting a sampled trajectory as a representation may be useful, but redundancy does not transfer constructive authority to the samples.

27.4. Why pose2 Is Not Yet Railway State

At the chosen s , pose2 answers planar location and forward orientation in one realization. It says nothing yet about elevation, gradient, vertical curvature, cant, or the orientation of the rail plane. Those require additional longitudinal laws and a moving-frame convention. The next chapter adds precisely those dependencies; it does not replace the Alignment or promote pose2 coordinates to identity.

Summary 27.4. The same $\kappa(s)$ combined with different starting poses yields different coordinate realizations of the same intrinsic construction. Pose2 is therefore a necessary derived state, not a complete asset, physical truth, observation, or decision.

28. From Planar Pose to Spatial Railway State

At the selected transition position, pose2 supplies planar location and heading. A railway cross-section still cannot be placed: the Alignment's horizontal curvature law contains neither elevation nor rail-plane rotation. Spatial interpretation therefore needs additional, independently qualified longitudinal laws.

28.1. Additional Laws, Not Additional Identity

Let $h(s)$ denote the elevation law in the chosen metric realization. Where supported, $h'(s)$ supplies gradient and $h''(s)$ contributes vertical-curvature information under the adopted profile convention. Let $u(s)$ denote cant as a rail-height difference:

Definition 28.1 (Cant height difference).

$$u: D_u \rightarrow \mathbb{R}$$

is a longitudinal cant law with its own domain D_u , units, partition, provenance, and applicability.

Cant is not horizontal curvature. The laws $\kappa(s)$, $h(s)$, and $u(s)$ may have different breakpoints and sources. Their common parameter allows synchronized evaluation; it does not make them one property or give one law authority over another.

Definition 28.2 (Cant reference convention). A cant convention states the sign, rotation axis, gauge interpretation, reference line or rail, and any induced reference-line displacement.

For the explanatory convention used here only, u is measured across a stated gauge $g_{\text{gauge}} > 0$, the centerline remains the reference, and positive roll follows the displayed right-handed frame. Then

$$\phi(s) = \arctan\left(\frac{u(s)}{g_{\text{gauge}}}\right). \quad (28.1)$$

This equation converts one qualified representation of cant into a roll angle. It is not a universal cant convention and says nothing about which cant law should be designed.

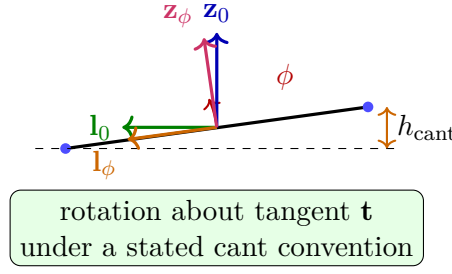


Figure 28.1.: Local cross-section: cant rotates the lateral–vertical pair about the tangent; the convention and gauge remain explicit inputs.

28.2. The Moving Railway Frame

Definition 28.3 (Uncanted geometric frame). At s , let

$$F_0(s) = (\mathbf{t}(s), \mathbf{l}_0(s), \mathbf{z}_0(s))$$

be an oriented orthonormal frame: $\mathbf{t}$ is longitudinal, while $\mathbf{l}_0$ and $\mathbf{z}_0$ are the lateral and vertical directions supplied by a stated realization and transport convention.

Horizontal curvature controls change of heading. Vertical geometry controls the slope of $\mathbf{t}$. Metric realization supplies the spatial reference and vertical interpretation. A continuity rule transports the cross-section through straights and low-curvature regions where a pure Frenet construction would be unstable.

Definition 28.4 (Railway frame). The cant-aware track frame is

$$F_{\text{rail}}(s) = (\mathbf{t}(s), \mathbf{l}_\phi(s), \mathbf{z}_\phi(s)), \quad (\mathbf{l}_\phi, \mathbf{z}_\phi) = R_{\mathbf{t}, \phi(s)}(\mathbf{l}_0, \mathbf{z}_0).$$

This local right-handed construction is selected to make dependencies visible; it does not establish a universal AIM frame convention. Source conventions must be transformed explicitly before signs or rail positions are compared.

Definition 28.5 (Vehicle frame). $F_{\text{veh}}(s)$ denotes an optional vehicle-dependent frame produced from the track frame together with a stated vehicle and suspension model.

A vehicle frame cannot be inferred from Alignment geometry alone.

28.3. What pose3 Adds

Definition 28.6 (Runtime pose3). For this Thesis-level explanation, the spatial railway pose at s is

$$\text{pose3}(s) = (\Gamma(s), F_{\text{rail}}(s), h'(s), h''(s), u(s)), \quad (28.2)$$

with unavailable or inapplicable profile terms omitted and all quantities qualified by their realization and convention.

$\Gamma(s)$ is the realized spatial reference trajectory assembled from pose2, profile, metric interpretation, and coordinate services. The frame operator $\mathcal{F}$ of theorem 11.5 expresses the typed evaluation. Pose3 adds the quantities needed to orient a railway cross-section; it is not the complete physical asset and does not include materials, condition, topology, vehicle behaviour, or engineering authority.

Proposition 28.7 (pose3 reconstruction principle). *Given $\kappa(s)$, $h(s)$, and $u(s)$, together with a starting pose, metric realization, gauge, orientation and frame conventions, the applicable pose3 fields can be evaluated. The result preserves its Alignment reference and the provenance of every additional input.*

28.4. Target, Realization, Physical State, and Observation

The intended Alignment and its design laws produce a *target pose*. A Metric Realization places that target in a metric space. Construction produces a physical asset whose state varies with time. Surveying produces time-stamped observations or observed-state estimates in represented coordinates. These are related, not interchangeable.

A CAD curve may represent the target trajectory. A survey polyline may represent observations of the rails. A Soll-Ist comparison must qualify both sides by object reference, time, realization, convention, provenance, and uncertainty before a deviation can be interpreted. Coordinate subtraction alone cannot decide which state is correct or acceptable.

28.5. Cant and Curvature Meet Downstream

Curvature and cant remain separate laws even when an operational calculation uses both. The relations in eqs. (9.10) to (9.12) are simplified, assumption-dependent interpretations introduced for the subsequent dynamics discussion. They are not authoritative design coupling rules. Applicable standards, vehicle models, speed profiles, and decisions remain separate inputs.

Summary 28.8. Pose3 is a qualified spatial state derived from the identified Alignment plus profile, cant, metric realization, and frame convention. Cant rotates the railway frame but neither changes the identity of $\kappa(s)$ nor supplies engineering authority.

Proposition 28.9 (Track state principle). *Pose3 describes a qualified railway pose. Vehicle states require additional vehicle-space operators and models.*

29. Moving Frames and Their Limits

The preceding chapter constructed one local railway frame because the running Alignment required a cross-section orientation. This chapter states what each direction depends on and where the explanatory convention ends.

29.1. Longitudinal Direction

For the realized spatial trajectory $\Gamma(s)$, the unit tangent is

$$\mathbf{t}(s) = \frac{\Gamma'(s)}{\|\Gamma'(s)\|}.$$

It follows from horizontal pose, vertical profile, metric interpretation, and the orientation of increasing s . Horizontal curvature alone does not determine its three-dimensional direction.

29.2. Uncanted Cross-Section

The realization supplies a usable vertical reference $\mathbf{z}_{\text{ref}}(s)$: locally this may be the vertical of a planar engineering realization; in an Earth realization it may involve surface normal and gravity conventions. Projecting that reference orthogonally to $\mathbf{t}$, normalizing it, and completing a right-handed triad gives one possible $F_0 = (\mathbf{t}, \mathbf{l}_0, \mathbf{z}_0)$. A continuous-transport rule is required where the projection becomes ill-conditioned.

This construction is deliberately qualified. The Thesis does not choose one universal railway frame for every realization. What must remain explicit is:

- the longitudinal direction follows the realized trajectory and parameter orientation;
- the uncanted lateral–vertical pair follows a stated vertical and transport convention;
- $u(s)$ or $\phi(s)$ rotates that pair about $\mathbf{t}$;
- coordinate components follow the selected representation.

29.3. Why a Pure Frenet Frame Is Insufficient

A Frenet normal derived only from curvature is undefined or numerically unstable as $\kappa \rightarrow 0$. The running Alignment contains an initial straight, so a railway frame must carry orientation continuously across that segment. This is not a new architectural object: it is an applicability condition of the frame-evaluation operation.

29.4. Degrees of Determination

At a fixed s , Alignment identity and the ordered law are preserved. The starting pose and metric realization fix the planar placement. The profile fixes elevation and slope where defined. The cant law fixes roll only under a gauge and sign convention. A residual yaw or cross-section convention may remain free until the selected frame rule resolves it. Vehicle body orientation remains free until a vehicle model and operating state are provided.

29.5. World–Track and Vehicle–Space Use

World–track projection consumes the qualified frame: $\mathbf{t}$ defines longitudinal direction, $\mathbf{l}_\phi$ lateral offset, and $\mathbf{z}_\phi$ cross-section height. A vehicle-space interpretation may then consume this track frame, but it also requires vehicle-specific data. The frame preserves the Alignment reference and provenance of its realization; it cannot decide whether a measured point belongs to the intended object or whether a clearance result is acceptable.

Summary 29.1. A moving railway frame is an evaluated relation between intrinsic geometry, vertical realization, cant, and convention. Its components are useful only with those dependencies attached.

30. Speed-Qualified Consequences

The running Alignment and its pose3 state now describe where the track frame is and how it is oriented. They still do not determine a physical response. Speed enters as a separate operating input; a vehicle model and evaluation rule enter separately again.

30.1. From Arc Length to Time

For constant $v > 0$,

$$\frac{ds}{dt} = v, \quad \frac{df}{dt} = v \frac{df}{ds}.$$

Thus the spatial curvature rate $\kappa'(s)$ acquires a time rate only after speed is supplied. The same holds for cant rate and higher derivatives.

30.2. A Reduced Illustrative Relation

For planar motion of a point following the reference trajectory,

$$a_\kappa(s) = v^2 \kappa(s).$$

Under the local right-handed frame, small-scope rigid-body assumptions, the cant convention of eq. (28.1), and neglect of suspension, wheel–rail, wind, and other effects, a reduced effective lateral component is

$$a_{\text{eff}}(s) = v^2 \kappa(s) - g \sin \phi(s).$$

This is an illustrative derived quantity, not a validated complete vehicle dynamics model and not an authoritative cant-design rule.

Case at the same s	$v^2 \kappa$	a_{eff} for $\phi = 4^\circ$
$v = 20 \text{ m/s}, \kappa = 0.001 \text{ m}^{-1}$	0.40 m/s^2	-0.28 m/s^2
$v = 40 \text{ m/s}, \kappa = 0.001 \text{ m}^{-1}$	1.60 m/s^2	0.92 m/s^2

Table 30.1.: Illustrative speed dependence for the same curvature and cant state ($g = 9.81 \text{ m/s}^2$, rounded).

Table 30.1 isolates the dependency: neither the Alignment nor $\kappa(s)$ changes, yet doubling speed multiplies the curvature-induced term by four. Changing cant would change the effective component and frame, not the horizontal curvature law.

30.3. Variation and Operational Meaning

With the same assumptions and constant speed,

$$j(s) = v^3 \kappa'(s) - gv \cos \phi(s) \phi'(s).$$

This relation explains why transition shape and cant transition both matter operationally while remaining distinct laws. If speed varies, additional time-derivative terms appear. Vehicle response further depends on mass, suspension, wheel–rail contact, and other model choices. None of those quantities is silently folded into Alignment geometry.

30.4. Evaluation Is a Later Operation

The dependency order is

$$\begin{aligned} \text{geometry} &\rightarrow \text{realized frame} \rightarrow \text{speed profile} \rightarrow \text{vehicle model} \\ &\rightarrow \text{derived dynamics} \rightarrow \text{purpose and rule} \rightarrow \text{evaluation}. \end{aligned}$$

A limit from a standard is applicable only under its scope and version. A computed exceedance is an evaluation result; acceptance, rejection, or authorization requires an Engineering Decision with the relevant authority.

Summary 30.1. Speed changes the consequence of the running Alignment without changing its intrinsic geometry. Cant changes the track frame and the reduced effective action, while geometry, operating input, vehicle model, derivation, evaluation, and decision remain separate.

31. Admissibility States

Admissibility is not an additional geometric state layer. It is the purpose-qualified evaluation that follows the dependency chain developed in the preceding chapters. It asks whether derived quantities remain inside an applicable operational envelope; it neither changes the running Alignment nor turns a computed result into a decision.

31.1. Admissibility State

Definition 31.1 (Admissibility state). An admissibility state is a runtime evaluation state describing whether operational quantities remain within allowed envelopes.

Admissibility states may include bounds on effective acceleration, jerk, cant deficiency, roll evolution, velocity, and realization quality.

These quantities are derived from runtime railway state and dynamic evaluation; they are not persisted as alignment identity.

31.2. Operational Envelopes

Definition 31.2 (Operational envelope state). An operational envelope state describes admissible regions in runtime state space.

Operational envelopes are typically defined by standards, infrastructure constraints, and operational policy. In AIM they act as constraints on derived runtime quantities rather than as replacements for intrinsic geometry or railway state.

31.3. Layer Position

Admissibility is evaluated after pose2/pose3 reconstruction, metric realization, frame construction, and speed-qualified quantity derivation. Rule, version, purpose, applicable object and state, validity time, and authority must be stated. The assessment remains downstream of railway state and does not redefine it.

Vehicle-level admissibility is evaluated relative to railway state and must not be conflated with the railway-state layer itself.

31.4. Connection to AIM

Summary 31.3. Admissibility states connect runtime dynamics, engineering rules, realization-aware behaviour, and optimization constraints while preserving the separation between persistent alignment information and derived operational interpretation.

32. World–Track Coordinate Transformation

Motivation

This chapter answers how intrinsic railway coordinates and external world coordinates are related in AIM without collapsing geometry, runtime evaluation, and coordinate representation into one operation. It establishes why world–track transformation is treated as an operator-level runtime service and what this implies for projection, ambiguity, and engineering interpretation.

The first question is directional: what exactly is mapped from world to track, and what is mapped from track to world?

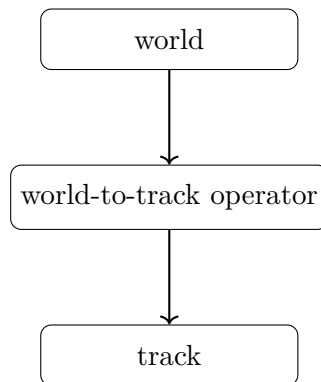


Figure 32.1.: Supporting operator view: world-to-track transformation as directed runtime mapping.

The figure fixes this directionality before formal definitions are introduced and prevents confusion between projection and generic coordinate conversion.

Railway alignments are formulated intrinsically using arc-length-based coordinates. Real-world data, by contrast, is typically represented in external coordinate reference systems such as WGS84, UTM, Gauss–Krüger, DBRef, or local engineering CRS definitions. A transformation between intrinsic alignment coordinates and external world coordinates is therefore required. Within AIM, this transformation is interpreted as a runtime evaluation service connecting intrinsic alignment states, measured data, visualization, realization, topology, and operational geometry.

Principle 32.1 (Intrinsic-space primacy). Within AIM, intrinsic railway geometry is primary.

World-space coordinates are interpreted as external embeddings generated through runtime evaluation.

32.1. Track Coordinate System

Definition 32.2 (Track space). The track space is the intrinsic railway coordinate domain:

$$\Omega_{\text{track}} = \{(s, d, h)\}.$$

It represents positions relative to an alignment rather than positions in an external coordinate reference system.

Definition 32.3 (Track coordinate system). The intrinsic track coordinate system consists of:

$$(s, d, h),$$

where:

- s denotes intrinsic arc-length,
- d denotes lateral offset,
- h denotes vertical offset.

Track coordinates belong to the intrinsic track-space domain Ω_{track} , introduced in theorem 32.2. In planar situations, only (s, d) is required. Track coordinates are intrinsic alignment coordinates and must not be confused with projected world coordinates.

32.1.1. Intrinsic Geometry versus External Coordinates

Intrinsic geometry is independent of any external coordinate reference system. The same intrinsic alignment may be embedded into projected engineering planes, ellipsoidal world models, local realization frames, or visualization coordinate systems. Thus, intrinsic alignment geometry and external CRS representation form distinct conceptual layers.

32.1.2. Arc-Length and Stationing

Intrinsic arc-length is not identical to engineering stationing. Operational kilometre systems may contain station equations, kilometre jumps, overlapping station ranges, topology-dependent references, and operational branch relations. Stationing systems are operational reference structures layered on top of intrinsic geometry. Consequently, the mapping $s \leftrightarrow \text{km}$ is generally neither linear nor globally unique.

32.2. Track-to-World Transformation

Definition 32.4 (Track-to-world mapping). The forward transformation is:

$$x(s, d) = \gamma(s) + d \mathbf{n}(s). \tag{32.1}$$

32. World–Track Coordinate Transformation

This relation specializes the canonical track-to-world operator $\mathcal{P}_{\text{tw}}$ introduced in theorem 11.6.

The complementary question is then geometric-operational: how is the intrinsic state emitted into world space once directionality is fixed?

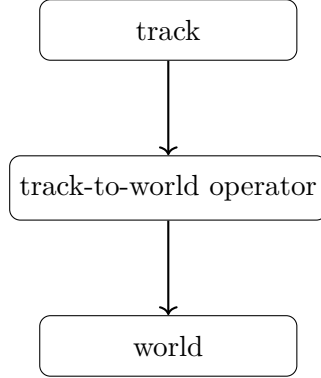


Figure 32.2.: Supporting operator view: track-to-world transformation as the complementary directed mapping.

This figure provides that forward-emission interpretation and closes the operator pair with fig. 32.1. The engineering consequence is that both mappings must be treated as coupled runtime services with distinct numerical behaviour, not as a single coordinate utility.

32.2.1. Planar Tangent and Normal Directions

In the planar case:

$$\mathbf{t}(s) = (\cos \theta(s), \sin \theta(s)),$$

and:

$$\mathbf{n}(s) = (-\sin \theta(s), \cos \theta(s)).$$

32.2.2. Spatial Runtime Evaluation

In spatial runtime evaluation, the mapping depends additionally on profile, cant, frame construction, metric interpretation, and runtime realization context. Consequently, the world embedding $x(s, d, h)$ is generally generated dynamically from runtime pose3 evaluation.

32.2.3. Reference-Line Interpretation

The lateral coordinate d depends on the chosen reference interpretation. Possible reference structures include the alignment centerline, track center, rail axis, gauge reference, or operational offset definitions. A more formal treatment of reference-line conventions for multi-track systems, cant-dependent offsets, and vehicle-space interpretation remains open.

32.3. World-to-Track Transformation

Definition 32.5 (World-to-track projection). The inverse transformation is:

$$(s^*, d^*) = \arg \min_{s, d} \|x - (\gamma(s) + d \mathbf{n}(s))\|. \quad (32.2)$$

This relation specializes the canonical world-to-track operator $\mathcal{P}_{\text{wt}}$ introduced in theorem 11.7.

The term projection is used operationally and does not necessarily imply a unique orthogonal projection in the classical geometric sense.

Within AIM, world-to-track projection is interpreted as an inverse runtime evaluation problem.

32.4. Coordinate Transformation versus Projection

Coordinate transformation and world-to-track projection are distinct operations within AIM. Coordinate transformations map between world-space representations, $\mathcal{C} : \Omega_{\text{world}} \rightarrow \Omega_{\text{world}}$, as introduced in theorem 11.8. World-to-track projection maps between Ω_{world} and Ω_{track} .

Principle 32.6 (Projection distinction principle). Coordinate transformation must not be confused with intrinsic world-to-track projection.

CRS transformations operate entirely within external coordinate representations, whereas projection accesses intrinsic railway geometry.

32.5. Numerical Evaluation

The inverse transformation is generally nonlinear. Typical numerical approaches include nearest-point search, local Newton iteration, segment-wise projection, topology-aware lookup, and hybrid runtime strategies. Projection quality depends strongly on curvature behaviour, topology, initialization, runtime context, and metric interpretation.

32.6. Ambiguity and Context Dependence

World-to-track mapping is generally not globally unique. Ambiguities may occur in loops, switches and crossings, parallel tracks, closely spaced alignments, station areas, and topology-rich railway networks. Multiple intrinsic states may correspond to nearly identical world-space locations. Consequently, projection is fundamentally context dependent. Runtime projection may require track identity, route identity, topology context, expected station range, operational direction, runtime interaction history, vehicle context, or network-state constraints.

Principle 32.7 (Projection ambiguity principle). World-to-track projection cannot generally be interpreted as a purely local geometric operation.

32.7. Topology and Operational References

Intrinsic geometry alone is insufficient for operational railway referencing. Operational interpretation additionally depends on topology, route membership, kilometre references, operational direction, and network semantics. A complete projection context may therefore require:

$$(x, \mathcal{T}, \mathcal{R}, \mathcal{O}),$$

where x denotes world-space input, $\mathcal{T}$ topology context, $\mathcal{R}$ route/reference context, and $\mathcal{O}$ operational context.

32.8. Relation to Curvature

Projection stability depends directly on curvature behaviour. High curvature increases projection sensitivity, whereas low curvature typically yields stable projection behaviour. Errors in intrinsic coordinates propagate nonlinearly into world space, and behaviour near transition regions may depend strongly on higher-order curvature.

32.9. Runtime Dependency

Within AIM, world–track transformation depends on runtime evaluation of the underlying alignment representation. In particular, cGeom defines intrinsic geometry, pose2 defines tangent evolution, pose3 defines runtime spatial interpretation, metric services define realized embedding, CRS services define external representation, and runtime operators evaluate resulting states. Projection therefore depends on the currently active runtime evaluation pipeline. Different runtime configurations may yield different projection results, for example projected versus ellipsoidal evaluation, target versus realized geometry, low-resolution versus high-resolution runtime, or simplified versus vehicle-aware interpretation. *Principle 32.8* (Runtime projection principle). World–track transformation is a runtime evaluation problem operating on alignment states rather than a static coordinate utility.

32.10. LOD and Hybrid Evaluation

Exact world-to-track projection is computationally expensive. Large-scale systems therefore require LOD-dependent evaluation, approximate projection, topology-aware filtering, and hybrid runtime architectures.

Proposition 32.9 (Selective projection principle). *Exact projection should only be evaluated where high precision is required.*

A hybrid strategy combines precomputed samples for coarse lookup, topology-aware candidate filtering, cached runtime states, and local parametric refinement for precision. Such

architectures are particularly important for large railway networks, interactive visualization, realtime vehicle-space evaluation, and multi-resolution runtime systems.

32.11. Connection to CRS and Realization

The complete external transformation chain is:

$$\text{CRS} \rightarrow \Omega_{\text{world}} \rightarrow \Omega_{\text{track}}.$$

For high-precision applications, the alignment itself may be defined on a reference ellipsoid rather than in a projected plane. Projection may also depend on realized geometry:

$$\gamma_{\text{real}}(s) \neq \gamma_{\text{target}}(s).$$

Physical measurements should therefore reference realized rather than purely analytical geometry. The distinction between

intrinsic geometry and external embedding

is fundamental within AIM.

32.12. Key Insight

Proposition 32.10. *World–track transformation is an inverse runtime service rather than a static coordinate conversion.*

Proposition 32.11. *Within AIM, intrinsic alignment space is primary, whereas world coordinates are interpreted as external embeddings of alignment states.*

Proposition 32.12. *Projection is generally:*

- *nonlinear,*
- *ambiguous,*
- *topology-dependent,*
- *runtime-dependent,*
- *and context-sensitive.*

32.13. Connection to AIM

Summary 32.13. World–track transformation connects intrinsic railway geometry with external coordinate systems, topology, runtime evaluation, and measured reality.

32. *World–Track Coordinate Transformation*

Its nonlinear, ambiguous, topology-dependent, and scale-dependent nature makes it a central runtime component of the AIM framework.

Within AIM, projection is therefore interpreted not as static coordinate conversion, but as contextual runtime reconstruction of intrinsic railway states from external observations. Efficient handling consequently requires:

- hybrid runtime architectures,
- topology-aware projection strategies,
- LOD-dependent evaluation,
- sparse-state reconstruction,
- and metric-aware runtime services.

Part VI.

Dynamic Railway State

Dynamics

The identified straight–transition–arc Alignment does not stop at the Part V boundary. At each intrinsic position s , its curvature law, realized pose, profile, cant law, and qualified moving frame already determine the railway state needed here. Part VI therefore introduces no second state model. It asks one new question:

How does traversal through this spatial state generate time-dependent quantities, and which additional models are required before an engineering consequence can be evaluated?

The new input is a traversal law

$$s = s(t), \quad v(t) = \frac{ds}{dt}.$$

Composition turns spatial laws into temporal exposure: $\kappa(s) \mapsto \kappa(s(t))$ and $u(s) \mapsto u(s(t))$. Neither law changes identity. What changes is the rate at which a vehicle or other moving system encounters them.

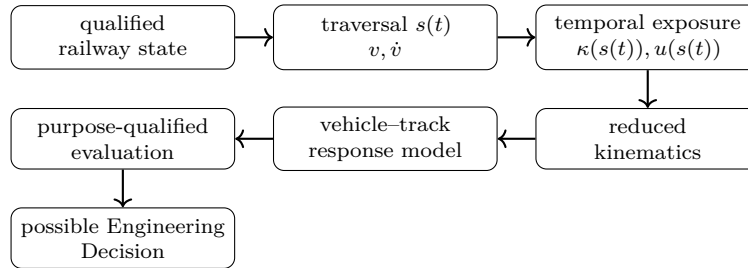


Figure 32.3.: From qualified railway state to temporal exposure, response modelling, evaluation, and possible decision.

Every arrow in fig. 32.3 adds information: the traversal law adds time; its derivatives add velocity and acceleration; a reduced relation adds stated kinematic assumptions; a response model adds vehicle and track physics; evaluation adds purpose and applicable rules; only an Engineering Decision adds authorized commitment.

Principle 32.14 (Dynamic-State Interpretation Principle). Time-dependent quantities are derived by composing the qualified railway state with a qualified traversal law and, where required, explicit response models. Reduced kinematics, response, evaluation, and decision remain distinct.

Summary 32.15. Geometry supplies the spatial law. Traversal supplies time. Models supply response. Purpose and rules supply evaluation. Authority supplies decision.

Layer	Required input	Result and boundary
Spatial law	identified Alignment, $s, \kappa(s), u(s)$	intrinsic geometry and separate cant law; no time
Traversal law	$s(t)$, source, applicability, validity	$v(t), \dot{v}(t)$, temporal exposure; no vehicle response
Reduced kinematics	frame, signs, cant and speed assumptions	indicators such as $v^2\kappa$; not comfort or safety
Response model	vehicle, contact, track and loading data	model-dependent response; not observation or decision
Evaluation	purpose, rule, scope, evidence	qualified assessment; no authority by itself
Decision	responsible authority and admitted evidence	accepted or rejected course of action

Table 32.1.: The dynamic argument adds one responsibility at a time.

33. Where the State Model Ends and Dynamics Begins

Part V already established pose2, pose3, moving frames, cant, Metric Realization, target and physical state, and observation. This chapter does not define them again. It fixes the narrower responsibilities of Part VI so that later chapters consume one railway-state model.

33.1. Inherited Railway State

The input is the qualified Part V state at s , including its Alignment reference, realization, frame convention, provenance, applicability, and validity. “Track state” in the following chapters is a short reader-facing name for the applicable subset of that structure, not a competing state type. Likewise, a represented coordinate tuple is not a replacement for pose3 and an observed pose is not a physical state.

33.2. New Responsibilities

Operational velocity qualifies the traversal law $s(t)$ and its derivatives.

Temporal exposure composes spatial curvature, cant, profile, and frame quantities with that traversal.

Reduced kinematics evaluates assumption-bound indicators without claiming complete response.

Vehicle space expresses vehicle-related quantities relative to the qualified track frame and a stated vehicle model.

Response modelling adds the physical parameters needed for a predicted vehicle–track response.

Dynamic balance applies a declared reduced relation under its stated frame, sign, speed, and cant conventions.

Realization-aware dynamics compares qualified target, realized, or observed inputs without merging them.

Optimization varies declared degrees of freedom against explicit objectives and constraints; it neither supplies objectives nor decides.

33.3. One State, Several Uses

The same qualified railway state may feed a clearance calculation, a reduced lateral indicator, a multibody model, or an optimization experiment. These uses produce different outputs because they add different models and purposes. They do not create alternative definitions of the Alignment or railway state.

Principle 33.1 (Single-definition principle). Pose2, pose3, the moving frame, cant, realization, physical state, and observation retain their established meanings. Part VI adds traversal, response models, and purpose-qualified uses by reference.

Summary 33.2. Part VI begins at $s(t)$. Everything spatial is inherited; everything temporal or response-related must identify the additional input that creates it.

34. State Terms as Qualified Uses

The Thesis uses the word *state* for several outputs and viewpoints. This chapter is not a second ontology or canonical hierarchy. It is a reader map: each term below inherits the object, time, realization, provenance, applicability, and purpose qualifications established in Parts III–V.

Principle 34.1 (State-layer principle). A state term is meaningful only with the engineering question, subject, context, and validity it answers. Shared use of the word “state” does not make different outputs interchangeable.

34.1. Terms Already Established

State. No new general definition is introduced here. In this part, state means a qualified answer about an identified subject under stated conditions.

pose2. Use the planar state from Definition 27.1: it is reconstructed from the Alignment’s ordered curvature law and a starting pose under a realization convention. It is neither Alignment identity nor a persistent canonical Kernel.

pose3. Use the qualified spatial railway pose from Definition 28.6. It adds profile, cant, metric realization and frame interpretation without becoming the complete physical asset.

Runtime state. This phrase indicates that an applicable result is evaluated when needed. It does not identify one universal state type or erase the result-producing operator.

Realized state. Use only for a state qualified by a stated realization. A design realization, constructed physical state, and observed-state estimate remain distinct.

34.2. Terms Introduced by a Use or Model

Operational state. An output used for an operational question; the name does not itself establish comfort, safety, admissibility, or authority.

Vehicle state. An output of an identified vehicle model relative to the qualified railway state and traversal, as explained in Chapter 36.

Centre-of-gravity state. A model output for a selected mass reference point. It is not the complete vehicle state and requires the relevant mass and vehicle model.

Dynamic state. The model-specific result trajectory x_M introduced in Definition 38.2; it is not a synonym for pose3 or physical state.

Optimization state. A variable or output deliberately selected for an optimization formulation. Its selection is an explicit engineering choice, and an optimum is not a decision.

Relative state. A result expressed relative to a stated reference state or frame. Both reference and convention remain part of its meaning.

34.3. Dependency Instead of Hierarchy

There is no universal chain $\kappa \rightarrow \text{pose2} \rightarrow \text{pose3} \rightarrow x_{\text{vehicle}} \rightarrow x_{\text{opt}}$. Pose2 and pose3 follow the spatial dependencies of Part V. Traversal adds time. A selected response model may add vehicle outputs. An optimization formulation may then select some qualified quantity as a variable, objective, or constraint. Other uses stop earlier. Dependency is use-specific, not a canonical state ladder.

Summary 34.2. This map prevents homonyms from becoming identity claims. It introduces no new state model: every downstream term names a qualified use or model output derived from the one railway-state account already established.

35. Runtime Operators

Runtime operators define how established engineering objects are interpreted during execution. They consume upstream concepts from Foundations and State and map between interpretation layers. They do not redefine alignment identity, pose2/pose3, metric realization, physical realization, or representation.

Principle 35.1 (Operator-layer principle). Within AIM, runtime interpretation is organized through explicit operators connecting distinct state layers.

35.1. Operator Overview

The following symbols denote families of qualified operations in later chapters:

$$\mathcal{R}, \quad \mathcal{D}, \quad \mathcal{O}, \quad \mathcal{V}.$$

They do not form one mandatory state ladder. A use may compare a target with a qualified realization, compose the railway state with $s(t)$, evaluate a selected model, express a vehicle-relative result, or apply a purpose-specific evaluation. Each operation identifies its own inputs, applicability and provenance.

35.2. Realization Operator

Definition 35.2 (Realization operator). The realization operator maps target states to realized states:

$$\mathcal{R} : x_{\text{target}} \mapsto x_{\text{real}}.$$

For pose3 states,

$$\mathcal{R}_{\text{pose3}} : \text{pose3}_{\text{target}} \mapsto \text{pose3}_{\text{real}}.$$

$\mathcal{R}$ is explanatory shorthand for a stated realization relation; it does not make construction, physical state, maintenance history and observation one process. A measured state remains an observation with its own provenance and uncertainty.

35.3. Dynamics Operator

Definition 35.3 (Dynamics operator). For an identified response model M , the dynamics operator maps a qualified railway state, traversal law and model inputs to a model-specific

result:

$$\mathcal{D}_M : (\mathcal{S}_{\text{rail}}(s(t)), s(t), Q_M) \mapsto x_M(t).$$

Possible outputs include reduced indicators or modelled responses. Their names do not establish comfort, safety or admissibility.

A realization-aware use supplies the selected qualified realization as one input to $\mathcal{D}_M$; it does not change the operator's model and traversal requirements.

35.4. Vehicle Interpretation Operator

Definition 35.4 (Vehicle interpretation operator). For an identified vehicle model M , the vehicle interpretation operator maps the qualified railway state, traversal and vehicle inputs to vehicle-relative results:

$$\mathcal{V}_M : (\mathcal{S}_{\text{rail}}(s(t)), s(t), Q_M) \mapsto x_{\text{vehicle},M}(t).$$

The qualified input set keeps vehicle type, loading, model fidelity, provenance and applicability distinct rather than hiding them in one generic context.

35.5. Operational Evaluation Operator

Definition 35.5 (Operational operator). The operational operator maps runtime dynamic states to operational evaluation states:

$$\mathcal{O} : x_{\text{op}} \mapsto x_{\text{eval}}.$$

$\mathcal{O}$ evaluates admissibility, standards compliance, comfort indicators, and optimization objectives without redefining the upstream state objects it consumes.

35.6. Operator Composition

Operators may be composed only when output and input types match and realization, traversal, model, purpose, provenance and applicability remain attached. There is no universal composition that produces vehicle response, evaluation or decision from pose3 alone.

35.7. Runtime Abstraction and Implementation

Principle 35.6 (Runtime abstraction principle). Runtime operators define interpretation roles first; detailed mechanics may be added later as specialized implementations.

Definition 35.7 (Operator role). An operator role specifies the conceptual transformation performed by an operator, independent of a particular numerical implementation.

This distinction keeps AIM structurally clear and implementation flexible while preserving compatibility with higher-fidelity models.

35.8. Canonical Interpretation

Proposition 35.8. *Within AIM, runtime interpretation is operator-based.*

Proposition 35.9. *The realization operator $\mathcal{R}$ maps target states to realized states.*

Proposition 35.10. *The dynamics operator $\mathcal{D}$ maps railway runtime states to operational dynamic state trajectories.*

Proposition 35.11. *The vehicle operator $\mathcal{V}$ maps railway runtime states to vehicle-relative interpretation states.*

Proposition 35.12. *The operational operator $\mathcal{O}$ maps operational states to evaluation and admissibility states.*

35.9. Connection to AIM

Summary 35.13. Runtime operators provide the structural connection between established engineering knowledge and runtime interpretation.

Within this framework, $\mathcal{R}$ represents realization, $\mathcal{D}$ runtime dynamic state generation, $\mathcal{V}$ vehicle-relative interpretation, and $\mathcal{O}$ operational evaluation. Their composition yields derived runtime engineering states without redefining upstream concepts.

36. Vehicle Space Relative to the Railway State

Vehicle Space does not introduce another railway state. It asks how vehicle-related quantities are expressed relative to the qualified pose3 frame inherited from Part V and the traversal $s(t)$ introduced here.

Principle 36.1 (Track–Vehicle Distinction). The qualified railway state may exist without a vehicle model. A vehicle state requires that reference plus an identified vehicle model and operating inputs.

Proposition 36.2 (State hierarchy).

$$\text{vehicleState}_M(t) = \mathcal{V}_M(\mathcal{S}_{\text{rail}}(s(t)), Q_M),$$

but the railway state is not derived from vehicleState_M . The model index and inputs remain part of the result's meaning.

36.1. Relative Quantities

The moving track frame supplies longitudinal, lateral, and vertical reference directions. A vehicle model may then express body or subsystem displacement, yaw, roll, acceleration, wheel load, or envelope position relative to that frame. These are examples of possible model outputs, not universally present fields and not validated merely because they are expressed in Vehicle Space.

$$F_{\text{vehicle}}(t) \neq F_{\text{rail}}(s(t)).$$

Their relative orientation is meaningful only after both frames, sign conventions, time relation, and relevant vehicle degrees of freedom are specified.

36.2. Subsystem and Reference-Point Uses

A wheelset, bogie, vehicle body, or selected centre-of-gravity point may require a different state vector and model. In particular, a centre-of-gravity trajectory is not the whole vehicle state. It may become an explicit optimization output only if the vehicle model, objective, constraints and applicability are declared.

36. Vehicle Space Relative to the Railway State

Summary 36.3. Vehicle Space is a relative interpretation domain downstream of the qualified railway state and traversal. It does not redefine `pose3`, the track frame, or Alignment identity.

37. Traversal Law and Operational Velocity

The two-speed comparison in Part V assumed constant test speeds to isolate the v^2 dependency. Operation is more general. A traversal of the running Alignment is a monotone function $s = s(t)$ over the interval where the chosen orientation and applicability conditions hold.

Principle 37.1 (Velocity-runtime principle). Velocity is a qualified derivative of a traversal law, not an intrinsic Alignment property.

37.1. Space Becomes Time

Definition 37.2 (Velocity relation). For a differentiable traversal,

$$v(t) = \dot{s}(t) = \frac{ds}{dt}, \quad a_s(t) = \dot{v}(t) = \frac{d^2s}{dt^2}.$$

For a differentiable spatial quantity $f(s)$,

$$\frac{d}{dt}f(s(t)) = v(t)f'(s(t)).$$

Applied to the separate curvature and cant laws,

$$\frac{d\kappa}{dt} = v\kappa'(s), \quad \frac{du}{dt} = vu'(s).$$

Their different partitions and provenance remain attached. Common traversal does not turn them into one law or authorize a coupling rule.

For curvature acceleration, the full chain rule is

$$\frac{d^2\kappa}{dt^2} = v^2\kappa''(s) + \dot{v}\kappa'(s). \quad (37.1)$$

The second term vanishes only under a stated constant-speed assumption. Spatial smoothness belongs to $\kappa(s)$; temporal excitation also depends on $s(t)$.

37.2. Which Velocity Is Meant?

Definition 37.3 (Design velocity). A design velocity is an assumption selected for a stated design purpose, source, route or element scope, and validity.

37. Traversal Law and Operational Velocity

Definition 37.4 (Operational velocity). An operational velocity is a planned, permitted, or actual traversal input whose role and qualifications are stated for the calculation.

The following roles must not be collapsed:

Design speed an assumption used to develop or check a design.

Permitted speed a rule- or authority-backed upper bound under its applicability and validity.

Planned operating speed a timetable or operating-plan input.

Actual measured speed a time-indexed observation with acquisition provenance and uncertainty.

Optimization variable a declared degree of freedom within an experiment, not an authorized operating value.

Constant test speed an explicit simplification used to isolate a dependency.

Definition 37.5 (Velocity envelope). A velocity envelope is a qualified set of permitted or investigated traversal values along an identified route or Alignment. Its source, scope, validity, and role determine how it may be used.

37.3. Compact Variable-Speed Example

At the same transition position, suppose

$$\kappa' = 2.0 \times 10^{-5} \text{ m}^{-2}, \quad \kappa'' = -1.0 \times 10^{-7} \text{ m}^{-3}.$$

At $v = 20 \text{ m/s}$,

$$\dot{\kappa} = 4.0 \times 10^{-4} \text{ m}^{-1} \text{ s}^{-1}.$$

If speed is constant, then $\ddot{\kappa} = v^2 \kappa'' = -4.0 \times 10^{-5} \text{ m}^{-1} \text{ s}^{-2}$. If instead $\dot{v} = 0.5 \text{ m/s}^2$, eq. (37.1) gives

$$\ddot{\kappa} = -4.0 \times 10^{-5} + 1.0 \times 10^{-5} = -3.0 \times 10^{-5} \text{ m}^{-1} \text{ s}^{-2}.$$

The Alignment law is identical in both cases. Acceleration changes temporal exposure by adding a term that a constant-speed simplification omits.

Summary 37.6. Every use of velocity must disclose source, applicability, validity, and calculation role. The symbol v is not sufficient qualification by itself.

38. From Temporal Exposure to Modelled Response

The traversal law now exposes the moving system to $\kappa(s(t))$, $u(s(t))$, the profile, and the qualified track frame. This chapter marks the boundary between quantities that follow from that kinematics and quantities that require a vehicle–track response model.

Principle 38.1 (Runtime-dynamics principle). Runtime dynamics consume a qualified railway state and traversal law. Any claimed response must additionally identify the model that produces it.

38.1. A Result, Not a Second State Definition

Definition 38.2 (Dynamic runtime state). For the narrow purpose of this part, a dynamic runtime state is the output of a declared response model evaluated for a qualified railway state, traversal, and model input set.

This definition classifies an output; it does not redefine pose3, physical state, or observation. A predicted body acceleration, an observed wheel load, and a reduced kinematic indicator belong to different result kinds even when they refer to the same $s(t)$.

Definition 38.3 (Runtime dynamics operator). Let

$$\mathcal{D}_M : (\mathcal{S}_{\text{rail}}(s(t)), s(t), Q_M) \mapsto x_M(t)$$

denote evaluation by an explicitly identified response model M , with qualified model inputs Q_M .

The model index matters. There is no universal $\mathcal{D}$ that silently turns pose3 into validated vehicle behaviour.

Definition 38.4 (Runtime state evolution). Runtime state evolution is the time- or space-indexed output trajectory x_M produced by the selected model over the applicable traversal interval.

38.2. Reduced Kinematics

Under the Part V right-handed track frame, its stated cant sign, rigid point-following motion, and neglect of suspension, contact, track flexibility, irregularities, wind, and other

effects,

$$a_{\kappa}(t) = v(t)^2 \kappa(s(t))$$

is a lateral kinematic component. With the stated cant-angle convention, the reduced effective component

$$a_{\text{eff}}(t) = v(t)^2 \kappa(s(t)) - g \sin \phi(s(t))$$

is an explanatory indicator. If speed varies, its time derivative is

$$\dot{a}_{\text{eff}} = 2v\dot{v}\kappa + v^3\kappa' - gv \cos \phi \phi', \quad (38.1)$$

with all spatial quantities evaluated at $s(t)$. The term $2v\dot{v}\kappa$ is absent from the constant-speed relation used in Part V. This indicator is not by itself comfort, safety, wear, wheel unloading, or vehicle response.

38.3. The Explicit Model Boundary

Kinematics end here. The railway state and traversal can supply trajectory, frame, curvature and cant exposure, and reduced acceleration or rate indicators under stated assumptions.

Predicted response begins only after a model adds vehicle mass and inertia, suspension, wheel–rail contact, track stiffness and damping, irregularities, operational loading, environmental state, and relevant maintenance condition. Model applicability, calibration, and uncertainty remain attached to every output.

Not every use requires every listed input, but omission must follow the stated scope of the selected model rather than disappear silently. A measured response remains an observation and does not become the model output or physical state by naming alone.

Principle 38.5 (Mechanics separation principle). Reduced railway kinematics and a complete vehicle–track mechanics simulation are different operations with different inputs, applicability, and evidence.

38.4. The Running Comparison

The two Part V cases use identical κ and cant but different constant test speeds, so their reduced $v^2\kappa$ terms differ. The variable-speed example adds $\dot{v}$ -dependent exposure. A different $u(s)$ would rotate the track frame and change the reduced effective component without changing the identity of $\kappa(s)$. Yet identical reduced indicators would still not guarantee identical response: vehicles, track condition, loading, and environment may differ.

38.5. AXTRAN and Declared Objectives

The Alignment calculation core associated with axtranNew/AXTRAN may propose geometry or free parameters. A declared reduced quantity may later be used as an objective or constraint, provided its assumptions, purpose, and applicability are explicit. The objective is an engineering choice supplied to the calculation, not discovered as authority by the solver. A calculated optimum is neither automatically applicable nor accepted.

The present implementation does not establish general joint optimization of geometry, cant, traversal, and vehicle response. The new continuity solver is therefore not described as a dynamics optimizer.

Summary 38.6. Temporal exposure follows from $s(t)$. Reduced kinematics follow from stated frame and motion assumptions. Vehicle–track response begins only with an identified model. Evaluation and decision remain downstream.

39. What *Schwerpunkt* Can Mean

A model reference, not a new canonical state

The railway state and traversal $s(t)$ expose a moving vehicle to geometry. They do not select a vehicle model or a point whose motion represents that vehicle. The German word *Schwerpunkt* is therefore retained only where its mathematical role is stated.

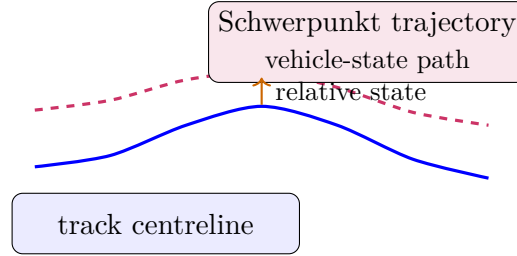


Figure 39.1.: A model-selected reference-point trajectory relative to the railway state; it is not Alignment geometry.

Principle 39.1 (Qualified Schwerpunkt use). An occurrence of *Schwerpunkt* shall identify its owning subject, model, reference frame and evidential status. It is not a canonical Kernel state.

Principle 39.2 (Schwerpunkt is not geometry). Changing or evaluating a model reference-point trajectory does not change Alignment identity or replace its intrinsic construction.

39.1. Center of mass and reduced reference point

The physical centre of mass is a property of a particular loaded vehicle. A reduced vehicle model M , however, may use a reference point P_M that coincides with an assumed centre of mass only under stated loading and rigid-body assumptions. A representative trajectory point, an optimization aggregate and a conceptual “focus” are different objects and shall not be denoted silently by the same symbol.

Definition 39.3 (Model-qualified reference-point state). For vehicle subject V , response model M , frame F , qualified railway state S_{rail} , traversal $s(t)$, parameters Q_M and initial conditions I_M , the predicted reference-point state is

$$x_{P_M}^{\text{pred}}(t) = \mathcal{D}_M(S_{\text{rail}}(s(t)), Q_M, I_M)_{P_M, F}.$$

It is a model output with the provenance and validity of those inputs. It is neither an observation nor proof of the physical centre-of-mass motion.

Definition 39.4 (Center-of-gravity trajectory). The notation $x_{CG}(t)$ may be used only when P_M has explicitly been identified with the modelled centre of mass. Otherwise $x_{P_M}(t)$ is used.

39.2. What may be derived

Definition 39.5 (Operational balance). Operational balance in this chapter means a declared model relation evaluated in a declared frame. It is not a generic vehicle condition.

Definition 39.6 (Balance trajectory). A balance trajectory is the time- or station-dependent result of that relation, with its model, frame, signs, velocity and cant assumptions attached.

Definition 39.7 (Effective operational acceleration). In the reduced track-frame test used in chapter 40, $a_{\text{eff}} = v^2\kappa - g \sin \phi$ is a residual indicator. It is not the acceleration of P_M unless a response model establishes that mapping.

Definition 39.8 (Roll evolution). Roll evolution is a response-model output requiring inertia, suspension, loading, initial conditions and a frame convention; cant alone does not determine it.

Definition 39.9 (Operational comfort). Comfort is a purpose- and rule-qualified evaluation of applicable evidence. Neither a reference-point trajectory nor a reduced balance residual is by itself a comfort statement.

Definition 39.10 (Operational trajectory shaping). Trajectory shaping is an optimization description only after variables, constraints, response model and objective have been declared. Its result is a candidate.

39.3. Schwerpunkt interpretation and realization

A target Alignment, its Metric Realization, a constructed asset, an observed-state estimate and a predicted reference-point response remain separate. A model may consume a qualified realization or an observed-state estimate, but the output inherits its uncertainty and applicability. The phrase *Schwerpunkt-Trassierung* is consequently historical or illustrative shorthand for model-qualified reference-point trajectory investigation; it names neither a Kernel concept nor a current AXTRAN capability.

39.4. Claim status

Expression	Status	Defensible use
physical centre of mass	physical/model parameter	particular vehicle and loading
P_M trajectory	model prediction	identified M, F, Q_M, I_M
Schwerpunkt State	non-canonical shorthand	only with the preceding qualification
Schwerpunkt optimization	candidate	declared optimization problem only

The next chapter uses the simplest residual relation deliberately. Its limitations show why a model output cannot become an evaluation or decision merely by receiving a state-like name.

40. Dynamic Balance as a Qualified Residual

One calculation, six different roles

Consider the reduced point-following test M_{red} in a local track frame F_{track} . Positive curvature and positive lateral action use the same declared side; positive cant angle ϕ compensates that action. The test assumes rigid traversal, prescribed $v(t)$, small local extent and a uniform gravity magnitude g . It omits vehicle mass and inertia, suspension, wheel–rail contact, track flexibility, irregularities, aerodynamics and control.

Principle 40.1 (Dynamic-balance principle). Dynamic balance is a model-, frame-, sign- and purpose-qualified relation.

40.1. Balance relation

Definition 40.2 (Balance relation). For M_{red} , the balance residual is

$$r_{\text{bal}}(t) = v(t)^2 \kappa(s(t)) - g \sin \phi(s(t)).$$

The reduced balance condition is $r_{\text{bal}} = 0$.

zero reduced balance residual $\nRightarrow$ zero vehicle-body acceleration

Equality cancels two terms in this one frame and model. It does not establish multibody equilibrium, predicted vehicle response, measured response, passenger perception, safety or comfort.

Definition 40.3 (Effective lateral acceleration). The historical notation a_{eff} denotes r_{bal} in this reduced test. Calling it an acceleration does not expand its evidential meaning.

40.2. Cant deficiency, excess and trajectory

Definition 40.4 (Cant deficiency). Within the declared signs and M_{red} , $r_{\text{bal}} > 0$ is reduced cant deficiency. A standard-specific quantity still requires its own definition and applicability.

Definition 40.5 (Excess cant). Within the same qualifications, $r_{\text{bal}} < 0$ is reduced excess cant.

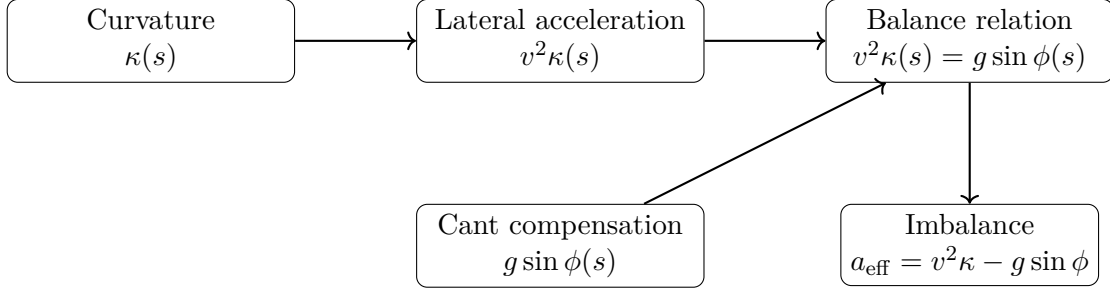


Figure 40.1.: The reduced track-frame residual; omitted response physics remains outside the relation.

Definition 40.6 (Balance trajectory). The balance trajectory is $r_{\text{bal}}(t)$, or $r_{\text{bal}}(s)$ only where velocity has been supplied as a function of station.

For variable speed,

$$\dot{r}_{\text{bal}} = 2v\dot{v}\kappa + v^3\kappa' - gv \cos \phi \phi',$$

under the same assumptions. The derivative adds exposure rate; it still adds no vehicle response.

40.3. Do not collapse the roles

Role	Produces	Does not establish
residual	r_{bal} from M_{red}	body response
prediction	$x_M^{\text{pred}}(t)$ from identified M	observation
observation	source- and uncertainty-qualified estimate	physical truth or cause
comparison	difference under matching identity, time and frame	acceptability
evaluation	criterion result for a purpose	selection or authority
Engineering Decision	documented selection or rejection	universal validity

$$\boxed{\text{predicted response} \not\Rightarrow \text{acceptable engineering state}}$$

The CAD curve and survey polyline make the separation concrete. Either may be realized into a comparable track-frame curvature estimate, but a deviation between them is not automatically the cause of a measured body acceleration. That causal claim requires an identified response model, matched observations and qualified evidence. The following Vienna example is read with exactly this restraint.

41. Vienna Functions as a Bounded Example

Evidence before interpretation

The name *Vienna* occurs in historical project material and in transition-function implementations. That evidence supports examining particular normalized function forms. It does not establish one authoritative curvature–cant coupling, exhaustive transition-family coverage, validated modern vehicle response or general joint optimization.

Principle 41.1 (Vienna reinterpretation principle). In this Thesis, a Vienna-related function is an identified historical or implemented example whose inputs, output law and limits must be stated.

Principle 41.2 (No special-curve principle). No Vienna-related function is elevated here to a canonical AIM construction.

41.1. Separate laws

Let $w = s/L \in [0, 1]$, and let $f_\kappa(w)$ and $f_u(w)$ be two declared normalized functions. An illustrative specialization is

$$\kappa(s) = \kappa_0 + \Delta\kappa f_\kappa(s/L), \quad u(s) = u_0 + \Delta u f_u(s/L).$$

This form is mathematically established once the two functions and domains are supplied. It does not say that $f_\kappa = f_u$, that their parameters are coupled, or that either function is optimal.

Definition 41.3 (Vienna-type state shaping). “Vienna-type state shaping” is retained only as historical project intent: using an identified transition function to investigate a downstream model output. It is not a Kernel definition.

Definition 41.4 (Half-wave interpretation). A half-wave is a compositional implementation or modelling form when the identified function actually uses it. It is not asserted as the grammar of all transition families.

Definition 41.5 (Dynamic state shaping). For an identified model M , changing a declared law and comparing the resulting x_M^{pred} is an illustrative sensitivity study. Calling it shaping does not supply an objective or validation.

41.2. Compact alternative-objective example

For the same boundary values and length L , consider two feasible pairs (f_κ, f_u) . Candidate A minimizes

$$J_\kappa = \int_0^L (\kappa'(s))^2 ds,$$

whereas candidate B minimizes

$$J_r = \int_0^{L/v} r_{\text{bal}}(t)^2 dt$$

for the explicitly assumed constant test speed v and M_{red} . The objectives have different units and meanings. Neither candidate dominates without a supplied purpose, scaling or explicit aggregation. Neither objective is a comfort, wear, cost or energy objective.

41.3. Claim-status audit

Claim	Status	Disposition
implemented function form	implementation-backed	cite exact implementation when used
historical dynamic intent	historical project intent	retain as provenance, not validation
separate curvature/cant laws	illustrative	state functions and coupling assumption
universal Vienna coupling	unsupported	removed
validated vehicle response	unsupported	requires identified model and evidence
general joint optimization	unsupported	open work, not present capability

The same restraint applies to realization. A smooth target function may be compared with qualified realization or observation evidence, but a coordinate transformation is not construction and a measured deviation is not a dynamic cause. The next chapter makes that comparison contract explicit.

42. Realization-Aware Prediction and Comparison

Which geometry enters the model?

The intended Alignment geometry, its Metric Realization, the constructed asset, physical state and observed-state estimate are related but not interchangeable. A coordinate transformation changes representation; it does not construct an asset. A survey polyline is an observation-bearing representation; it is not physical truth.

Principle 42.1 (Realization-aware dynamics principle). A prediction shall identify which qualified realization or observed-state estimate enters response model M . The result remains a prediction of M , not an observation.

42.1. Typed comparison chain

Definition 42.2 (Realized runtime state). In this chapter, a realized runtime state is a response-model input derived from an explicitly identified realization or observed-state estimate, qualified by object identity, time, frame, method, provenance, applicability and uncertainty. It is not a new canonical state class.

For source $q \in \{\text{intent}, \text{realization}, \text{observation}\}$,

$$\hat{S}_q \xrightarrow[Q_M, I_M]{\mathcal{D}_M} x_{M,q}^{\text{pred}}(t).$$

Only like-for-like outputs may be compared. A measured response $x^{\text{obs}}(t)$ remains separately qualified:

$$\Delta_M(t) = \mathcal{C}\left(x_{M,q}^{\text{pred}}(t), x^{\text{obs}}(t) \mid \text{matched subject, time, frame and quantity}\right).$$

Δ_M is a comparison result, not a cause or decision.

42.2. Realization filtering

Proposition 42.3 (Realization-filter principle). *Changing the qualified model input can change a prediction while preserving Alignment identity. The change must not be attributed to construction, degradation or measurement until evidence distinguishes those explanations.*

For the CAD-curve/survey-polyline discrepancy, the CAD realization and the polyline-derived estimate may each feed the same M . Their output difference is a sensitivity result. Causal attribution requires further evidence.

42.3. Calibration and uncertainty boundary

A model parameter belongs to Q_M . Calibration evidence supports selecting or estimating such a parameter for a stated domain. Observation uncertainty qualifies measured evidence; construction variability concerns the asset; operational variability concerns use; a numerical residual concerns a calculation; model-form uncertainty concerns the adequacy of M . These sources shall not be collapsed into one error term merely because all affect a comparison.

Matching one observation can support a local calibration statement. It does not establish universal model validity.

Definition 42.4 (Realization-aware admissibility). Realization-aware admissibility is an evaluation result obtained by applying an identified rule and purpose to qualified comparison evidence. It is not an intrinsic property produced by $\mathcal{D}_M$.

Definition 42.5 (Runtime envelope). A runtime envelope is a declared set of limits from an identified source, with units, applicability and validity. It is not supplied by the state model itself.

This yields inputs for optimization, but it does not choose an objective. That responsibility is made explicit next.

43. Optimization Produces Candidates

An objective must have an owner

The preceding chapters can produce qualified model outputs and comparisons. They do not make one quantity self-evidently desirable. Short transition length, reduced balance residual, comfort, wear, cost and energy are distinct objectives unless a responsible source explicitly aggregates them.

Principle 43.1 (Qualified optimization principle). Optimization begins only after problem identity, purpose, decision variables, fixed inputs, feasible set, model and objective have been declared.

Principle 43.2 (Candidate principle). A solver returns a calculation candidate under that declaration, not an Engineering Decision.

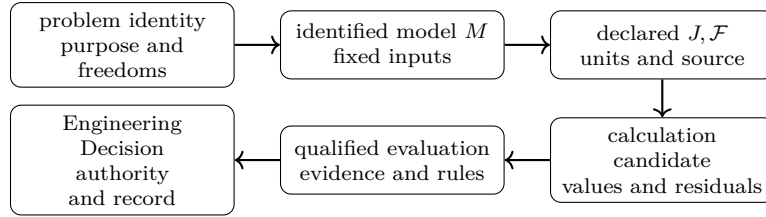


Figure 43.1.: Dependency from an identified problem and model to a non-authoritative candidate, evaluation and possible Engineering Decision.

43.1. Problem contract

Definition 43.3 (Runtime state trajectory). A runtime state trajectory is an identified model output $x_M(t)$ used by a declared optimization problem. It is not a universal AIM state.

Definition 43.4 (Runtime state evolution). Its evolution is generated by $\mathcal{D}_M$ from fixed and variable inputs under stated initial conditions and applicability.

Only then is the familiar form meaningful:

$$\min_{x \in \mathcal{F}} J(x).$$

43. Optimization Produces Candidates

Here x denotes the declared decision vector, not an unspecified state. The problem record shall identify:

$$\mathcal{P} = (id, purpose, x, b, f, d, M, \mathcal{F}, J, \\ units, scaling, source, applicability, provenance),$$

where b are bounds, f fixed inputs and d derived quantities. Unresolved freedoms remain explicit rather than being filled silently.

43.2. Degrees of freedom

For a bounded continuity edit, a defensible example is

$$x = (L, \alpha), \quad \mathcal{F} = \{x \mid c_{\text{continuity}}(x) = 0, L_{\min} \leq L \leq L_{\max}, \alpha_{\min} \leq \alpha \leq \alpha_{\max}\}.$$

Known endpoint states and the selected transition family are fixed; L, α are free within bounds; coefficients and terminal pose are derived; continuity residuals are constrained. Cant, vehicle parameters and an operational velocity profile remain unresolved unless deliberately added.

The current ufAIM/AXTRAN continuity solver is implementation evidence for such bounded continuity solving. It is not evidence of general vehicle, cant, comfort, network or automatic engineering-selection optimization.

43.3. Objective and scaling

Definition 43.5 (Runtime-state functional). A runtime-state functional is a declared objective computed from an identified model output, with units, scaling, source and purpose recorded.

Definition 43.6 (Operational trajectory shaping). Operational trajectory shaping is a candidate optimization formulation, not a canonical AIM interpretation.

The two objectives J_κ and J_r in chapter 41 illustrate why scalar values cannot be compared across purposes. A weighted aggregation

$$J = w_\kappa J_\kappa / J_{\kappa,0} + w_r J_r / J_{r,0}$$

is meaningful only when the reference scales, weights and their responsible source are recorded. The solver does not own them.

43.4. Candidate contract

A calculation candidate contains:

$$C = (\mathcal{P}.id, M.id, x, b, \mathcal{F}, J, x^*, \\ residuals, convergence, provenance, applicabilityStatus).$$

The applicability status remains unevaluated on solver return, and the candidate is explicitly non-authoritative.

$\text{calculated} \neq \text{applicable} \neq \text{accepted} \neq \text{authoritative}$

Multiple candidates may be mathematically feasible, optimal under different objectives, incomparable without purpose, invalid under later evidence, or stale after an Alignment or realization change. One scalar score supplies no universal engineering ordering.

Definition 43.7 (Sparse runtime optimization). Sparse runtime optimization is only a candidate computational technique that exploits a stated sparse dependency structure; no general performance or capability claim is made here.

Definition 43.8 (Operationally admissible trajectory). An operationally admissible trajectory is an evaluation result under identified rules, purpose, evidence and applicability. Feasibility in $\mathcal{F}$ alone does not establish it.

43.5. Evaluation and Engineering Decision

Evaluation asks whether a candidate and its evidence are applicable to a purpose under identified rules, lifecycle and validity. An Engineering Decision may then document selection or rejection together with authority. The solver, AXTRAN implementation, App and Thesis do not hold that authority.

Thus the complete sequence is

$$S_{\text{rail}} \rightarrow s(t) \rightarrow M \rightarrow x_M^{\text{pred}} \rightarrow \text{comparison} \rightarrow J, \mathcal{F} \rightarrow C \\ \rightarrow \text{evaluation} \rightarrow \text{Engineering Decision}.$$

Geometry, model, objective, candidate and decision remain separately owned.

Part VII.

Runtime and Evaluation

Runtime

This part establishes how approved engineering semantics are evaluated operationally without transferring conceptual ownership.

Foundations and State established the persistent engineering information and the canonical state layers. Runtime consumes these stabilized concepts and evaluates them in operational contexts. It does not redefine alignment semantics, pose2/pose3, metric realization, physical realization, or representation.

The runtime part therefore follows the canonical progression

persistent engineering information → runtime evaluation
→ runtime services
→ derived engineering state
→ representation
→ application-specific
 interpretation

In this progression, runtime services are operator-based evaluation systems. They reconstruct and interpret quantities from established state and realization layers rather than introducing new engineering objects.

Relation to the AIM Roadmap

Within the AIM roadmap, the present part corresponds to the transition from state-space models to operational evaluation and interpretation.

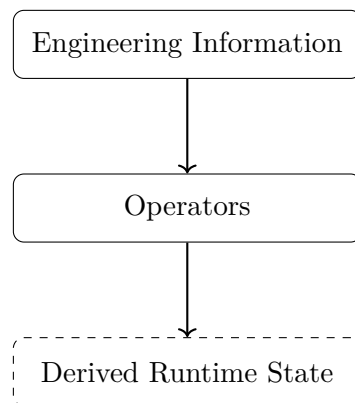


Figure 43.2.: Runtime icon: operator-mediated derivation from engineering information to runtime state.

The runtime chapters now specialize this operator chain for concrete services and projections.

The guiding question for this figure is: which operator chain must be kept explicit so runtime remains reproducible, interpretable, and architecturally consistent?

Principle 43.9 (Runtime Layer Principle). Within AIM, operational quantities are generated through runtime operators acting on established state and realization layers rather than stored redundantly as independent truth.

Summary 43.10. The runtime layer is the operator-based evaluation layer of AIM.

It consumes persistent engineering information, produces derived runtime state, and enables downstream representation and application-specific interpretation without redefining upstream semantics.

This runtime architecture prepares the Reality part, where evaluated state is related to physical realization, measurement, and comparison.

44. Runtime Services

Runtime services are the execution layer that evaluates approved engineering knowledge in operational contexts. They consume established objects from Foundations and State and generate derived runtime quantities. They are not engineering objects themselves and do not redefine upstream semantics.

This chapter answers how runtime evaluation is organized so that projection, frame construction, dynamic quantities, and representation boundaries remain consistent with canonical AIM semantics. It establishes the runtime service architecture and its engineering consequences for downstream interpretation.

44.1. Runtime Interpretation

Definition 44.1 (Runtime service). A runtime service is an operator-based evaluation process acting on alignment states during operational use.

Principle 44.2 (Runtime evaluation principle). Operational quantities should be evaluated from underlying state descriptions rather than stored redundantly.

Runtime services evaluate geometry, projections, frames, realization-dependent states, dynamic quantities, and vehicle-space interpretation through explicit operator composition.

The remaining sections now follow the same reader path: from runtime state evaluation, to concrete services, to pipeline composition, and finally to representation boundaries and architectural consequence.

44.2. Runtime State Evaluation

Runtime evaluation is governed by the canonical evaluation operator in theorem 11.10.

Definition 44.3 (Runtime evaluation). Runtime evaluation is represented by

$$\mathcal{E} : \mathcal{X} \times \Omega_{\text{track}} \rightarrow \mathcal{Y}.$$

Here $\mathcal{X}$ denotes railway state space, Ω_{track} intrinsic track space, and $\mathcal{Y}$ runtime-evaluated quantities.

Typical runtime outputs include world coordinates, tangent and frame quantities, curvature/cant-derived quantities, projection states, and vehicle-space-derived quantities.

44.3. Persistent Information versus Derived Runtime State

Within AIM, persistent data stores sparse, semantically stable engineering information. Runtime services reconstruct derived quantities from that information.

Principle 44.4 (Runtime reconstruction principle). Runtime geometry should be reconstructed from sparse alignment semantics rather than stored redundantly.

This separation avoids synchronization instability, redundant storage, and realization inconsistencies.

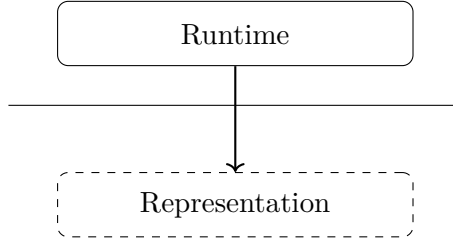


Figure 44.1.: Supporting boundary: runtime evaluation remains upstream of representation.

The figure makes the ownership boundary explicit: runtime produces derived engineering state, while representation consumes that state for encoding or visualization. The engineering consequence is that runtime services must be validated against evaluation correctness, not against format-specific rendering choices.

44.4. Runtime pose3 Evaluation

The foundational pose3 definition is established in theorems 10.3 and 28.6. Runtime services consume these concepts and evaluate them operationally.

Definition 44.5 (Runtime pose3 evaluation). Runtime pose3 evaluation may be represented schematically as

$$\text{pose3}_{\text{run}}(s) = \mathcal{E}_{\text{pose3}}(x, s).$$

The runtime pose3 state is therefore a derived evaluation state, not a persistent engineering identity.

44.5. Projection Services

Runtime projection services consume the canonical operators in theorems 11.6 and 11.7.

The next two figures answer a directional question before the service definitions are stated: which operator maps from world to track, and which provides the complementary mapping from track to world?

Read together, the pair establishes a directional runtime contract before formal service definitions: inverse projection and forward projection are related but non-identical operators.

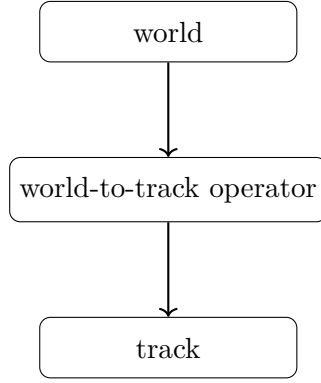


Figure 44.2.: Supporting operator view: world-to-track mapping as a directed runtime operator.

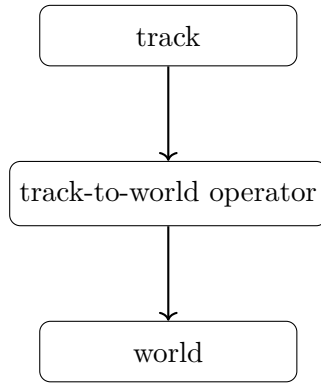


Figure 44.3.: Supporting operator view: track-to-world mapping as the complementary directed runtime operator.

The engineering consequence is explicit separation of numerical strategy, ambiguity handling, and quality criteria for each mapping direction.

Definition 44.6 (Track-to-world runtime service). The track-to-world runtime service evaluates

$$\mathcal{P}_{\text{tw}} : \Omega_{\text{track}} \rightarrow \Omega_{\text{world}}.$$

Definition 44.7 (World-to-track runtime service). The inverse runtime projection service evaluates

$$\mathcal{P}_{\text{wt}} : \Omega_{\text{world}} \rightarrow \Omega_{\text{track}}.$$

The inverse service is nonlinear, potentially ambiguous, metric-dependent, and context-dependent. Projection remains distinct from CRS transformation, as established in theorem 11.8.

This distinction is the practical boundary condition for all later runtime services: projection interprets intrinsic geometry, while CRS transformation only changes world-space representation.

44.6. Frame Services

Runtime frame construction consumes the canonical frame operator in theorem 11.5.

Definition 44.8 (Frame service). The frame service evaluates

$$\mathcal{F}(x_{\text{metric}}, s) = (\mathbf{t}(s), \mathbf{n}(s), \mathbf{b}(s)).$$

Frame services support cant interpretation, pose3 evaluation, vehicle-space interpretation, projection interpretation, and runtime dynamics. Their evaluation may depend on metric realization.

44.7. Derived Dynamic Runtime Quantities

Many operational quantities are derived at runtime and are not persistent alignment knowledge.

Definition 44.9 (Effective lateral acceleration).

$$a_{\text{eff}} = v^2 \kappa - g \sin \phi.$$

This is the runtime form of the canonical relation in eq. (9.10).

Definition 44.10 (Runtime jerk).

$$j = v^3 \kappa' - g v \cos \phi \phi'.$$

This is the runtime form of the canonical jerk relation in eq. (9.12).

44.8. Hybrid and LOD-Dependent Evaluation

Principle 44.11 (Hybrid runtime principle). Efficient runtime systems combine approximate coarse evaluation, cached representations, and exact local refinement.

Definition 44.12 (LOD-dependent evaluation). Runtime evaluation depends on scale, precision requirements, visibility, operational context, interaction state, and runtime cost.

This combination is central for scalable projection, frame evaluation, visualization, and vehicle-space interpretation.

44.9. Caching and Consistency

Definition 44.13 (Runtime cache). A runtime cache stores previously evaluated quantities to avoid repeated computation.

Caching can accelerate projection, frame, sampled geometry, and dynamic queries, while introducing consistency and invalidation requirements.

44.10. Metric-Aware Runtime

Runtime evaluation depends on metric realization (chapter 12). Different realizations affect distance interpretation, curvature-related quantities, frame construction, projection behaviour, and stationing interpretation.

Principle 44.14 (Metric-aware runtime principle). Runtime systems should evaluate geometry through metric-aware operator services rather than through hardcoded Euclidean assumptions.

44.11. Runtime Pipelines and Service Interaction

Definition 44.15 (Runtime operator pipeline). A runtime pipeline is a composed operator chain such as

$$\mathcal{E}_{\text{run}} \sim \mathcal{P} \circ \mathcal{F} \circ \mathcal{G}.$$

The following figures are read as progressively richer answers to the same question: how does the operator chain change when realized geometry must be included explicitly?

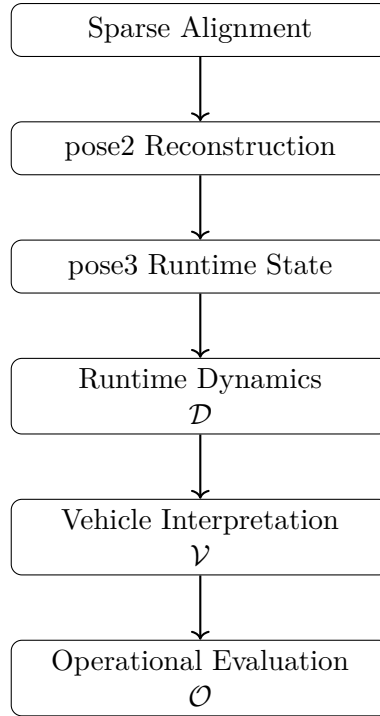


Figure 44.4.: Conceptual runtime evaluation pipeline.

This figure is interpreted as the baseline operator composition used for target-state runtime evaluation.

Definition 44.16 (Realized runtime pipeline). When realized geometry is required, runtime evaluation may conceptually depend on

$$\mathcal{E}_{\text{real}} \sim \mathcal{P} \circ \mathcal{F} \circ \mathcal{G} \circ \mathcal{R}.$$

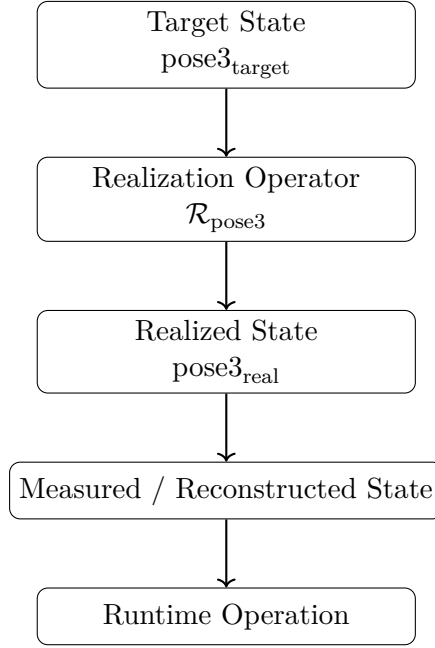


Figure 44.5.: Realization-aware runtime evaluation pipeline.

In contrast, the realization-aware figure adds $\mathcal{R}$ explicitly to represent realized-state dependence. The engineering consequence is that service selection must declare whether target or realized state is required, because this choice changes the active operator chain.

Different runtime services activate different operator chains, depending on precision, context, and purpose.

This provides the transition to the final boundary statement: runtime evaluation may vary operationally, but it must not alter conceptual ownership of engineering meaning.

44.12. Runtime, Realization, and Representation Boundaries

Runtime must distinguish target, realized, measured, and temporary interaction states. Runtime evaluation and physical realization are coupled but conceptually distinct: realization transforms target states, while runtime evaluates states.

Representation remains downstream. Runtime services provide derived state and quantities; representation services display, encode, or exchange those results for specific applications.

44.13. Conceptual Runtime Architecture

This architecture figure summarizes the chapter argument in executable form: persistent semantics feed metric-aware evaluation, and evaluation fans out into service families that converge into derived runtime state. The engineering consequence is a stable integration contract for downstream Reality, Modelling, and Optimization chapters.

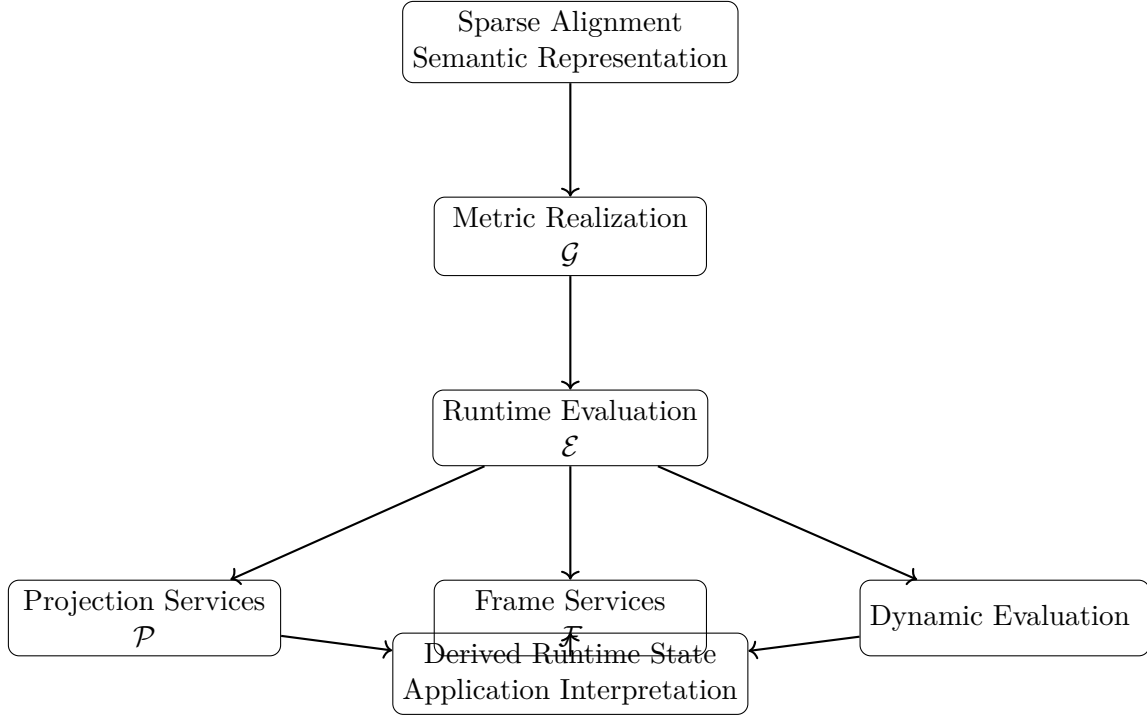


Figure 44.6.: Runtime services transform persistent engineering information through metric-aware evaluation into derived runtime state for downstream interpretation.

44.14. Canonical Interpretation

Proposition 44.17. *Within AIM, runtime services are operator-based evaluation systems acting on established engineering and state concepts.*

Proposition 44.18. *Derived runtime quantities are evaluation results and are not persistent engineering identity.*

Proposition 44.19. *Projection, frame construction, metric realization, and runtime dynamics form a coupled service architecture.*

Proposition 44.20. *Representation remains downstream of runtime evaluation.*

44.15. Connection to AIM

Summary 44.21. AIM runtime services consume persistent engineering information and state-layer concepts and generate derived runtime engineering state through operator-based evaluation.

This architecture keeps semantics stable, keeps evaluation explicit, maintains representation boundaries, and enables application-specific interpretation without redefining upstream knowledge.

Therefore, the runtime part closes with a precise consequence for the next parts: downstream reality, modelling, optimization, and applications may specialize runtime services, but they do so by consuming this architecture rather than redefining it.

Part VIII.

Reality and Data

Reality

This part establishes how canonical and runtime state are connected to constructed and observed railway reality.

The preceding parts established constructive alignment semantics, metric realization, railway state, and runtime evaluation. The present part explains how these approved concepts are embedded, observed, realized, and compared in engineering practice.

Reality does not redefine upstream concepts. It consumes them and establishes the progression

constructive alignment semantics → metric realization
→ world embedding
→ physical realization
→ measurement and observation → $\xrightarrow{\text{target-realized comparison}}$ → engineering interpretation.

This progression preserves the distinction between constructive identity, metric realization, CRS context, physical realization, realized railway state, measurement, representation, and operational interpretation.

Relation to the AIM Roadmap

Within the AIM roadmap, Reality is the layer where runtime-evaluated state meets measured and constructed infrastructure.

The orienting question for the icon below is: which layer order prevents target, realized, and observed states from collapsing into one another?

The following chapters use this separation to structure embedding, measurement, and comparison.

Principle 44.22 (Reality Principle). Within AIM, model state and physical railway state are related but conceptually distinct; they are connected through realization, measurement, and interpretation operators.

Summary 44.23. Reality establishes how approved alignment semantics and runtime state are embedded in the engineering world, observed through data, and interpreted through target–realized comparison.

This separation prepares the Modelling part, where persistent structures are designed to preserve exactly these distinctions in operational information systems.

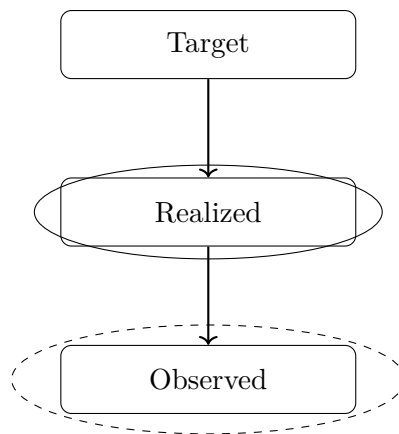


Figure 44.7.: Reality icon: target, realized, and observed railway state as dependent layers.

45. Measurement Data and Observation Context

Measurement data is the observation layer of AIM. It provides evidence about physical infrastructure and runtime state but does not define engineering identity by itself.

45.1. Role of Data

Definition 45.1 (Data source). A data source is any origin of geometry-related or state-related observation used in railway modelling.

Typical sources include geodetic surveys, track geometry measurements, legacy engineering datasets, structured exchange formats, and point cloud-derived products.

Principle 45.2 (Observation principle). Measurement data is engineering observation evidence and must be interpreted through explicit model and context assumptions.

45.2. Data Characteristics

Real-world railway data is heterogeneous and typically exhibits incompleteness, noise, ambiguity, and cross-source inconsistency. As a result, data cannot be equated directly with constructive identity, metric realization, or realized state.

Definition 45.3 (Measurement uncertainty). Measurement uncertainty describes deviation between observed quantities and the corresponding physical quantities.

Uncertainty propagates through interpretation, reconstruction, and runtime evaluation.

45.3. Interpretation Layer

Definition 45.4 (Interpretation). Interpretation is the process of assigning geometric and semantic meaning to observed data in a specified context.

Interpretation includes segmentation, topology assignment, station context assignment, and association with profile/cant/runtime quantities. The process is context-dependent and generally non-unique.

45.4. From Data to Structured Model Input

Data processing in AIM maps

observation data $\rightarrow$ interpreted model input $\rightarrow$ runtime evaluation context.

Definition 45.5 (Reconstruction from observation data). Reconstruction is the process of deriving structured model input from observed data while preserving provenance and uncertainty context.

This reconstruction may involve filtering, segmentation, station/topology alignment, and sparse-compatible approximation.

45.5. Geometric and Topological Evidence

Observation datasets may contain geometric evidence (positions, directions, curvature-related samples) and topological evidence (connectivity, branch context, route context). These layers must be kept conceptually distinct while interpreted jointly.

45.6. Container and Exchange Context

Container formats can carry mixed geometry, metadata, and semantic content. AIM therefore treats container content as import evidence that must be normalized into explicit model-layer structures before runtime or realization comparison.

45.7. Connection to AIM

Summary 45.6. In the Reality part, data is treated as observation context for realization-aware and runtime-aware interpretation.

It is neither constructive identity nor final engineering truth. Engineering interpretation arises only after explicit reconstruction, context assignment, and target-realized comparison.

46. Coordinate Systems and World Embedding

This chapter owns the coordinate and CRS explanation of the Reality part. In AIM, CRS and coordinate systems define embedding and transformation context; they do not define constructive alignment identity.

CRS Is Not Metadata

In railway alignment modelling, CRS cannot be treated as auxiliary metadata. CRS assumptions affect metric interpretation, coordinate transformations, projection behaviour, and comparison of target and observed states.

Principle 46.1 (CRS structural principle). CRS is a structural component of realization and embedding context. Ignoring it breaks consistency of engineering interpretation.

Consequently, CRS handling must be explicit in import, runtime projection, reconstruction, and realization-aware comparison workflows.

46.1. Coordinate Reference Systems

Definition 46.2 (Coordinate reference system). A coordinate reference system (CRS) is a mathematical framework that maps spatial coordinates to positions in a defined world embedding.

A CRS combines reference surface assumptions, coordinate definitions, and transformation conventions.

46.2. Embedding Context

Railway projects commonly use multiple systems simultaneously, including ellipsoid-referenced systems, projected systems, and local engineering frames. These systems provide embedding context for geometry and measurements.

Principle 46.3 (CRS context principle). CRS belongs to realization and embedding context, not to constructive identity.

46.3. Transformations and Consistency

Definition 46.4 (Coordinate transformation). A coordinate transformation is a mapping

$$T : \mathbb{R}^n \rightarrow \mathbb{R}^n$$

between coordinate systems.

Transformation handling must be explicit and auditable, since small transformation inconsistencies can create large interpretation errors in curvature-related or projection-related quantities.

46.4. Local Frames and Numerical Stability

Definition 46.5 (Local frame). A local frame is a coordinate system anchored near an engineering region of interest.

Local frames improve numerical stability for runtime evaluation, projection, and reconstruction, while preserving linkage to global reference context.

46.5. Stationing versus CRS Coordinates

CRS coordinates express embedded position. Stationing expresses route address along railway reference structures. These are distinct layers and must not be conflated.

46.6. Relation to Metric Realization

Coordinate handling consumes metric realization (chapter 12) and realization operators; it does not replace them. World embedding and world-track relations remain realization-dependent services.

46.7. Connection to AIM

Summary 46.6. Within AIM, coordinate systems and CRS are explicit embedding context for realized and observed railway states.

They are essential for consistency and interoperability, but they are not alignment identity.

47. Kilometre Line and Stationing

Stationing is the operational addressing layer of railway systems. It is not identical to constructive alignment identity, metric embedding, or realized geometry.

47.1. Stationing Role

Definition 47.1 (Kilometre line). A kilometre line is a longitudinal reference structure that assigns station values within a route context.

Definition 47.2 (Stationing layer). The stationing layer maps railway positions to operational address values.

Stationing answers how an infrastructure position is addressed; geometry answers what shape is realized; CRS answers where embedding is expressed.

47.2. Relation to Geometry

A kilometre line may coincide with a physical track centerline, but this is not required. In multi-track and junction contexts, several physical alignments can reference one stationing structure.

Definition 47.3 (Outer stationing). Outer stationing describes mapping between a physical track alignment coordinate and an external kilometre reference.

Outer stationing is required when station reference and physical track reference do not coincide.

47.3. Layer Separation

Proposition 47.4. *Railway modelling requires separation of*

$$\textit{spatial embedding} \neq \textit{geometric alignment} \neq \textit{stationing reference}.$$

Confusing these layers yields systematic errors in interpretation, network addressing, and target–realized comparison.

47.4. Topology and Operational Continuity

Kilometre references provide operational continuity across track changes, branches, station areas, and route transitions, even where geometric continuity is represented by multiple physical alignments.

47.5. Connection to AIM

Summary 47.5. In AIM, stationing is an explicit reference layer used for operational addressing and comparison workflows.

It remains distinct from constructive identity, CRS embedding, and realized track geometry.

48. Measurement and Imperfect Data

Measurement provides observation evidence about realized infrastructure. It does not replace constructive identity or realized-state modelling.

48.1. Observation Context

Measurement data in railway projects is heterogeneous in resolution, accuracy, provenance, and coordinate context.

Definition 48.1 (Observed data). Observed data denotes measured quantities acquired from sensors, surveys, or legacy datasets in a specified observation context.

Principle 48.2 (Observation context principle). Observed data must be interpreted together with sensor, stationing, coordinate, and topology context.

48.2. Imperfect Data Characteristics

Observed data typically includes random noise, systematic bias, incompleteness, and cross-source inconsistency.

Definition 48.3 (Systematic error). A systematic error is a persistent observation deviation induced by sensor, calibration, or transformation effects.

Definition 48.4 (Measurement uncertainty). Measurement uncertainty quantifies the interval or distribution of plausible physical values compatible with an observation.

48.3. Inference from Observation

Geometry and state are inferred from observation through reconstruction operators and interpretation rules.

Definition 48.5 (Discrete curvature estimate). Given local directional samples, a discrete curvature estimate may be written as

$$\kappa_i \approx \frac{\theta_{i+1} - \theta_i}{\Delta s_i}.$$

Such estimates are sampling-dependent and uncertainty-sensitive; filtering and regularization are therefore required in practice.

48.4. Observation and Runtime Reconstruction

Observation enters AIM through runtime reconstruction pipelines that combine data with metric realization, stationing context, and topology context.

Observed values are interpreted as evidence for realized state, not as canonical object identity.

48.5. Engineering Use

In engineering workflows, imperfect data affects validation, maintenance planning, and target–realized comparison. Robust workflows therefore require provenance tracking, uncertainty-aware evaluation, and explicit assumption management.

48.6. Connection to AIM

Summary 48.6. In AIM, measurement and imperfect data form the observation layer of the Reality part.

This layer supports realization-aware reconstruction and interpretation while preserving the distinction between observed evidence and engineering identity.

49. Topology and Special Elements

Railway infrastructure is networked. Reality interpretation therefore requires explicit topology context in addition to geometric state.

49.1. Topology Context

Definition 49.1 (Railway network context). Railway network context is the set of connectivity relations between aligned track segments, branches, merges, and route continuations.

Topology context governs route continuity and interpretation of station, projection, and observation data in multi-track systems.

49.2. Special Elements

Switches, crossings, station throats, and yard structures are special elements where geometric and operational branching is explicit.

Definition 49.2 (Connection point). A connection point is a location where two or more alignment segments are topologically linked.

These elements do not redefine constructive alignment identity; they provide network context in which identities are interpreted.

49.3. Geometry–Topology Separation

A useful abstraction separates geometry from connectivity while allowing explicit coupling at interpretation time.

Definition 49.3 (Graph representation). A railway network may be represented as a graph in which nodes denote connection points and edges denote alignment segments.

This representation clarifies that topology context and geometric state are distinct but jointly required in Reality workflows.

49.4. Data Ambiguity and Inference

Observed datasets frequently under-specify topology. Inference of route continuity and branch assignment is therefore part of reality interpretation and must be handled explicitly.

49.5. Connection to AIM

Summary 49.4. Topology and special elements provide the network context required for real-world interpretation of measured and realized railway states.

Within AIM, topology is consumed as contextual structure and does not replace constructive alignment semantics.

50. Construction and Physical Realization

This chapter answers how target engineering state becomes physically built railway state under construction and maintenance constraints. It establishes the realization operator view used in Reality and clarifies the consequence for runtime reconstruction and realization-aware design.

The opening question is therefore comparative: what changes when analytic target state is transformed into realized state?

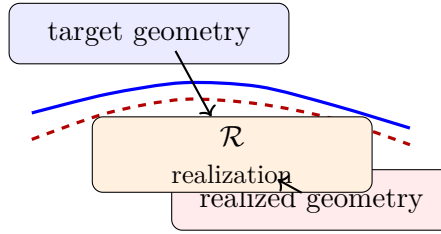


Figure 50.1.: Target geometry and realized geometry connected by the realization operator.

The figure provides this comparison at a glance and motivates the operator formulation used in the formal sections below.

This chapter owns physical realization in the Reality part. Physical realization transforms target engineering state into physically built state under construction, maintenance, and structural constraints.

Principle 50.1 (Realization principle). Within AIM, physical realization is an operator layer that transforms target states into physically realized states.

50.1. Physical Realization versus Metric Realization

Metric realization (chapter 12) defines measurable embedding context. Physical realization describes construction and maintenance effects on built infrastructure. The two layers are coupled but distinct.

Definition 50.2 (Realized alignment). The realized alignment is the physically constructed railway geometry, denoted by

$$\gamma_{\text{real}}(s).$$

Definition 50.3 (Realized curvature). The realized curvature is

$$\kappa_{\text{real}}(s) = \frac{d\theta_{\text{real}}}{ds}.$$

50.2. Realization Operator View

$$x_{\text{real}} = \mathcal{R}(x_{\text{target}}),$$

where $\mathcal{R}$ is the canonical realization operator (theorem 11.9).

Realization error can be assessed in state-dependent spaces. A curvature-based measure is

$$E_{\kappa} = \int_0^L (\kappa_{\text{real}} - \kappa_{\text{target}})^2 \, ds.$$

50.3. Construction and Adjustment Process

Construction and maintenance are iterative local processes.

The following definitions and propositions formalize this practical workflow as an iterative realization process whose fixed-point view connects engineering practice to operator semantics.

Definition 50.4 (Adjustment operator). The adjustment operator $\mathcal{A}$ denotes a localized construction/maintenance update applied to an existing alignment state.

Definition 50.5 (Iterative realization). Starting from γ_0 , an iterative realization process can be represented by

$$\gamma_{k+1} = \mathcal{A}_{\gamma_{\text{target}}}(\gamma_k).$$

Definition 50.6 (Realized alignment as limit). If this process converges,

$$\gamma_{\text{real}} = \lim_{k \rightarrow \infty} \gamma_k.$$

Proposition 50.7 (Fixed point interpretation). *If iterative realization converges,*

$$\gamma_{\text{real}} = \mathcal{A}_{\gamma_{\text{target}}}(\gamma_{\text{real}}).$$

50.4. Sampling, Locality, and Filtering

Definition 50.8 (Control points). Let

$$0 = s_0 < s_1 < \cdots < s_N = L$$

be control-point stations used in realization and maintenance workflows.

Proposition 50.9 (Adaptive sampling principle). *Sampling density should increase in regions with strong curvature variation.*

Definition 50.10 (Local correction). A local correction form is

$$\mathcal{A}_{\gamma_{\text{target}}}(\gamma) = \gamma + \Delta(\gamma, \gamma_{\text{target}}).$$

Proposition 50.11 (Locality). *Adjustment depends on local neighbourhood context rather than global alignment state.*

Definition 50.12 (Kernel-based adjustment). A linearized local model is

$$(\mathcal{A}_{\gamma_{\text{target}}}\gamma)(s) = \gamma(s) + \int_I K(s, \sigma)(\gamma_{\text{target}}(\sigma) - \gamma(\sigma)) \, d\sigma.$$

Definition 50.13 (Discrete adjustment model). For control points,

$$\gamma_i^{(k+1)} = \gamma_i^{(k)} + \sum_j K_{ij} (\gamma_j^{\text{target}} - \gamma_j^{(k)}).$$

Proposition 50.14. *Physical realization acts approximately as a low-pass filter on curvature and related high-frequency components.*

Principle 50.15 (Realization filter principle). Construction and maintenance transform target state through physical sampling, machine response, and structural filtering.

50.5. Runtime and Optimization Implications

The realization layer is compatible with sparse and runtime evaluation: realized geometry can be interpreted as measured samples, reconstructed runtime state, or sparse-compatible approximation.

This section therefore transitions from construction mechanics to engineering consequence: realized-state performance becomes the relevant design criterion for downstream optimization.

Principle 50.16 (Sparse realization compatibility). Realization-aware workflows must preserve compatibility with sparse and runtime state architectures.

Definition 50.17 (Realization-aware objective). A realization-aware design objective can be written as

$$\min_{x_{\text{target}}} J(\mathcal{R}(x_{\text{target}})).$$

Proposition 50.18 (Realization-aware design principle). *The relevant design criterion is performance of realized state, not only analytic smoothness of target state.*

Proposition 50.19. *The physical design space is the image of the realization operator.*

50.6. Connection to AIM

Summary 50.20. Construction and physical realization connect target engineering state with built infrastructure state.

Within AIM, physical realization is an explicit downstream layer that consumes constructive semantics and metric realization context and produces realized state for measurement, runtime reconstruction, and engineering interpretation.

50. Construction and Physical Realization

Accordingly, the next Reality chapter can treat target–realized comparison as an evidence-based assessment layer built on this realization operator foundation.

51. Realized pose3 and Target–Realized Comparison

This chapter consumes the canonical pose3 definitions from Foundations and State (theorems 10.3 and 28.6) and defines the realized and measured counterparts used in Reality workflows.

Principle 51.1 (Realized-state principle). Within AIM, operational engineering interpretation depends on realized runtime state rather than on target state alone.

51.1. Target, Realized, and Measured pose3

Definition 51.2 (Target pose3). The target railway state is denoted by

$$\text{pose3}_{\text{target}}(s).$$

Definition 51.3 (Realized pose3). The realized railway state is denoted by

$$\text{pose3}_{\text{real}}(s).$$

Definition 51.4 (Measured pose3 state). A measured pose3 state is an observation-derived approximation

$$\text{pose3}_{\text{meas}}(s_i) \approx \text{pose3}_{\text{real}}(s_i).$$

Principle 51.5 (Measurement separation principle). Target state, realized state, and measured state are distinct layers.

The first figure addresses the layer question directly: how should target and realized state be compared without erasing their conceptual separation?

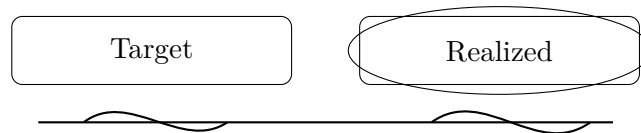


Figure 51.1.: Supporting comparison: target and realized states are related but non-identical layers.

The engineering consequence is that all downstream assessment must be formulated as explicit target–realized comparison, not as implicit assumption of geometric identity.

51.2. Realized State Components

Definition 51.6 (Realized spatial trajectory). The realized spatial trajectory is denoted by

$$\Gamma_{\text{real}}(s).$$

Definition 51.7 (Realized curvature).

$$\kappa_{\text{real}}(s) = \frac{d\theta_{\text{real}}}{ds}.$$

Definition 51.8 (Realized cant).

$$\phi_{\text{real}}(s).$$

In general,

$$\text{pose3}_{\text{real}} \neq \text{pose3}_{\text{target}}.$$

51.3. Runtime Reconstruction of Realized pose3

Realized pose3 is typically reconstructed at runtime from observed data, realization models, and embedding context.

Definition 51.9 (Runtime realization reconstruction). A schematic form is

$$\text{pose3}_{\text{real}}(s) = \mathcal{E}_{\text{real}}(s, x_{\text{target}}, m, \mathcal{R}, \mathcal{C}),$$

where m is observation data, $\mathcal{R}$ realization-model information, and $\mathcal{C}$ metric/coordinate context.

Principle 51.10 (Measured-state ambiguity). Measured data is evidence for realized pose3, not realized pose3 itself.

51.4. Observation Operators and Sensor Integration

Definition 51.11 (Sensor observation model).

$$y_i = \mathcal{H}_i(\text{pose3}_{\text{real}}(s_i)) + \varepsilon_i,$$

where $\mathcal{H}_i$ is a sensor observation operator and ε_i observation error.

Realized-state inference is therefore an inverse reconstruction problem from heterogeneous observations.

The second figure then answers the evidence question: how does observed data mediate assessment rather than replacing canonical target meaning?

Its consequence is methodological: observation enters as evidence in an assessment chain, so model validation must preserve separation between target semantics, realized state, and measured approximation.

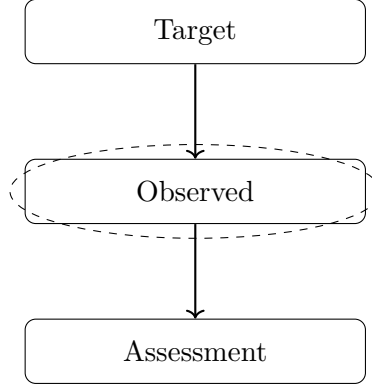


Figure 51.2.: Supporting realization view: assessment depends on observed evidence relative to target semantics.

51.5. Dynamic Comparison Quantities

Definition 51.12 (Realized equilibrium condition). A realized track is dynamically balanced if

$$v^2 \kappa_{\text{real}}(s) = g \sin \phi_{\text{real}}(s).$$

Definition 51.13 (Realized effective lateral acceleration).

$$a_{\text{eff,real}}(s) = v^2 \kappa_{\text{real}}(s) - g \sin \phi_{\text{real}}(s).$$

Definition 51.14 (Realized jerk).

$$j_{\text{real}}(s) = v^3 \kappa'_{\text{real}}(s) - g v \cos \phi_{\text{real}}(s) \phi'_{\text{real}}(s).$$

These quantities support target–realized engineering comparison beyond position-only deviation.

51.6. Realization Operator on pose3

Definition 51.15 (Pose3 realization operator).

$$\mathcal{R}_{\text{pose3}} : \text{pose3}_{\text{target}} \mapsto \text{pose3}_{\text{real}}.$$

This operator acts on full railway state interpretation (geometry, frame, cant, and derived dynamic behaviour), not on coordinates alone.

51.7. Connection to AIM

Summary 51.16. Realized pose3 is the central comparison layer between target engineering state and observed railway reality.

51. Realized pose3 and Target–Realized Comparison

Within AIM, the chapter preserves separation between target, realized, and measured state while consuming canonical pose3 and realization operators from prior parts.

Part IX.

Alignment Modelling

Modelling

This part establishes how canonical railway semantics are encoded as compact, reconstructable, and operational information structures.

What do we already know?

The previous parts established the conceptual foundations of AIM.

Intrinsic geometry, railway states, runtime evaluation, realization, and reality-aware interpretation have been developed as coherent theoretical constructs.

These concepts define what railway alignments are, how they evolve, and how they interact with the physical world.

However, concepts alone do not yet define how alignment information is represented within an operational information model.

What is still missing?

An information model requires explicit internal structures.

Geometric concepts must be transformed into representations that can be stored, exchanged, reconstructed, analyzed, and optimized.

Traditional railway systems frequently rely on coordinate lists, geometric primitives, or implementation-specific data structures.

While such representations may be sufficient for storage purposes, they often obscure the intrinsic structure of the alignment and limit subsequent analysis.

The missing element is therefore a modelling framework capable of capturing railway semantics while remaining compact, interpretable, and computationally efficient.

What comes next?

This part introduces the modelling layer of AIM.

The central objective is the construction of sparse, semantic alignment models.

Within AIM, an alignment is not primarily represented as sampled coordinates.

Instead, it is represented through a compact collection of geometric states, transition descriptions, and semantic relationships from which geometry can be reconstructed whenever required.

The chapters that follow introduce:

- sparse alignment representations,
- transition modelling,
- transition classification,
- higher-level alignment semantics,
- and Schwerpunkt-based modelling concepts.

These structures form the operational core of alignment-based information modelling.

They also reveal why the system grew beyond the originating mathematics. Berlinish supplies a generalized transition grammar. axtranNew consumes functions, boundary conditions, and degrees of freedom to calculate concrete alignment structures. transitionDB preserves reusable transition knowledge, and TransEd makes that knowledge examinable. SPOT is then needed to carry the identity and state of concrete engineering objects, while alignmentOS provides the operational environment in which the calculated structure can be edited, evaluated, and related to other work. These are dependent responses to one engineering problem, not a flat catalogue of products.

Relation to the AIM Roadmap

Within the AIM roadmap, the present part corresponds to the transition from conceptual railway descriptions toward explicit alignment models.

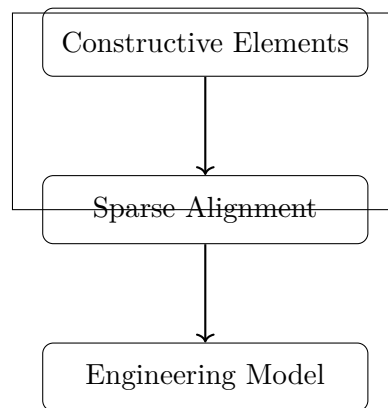


Figure 51.3.: Modelling icon: constructive elements form sparse alignment and engineering model semantics.

The progression

Concepts $\rightarrow$ Sparse Model $\rightarrow$ Alignment Model

marks the transition from theoretical railway descriptions toward operational information structures.

The figure above answers the orienting question for this transition: which constructive elements must remain explicit so sparse models stay both compact and semantically faithful?

These models subsequently become the basis for runtime services, analysis pipelines, optimization procedures, and engineering applications.

Principle 51.17 (Sparse Modelling Principle). Within AIM, railway alignments should be represented through compact semantic structures from which geometric, operational, and realization- aware information can be reconstructed when required.

Summary 51.18. Concepts explain railway structure.

Reality explains interaction with the physical world.

Modelling explains how railway knowledge is represented.

The present part therefore establishes the sparse and semantic information structures that form the core of alignment-based Alignment-Based Information Modelling.

With this modelling layer established, the Analysis part can examine how these structures are transformed through pipelines, services, and system interactions.

52. Sparse Alignment Model

Motivation

This chapter answers how continuous curvature theory is converted into a compact engineering representation that remains reconstructable, interpretable, and operationally useful. It establishes the sparse alignment model, clarifies ownership between adjacent modelling layers, and prepares the subsequent transition and classification chapters.

The guiding question is therefore: which information must be persisted, and which information should be reconstructed at runtime?

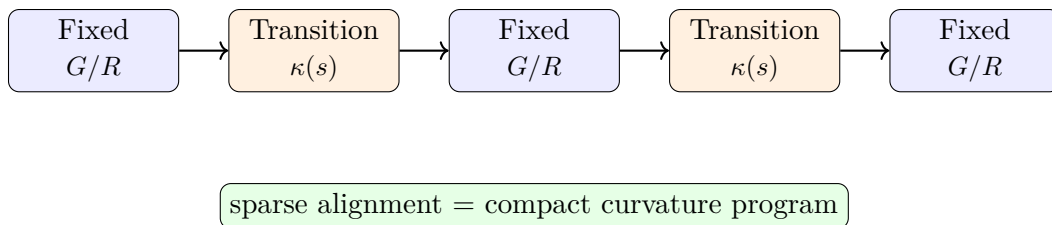


Figure 52.1.: Sparse alignment as an alternating sequence of fixed and transition elements.

This first figure provides the structural answer: sparse alignment is an ordered semantic sequence, not a dense geometric snapshot.

A continuous curvature law is mathematically natural, but it is not by itself an operational representation for editing, exchange, validation, or reconstruction from heterogeneous engineering data. For this reason, a sparse model is introduced as a structured, piecewise-defined representation of an alignment.

The sparse model is not intended as a denial of continuous geometry. On the contrary, it serves as an intermediate layer between continuous curvature theory and practical modelling tasks such as import, computation, optimization, and interoperability.

Dense geometric representations such as point clouds or sampled polylines preserve positional information, but largely destroy the structural semantics of the alignment itself.

The second question is responsibility: how are persistent data, structural alignment semantics, and downstream engineering objects kept distinct?

This distinction prevents semantic drift between storage, canonical alignment semantics, and application-specific runtime products.

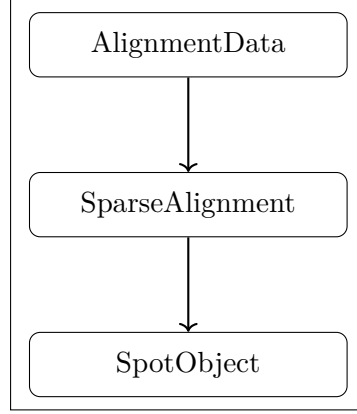


Figure 52.2.: Supporting responsibility view: AlignmentData, SparseAlignment, and SpotObject form distinct ownership layers.

52.1. General Idea

Definition 52.1 (Sparse alignment model). A sparse alignment model is a structured longitudinal representation consisting of synchronized alignment bands together with sufficient reconstruction information.

The adjective *sparse* refers here to the fact that the alignment is not stored as a dense point cloud, but as a compact sequence of semantically meaningful elements.

Within AIM, the sparse model is not interpreted as a static 3D geometry, but as a persistent structural kernel from which geometry and dynamics are derived.

52.2. Element Types

Definition 52.2 (Alignment element). An alignment element is a segment of an alignment with a prescribed local curvature behaviour on a parameter interval.

Definition 52.3 (Fixed element). A fixed element is an alignment element with constant curvature

$$\kappa(s) = \kappa_0.$$

Typical special cases are straight segments with $\kappa_0 = 0$ and circular arcs with $\kappa_0 \neq 0$.

Definition 52.4 (Transition element). A transition element is an alignment element with non-constant curvature, i.e.

$$\kappa(s) = f(s)$$

for a suitable curvature law f .

Within AIM, transition elements are primarily interpreted through their curvature evolution rather than through explicit geometric shape.

52. Sparse Alignment Model

At this level of abstraction, no commitment is yet made to a specific family of transition laws. This includes classical clothoids as well as polynomial, lookup-based, or hybrid constructions.

52.3. Minimal Parameterization

Definition 52.5 (Arc length of an element). Each element e_i is associated with an arc length

$$L_i > 0.$$

Definition 52.6 (Local curvature law). Each element e_i is associated with a curvature law

$$\kappa_i : [0, L_i] \rightarrow \mathbb{R}.$$

For fixed elements, κ_i is constant. For transition elements, κ_i varies along the local parameter.

Definition 52.7 (Sparse alignment). A sparse alignment is an ordered tuple

$$A = (e_1, e_2, \dots, e_n)$$

together with sufficient initial data for geometric reconstruction.

52.4. Initial Data and Pose

Definition 52.8 (Planar start pose). A planar start pose is a tuple

$$P_0 = (x_0, y_0, \theta_0)$$

with position $(x_0, y_0) \in \mathbb{R}^2$ and initial direction $\theta_0 \in \mathbb{R}$.

Proposition 52.9. *The planar alignment is reconstructed by successive integration of the curvature law:*

$$\kappa(s) \rightarrow \theta(s) \rightarrow \gamma(s).$$

Together with the ordered element sequence and the local curvature laws, the start pose determines the alignment geometry by successive integration.

Definition 52.10 (Reconstructable sparse alignment). A sparse alignment is called reconstructable if its element sequence and its initial data suffice to determine a unique planar alignment up to the intended modelling conventions.

52.5. Sparse Longitudinal Band Structure

Principle 52.11 (Sparse longitudinal band principle). Railway alignments are represented as synchronized longitudinal bands rather than as monolithic geometric objects.

52. Sparse Alignment Model

Within the AIM framework, the persistent alignment model is intentionally kept sparse and decomposed into synchronized longitudinal bands.

52.5.1. cGeom

Definition 52.12 (Core geometric band). The central geometric band is denoted by

cGeom.

It represents the curvature-driven planar alignment core and consists of:

- fixed elements,
- transition elements,
- pose2-based reconstruction anchors.

The cGeom band forms the primary integration core of the alignment.

52.5.2. Cant Band

Definition 52.13 (Cant band). Cant is represented as an independent longitudinal band correlated with fixed alignment elements.

The cant band:

- does not define an independent stationing system,
- stores rail height differences rather than roll angles,
- follows the same normalized curvature family as the associated cGeom element,
- may contain local station offsets at element boundaries.

Since cant evolution follows the same normalized curvature family as the associated cGeom element, no independent geometric evolution model is required for the cant band itself.

The corresponding roll angle is derived via:

$$\phi(s) = \arctan\left(\frac{u(s)}{g}\right),$$

where:

- $u(s)$: rail height difference,
- g : gauge.

52.5.3. Profile Band

Definition 52.14 (Profile band). Vertical geometry is represented independently through a profile band.

Thus, cGeom, cant, and profile are treated as synchronized but distinct bands sharing a common arc-length domain.

52.5.4. Runtime State

Proposition 52.15. *Pose3 is not treated as persistent truth within the sparse model.*

Persisting pose3 directly would introduce redundancy and synchronization instabilities between geometry, cant, and profile representations.

Instead, pose3 is derived dynamically from:

- cGeom,
- cant,
- profile,
- and runtime evaluation services.

Consequently, the primary intelligence of the AIM framework resides not in the storage format itself, but in the mathematical operators and evaluation services acting on the sparse model.

52.6. Concatenation

Definition 52.16 (Element concatenation). Two consecutive elements are concatenated if the end state of the first element provides the start state of the second element.

Definition 52.17 (Positional continuity). A sparse alignment is positionally continuous if consecutive elements meet in a common endpoint.

Definition 52.18 (Directional continuity). A sparse alignment is directionally continuous if consecutive elements share the same tangent direction at their common boundary.

In most engineering contexts, positional and directional continuity are minimal requirements. Higher continuity requirements may depend on the chosen transition families and on the intended application.

52.7. Normal Form and Alternation

In practical sparse representations, a frequent pattern is the alternation between fixed elements and transition elements. This corresponds closely to engineering intuition and to many traditional alignment descriptions.

It is plausible to regard alternation as a normal form rather than a fundamental axiom. Multiple neighbouring fixed elements may be merged, and certain degenerate transition cases may collapse into fixed elements.

52.8. Normalized Representation of Elements

Definition 52.19 (Normalized local parameter). A normalized local parameter is a variable

$$u \in [0, 1]$$

used to represent an element independently of its physical length.

Definition 52.20 (Normalized curvature law). A normalized curvature law is a law defined on the unit interval, typically of the form

$$\hat{\kappa} : [0, 1] \rightarrow \mathbb{R}.$$

Normalization separates shape from scale. It allows the same transition family to be reused for different lengths and different curvature magnitudes.

Normalized representations form the basis for reusable transition registries, family classification, and implementation-independent comparison of alignment elements.

52.9. Structural Role of the Sparse Model

The sparse model occupies an intermediate conceptual level:

- below the purely abstract curvature-theoretic foundation,
- but above dense numerical sampling, imported point lists, or format-specific container structures.

The sparse model is intentionally not a dense geometric storage format, nor a direct copy of external exchange containers.

This intermediate status makes the sparse model suitable as

- an internal modelling layer,
- a reconstruction target from imported data,
- a source for numerical evaluation,
- and a bridge toward standards and BIM-related representations.

52.10. Relation to Imported and Standardized Data

Existing engineering data rarely arrives directly in sparse form. Instead, it is typically given through container formats, survey data, legacy descriptions, or derived coordinate sequences.

Such data may contain:

- inconsistent stationing,
- incomplete cant information,
- broken coordinate reference systems,
- discontinuities,
- partial geometry fragments,
- or non-synchronized alignment components.

The sparse model therefore should not be identified with any one exchange standard. It is more appropriate to view it as a compact, internal representation that can be derived from, and mapped back to, multiple external formats.

52.11. Relation to BIM and BIMalignment

A sparse alignment model may support BIM-oriented workflows, but it should not merely imitate BIM container logic. Its primary responsibility is to represent alignment structure in a mathematically and operationally coherent way.

52.12. Summary

Summary 52.21. The sparse alignment model is a compact, structured representation of an alignment by means of synchronized longitudinal bands together with reconstruction data and concatenation rules.

Within AIM, the sparse alignment model is not interpreted as a static 3D geometry, but as a persistent structural kernel from which geometry, runtime state, and dynamic interpretation are derived.

It serves as a bridge between continuous curvature theory and practical engineering operations such as import, validation, editing, numerical evaluation, optimization, and interoperability.

53. Transition Functions

Motivation

Transitions are the decisive element type for curvature-driven alignment modelling. While fixed elements describe geometric states of constant curvature, transitions describe the controlled change between such states.

In railway engineering, transitions are essential for ensuring smooth vehicle motion, limiting dynamic forces, and satisfying comfort and safety requirements, as discussed in both engineering-oriented studies and review literature [28, 4].

Within AIM, transitions are not primarily interpreted through explicit geometric shape, but through the evolution of curvature.

53.1. Definition by Curvature Evolution

Principle 53.1 (Curvature evolution principle). Within AIM, transitions are classified primarily by their curvature evolution rather than by explicit geometric shape.

Definition 53.2 (Transition). A transition is an alignment element whose curvature varies over its parameter interval.

Definition 53.3 (Transition curvature law). Let $L > 0$. A transition curvature law is a function

$$\kappa : [0, L] \rightarrow \mathbb{R}$$

with prescribed boundary values

$$\kappa(0) = \kappa_0, \quad \kappa(L) = \kappa_1.$$

This formulation directly follows the curvature-based description of planar curves [9].

53.2. Normalized Representation

Definition 53.4 (Normalized transition law). A normalized transition is defined by

$$\hat{\kappa} : [0, 1] \rightarrow \mathbb{R}$$

with

$$\hat{\kappa}(0) = 0, \quad \hat{\kappa}(1) = 1.$$

53. Transition Functions

Normalization separates the geometric shape of a transition from its scale and allows systematic comparison of transition families.

Normalization separates intrinsic transition behaviour from physical scale and therefore enables reusable transition registries and family-based modelling.

The following comparison figure answers the engineering question for transition modelling: when geometric paths appear similar, where do operationally relevant differences still enter the model?

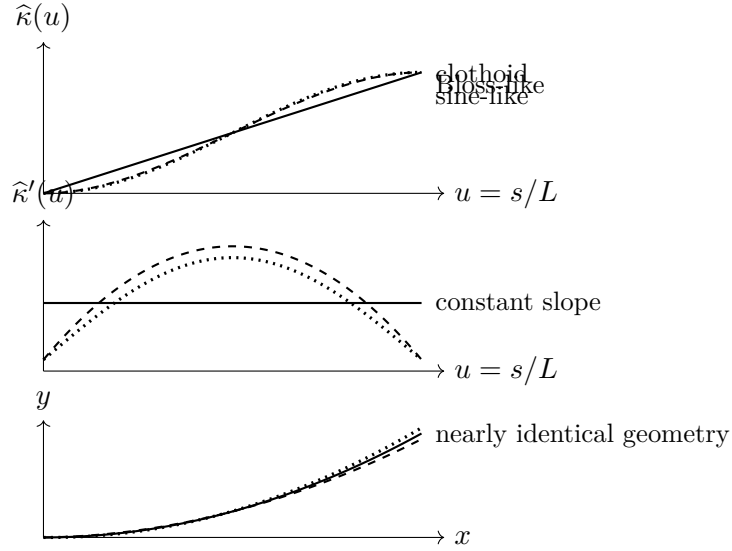


Figure 53.1.: Transition families may produce very similar geometric paths while differing strongly in curvature evolution and curvature derivatives. This is why transition design must be evaluated in curvature and dynamic terms, not only in position space.

Transitions that appear nearly identical in position space may differ substantially in curvature derivatives and therefore in dynamic behaviour.

The engineering consequence is methodological: transition selection cannot be justified by geometric fit alone; it must include curvature evolution and derivative-level runtime criteria.

53.3. Scaling

Proposition 53.5. *Let $\hat{\kappa}$ be a normalized transition law. Then a physical transition is given by*

$$\kappa(s) = \kappa_0 + (\kappa_1 - \kappa_0) \hat{\kappa}\left(\frac{s}{L}\right).$$

This separates shape from scale and provides the basis for reusable transition families and parameterized design.

53.4. Regularity and Smoothness

Definition 53.6 (Regularity class). A transition is of class C^k if its curvature function is k -times continuously differentiable.

In engineering practice, higher-order properties such as curvature derivatives are directly related to ride comfort and dynamic loading, as shown in both theoretical and engineering studies [36, 29].

53.5. Transition Families

Definition 53.7 (Transition family). A transition family is a class of curvature laws sharing a common functional structure.

53.5.1. Classical Families

The classical transition curve in railway engineering is the clothoid, characterized by linear curvature evolution:

$$\kappa(s) \propto s.$$

Historically, this line of development can be traced through early work by Glover [10], Horsburgh [16], and Higgins [14].

53.5.2. Polynomial and Alternative Families

Polynomial transition curves provide additional flexibility in shaping curvature evolution and its derivatives.

Such approaches are associated with modern polynomial smoothing research [55] as well as control-oriented constructions proposed by Klauder [21, 24].

53.5.3. Geometric Families

Some transition curves are introduced through geometric or implicit relations rather than explicit curvature laws.

Example 53.8. Lemniscate-type or sinusoid-related constructions belong to this class after suitable reparameterization into curvature space [40].

These curves require transformation into $\kappa(s)$ representation before they can be integrated into the present framework.

53.5.4. Engineering and Practice-Oriented Families

Engineering practice often introduces transition curves through design rules and operational considerations rather than through purely theoretical derivation.

The Bloss transition is a notable example of a practical engineering family, originally described by Bloss [3] and further discussed in modern summaries [8].

53.5.5. Modern and Hybrid Families

Modern engineering practice considers a variety of transition families with improved dynamic behaviour, smoother endpoint properties, or higher-order control.

Examples include smoothed-curvature transitions proposed by Koc [29, 31, 30] and Wiener-Bogen-related approaches [54].

53.6. Decomposition Approaches

A transition need not always be treated as a single indivisible object. It may be useful to represent it as a composition of sub-elements.

The Berlinish framework introduced in section 25.8 provides the concrete decomposition

$$T = H_{\text{in}} + C + H_{\text{out}}, \quad L_{H_{\text{in}}}, L_C, L_{H_{\text{out}}} \geq 0.$$

It is a mathematical composition grammar: zero component lengths and the choice of half-wave functions determine pure, one-sided, asymmetric, and compound cases. A sparse implementation may encode the same grammar through a family identifier, normalized partitions, function references, and parameters. The mathematical interpretation and its particular data encoding are related, but not identical.

53.7. Operator View

Principle 53.9 (Transition operator principle). A transition element acts as a local curvature evolution operator transforming one geometric state into another.

Transitions can therefore be interpreted as state evolution operators rather than merely as predefined geometric curve shapes.

53.8. Implementation Perspective

In computational systems, transition curves may be represented using:

- analytic expressions,
- polynomial approximations,
- lookup tables,
- compiled hybrid representations.

Transition families may be represented through registries containing normalized curvature laws together with derivative and integration operators.

In the originating system line, *transitionDB* denotes the structured knowledge base of function identities, half-wave families, parameters, and known transition forms. The

current reference implementation realizes a substantial part of that role through a validated transition registry: it contains normalized prototype functions, half-wave descriptors, family partitions, and named transition entries. This is implementation evidence for the composition mechanism, not proof that every historical or proprietary form has been captured.

TransEd is the corresponding examination and editing workspace. Its purpose is to inspect normalized curvature and derivative behaviour and to work with reusable transition definitions; it does not own Alignment identity and does not replace alignment-level editing. Analytic expressions and lookup representations remain alternative realizations of a function law, with the usual trade-off between traceable exact structure and runtime efficiency.

53.9. Relation to Railway Engineering

In railway applications, transitions are tightly coupled to operational parameters such as speed and cant, and must satisfy strict engineering constraints [32, 26].

The review literature confirms that transition curves must be understood not only geometrically, but also with respect to comfort, maintenance, and dynamic response [4].

53.10. Summary

Summary 53.10. Transitions define the controlled evolution of curvature between geometric states.

Within AIM, they are interpreted as normalized curvature evolution operators rather than merely as predefined geometric curve types.

Their normalized representation enables reusable transition families, operator-based modelling, and runtime reconstruction of alignment geometry and dynamics.

54. Transition Families and Classification

Motivation

Transition curves form a central component of alignment design, yet their treatment in the literature remains fragmented across geometric, engineering, and dynamic perspectives.

A unified classification is therefore required in order to compare, evaluate, and optimize transition concepts systematically.

54.1. Curvature-Based Representation

Proposition 54.1. *Any transition curve can be represented by a curvature function*

$$\kappa : [0, L] \rightarrow \mathbb{R}.$$

This representation is independent of the original definition of the curve and provides a common mathematical framework.

54.1.1. Normalization

Definition 54.2. A normalized transition is defined on

$$s \in [0, 1]$$

with

$$\kappa(0) = 0, \quad \kappa(1) = 1.$$

Normalization removes scale and allows direct comparison of transition families.

54.2. Classification by Functional Form

54.2.1. Linear Curvature: Clothoid

Definition 54.3.

$$\kappa(s) = as.$$

The clothoid is characterized by constant curvature derivative

$$\kappa'(s) = \text{const.}$$

This simplicity explains its widespread adoption in railway engineering.

54.2.2. Polynomial Curvature Families

Definition 54.4.

$$\kappa(s) = \sum_{i=0}^n a_i s^i.$$

Polynomial transitions allow explicit control of higher-order derivatives of curvature.

Such flexibility enables optimization with respect to dynamic criteria.

Klauder-type transitions belong to this class, incorporating dynamic considerations directly into the curvature design.

54.2.3. Geometric Families

Some transition curves are defined implicitly or through geometric relations rather than explicit curvature functions.

These curves must be transformed into curvature representation to be integrated into the present framework.

54.2.4. Engineering-Based Families

Engineering approaches often define transition curves through design rules or empirical considerations.

The Bloss transition represents a notable example of such a practical engineering compromise.

54.2.5. Optimized and Hybrid Families

Modern approaches introduce transition curves designed explicitly to optimize dynamic performance.

These include smoothed curvature transitions and jerk-optimized polynomial curves.

54.3. Classification by Higher-Order Behaviour

54.3.1. Curvature Derivatives

Definition 54.5.

$$\kappa'(s), \quad \kappa''(s)$$

denote the first and second derivatives of curvature.

These quantities determine dynamic behaviour, including acceleration variation and smoothness of force development.

54.3.2. Smoothness Classes

Definition 54.6. A transition is of class C^k if κ is k -times continuously differentiable.

Higher smoothness typically improves dynamic performance and reduces wear.

54.4. Comparison of Families

Family	κ	κ'	κ''
Clothoid	linear	constant	0
Polynomial	flexible	flexible	flexible
Engineering	implicit	implicit	implicit
Optimized	designed	controlled	controlled

54.5. Interpretation as Control Functions

Proposition 54.7. *Transition curves can be interpreted as control functions acting on system dynamics through curvature evolution.*

This interpretation links geometric design directly to dynamic performance.

54.6. Connection to Railway Engineering

Railway transition design is governed by dynamic constraints such as acceleration, jerk, and wear.

These constraints translate directly into requirements on

$$\kappa(s), \quad \kappa'(s), \quad \kappa''(s).$$

54.7. Main Insight

Theorem 54.8. *Transition curves are not merely geometric connectors, but control functions governing system behaviour.*

54.8. Conclusion

Summary 54.9. Transition curves form a unified class of curvature functions.

Their classification by functional form and higher-order behaviour provides a systematic framework for comparison, optimization, and integration into engineering systems.

This perspective establishes a direct link between geometry and system dynamics.

55. Schwerpunkt-Trassierung as a Qualified Model Idea

The idea in one page

The historical attraction of *Schwerpunkt-Trassierung* is easy to state. A track centreline is not the path of every dynamically relevant point on a vehicle. If an Alignment edit changes curvature, the motion predicted for a chosen vehicle reference point can change as well. Berlinish and axtranNew project thinking used this insight to look beyond isolated curve formulae; engineering and patent literature likewise relates alignment to vehicle centre-of-mass motion [13].

That origin is strong and useful, but it is not validation. The word *Schwerpunkt* can refer to several different things:

Use	Required qualification	Status here
physical centre of mass	particular vehicle and loading	physical property
model reference point P_M	model M , frame and parameters	predicted model quantity
Berlinish/axtranNew idea	source and historical purpose	project provenance
optimization aggregate	variables, objective and scaling	problem-specific construction
“centre” as emphasis	no mathematical object	metaphor only

This chapter investigates the second and fourth meanings. It neither creates a canonical *Schwerpunkt State* nor changes Alignment identity. The reader arrives here from the contract established in chapter 43:

model → objective → candidate → evaluation → Engineering Decision.

55.1. Historical proposition, present boundary

The original proposition was that transition design might be judged through a representative vehicle-related trajectory rather than through track geometry alone. Its originality lies in connecting constructive curvature, cant and vehicle-oriented consequence within one computational question. It remains a research and modelling proposition: no active Kernel

authority establishes a Schwerpunkt principle, one universal reference point, one response model or one objective.

The physically relevant centre of mass belongs to a particular loaded vehicle and changes with loading. A model reference point P_M belongs instead to an identified model M . It may coincide with an assumed centre of mass only under explicit rigid-body, loading and frame assumptions. A representative trajectory or weighted optimization aggregate need not coincide with either.

55.2. A deliberately reduced reference-point model

Let $\gamma(s)$ be a planar realization of the Alignment, with tangent $t(s)$, normal $n(s)$, heading $\theta(s)$ and curvature $\kappa(s)$:

$$t = (\cos \theta, \sin \theta), \quad n = (-\sin \theta, \cos \theta), \quad n' = -\kappa t.$$

Choose the illustrative model

$$M_{\text{off}} = (P_M, d, F_{\text{track}}, \text{rigid offset, planar realization}),$$

where d is a fixed signed offset. Its represented reference-point path is

$$\gamma_{P_M}(s) = \gamma(s) + d n(s).$$

This is a geometric output of M_{off} , not a measured vehicle trajectory and not automatically a physical centre-of-mass path. Cant, suspension, inertia, wheel–rail contact, load transfer, track flexibility and irregularities are omitted.

Differentiation with respect to station gives

$$\gamma'_{P_M} = (1 - d\kappa)t,$$

$$\gamma''_{P_M} = -d\kappa' t + (1 - d\kappa)\kappa n.$$

These are station derivatives. They become temporal kinematics only after a qualified traversal $s(t)$ is supplied:

$$\dot{\gamma}_{P_M} = v \gamma'_{P_M}, \quad \ddot{\gamma}_{P_M} = \dot{v} \gamma'_{P_M} + v^2 \gamma''_{P_M}.$$

Even then, they describe the prescribed point path of the reduced model, not a vehicle-body response solved from mechanics.

55.3. From model output to an optimization problem

An illustrative investigation can ask whether changing a transition reduces variation of this model path. It must first declare the problem:

Contract field	Declaration in the example
problem identity	reference-point path sensitivity for one transition
model	M_{off} with stated d and planar track frame
decision variables	selected transition parameters $x = (L, \alpha)$
fixed inputs	endpoint states, family, realization, d , traversal test
derived quantities	$\kappa, \gamma, \gamma_{P_M}$ and their derivatives
constraints	continuity, parameter bounds and declared engineering bounds
unresolved freedoms	cant and vehicle response unless explicitly added
provenance	Berlinish/axtranNew-inspired modelling experiment

One possible geometric sensitivity objective is

$$J_P(x) = \int_0^{L(x)} \left\| \gamma_{P_M}''(s; x) \right\|^2 ds.$$

Its source is the stated experiment; it measures a station-domain path property. It is not self-evidently a comfort, safety, wear, cost or energy objective.

If the reduced balance residual is also examined, a separate objective may be

$$J_r(x) = \int_0^{T(x)} r_{\text{bal}}(t; x)^2 dt, \quad r_{\text{bal}} = v^2 \kappa - g \sin \phi,$$

under the assumptions of chapter 40. The two objectives have different units and meanings. A combined objective

$$J = w_P J_P / J_{P,0} + w_r J_r / J_{r,0}$$

is defined only when a responsible source supplies the scales, weights, purpose and applicability. Neither the model nor the solver owns those choices.

55.4. Degrees of freedom and candidate status

In the bounded example, L and α are free within declared bounds. Endpoint states and the chosen family are fixed; coefficients and terminal pose are derived; continuity is constrained. Cant, traversal, d , vehicle parameters or realization data may be fixed, made variable, or left unresolved, but their role must be recorded. They do not become variables merely because the word *Schwerpunkt* is used.

A solver result is therefore

$$C_{\text{SP}} = (\mathcal{P}.id, M_{\text{off}}.id, x^*, J, \text{residuals}, \text{convergence}, \text{provenance}),$$

a calculation candidate. It remains subject to applicability evaluation against qualified

evidence, rules, lifecycle and purpose. Selection or rejection requires an Engineering Decision with authority.

calculated $\neq$ applicable $\neq$ accepted $\neq$ authoritative

The current continuity solver supports only the bounded continuity claim already stated in chapter 43. This chapter does not claim current AXTRAN vehicle, cant, comfort, network or automatic engineering-selection optimization.

55.5. Scientific potential without canonization

The model idea survives with qualification. More capable work may replace M_{off} with an identified rigid-body or multibody model, calibrate parameters from qualified evidence, propagate observation and model-form uncertainty, or compare several vehicle reference points. Each extension must state its subject, model, frame, inputs, validity and purpose.

The historical metaphor remains productive because it asks a better downstream question: what consequence does an Alignment construction produce for a specified vehicle-related model? The answer remains model-specific evidence. It does not redefine Alignment, establish universal validity or decide what engineering shall accept.

Part X.

System and Pipeline

Analysis

This part establishes how AIM operates as a coherent processing system rather than as a disconnected set of models.

What do we already know?

The previous parts established the conceptual, geometric, operational, realization-aware, and modelling foundations of AIM.

Railway alignments can now be represented through sparse semantic models, reconstructed through runtime services, interpreted through state operators, and related to physical reality through realization concepts.

The individual building blocks of the framework therefore exist.

However, a collection of components does not yet constitute a complete system.

The system also has a development lineage that clarifies why these components exist. Berlinish began as an originating research grammar for transition functions. axtranNew was intended to turn that grammar into solvable alignment calculations. transitionDB and TransEd address preservation and inspection of the reusable function knowledge. SPOT and alignmentOS address concrete object identity, state, editing, and operational use. The Knowledge Kernel provides the stable terminology and responsibility boundaries that the originating calculation work did not by itself supply. Research challenges and extends claims; the Thesis explains, relates, and transfers them.

These responsibilities form an execution and knowledge chain. They must not be read as a claim that every named component is a canonical Kernel concept or a complete implementation.

What is still missing?

Practical railway information modelling requires more than isolated models and operators.

Data must move through processing chains.

States must be transformed.

Services must interact.

Failures must be detected.

Results must be generated from intermediate representations.

The missing element is therefore the processing architecture that connects individual concepts into a coherent operational framework.

AIM must be understood not only as a collection of models, but as a pipeline of interacting transformations.

What comes next?

This part introduces the system-level interpretation of AIM.

The focus shifts from individual concepts toward the interaction between concepts.

The chapters that follow investigate:

- transformation pipelines,
- runtime services,
- operator chains,
- system interactions,
- failure modes,
- and architectural design principles.

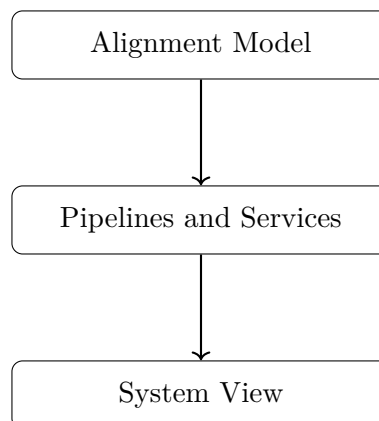
The objective is to explain how railway information flows through the AIM framework from acquisition to evaluation, realization, and optimization.

This perspective ultimately reveals AIM as an operator-based railway information system rather than a mere collection of geometric methods.

Relation to the AIM Roadmap

Within the AIM roadmap, the present part corresponds to the transition from alignment models toward processing pipelines and system services.

The roadmap sketch below answers the guiding question: how does the argument progress from model structure to operational system behaviour?



The progression

Alignment Model → Pipelines and Services → System View

marks the transition from static railway representations toward operational processing architectures.

This system perspective prepares the subsequent treatment of optimization, engineering workflows, and practical deployment.

Principle 55.1 (Pipeline Principle). Within AIM, railway information is processed through explicit operator pipelines in which states, models, realizations, and evaluations are connected by well-defined transformations.

Summary 55.2. Modelling explains how railway information is represented.

Analysis explains how railway information is processed.

The present part therefore establishes the system architecture that connects models, operators, services, and transformations into a coherent AIM framework.

This system-level interpretation prepares the Optimization part, where pipeline outputs are used for structured improvement and decision support.

55.6. AIM Processing Pipeline

Motivation

Railway alignment modelling involves multiple domains:

- global coordinate systems (CRS),
- geometric alignment representation,
- physical realization,
- measurement data,
- optimization.

These domains are often treated separately, leading to fragmented models and inconsistent results.

Within the AIM framework, these components are unified into a single processing pipeline.

55.6.1. Operator Chain

Definition 55.3 (AIM pipeline). The complete modelling and optimization process is described as a composition of operators:

$$\mathcal{P} = \mathcal{W}^{-1} \circ \mathcal{R} \circ \mathcal{M} \circ \mathcal{W} \circ \mathcal{C}$$

The operators are:

- $\mathcal{C}$: CRS transformation,
- $\mathcal{W}$: world-to-track transformation,
- $\mathcal{M}$: alignment model (parametric or hybrid),
- $\mathcal{R}$: realization operator,
- $\mathcal{W}^{-1}$: track-to-world transformation.

55.6.2. Detailed Structure

The pipeline maps raw data to realized geometry:

$$\text{CRS data} \xrightarrow{\mathcal{C}} \text{world coordinates} \xrightarrow{\mathcal{W}} (s, d) \xrightarrow{\mathcal{M}} \kappa(s) \xrightarrow{\mathcal{R}} \gamma_{\text{real}}(s) \xrightarrow{\mathcal{W}^{-1}} \text{world geometry}.$$

55.6.3. Hybrid Model Integration

The alignment model operator is hybrid:

$$\mathcal{M} : (\mathbf{T}, \delta\kappa) \mapsto \kappa(s).$$

Thus, the pipeline operates on both:

- structured parameters,
- local corrections.

55.6.4. Inverse Problem

Definition 55.4 (Pipeline inversion). Given observed data x_i , the inverse problem is:

$$\min_{\mathbf{x}} \sum_i \|\mathcal{P}(\mathbf{x})(s_i) - x_i\|^2.$$

This corresponds to fitting a model such that the realized geometry matches observations.

55.6.5. Optimization Embedding

The pipeline is embedded into an optimization problem:

$$\min_{\mathbf{x}} J(\mathcal{P}(\mathbf{x})).$$

The objective is evaluated on the realized geometry, not on the parametric model.

55.6.6. Jacobian Structure

The derivative of the pipeline follows from the chain rule:

$$\frac{\partial \mathcal{P}}{\partial \mathbf{x}} = D\mathcal{W}^{-1} \cdot D\mathcal{R} \cdot D\mathcal{M} \cdot D\mathcal{W}.$$

Each component has specific properties:

- $D\mathcal{M}$: sparse and structured,
- $D\mathcal{R}$: smoothing operator,
- $D\mathcal{W}$: nonlinear projection,
- $D\mathcal{C}$: linear transformation.

55.6.7. Computational Implications

The pipeline exhibits:

- sparsity (model level),
- nonlinearity (projection level),
- smoothing (realization level),
- scaling effects (CRS level).

Efficient implementation requires:

- hybrid representations,
- block-structured solvers,
- LOD-dependent evaluation.

55.6.8. Level-of-Detail Interpretation

Different parts of the pipeline dominate at different scales:

- large scale: CRS and world geometry,
- medium scale: parametric alignment,
- small scale: corrections and realization.

55.6.9. Key Insight

Proposition 55.5. *Alignment modelling is not a single mapping, but a composition of geometric, physical, and computational operators.*

55.6.10. Connection to AIM

Summary 55.6. The AIM framework is fundamentally defined by its operator pipeline.

It integrates coordinate systems, geometry, physics, and optimization into a single coherent model.

This unified perspective enables:

- consistent data integration,
- physically meaningful optimization,
- scalable computation.

Thus, AIM transforms alignment modelling from a collection of isolated methods into a structured computational system.

55.7. AIM Pipeline Overview

This section answers a system-level question: which operator sequence connects CRS context, intrinsic modelling, realization, and feedback without breaking conceptual ownership across layers?

The figure is introduced as this operator-sequencing answer.

Interpreted in chapter context, the diagram establishes a forward processing spine with two explicit feedback channels: measurement data stabilizes projection interpretation, and optimization feedback updates the curvature model.

The engineering consequence is that pipeline correctness depends on operator composition discipline, not on isolated local computations. This prepares the following sections, where pipeline behaviour is discussed through interaction patterns and failure modes.

55.8. AIM Master Architecture

This section addresses the architectural integration question of the Analysis part: how are modelling, runtime evaluation, realization, and feedback mechanisms composed into one coherent operator system?

The figure below is read as the consolidated architecture answer.

The interpretation is structural: each layer preserves a distinct responsibility, while feedback loops connect evidence and improvement without relocating conceptual ownership.

The engineering consequence is that AIM behaves as a governed transformation architecture whose stability depends on maintaining these layer and feedback contracts in implementation and workflow design.

55.9. Pipeline Story: From Measurement to Alignment

55.9.1. A Single Point

Consider a single measured point x in the real world.

It may originate from:

- a surveying campaign,
- a track recording vehicle,
- or a design dataset.

At this stage, the point exists only in a global coordinate system.

55.9.2. Step 1: CRS Transformation

The point is first interpreted within a coordinate reference system:

$$x_{\text{crs}} \xrightarrow{\mathcal{C}} x_{\text{world}}.$$

This step ensures that all data is expressed in a consistent spatial frame.

55.9.3. Step 2: Projection onto the Track

The point is projected onto the alignment:

$$x_{\text{world}} \xrightarrow{\mathcal{W}} (s, d).$$

This is a nonlinear operation:

- the longitudinal position s is determined,
- the lateral deviation d is computed.

At this moment, the point becomes part of the intrinsic alignment coordinate system.

55.9.4. Step 3: Interpretation through the Model

The alignment model provides a curvature description:

$$(s, d) \xrightarrow{\mathcal{M}} \kappa(s).$$

Here, two components interact:

- the parametric structure (design intent),
- local corrections (data-driven adjustments).

The point is no longer treated as isolated data, but as part of a structured geometric system.

55.9.5. Step 4: Physical Realization

The theoretical model is transformed by physical processes:

$$\kappa(s) \xrightarrow{\mathcal{R}} \gamma_{\text{real}}(s).$$

This step reflects:

- construction constraints,
- mechanical behaviour,
- maintenance operations.

The resulting geometry is not identical to the theoretical model.

55.9.6. Step 5: Back to World Coordinates

The realized alignment is mapped back into world space:

$$\gamma_{\text{real}}(s) \xrightarrow{\mathcal{W}^{-1}} x_{\text{real}}.$$

This produces the geometry that is visible and measurable in reality.

55.9.7. Step 6: Comparison and Feedback

The original measurement and the realized position are compared:

$$x_{\text{real}} \approx x_{\text{world}}.$$

The deviation drives optimization:

- parameters are adjusted,
- corrections are updated,
- the pipeline is evaluated again.

55.9.8. Interpretation

The point has passed through multiple representations:

- global coordinates,
- intrinsic alignment coordinates,
- parametric model,
- realized geometry.

Each transformation introduces structure, constraints, and interpretation.

55.9.9. Key Insight

Proposition 55.7. *A measured point is not directly part of the alignment.*

It becomes part of the alignment only after passing through the world-to-track transformation and the alignment model.

55.9.10. Connection to AIM

Summary 55.8. The AIM framework transforms raw spatial data into structured alignment information through a sequence of operators.

This process integrates:

- coordinate systems,
- geometric modelling,
- physical realization,
- optimization.

Thus, alignment modelling is not a static representation, but a dynamic process linking measurement, theory, and construction.

55.10. Failure Modes and Limitations of the AIM Pipeline

Motivation

Any modelling and optimization framework must account not only for its capabilities, but also for its limitations.

Within the AIM pipeline, failure modes arise from the interaction of:

- coordinate systems,
- geometric transformations,
- physical realization,
- numerical optimization.

55.10.1. CRS-Related Errors

Errors in coordinate reference systems include:

- incorrect CRS identification,
- inconsistent transformations,
- mixed coordinate systems within datasets.

Proposition 55.9. *CRS inconsistencies lead to systematic spatial distortions that cannot be corrected by local alignment optimization.*

55.10.2. Projection Errors (World-to-Track)

The world-to-track transformation may fail due to:

- ambiguous nearest points,
- high curvature regions,
- sparse or noisy alignment representations.

Proposition 55.10. *Projection errors introduce nonlocal distortions in the intrinsic coordinate s , affecting all downstream computations.*

55.10.3. Model Mis-Specification

The parametric model may fail to represent the true alignment due to:

- insufficient transition families,
- incorrect parameterization,
- overly restrictive structure.

Hybrid correction terms mitigate but do not eliminate this issue.

55.10.4. Overfitting vs. Underfitting

Hybrid models introduce a trade-off:

- underfitting: parametric model too rigid,
- overfitting: correction terms capture noise.

Proposition 55.11. *Regularization is required to balance model fidelity and stability.*

55.10.5. Realization Effects

The realization operator introduces:

- smoothing of curvature,
- loss of high-frequency information,
- dependence on construction processes.

Proposition 55.12. *Certain features of the target alignment are not physically realizable and will be systematically suppressed.*

55.10.6. Inverse Problem Instability

The inverse realization problem is ill-posed:

- multiple target alignments yield similar realized geometry,
- small changes in data may cause large parameter variations.

Proposition 55.13. *Regularization and prior knowledge are essential for stable solutions.*

55.10.7. Numerical Optimization Issues

Optimization may fail due to:

- poor initial guesses,
- nonconvex objective functions,
- ill-conditioned Jacobians.

Despite structured sparsity, convergence is not guaranteed.

55.10.8. Block Structure Degeneracy

The block structure may become degenerate if:

- coupling terms dominate,
- insufficient data separates variables,
- parameters become redundant.

Proposition 55.14. *Loss of block separability reduces computational efficiency and interpretability.*

55.10.9. Measurement Noise and Bias

Measurement data may contain:

- random noise,
- systematic bias,
- missing or inconsistent segments.

Proposition 55.15. *Noise propagates through the pipeline and may be amplified by inverse operations.*

55.10.10. Scale-Dependent Failures

Different failure modes dominate at different scales:

- large scale: CRS and projection errors,
- medium scale: model mis-specification,
- small scale: realization and measurement noise.

55.10.11. Non-Uniqueness of Solutions

Different alignment models may produce nearly identical realized geometry.

Proposition 55.16. *The mapping*

$$\kappa_{\text{target}} \mapsto \gamma_{\text{real}}$$

is not injective.

55.10.12. Practical Engineering Limits

Certain effects are outside the scope of the model:

- material-dependent behaviour,
- long-term degradation,
- construction variability.

These factors must be treated as external constraints.

55.10.13. Key Insight

Proposition 55.17. *The AIM pipeline is inherently approximate and limited by both physical and computational constraints.*

55.10.14. Connection to AIM

Summary 55.18. Failure modes in the AIM pipeline arise from the interaction of geometry, physics, data, and computation.

Understanding these limitations is essential for:

- robust model design,
- stable optimization,
- reliable engineering application.

Thus, failure analysis is not an auxiliary component, but an integral part of the AIM framework.

55.11. Design Principles of Alignment-Based Information Modelling

Motivation

The AIM framework integrates geometry, coordinate systems, realization, and optimization into a unified model.

From this integration, a set of fundamental design principles emerges.

55.11.1. Principle 1: Curvature-Centric Modelling

Proposition 55.19. *Railway alignments should be modelled primarily through curvature functions $\kappa(s)$, not through point-based geometry.*

Curvature provides:

- intrinsic representation,
- direct link to dynamics,
- natural integration with optimization.

55.11.2. Principle 2: Separation of Representation and Realization

Proposition 55.20. *The theoretical **alignment** model must be separated from its physical realization.*

The realization operator introduces:

- smoothing,
- constraints,
- deviations from the ideal model.

Ignoring this separation leads to systematic modelling errors.

55.11.3. Principle 3: Hybrid Modelling

Proposition 55.21. *Alignment models should combine parametric structure with local correction terms.*

This allows:

- interpretable design intent,
- robust handling of imperfect data,
- flexible refinement.

55.11.4. Principle 4: Operator-Based Architecture

Proposition 55.22. *Alignment modelling should be formulated as a composition of operators:*

$$\mathcal{P} = \mathcal{W}^{-1} \circ \mathcal{R} \circ \mathcal{M} \circ \mathcal{W} \circ \mathcal{C}.$$

Each operator represents a distinct domain:

- coordinate systems,
- geometric modelling,
- physical processes,
- data transformation.

55.11.5. Principle 5: Design in Realized Space

Proposition 55.23. *Optimization objectives must be evaluated on realized geometry, not on theoretical models.*

The relevant mapping is:

$$\kappa_{\text{target}} \mapsto \gamma_{\text{real}}.$$

Design must account for the transformation induced by realization.

55.11.6. Principle 6: Exploitation of Structure

Proposition 55.24. *The intrinsic block and sparsity structure of the problem must be exploited for efficient computation.*

This includes:

- locality in arc-length,
- block decomposition (curvature vs. cant),
- sparse Jacobians.

55.11.7. Principle 7: Scale Awareness

Proposition 55.25. *Different components of the pipeline must be evaluated at appropriate scales.*

- CRS dominates at large scale,
- parametric modelling at medium scale,
- corrections and realization at small scale.

55.11.8. Principle 8: Measurement Integration

Proposition 55.26. *Measured data must be integrated through the world-to-track transformation, not directly in world space.*

This ensures consistency with the intrinsic alignment representation.

55.11.9. Principle 9: Regularization as Necessity

Proposition 55.27. *Regularization is an essential component of alignment modelling and optimization.*

It stabilizes:

- inverse problems,
- hybrid corrections,
- numerical optimization.

55.11.10. Principle 10: Acceptance of Non-Uniqueness

Proposition 55.28. *Alignment solutions are not unique and must be evaluated based on engineering criteria.*

Different models may produce nearly identical realized geometry.

Thus, solution quality must be defined through:

- dynamics,
- constraints,
- robustness.

55.11.11. Summary

Summary 55.29. The AIM framework is governed by a set of design principles that emerge from the interaction of geometry, physics, data, and computation.

These principles define:

- how alignments should be represented,
- how they should be transformed,
- how they should be optimized,
- and how their limitations must be handled.

Together, they provide a coherent foundation for alignment-based information modelling as a unified engineering and computational discipline.

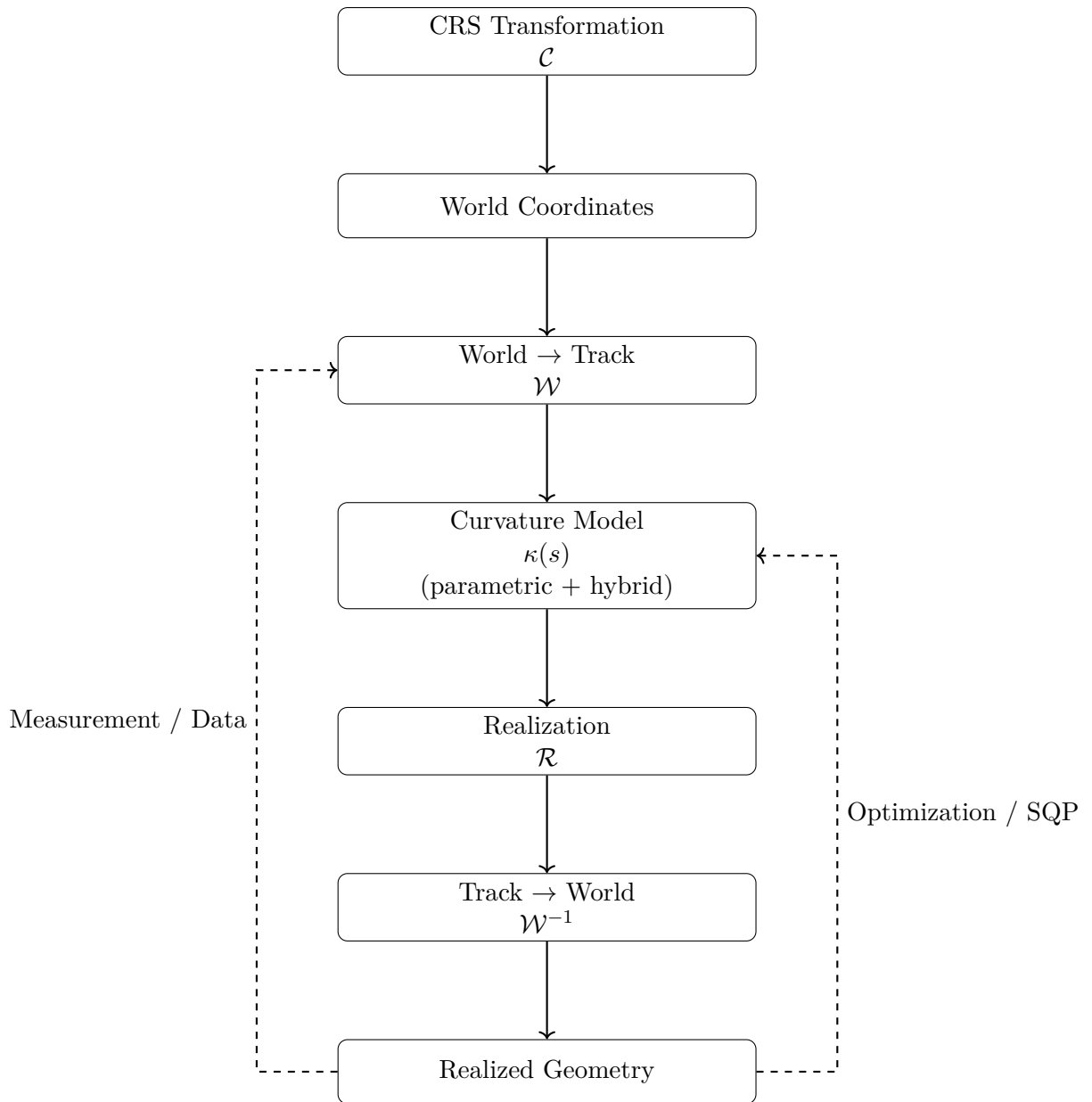


Figure 55.1.: AIM processing pipeline as composition of operators linking coordinate systems, alignment modelling, realization, and optimization.

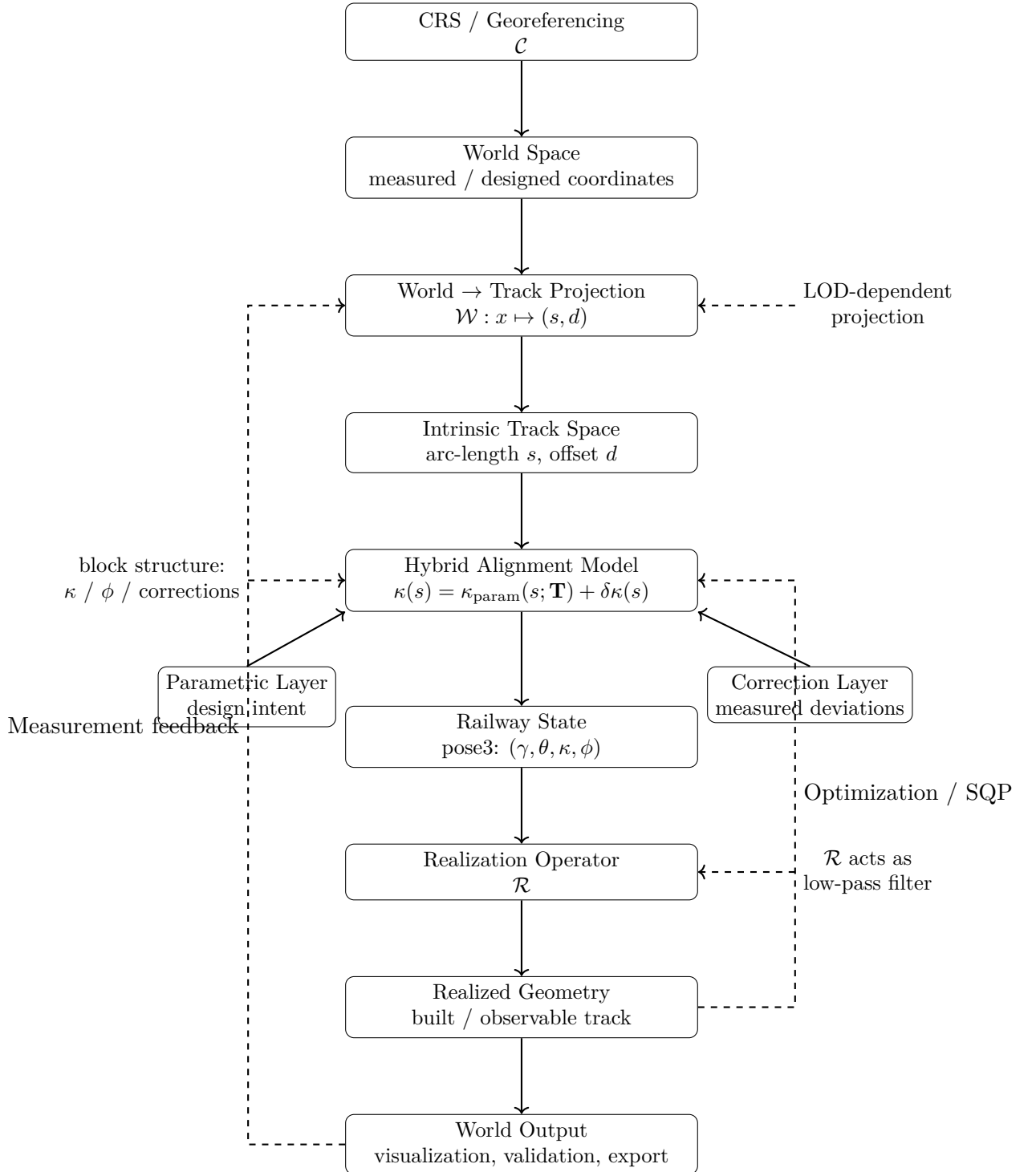


Figure 55.2.: Level-2 AIM master architecture. The framework combines CRS handling, nonlinear world-to-track projection, hybrid curvature modelling, pose3 state reconstruction, realization, and optimization feedback. The central modelling object is not the geometric curve itself, but the curvature-driven and realization-aware operator system producing it.

Part XI.

Optimization

Optimization

Reader abstract

Optimization helps when an engineering problem admits more than one possible change. It searches declared freedoms for candidates that satisfy declared constraints and compare well under a declared objective. It cannot supply the purpose of that objective, establish the applicability of its inputs, accept a candidate, or make an Engineering Decision.

This part follows one example. A survey polyline and a CAD curve disagree in a transition region. The Alignment identity is fixed. The engineer permits a bounded edit of selected intrinsic curvature parameters, while the endpoints, end directions, project context, and source observations remain fixed. The question is not “Which curve is true?” but:

Which curvature edit is a reviewable candidate for this stated purpose, under these constraints, assumptions, and observations?

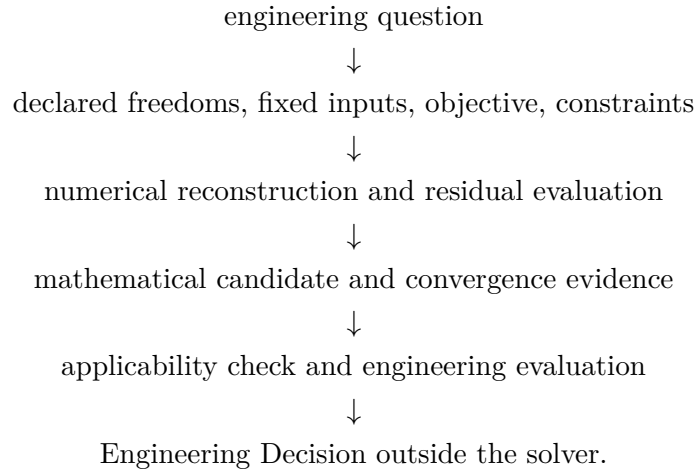
The contract before the calculation

An optimization problem is meaningful only after its responsibilities have been declared. For the running example, let

$$\mathcal{P} = (I, P, M, x, q, \mathcal{F}, J, \Sigma),$$

where I identifies the Engineering Object, P states the purpose, M is the reconstruction and evaluation model, x contains the free variables, q contains fixed inputs and context, $\mathcal{F}$ is the feasible set, J is the objective with units and scaling, and Σ states the uncertainty boundary. None of these entries may be inferred from a solver result.

The reader path now follows the dependencies of this contract:



Why the technical chapters follow in this order

Numerical reconstruction appears first because every trial curvature edit must be turned into geometry before endpoint and observation residuals can be evaluated. Alignment optimization then states the continuous and discrete problem and introduces SQP, derivatives, and sparsity only when the growing problem makes them necessary. The architecture chapter describes the responsibility boundaries of a possible solver service and distinguishes them from the limited experimental AXTRAN evidence. The final specialization asks how a declared realization model may enter the evaluation without collapsing model, realization, observation, or decision.

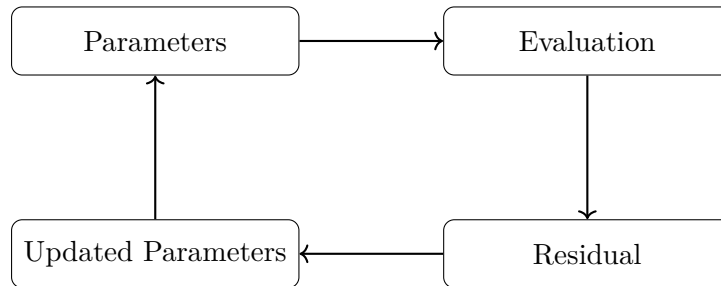


Figure 55.3.: Optimization produces a revised candidate and evidence for review; the loop does not accept the candidate.

Principle 55.30 (Optimization Principle). An AIM optimization formulation shall declare its Engineering Object and purpose, freedoms and fixed inputs, model, objective and scaling, constraints, uncertainty boundary, and provenance. Its numerical output is a candidate with residual and convergence evidence, not an Engineering Decision.

Summary 55.31. Optimization does not turn evaluation into authority. It makes a declared engineering question computationally searchable and returns candidates that can be inspected, compared, and evaluated.

calculated $\neq$ applicable $\neq$ accepted $\neq$ authoritative

56. Numerical Reconstruction for a Reviewable Candidate

56.1. Why a curvature edit needs numerics

In the running example, the free variables change selected coefficients of an intrinsic curvature law. Yet the fixed endpoint, direction, and survey observations are spatial quantities. Each trial vector x must therefore be reconstructed before the problem can say how well it fits.

Let

$$\kappa_x : [0, L] \rightarrow \mathbb{R}$$

be the candidate curvature, and let the fixed start pose be (γ_0, θ_0) . The model M produces

$$\theta'_x(s) = \kappa_x(s), \quad \gamma'_x(s) = (\cos \theta_x(s), \sin \theta_x(s)).$$

This reconstruction produces a representation used for evaluation. It neither changes Alignment identity nor turns the represented curve into an accepted engineering state.

56.2. From a continuous model to computable samples

Reader bridge. The equations state the dependency exactly, but a solver must evaluate it finitely and repeatedly. Discretization is the declared approximation that makes this possible.

Choose

$$0 = s_0 < s_1 < \cdots < s_N = L, \quad \Delta s_i = s_{i+1} - s_i.$$

For example, a midpoint step gives

$$\begin{aligned} \theta_{i+\frac{1}{2}} &= \theta_i + \frac{1}{2} \kappa_x(s_i) \Delta s_i, \\ \gamma_{i+1} &= \gamma_i + \Delta s_i (\cos \theta_{i+\frac{1}{2}}, \sin \theta_{i+\frac{1}{2}}), \quad \theta_{i+1} = \theta_i + \kappa_x(s_i) \Delta s_i. \end{aligned}$$

Other integration schemes may be appropriate. The method, tolerance, and sampling rule are part of M ; they are not invisible implementation detail.

56.3. Residuals answer declared questions

For the fixed target end pose $(\bar{\gamma}_L, \bar{\theta}_L)$, a model may produce the endpoint residual

$$r_{\text{end}}(x) = \begin{bmatrix} \gamma_x(L) - \bar{\gamma}_L \\ \text{wrap}(\theta_x(L) - \bar{\theta}_L) \end{bmatrix}.$$

For survey observations y_j , an observation model h_j may produce

$$r_{\text{obs},j}(x) = h_j(\gamma_x) - y_j.$$

These residuals answer different questions and can have different units, uncertainties, and responsibilities. A constraint residual records feasibility; an observation residual records discrepancy under an observation model. They must not be silently pooled.

If residual families enter one least-squares objective, scaling must be declared, for example

$$J(x) = \frac{1}{2} \|W_{\text{end}} r_{\text{end}}(x)\|_2^2 + \frac{1}{2} \sum_j \|W_{\text{obs},j} r_{\text{obs},j}(x)\|_2^2 + J_{\text{reg}}(x).$$

The matrices W make units and relative influence explicit; their values come from the stated purpose, uncertainty model, or responsible engineering choice, not from the numerical method.

56.4. Accuracy belongs to the result evidence

Reader bridge. A candidate may appear feasible because of a coarse grid rather than because the underlying continuous model satisfies the constraint. Numerical error must therefore travel with the result.

Relevant contributions include:

- integration and discretization error;
- interpolation or quadrature error in observation residuals;
- floating-point and linear-solve error;
- sensitivity of reconstructed position to curvature perturbations;
- model-form and observation uncertainty, which numerical refinement cannot remove.

A defensible run records the method and mesh, tolerances, refinement checks, and residuals recomputed on an independent or finer evaluation grid. A floating origin or local metric frame can reduce numerical loss for large coordinate values, but such a representation change does not alter the intrinsic Alignment.

56.5. Why derivatives and sparsity appear next

Repeated reconstruction makes the dependence

$$x \longmapsto \gamma_x \longmapsto r(x) \longmapsto J(x)$$

computationally visible. Derivatives quantify how a permitted parameter change affects the residuals. Sparsity records that a local curvature parameter usually affects only structured portions of the assembled problem. Both are responses to cost and traceability needs; neither selects the engineering objective.

The numerical chapter has therefore supplied the evaluator required by the optimization problem:

$$(x; M, q) \longmapsto (\hat{\gamma}_x, r(x), J(x), \varepsilon_{\text{num}}).$$

Its output is evidence about a trial, not a decision about the Alignment.

Summary 56.1. Numerical reconstruction makes a declared curvature edit evaluable. Its discretization, scaling, tolerances, and error estimates remain visible so that an apparently good solver result can be reviewed. The next chapter can now ask how to search the freedoms efficiently without changing who owns the problem or the decision.

57. From an Alignment Problem to a Mathematical Candidate

Reader bridge: the question carried into the mathematics

The numerical evaluator can now reconstruct and test any permitted curvature edit. The remaining task is to search those edits without forgetting the engineering declaration that gave the numbers meaning.

For the CAD-curve/survey-polyline discrepancy, the chapter bundle uses the following fixed reading contract:

Engineering Object	the identified Alignment; identity is preserved
Purpose	obtain candidates for reconciling a declared transition-region discrepancy
Free variables	selected intrinsic curvature parameters and only those explicitly released
Fixed inputs	start/end pose, locked elements, metric and observation context, source data
Constraints	continuity, fixed boundary conditions, declared bounds and applicability conditions
Objective	declared, unit-aware residual terms with recorded scaling and source
Output	candidate parameters, reconstructed representation, residuals, convergence and numerical evidence

This contract is an illustrative formulation, not a universal AIM objective.

The route through the deep dive

The following sections are cumulative. First, the engineering question is written as a continuous curvature problem. Second, discretization exposes a finite vector of freedoms and constraints. Third, SQP appears because the reconstruction makes the problem nonlinear while equality and inequality conditions must remain visible. Initial guesses matter because local methods do not erase non-convexity. Finally, analytic derivatives, sparsity, and hybrid parameterizations appear because they reduce computational work and make influence traceable.

Motivation

In practical applications, alignments are not only reconstructed from data, but also designed, modified, and improved.

This leads naturally to optimization problems in which alignment parameters are adjusted to satisfy geometric, physical, and regulatory constraints, as already emphasized in railway alignment literature [32, 26].

57.1. General Optimization Problem

Definition 57.1 (Alignment optimization problem). An alignment optimization problem consists of:

- a parameterized alignment model,
- an objective function,
- a set of constraints.

Definition 57.2 (Objective function). An objective function is a mapping

$$J : \mathcal{A} \rightarrow \mathbb{R}$$

that assigns a cost or quality measure to an alignment $\mathcal{A}$.

Typical objectives include:

- minimizing curvature variation,
- minimizing deviation from measurement data,
- minimizing construction cost,
- maximizing comfort.

Such objective classes are consistent with both general numerical optimization literature [38] and railway-specific optimization approaches [32].

57.2. Parameterization

The choice of parameters is central to the formulation of the optimization problem.

Definition 57.3 (Parameter vector). An alignment is described by a parameter vector

$$\mathbf{x} \in \mathbb{R}^n$$

encoding quantities such as:

- element lengths,
- curvature values,
- transition parameters,
- geometric constraints.

The sparse alignment model provides a natural parameterization by element-wise variables.

57.3. Constraints

Definition 57.4 (Constraint). A constraint is a condition of the form

$$g_i(\mathbf{x}) \leq 0, \quad h_j(\mathbf{x}) = 0.$$

Constraints arise from:

- geometric continuity,
- engineering design rules,
- physical limitations,
- regulatory requirements.

57.3.1. Geometric Constraints

- continuity of position and direction,
- continuity of curvature.

57.3.2. Engineering Constraints

- minimum radius,
- maximum cant,
- limits on curvature derivatives.

Such conditions reflect standard railway design practice and are closely related to design rules discussed in engineering literature and standards [26, 7].

57.4. Relation to Numerical Reconstruction

Evaluation of the objective function requires reconstruction of the alignment geometry from the parameter vector.

Thus, optimization is intrinsically linked to numerical reconstruction, as discussed in the previous chapter. From a numerical perspective, this places alignment optimization within the broader context of differential equations and numerical approximation [12, 44].

57.5. Sequential Quadratic Programming

Sequential Quadratic Programming (SQP) is a powerful method for solving nonlinear constrained optimization problems [38].

Definition 57.5 (SQP iteration). An SQP method approximates the original problem by a sequence of quadratic subproblems.

Each iteration solves a quadratic approximation of the objective function subject to linearized constraints.

57.6. Alignment Optimization as Control Problem

The curvature function $\kappa(s)$ may be interpreted as a control variable in a continuous system.

Definition 57.6 (Control formulation). The alignment problem can be formulated as an optimal control problem with state variables (γ, θ) and control κ .

This viewpoint is compatible with the curvature-centred perspective on railway alignment advocated by Kufver [33, 32].

57.7. Network-Level Optimization

Real-world railway systems involve not only single alignments, but networks with branching and merging structures.

57.8. Robustness and Uncertainty

Optimization must account for uncertainty in input data.

57.9. Implementation Perspective

Practical optimization systems must balance:

- numerical stability,

- computational efficiency,
- model expressiveness.

This balance is central both in generic numerical optimization [38] and in railway-specific optimization settings [32].

57.10. Historical Context

Classical systems such as AXTRAN have demonstrated the feasibility of optimization-based alignment design.

Modern approaches aim to extend these ideas using more flexible models and improved numerical methods, in line with the development from classical railway design methods toward explicit optimization formulations [32].

Reader bridge to the continuous formulation. The purpose and responsibilities are now fixed. The next section expresses smoothness or fit over the intrinsic longitudinal domain; it does not yet select a numerical algorithm.

57.11. Formal Optimization in Curvature Space

57.11.1. Problem Setting

Let $L > 0$ be fixed and let

$$\kappa : [0, L] \rightarrow \mathbb{R}$$

denote the curvature function of a transition.

We assume the boundary conditions

$$\kappa(0) = 0, \quad \kappa(L) = \kappa_1,$$

and optionally

$$\kappa'(0) = 0, \quad \kappa'(L) = 0.$$

57.11.2. Admissible Set

Definition 57.7 (Admissible transition set).

$$\mathcal{A} = \left\{ \kappa \in H^1(0, L) \mid \kappa(0) = 0, \kappa(L) = \kappa_1 \right\}.$$

57.11.3. Variational Problem

Definition 57.8 (Quadratic curvature-gradient functional).

$$J[\kappa] = \int_0^L (\kappa'(s))^2 \, ds.$$

Definition 57.9 (Optimization problem). Find $\kappa^* \in \mathcal{A}$ such that

$$J[\kappa^*] = \min_{\kappa \in \mathcal{A}} J[\kappa].$$

57.11.4. Euler–Lagrange Equation

Proposition 57.10. *If κ^* is a minimizer, then*

$$\kappa^{*''}(s) = 0.$$

57.11.5. Interpretation

Proposition 57.11. *The minimizer is*

$$\kappa^*(s) = \frac{\kappa_1}{L} s.$$

Thus, the clothoid appears as the solution of a variational problem. This connects classical railway transition design with an optimization view in curvature space and provides a formal counterpart to the historical role of the clothoid in railway engineering [10, 14].

57.11.6. Higher-Order Model

Definition 57.12.

$$J_2[\kappa] = \int_0^L (\kappa''(s))^2 \, ds.$$

Proposition 57.13. *Minimizers satisfy*

$$\kappa^{(4)}(s) = 0.$$

This explains the occurrence of cubic transition laws and links them to modern polynomial transition design [55].

57.11.7. Generalized View

General functionals of the form

$$J[\kappa] = \int_0^L F(\kappa, \kappa', \kappa''; v, c) \, ds$$

allow incorporation of dynamic and engineering criteria.

57.11.8. Connection to the Main Thesis

Summary 57.14. Transition design can be formulated as an optimization problem in curvature space.

Classical transition curves arise as solutions of such problems, supporting the interpretation of transitions as curvature control functions.

57.12. Coupled Optimization of Curvature and Cant

57.12.1. Motivation

In railway applications, curvature and cant cannot be optimized independently. Both jointly determine the effective lateral acceleration and its rate of change.

This motivates a coupled optimization problem in which both

$$\kappa(s) \quad \text{and} \quad \phi(s)$$

are treated as design variables.

Such coupling is fully consistent with railway alignment design standards, where curvature, speed, and cant are not independent design quantities [7, 26].

57.12.2. Effective Acceleration Functional

Definition 57.15 (Effective lateral acceleration). The effective lateral acceleration is

$$a_{\text{eff}}(s) = v^2 \kappa(s) - g \sin \phi(s).$$

A natural optimization objective is to reduce the magnitude of unbalanced lateral acceleration.

Definition 57.16 (Acceleration-based functional). Define

$$J_a[\kappa, \phi] = \int_0^L a_{\text{eff}}(s)^2 ds = \int_0^L (v^2 \kappa(s) - g \sin \phi(s))^2 ds.$$

57.12.3. Jerk-Based Functional

Definition 57.17 (Jerk). The jerk is given by

$$j(s) = v^3 \kappa'(s) - gv \cos \phi(s) \phi'(s).$$

Definition 57.18 (Jerk functional). Define

$$J_j[\kappa, \phi] = \int_0^L j(s)^2 ds.$$

That is,

$$J_j[\kappa, \phi] = \int_0^L (v^3 \kappa'(s) - gv \cos \phi(s) \phi'(s))^2 ds.$$

This functional penalizes rapid variation of effective acceleration and therefore provides a natural dynamic smoothness criterion. Such criteria are closely related to engineering concerns about comfort and dynamic compatibility [7, 4].

57.12.4. Combined Objective

Definition 57.19 (Coupled objective functional). A general coupled objective may be written as

$$J[\kappa, \phi] = \alpha J_a[\kappa, \phi] + \beta J_j[\kappa, \phi] + \gamma J_\kappa[\kappa] + \delta J_\phi[\phi],$$

where:

$$J_\kappa[\kappa] = \int_0^L (\kappa'(s))^2 ds, \quad J_\phi[\phi] = \int_0^L (\phi'(s))^2 ds.$$

The weights

$$\alpha, \beta, \gamma, \delta \geq 0$$

determine the relative importance of equilibrium, comfort, curvature smoothness, and cant smoothness.

57.12.5. Admissible Set

Definition 57.20 (Admissible railway state set). Let

$$\mathcal{B} = \left\{ (\kappa, \phi) \mid \kappa \in H^1(0, L), \phi \in H^1(0, L), \kappa(0) = 0, \kappa(L) = \kappa_1 \right\}.$$

Additional boundary conditions may be imposed, for example:

$$\phi(0) = 0, \quad \phi(L) = \phi_1,$$

or derivative constraints such as

$$\kappa'(0) = \kappa'(L) = 0, \quad \phi'(0) = \phi'(L) = 0.$$

57.12.6. Coupled Optimization Problem

Definition 57.21 (Coupled railway optimization problem). Find

$$(\kappa^*, \phi^*) \in \mathcal{B}$$

such that

$$J[\kappa^*, \phi^*] = \min_{(\kappa, \phi) \in \mathcal{B}} J[\kappa, \phi].$$

57.12.7. Interpretation

This problem extends classical transition design in two directions:

- from pure geometry to coupled geometry–cant design,
- from curve selection to functional optimization.

In this framework, the transition curve is no longer chosen from a catalogue, but emerges as part of an optimal coupled state.

57.12.8. Special Cases

Proposition 57.22. *If $\phi(s) \equiv 0$, the coupled problem reduces to a pure curvature optimization problem.*

Proposition 57.23. *If $a_{\text{eff}}(s) = 0$ for all s , then*

$$v^2 \kappa(s) = g \sin \phi(s),$$

and the design is perfectly balanced.

These special cases show that classical equilibrium design appears as a subcase of the coupled optimization framework.

57.12.9. Relation to Optimal Control

The coupled optimization problem may be interpreted as an optimal control problem with controls

$$\kappa(s), \quad \phi'(s),$$

state variables

$$x(s), y(s), \theta(s), \phi(s),$$

and cost functional $J[\kappa, \phi]$.

This provides a direct bridge from railway alignment design to modern control-theoretic and variational methods.

57.12.10. Connection to the Main Thesis

Summary 57.24. The coupled optimization problem shows that railway alignment design is not a purely geometric task.

Instead, it is the optimization of a coupled state system in which curvature and cant jointly determine dynamic behaviour.

This supports the central thesis of the manuscript that transition curves and cant evolution should be understood as control functions in a curvature-based railway state model.

57.13. Summary

Summary 57.25. Alignment optimization treats geometry as a variable rather than a fixed input.

It combines parameterization, numerical reconstruction, and constrained optimization methods to produce alignments that satisfy both geometric and engineering requirements.

Reader bridge to discretization. The functional becomes computable only after a parameterization and grid declare which quantities may move. Refining that grid changes the numerical model, not the Engineering Object.

57.14. Discrete Formulation of Alignment Optimization

57.14.1. Discretization of the Domain

Definition 57.26 (Discretization). Let

$$0 = s_0 < s_1 < \cdots < s_N = L$$

be a partition of the interval $[0, L]$.

We denote

$$\Delta s_i = s_{i+1} - s_i.$$

57.14.2. Discrete Variables

Definition 57.27 (Discrete curvature). Let

$$\kappa_i \approx \kappa(s_i)$$

denote the curvature values at the nodes.

Definition 57.28 (Discrete cant). Let

$$\phi_i \approx \phi(s_i)$$

denote the cant values.

Definition 57.29 (Parameter vector). The optimization variable is

$$\mathbf{x} = (\kappa_0, \dots, \kappa_N, \phi_0, \dots, \phi_N) \in \mathbb{R}^{2(N+1)}.$$

57.14.3. Discrete Pose Reconstruction

The pose3 state is reconstructed by numerical integration.

Definition 57.30 (Discrete integration). Given initial conditions

$$\gamma_0, \theta_0, \phi_0,$$

we compute iteratively:

$$\begin{aligned} \theta_{i+1} &= \theta_i + \kappa_i \Delta s_i, \\ \gamma_{i+1} &= \gamma_i + \Delta s_i (\cos \theta_i, \sin \theta_i), \end{aligned}$$

$$\phi_{i+1} = \phi_i + \omega_i \Delta s_i.$$

Here, ω_i may be expressed via differences of ϕ_i . This discrete reconstruction corresponds to standard numerical integration ideas [12, 44].

57.14.4. Finite Differences

Definition 57.31 (First derivative). Approximate derivatives by

$$\kappa'(s_i) \approx \frac{\kappa_{i+1} - \kappa_i}{\Delta s_i}, \quad \phi'(s_i) \approx \frac{\phi_{i+1} - \phi_i}{\Delta s_i}.$$

Definition 57.32 (Second derivative).

$$\kappa''(s_i) \approx \frac{\kappa_{i+1} - 2\kappa_i + \kappa_{i-1}}{(\Delta s_i)^2}.$$

57.14.5. Discrete Objective Functions

Curvature Smoothness

$$J_\kappa(\mathbf{x}) = \sum_{i=0}^{N-1} \left(\frac{\kappa_{i+1} - \kappa_i}{\Delta s_i} \right)^2 \Delta s_i.$$

Cant Smoothness

$$J_\phi(\mathbf{x}) = \sum_{i=0}^{N-1} \left(\frac{\phi_{i+1} - \phi_i}{\Delta s_i} \right)^2 \Delta s_i.$$

Dynamic Objective

$$J_{\text{dyn}}(\mathbf{x}) = \sum_{i=0}^N (v^2 \kappa_i - g \sin \phi_i)^2 \Delta s_i.$$

Model-reference-point candidate objective

For an explicitly identified reference-point model M , finite differences may approximate

$$\|\gamma_{P_M}''(s_i)\|^2$$

by second-order differences of γ_{P_M} :

$$J_{P_M}(\mathbf{x}) = \sum_{i=1}^{N-1} \|\gamma_{P_M,i+1} - 2\gamma_{P_M,i} + \gamma_{P_M,i-1}\|^2.$$

This candidate objective inherits the model, frame, discretization and applicability limits stated in the Schwerpunkt example. It is not a physical centre-of-mass, comfort or safety objective unless an identified model and responsible source establish that interpretation.

57.14.6. Combined Discrete Problem

Definition 57.33 (Discrete objective).

$$J(\mathbf{x}) = \alpha J_{P_M} + \beta J_{\text{dyn}} + \gamma J_{\kappa} + \delta J_{\phi}.$$

The four terms have different meanings and may have different units. Reference scales, weights $\alpha, \beta, \gamma, \delta$, their source and purpose must be declared before this expression defines an optimization problem. A solver result remains a candidate.

57.14.7. Constraints

Definition 57.34 (Equality constraints).

$$\kappa_0 = 0, \quad \kappa_N = \kappa_1.$$

Definition 57.35 (Inequality constraints).

$$|\kappa_i| \leq \kappa_{\max}, \quad |\phi_i| \leq \phi_{\max}.$$

Further constraints may include:

- bounds on derivatives,
- continuity conditions,
- alignment anchoring points.

57.14.8. Final Optimization Problem

Definition 57.36 (Discrete optimization problem). Find

$$\mathbf{x}^* \in \mathbb{R}^{2(N+1)}$$

such that

$$J(\mathbf{x}^*) = \min_{\mathbf{x}} J(\mathbf{x})$$

subject to the given constraints.

57.14.9. Structure of the Problem

The problem has the structure of a nonlinear least-squares problem.

It is therefore well suited for:

- Sequential Quadratic Programming,
- Gauss–Newton methods,
- sparse optimization techniques.

57.14.10. Connection to Implementation

The discrete formulation directly corresponds to a computational model:

- κ_i correspond to element parameters,
- ϕ_i correspond to cant samples,
- reconstruction matches numerical evaluation in software.

Summary 57.37. The continuous alignment optimization problem can be reduced to a finite dimensional nonlinear optimization problem.

This provides the direct link between theoretical modelling and practical implementation.

Reader bridge to SQP. Reconstruction couples curvature parameters nonlinearly to endpoint and observation residuals. SQP is introduced as one local constrained method for this visible need, not as the definition of optimization and not as evidence of a general production solver.

57.15. Sequential Quadratic Programming (SQP)

57.15.1. Problem Structure

We consider the discrete optimization problem

$$\min_{\mathbf{x}} J(\mathbf{x}) \quad \text{subject to} \quad g(\mathbf{x}) = 0, \quad h(\mathbf{x}) \leq 0.$$

Definition 57.38 (Parameter vector).

$$\mathbf{x} = (\kappa_0, \dots, \kappa_N, \phi_0, \dots, \phi_N).$$

57.15.2. Residual Formulation

Definition 57.39 (Residual vector).

$$J(\mathbf{x}) = \frac{1}{2} \|\mathbf{r}(\mathbf{x})\|^2.$$

Typical residual components:

$$r_i^{(\kappa)} = \sqrt{\gamma} \frac{\kappa_{i+1} - \kappa_i}{\Delta s}, \quad r_i^{(\phi)} = \sqrt{\delta} \frac{\phi_{i+1} - \phi_i}{\Delta s},$$

$$r_i^{(\text{dyn})} = \sqrt{\beta} (v^2 \kappa_i - g \sin \phi_i).$$

57.15.3. Gauss–Newton Step

Definition 57.40 (SQP step). At iteration k , compute $\mathbf{p}_k$ from:

$$(J_r^T J_r) \mathbf{p}_k = -J_r^T \mathbf{r}.$$

The Hessian is approximated by:

$$H \approx J_r^T J_r.$$

57.15.4. Sparse Structure

The system matrix has band structure:

- tridiagonal coupling for smoothness terms,
- diagonal contributions from dynamic terms.

57.15.5. Constraints Handling

Constraints are handled via:

- projection,
- penalty terms,
- Lagrange multipliers.

57.15.6. Iteration Scheme

Algorithm 1 SQP iteration

```

initialize  $\mathbf{x}_0$ 
for  $k = 0, 1, \dots$  do
    compute  $\mathbf{r}(\mathbf{x}_k)$ 
    compute  $J_r(\mathbf{x}_k)$ 
    solve  $(J_r^T J_r) \mathbf{p}_k = -J_r^T \mathbf{r}$ 
    choose step size  $\alpha_k$ 
    update  $\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{p}_k$ 
end for
```

57.15.7. Interpretation

Summary 57.41. SQP provides a practical and efficient method for solving the discrete alignment optimization problem.

Its efficiency follows from the sparse and local structure of the underlying model.

Reader bridge to initialization. A converged local run answers a question near its starting point. Initial guesses and multiple starts therefore belong to the provenance of a candidate and cannot be hidden behind a success flag.

57.16. Construction of Initial Guesses

57.16.1. Motivation

The performance of SQP methods strongly depends on the choice of the initial parameter vector.

In railway applications, initial data is often available in the form of:

- measurement polylines,
- existing alignments,
- partially structured design data.

57.16.2. Input Representation

Definition 57.42 (Input polyline). Let

$$P = (p_0, \dots, p_M), \quad p_i \in \mathbb{R}^2$$

denote a measured or imported polyline.

57.16.3. Arc-Length Parameterization

Definition 57.43. Define cumulative arc-length

$$s_0 = 0, \quad s_{i+1} = s_i + \|p_{i+1} - p_i\|.$$

This defines a parameterization of the input data.

57.16.4. Initial Curvature Estimation

Definition 57.44 (Discrete curvature). For three consecutive points, define:

$$\kappa_i \approx \frac{\theta_{i+1} - \theta_i}{\Delta s_i},$$

where

$$\theta_i = \arg(p_{i+1} - p_i).$$

This provides a first estimate of curvature from geometric data.

57.16.5. Smoothing

Raw curvature estimates are typically noisy.

Definition 57.45 (Smoothing operator). Apply a smoothing filter:

$$\kappa_i^{(0)} = \sum_j w_{ij} \kappa_j.$$

Possible choices:

- moving average,
- Gaussian filter,
- low-pass filtering.

57.16.6. Initial Cant Estimation

Cant may be initialized based on equilibrium:

$$\phi_i^{(0)} = \arcsin \left(\frac{v^2 \kappa_i^{(0)}}{g} \right).$$

Alternatively, cant may be:

- set to zero,
- taken from input data,
- approximated by design rules.

57.16.7. Projection to Discrete Grid

Definition 57.46. Interpolate the values $\kappa_i^{(0)}, \phi_i^{(0)}$ onto the optimization grid:

$$s_0, \dots, s_N.$$

57.16.8. Heuristic Segmentation

The input alignment may be segmented into:

- straight sections,
- circular arcs,
- transition regions.

Definition 57.47 (Segment detection). A segment is classified based on curvature:

$$\kappa_i \approx 0 \quad \Rightarrow \text{straight},$$

$$\kappa_i \approx \text{const} \quad \Rightarrow \text{arc}.$$

This structure can be used to improve the initial guess and is compatible with railway alignment reconstruction approaches such as those discussed by Kufver [33, 32].

57.16.9. Final Initial Guess

Definition 57.48 (Initial parameter vector). The initial guess is

$$\mathbf{x}_0 = (\kappa_0^{(0)}, \dots, \kappa_N^{(0)}, \phi_0^{(0)}, \dots, \phi_N^{(0)}).$$

57.16.10. Interpretation

The initial guess combines:

- measured geometry,
- smoothing,
- physical approximation.

Summary 57.49. A robust initial guess transforms raw geometric input into a structured parameter vector suitable for optimization.

This step is essential for bridging real-world data and numerical methods.

Reader bridge to derivatives and sparsity. As the number of released curvature parameters grows, dense numerical differentiation repeats expensive reconstruction and obscures local dependency. Analytic structure and sparsity answer that cost and auditability problem.

57.17. Analytical Structure of the Optimization Problem

57.17.1. Motivation

In classical numerical optimization, Jacobians are often computed by finite differences or automatic differentiation.

Within the AIM framework, this is not necessary. The alignment model is inherently differentiable.

57.17.2. Fundamental Observation

Proposition 57.50. *All state variables are obtained via arc-length integration:*

$$\theta(s) = \theta_0 + \int_0^s \kappa(\sigma) \, d\sigma,$$

$$\gamma(s) = \gamma_0 + \int_0^s (\cos \theta, \sin \theta) d\sigma.$$

57.17.3. Parameter Types

Definition 57.51. Primary parameter classes:

- curvature parameters dK ,
- length parameters dL ,
- transition parameters dT .

57.17.4. Analytical Derivatives

Proposition 57.52.

$$\frac{\partial \theta(s)}{\partial p} = \int_0^s \frac{\partial \kappa}{\partial p} d\sigma.$$

Proposition 57.53.

$$\frac{\partial \gamma(s)}{\partial p} = \int_0^s \begin{pmatrix} -\sin \theta \\ \cos \theta \end{pmatrix} \frac{\partial \theta}{\partial p} d\sigma.$$

All derivatives reduce to arc-length integrals.

57.17.5. Key Insight

Proposition 57.54. *The optimization problem admits exact Jacobians without numerical approximation.*

This eliminates:

- finite differences,
- numerical instability,
- high computational cost.

57.17.6. Block Structure

The parameter vector decomposes as:

$$(\kappa, \phi).$$

This induces a block structure:

$$H = \begin{pmatrix} H_{\kappa\kappa} & H_{\kappa\phi} \\ H_{\phi\kappa} & H_{\phi\phi} \end{pmatrix}.$$

57.17.7. Interpretation

Summary 57.55. The structure of the optimization problem is not imposed numerically, but follows directly from the curvature-based modelling approach.

This yields:

- exact derivatives,
- sparse matrices,
- natural block structure.

This is a central advantage of the AIM framework.

Reader bridge to alternative parameterizations. No single parameterization is best for every declared purpose. The final section compares parametric and hybrid choices as model alternatives whose freedoms and approximations must remain recorded.

57.18. Parametric Representation instead of Sampling

57.18.1. Motivation

Classical discretization represents curvature and cant by samples:

$$\kappa_i \approx \kappa(s_i), \quad \phi_i \approx \phi(s_i).$$

This leads to high-dimensional optimization problems with weak structure and limited interpretability.

Within the AIM framework, curvature and cant are instead represented by parameterized functions.

57.18.2. Parametric Curvature Model

Definition 57.56 (Parametric curvature). A transition is described by:

$$\kappa(s) = \kappa_0 + (\kappa_1 - \kappa_0) \hat{\kappa}\left(\frac{s}{L}, \mathbf{T}\right),$$

where:

- L is the element length,
- $\mathbf{T}$ is a vector of transition parameters,
- $\hat{\kappa}$ is a normalized prototype function.

57.18.3. Parametric Cant Model

Definition 57.57 (Parametric cant). Cant is represented analogously:

$$\phi(s) = \phi_0 + (\phi_1 - \phi_0) \hat{\phi}\left(\frac{s}{L}, \mathbf{T}_\phi\right).$$

57.18.4. Parameter Vector

Definition 57.58 (Reduced parameter vector). The optimization variable becomes:

$$\mathbf{x} = (L, \kappa_0, \kappa_1, \mathbf{T}, \phi_0, \phi_1, \mathbf{T}_\phi).$$

The dimension is independent of discretization density.

57.18.5. Dimensional Reduction

A sampled representation uses $O(N)$ variables.

The parametric representation uses:

$O(1)$ parameters per element.

This yields a drastic reduction in problem size.

57.18.6. Derivative Structure

Proposition 57.59. *Derivatives reduce to:*

$$\frac{\partial \kappa}{\partial p} = (\kappa_1 - \kappa_0) \frac{\partial \hat{\kappa}}{\partial p}.$$

Thus, all derivatives are computed analytically from prototype functions.

57.18.7. Integration into Reconstruction

The parametric representation is evaluated through:

$$\theta(s) = \int \kappa(s) \, ds, \quad \gamma(s) = \int (\cos \theta, \sin \theta) \, ds.$$

This yields a smooth and consistent geometry without discretization artifacts.

57.18.8. Comparison to Sample-Based Methods

	Sample-based	Parametric
dimension	high	low
smoothness	enforced numerically	intrinsic
interpretability	low	high
constraints	many local	few global

57.18.9. Interpretation

The parametric approach shifts the problem from:

- fitting values

to:

- designing functions.

57.18.10. Connection to Transition Families

Different transition curves correspond to different prototype functions $\hat{\kappa}$.

Thus, the optimization problem includes selection and tuning of transition families.

57.18.11. Computational Implications

- smaller optimization problems,
- better conditioning,
- faster convergence,
- direct integration with analytical Jacobians.

57.18.12. Connection to Classical Systems

This formulation is conceptually close to classical systems such as AXTRAN, which operate on element parameters rather than sampled data.

57.18.13. Connection to AIM

Summary 57.60. The parametric representation replaces discrete sampling by structured functions.

It enables a compact, interpretable, and analytically differentiable model, which forms the basis for efficient and robust alignment optimization.

This approach is central to the AIM philosophy.

57.19. Hybrid Parametric–Discrete Model

57.19.1. Motivation

Purely parametric models provide low-dimensional structure but may lack flexibility for fitting real-world data.

Purely sampled models provide flexibility but lead to large and ill-conditioned optimization problems.

A hybrid approach combines both advantages.

57.19.2. Hybrid Representation

Definition 57.61 (Hybrid curvature model).

$$\kappa(s) = \kappa_{\text{param}}(s; \mathbf{T}) + \delta\kappa(s),$$

where:

- κ_{param} is a parametric base model,
- $\delta\kappa(s)$ is a correction term.

Definition 57.62 (Hybrid cant model).

$$\phi(s) = \phi_{\text{param}}(s; \mathbf{T}_\phi) + \delta\phi(s).$$

57.19.3. Discretization of Correction

The correction term is represented discretely:

$$\delta\kappa_i = \delta\kappa(s_i), \quad \delta\phi_i = \delta\phi(s_i).$$

57.19.4. Extended Parameter Vector

Definition 57.63.

$$\mathbf{x} = (\mathbf{T}, \mathbf{T}_\phi, \delta\kappa, \delta\phi).$$

57.19.5. Regularization of Corrections

To avoid overfitting, the correction terms are penalized:

$$J_\delta = \int_0^L (\delta\kappa'(s))^2 ds + \int_0^L (\delta\phi'(s))^2 ds.$$

This enforces smooth, small deviations from the parametric model.

57.19.6. Residual Structure

The residual now consists of:

- parametric contributions,
- correction contributions,
- data fitting terms.

57.19.7. Block Structure

The parameter vector decomposes as:

$$(\mathbf{T}, \delta\boldsymbol{\kappa}) \quad \text{and} \quad (\mathbf{T}_\phi, \delta\boldsymbol{\phi}).$$

This yields a multi-level block structure:

- coarse scale (parameters),
- fine scale (corrections).

57.19.8. Interpretation

The parametric model captures:

- geometric intent,
- transition structure,
- engineering semantics.

The correction term captures:

- measurement noise,
- local deviations,
- modelling imperfections.

57.19.9. Connection to Measurement Data

Measured polylines can be fitted by minimizing:

$$\sum_i \|\gamma(s_i) - p_i\|^2,$$

with the hybrid representation.

The parametric part explains the global structure, while the correction absorbs residual errors.

57.19.10. Computational Strategy

- first optimize parametric variables,
- then refine using correction terms,
- optionally alternate between both levels.

57.19.11. Relation to LOD

The hybrid model naturally supports level-of-detail concepts:

- coarse LOD: parametric model only,
- fine LOD: parametric + corrections.

57.19.12. Connection to AIM

Summary 57.64. The hybrid representation combines structured modelling and local flexibility.

It enables robust alignment reconstruction from imperfect data while preserving the interpretability of parametric models.

This approach bridges engineering design and data-driven fitting within the AIM framework.

What leaves the mathematical chapter

The mathematical result is not merely x^* . A reviewable handover is

$$\mathcal{C} = (\mathcal{P}.id, M.id, x^*, r(x^*), J(x^*), \\ \text{constraint margins, termination, } \varepsilon_{\text{num}}, \text{ provenance}).$$

Termination reports why the algorithm stopped; convergence evidence reports what the numerical method established. Neither proves applicability to a different context or acceptance by an engineer.

The next architecture chapter therefore asks how a service can preserve this contract across ingestion, calculation, and review without silently committing the candidate to the authoritative model.

57.20. Hybrid Representation and Block Structure

57.20.1. Motivation

Purely parametric models provide a low-dimensional and interpretable representation of alignments, but lack flexibility when fitting real-world data.

Purely sampled models provide maximum flexibility, but lead to high-dimensional and often ill-conditioned optimization problems.

A hybrid formulation combines both advantages.

57.20.2. Hybrid Representation

Definition 57.65 (Hybrid curvature model).

$$\kappa(s) = \kappa_{\text{param}}(s; \mathbf{T}) + \delta\kappa(s),$$

where:

- κ_{param} is a structured parametric model,
- $\delta\kappa(s)$ is a correction term.

Definition 57.66 (Hybrid cant model).

$$\phi(s) = \phi_{\text{param}}(s; \mathbf{T}_\phi) + \delta\phi(s).$$

57.20.3. Discretization of Corrections

The correction terms are represented discretely:

$$\delta\kappa_i = \delta\kappa(s_i), \quad \delta\phi_i = \delta\phi(s_i).$$

Definition 57.67 (Extended parameter vector).

$$\mathbf{x} = (\mathbf{T}, \mathbf{T}_\phi, \delta\kappa, \delta\phi).$$

57.20.4. Regularization

To avoid overfitting, correction terms are regularized:

$$J_\delta = \int_0^L (\delta\kappa'(s))^2 ds + \int_0^L (\delta\phi'(s))^2 ds.$$

Thus, the correction captures only smooth, physically meaningful deviations.

57.20.5. Residual Structure

The residual vector consists of:

- parametric contributions,
- correction contributions,
- physical constraints,
- data fitting terms.

57.20.6. Block Structure of Variables

The parameter vector decomposes naturally as:

$$(\mathbf{T}, \delta\boldsymbol{\kappa}) \quad \text{and} \quad (\mathbf{T}_\phi, \delta\boldsymbol{\phi}).$$

This yields a two-level structure:

- coarse scale: parametric variables,
- fine scale: correction variables.

57.20.7. Jacobian Block Structure

Definition 57.68 (Block Jacobian). The Jacobian takes the form:

$$J_r = \begin{pmatrix} J_{\kappa\kappa}^{\text{param}} & J_{\kappa\kappa}^{\text{corr}} & 0 & 0 \\ 0 & 0 & J_{\phi\phi}^{\text{param}} & J_{\phi\phi}^{\text{corr}} \\ J_{\text{dyn},\kappa} & J_{\text{dyn},\kappa}^\delta & J_{\text{dyn},\phi} & J_{\text{dyn},\phi}^\delta \end{pmatrix}.$$

Key properties:

- parametric and correction terms are weakly coupled,
- curvature and cant are largely separable,
- coupling occurs mainly through dynamics.

57.20.8. Hierarchical Interpretation

The hybrid model introduces a hierarchical structure:

- parametric layer defines global alignment structure,
- correction layer refines local deviations.

This reflects engineering practice:

- design defines intent,
- construction introduces deviations.

57.20.9. Computational Strategy

- optimize parametric variables first,
- introduce correction terms for refinement,
- optionally alternate between both levels.

57.20.10. Connection to Level-of-Detail

The hybrid representation naturally supports LOD concepts:

- coarse LOD: parametric model only,
- fine LOD: parametric + corrections.

57.20.11. Key Insight

Proposition 57.69. *The hybrid formulation separates structural modelling from data fitting while preserving a sparse and well-conditioned optimization problem.*

57.20.12. Connection to AIM

Summary 57.70. The combination of hybrid modelling and block structure is a central feature of the AIM framework.

It enables:

- interpretable parametric modelling,
- robust handling of imperfect data,
- efficient optimization due to sparsity and locality.

Thus, it bridges the gap between engineering design and data-driven reconstruction.

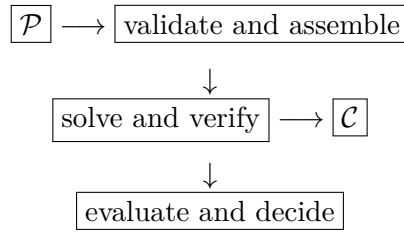
58. A Solver Architecture for Reviewable Candidates

58.1. Architecture begins at a responsibility boundary

Reader bridge. The previous chapter produced a candidate contract. Architecture is needed because that contract can be lost when parsing, solving, visualization, and model editing are treated as one operation.

A solver service may consume a read-only problem declaration and return candidate evidence. It must not infer the engineering purpose, broaden the released freedoms, replace source observations, decide applicability, or write a result into an authoritative Alignment merely because a numerical run terminated.

For the running discrepancy, the conceptual flow is



Only the middle operations belong to the solver responsibility. The final arrow is an engineering workflow boundary.

58.2. The problem package

A portable problem package needs more than geometry:

$$\mathcal{P} = (id, I, P, M, x_0, \mathcal{X}_{\text{free}}, q, \mathcal{F}, J, W, \Sigma, \pi),$$

where W records scaling and π records provenance. Validation checks that every free variable is declared, fixed inputs remain locked, units are compatible, objective terms name their source, and all referenced models and contexts are available. Failure of this check is not solver non-convergence; it is an incomplete or inapplicable problem declaration.

58.3. Assembly makes mathematical structure explicit

Reader bridge. Once the engineering contract is complete, the architecture can translate it into numerical objects without changing its meaning.

Assembly constructs:

- the parameter vector and bounds from the released freedoms;
- equality and inequality functions from declared constraints;
- separately typed objective and diagnostic residuals;
- reconstruction and observation evaluators;
- scaling, derivative providers, and sparsity patterns;
- termination tolerances and independent verification settings.

Residuals are not interchangeable merely because they are stored in one vector. Type and provenance must survive assembly so that a reviewer can distinguish an endpoint constraint, an observation mismatch, a regularizer, and a numerical diagnostic.

58.4. Why SQP, derivatives, and sparsity belong behind interfaces

Reader bridge. SQP is useful for a smooth nonlinear problem with visible equality and inequality constraints. It is one replaceable numerical method, not the owner of the problem.

At iterate x_k , an SQP implementation forms a local quadratic model and linearized constraints, obtains a step p_k , and applies a globalization rule such as a line search or trust region. The architecture must expose:

$$x_k, \quad J(x_k), \quad r(x_k), \quad \nabla J(x_k), \quad A(x_k), \quad p_k,$$

together with constraint margins, derivative provenance, scaling, and solver diagnostics.

Analytic, automatic, or finite-difference derivatives may be supplied if their method and validation are recorded. Sparsity is a computational property of the declared dependency graph. Exploiting it can reduce work; it does not justify an undeclared objective or constraint.

58.5. Termination, convergence, and verification

A solver may stop because a tolerance was met, progress stalled, an iteration limit was reached, or evaluation failed. These outcomes must remain distinct. A nominal convergence flag is credible only with the corresponding evidence, including:

- objective and step histories;

- equality residuals and inequality margins;
- a stationarity or optimality measure appropriate to the method;
- derivative and scaling checks where applicable;
- reconstruction on the declared verification grid;
- warnings, active assumptions, and numerical error estimates.

This evidence establishes what happened within M and $\mathcal{P}$. It does not establish that the observation model is correct, that the candidate is constructible, or that the purpose warrants adoption.

58.6. The candidate package and review boundary

The service returns, without applying,

$$\mathcal{C} = (\mathcal{P}.id, M.id, x^*, \hat{\gamma}^*, r^*, J^*, \text{margins}, \text{termination}, \text{verification}, \Sigma, \pi).$$

A review surface may compare candidates, inspect residuals and sensitivities, and preview a representation. Preview is not Physical Realization; comparison is not evaluation under every relevant purpose; an “Apply” operation, if a future workflow provides one, requires a separate authorized command and preserved provenance.

58.7. Current AXTRAN evidence and the general architecture

The architecture above is a scientific responsibility model and a direction for implementation. Current repository evidence for AXTRAN is narrower: AXTRAN is experimental; a single SQP step and a finite repeated loop are evidence of a numerical path. They do not demonstrate a general optimizer architecture, production robustness, live optimization, network optimization, vehicle-aware objectives, cant optimization, or automatic model integration.

Accordingly, examples of richer sessions, visualization, alternative solvers, or network assembly are future or problem-specific possibilities. They must not be read as present AIM or AXTRAN capability.

58.8. What the engineer receives

For the CAD/survey discrepancy, the architecture returns one or more traceable curvature candidates and the evidence needed to question them: which parameters moved, what stayed fixed, which residuals improved, which constraints are close to active, why the run stopped, and which uncertainties remain. Applicability is then checked under the intended project and realization context. Evaluation compares consequences under declared

58. *A Solver Architecture for Reviewable Candidates*

purposes. An authorized Engineering Decision may accept, reject, or request a new problem formulation.

Summary 58.1. A solver architecture is trustworthy when it preserves the engineering problem on the way in and returns a reviewable candidate on the way out.

calculated $\neq$ applicable $\neq$ accepted $\neq$ authoritative

The solver owns numerical work inside the declared contract. It does not own Alignment identity, applicability, acceptance, or authority.

59. Qualified Realization-Aware Optimization

Abstract for the general reader

An analytically smooth design is not guaranteed to be constructed exactly as drawn. Machines, materials, construction choices, maintenance and uncertainty can make the physical asset differ from design intent. Realization-aware optimization asks a useful question: if a declared realization model predicts such differences, should candidate design parameters be chosen with those predicted consequences in view?

The answer is conditional. A coordinate transformation is not construction. An observation is not the physical asset. A model prediction is not an observation, and an optimum is not an Engineering Decision. This chapter therefore develops a candidate formulation, not a universal or continuously operating AIM capability.

59.1. Five objects that must remain separate

Object	Role	Not interchangeable with
Metric Realization	makes intrinsic construction spatially measurable	physical construction
Physical Realization	relation from intent and process to an asset	coordinate transformation
Physical State	asset condition at a stated time	observation
Observation	source- and uncertainty-qualified evidence	physical truth
Representation	carries coordinates or records	identity, reality or authority

Let $S_{\text{int}}(x)$ be intended state produced by design variables x . A qualified physical-realization model is written

$$\hat{S}_{\text{phys}} \in \mathcal{R}_M(S_{\text{int}}(x), q, \xi),$$

where M identifies the model, q contains fixed process and asset inputs, and ξ represents declared variability or uncertainty. $\mathcal{R}_M$ may be deterministic, stochastic or set-valued. The notation does not establish that construction itself is an executable AIM operator.

An observation model remains separate:

$$y^{\text{obs}} = \mathcal{O}_N(S_{\text{phys}}, \eta),$$

with observation method N and uncertainty η . Coordinate conversion may be required to compare quantities in one frame, but it changes representation rather than physical state.

59.2. Problem contract before the minimum

A realization-aware optimization problem is defined only after recording:

Contract field	Required declaration
problem identity and purpose	engineering question and lifecycle use
realization relation	M , inputs, output, frame, validity and status
decision variables	known, fixed, free, bounded and unresolved quantities
fixed inputs	intended endpoints, context, process assumptions and evidence
feasible set $\mathcal{F}$	geometric, continuity, rule and model-applicability constraints
objective J	source, units, scales, aggregation and purpose
uncertainty boundary	variability represented and effects omitted
status and provenance	implementation-backed, Research-supported or illustrative

Only then may a candidate problem be written:

$$\min_{x \in \mathcal{F}} J\left(x, \hat{S}_{\text{phys}}, \mathcal{D}_K(\hat{S}_{\text{phys}}, s(t))\right),$$

where K is an identified response or comparison model when one is needed. If $\mathcal{R}_M$ is uncertain, the problem must state whether J uses an expectation, scenarios, a bound, a risk measure or another declared construction. No uncertainty treatment is implied by the word *realization-aware*.

59.3. Curvature and cant: coupled question, separate ownership

Curvature $\kappa(s)$ belongs to intrinsic horizontal construction. Cant $u(s)$, or its angle $\phi(s)$, is a separate longitudinal law with its own provenance and responsibility. A model may couple their consequences, for example through the reduced residual

$$r_{\text{bal}} = v^2 \kappa - g \sin \phi,$$

without making the two laws one quantity.

The decision vector may therefore be any explicitly selected partition:

$$x = (x_\kappa, x_u, x_p).$$

In one problem, x_κ may be free while cant is fixed. In another, curvature is fixed and cant is investigated. Both may be free only when the purpose, constraints and responsible objective source support joint investigation. There is no universal AIM rule requiring joint optimization.

59.4. Objectives, units and scaling

Possible objective components include curvature variation, cant variation, the reduced balance residual, response-model output, cost or realization sensitivity. They are distinct objectives. For example,

$$J_\kappa = \int_0^L (\kappa')^2 ds, \quad J_r = \int_0^T r_{\text{bal}}^2 dt$$

have different domains, units and meanings. J_r is not comfort or vehicle response without an applicable model and criterion.

A combined objective

$$J = \sum_i w_i J_i / J_{i,0}$$

is meaningful only when the reference scales $J_{i,0}$, weights w_i , their source and their purpose are recorded. The solver supplies none of these engineering choices.

59.5. Constraints and applicability

Candidate constraints may describe machine limits, continuity, permitted parameter ranges, process envelopes or applicable rules. Each constraint requires a source, units, validity and lifecycle scope. Measurement resolution is an observation qualification, not automatically a construction constraint. Tamping behaviour, ballast response or correction kernels enter only through an identified and supported realization model.

The feasible set $\mathcal{F}$ establishes mathematical feasibility under that declaration. It does not establish constructability, operational acceptability or authority outside the model's applicability.

59.6. Uncertainty and calibration boundary

The minimum useful separation is:

- process and construction variability in ξ ;
- observation uncertainty in η ;
- operational variability in $s(t)$ and other use inputs;
- numerical residual from the solver;
- parameter uncertainty inside M or K ;
- model-form uncertainty about whether M or K is adequate.

Calibration evidence may support parameters for a stated asset, process and domain. Agreement with one observation does not prove a universal realization law. An unexplained target–observation deviation does not identify construction as its cause.

59.7. Inverse and frequency interpretations

The expression

$$\mathcal{R}_M(S_{\text{int}}(x), q, \xi) \approx S_{\text{desired}}$$

defines an inverse problem only after the comparison quantity, norm, uncertainty treatment and applicability are declared. A solution is one candidate pre-image under M , not proof of what the physical process will produce.

Likewise, describing $\mathcal{R}_M$ as a low-pass filter is a possible linearized or empirical model, not a property of Physical Realization in general. Frequency penalties are defensible only when evidence supports that model over the stated spatial or temporal range.

59.8. Repeated optimization is a workflow hypothesis

Construction, maintenance or observation feedback may motivate repeated problem formulation and recalculation:

new evidence $\rightarrow$ qualified model update $\rightarrow$ new candidate $\rightarrow$ evaluation.

That sequence does not imply a continuously operating optimizer, automatic control or current AXTRAN runtime functionality. Such a workflow requires identified triggers, data qualification, model versioning, convergence handling, authority and implementation evidence. In this Thesis it remains a possible workflow and Research direction.

59.9. Candidate, evaluation and decision

A result shall preserve at least

$$C_R = (\mathcal{P}.id, M.id, K.id, x^*, \mathcal{F}, J, \text{residuals, convergence, scenarios, provenance}).$$

It is a non-authoritative calculation candidate. Later evidence may make it inapplicable or stale. Purpose-qualified evaluation may compare it with rules and evidence; an Engineering Decision may then select or reject it under documented authority. The dependency is the one shown in fig. 43.1.

calculated $\neq$ applicable $\neq$ accepted $\neq$ authoritative

59.10. Vehicle, Earth and implementation boundaries

Where a purpose requires vehicle-related quantities, an identified response or clearance model supplies them. The qualified Schwerpunkt example is referenced without redefining it: physical centre of mass, model reference point, envelope and clearance remain different subjects.

Ellipsoidal Metric Realization may matter at large scale or high precision, but it does not make physical construction an ellipsoid operator. Coordinate and surface models qualify where quantities are evaluated; they do not create an optimization objective.

The formulations in this chapter are scientifically usable problem contracts and candidates. They are not evidence that current AIM, AXTRAN, transitionDB or TransEd implements general realization-aware, vehicle, construction or runtime optimization.

59.11. Claim-status summary

Former claim	Disposition	Qualified result
deterministic realization operator	reformulated	identified relation $\mathcal{R}_M$
mandatory joint curvature/cant optimization	removed	problem-dependent freedom selection
universal dynamic objectives	reformulated	model- and purpose-qualified components
continuous runtime optimization	unsupported	possible repeated workflow/open work
optimum as applicable result	contradictory	non-authoritative candidate

The potential remains substantial: design candidates can be tested against explicit realization hypotheses instead of assuming perfect construction. The scientific gain comes from making the hypothesis, uncertainty and responsibility inspectable, not from calling the result universally realization-aware.

Part XII.

Engineering and Standards

Engineering

This part establishes how optimization results become admissible railway solutions under explicit engineering rules.

What do we already know?

The previous parts established the mathematical, geometric, operational, realization-aware, and optimization foundations of AIM.

Railway alignments can now be represented, evaluated, analyzed, and improved through structured operator-based methods.

Optimization provides mechanisms for generating improved solutions according to specified objectives.

However, optimization alone does not determine whether a solution is acceptable from an engineering perspective.

What is still missing?

Railway engineering operates within a framework of rules, recommendations, standards, regulations, and operational constraints.

Not every mathematically feasible alignment is an admissible railway alignment.

Track geometry must satisfy engineering limits.

Operational states must remain safe.

Construction solutions must remain practical.

Design alternatives must comply with applicable standards.

The missing element is therefore the engineering constraint layer that connects mathematical models with railway practice.

What comes next?

This part introduces engineering constraints as explicit components of the AIM framework.

Rather than treating standards as external documents consulted after design, AIM interprets engineering rules as structured constraints that can participate directly in analysis and optimization.

The chapters that follow investigate:

- standards and regulations,

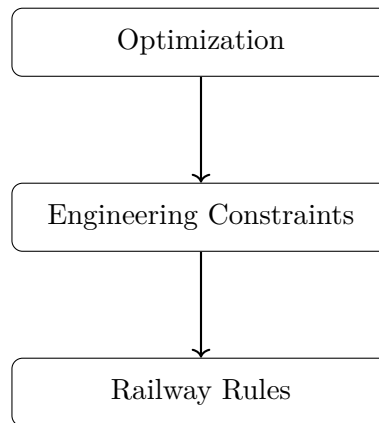
- admissibility constraints,
- railway-specific engineering rules,
- and formal representations of operational requirements.

The objective is to transform railway standards from passive reference material into active components of railway information modelling.

Relation to the AIM Roadmap

Within the AIM roadmap, the present part corresponds to the transition from optimization toward engineering admissibility.

The roadmap sketch answers the part-level question: which constraint layers convert mathematically improved states into engineeringly valid states?



The progression

Optimization $\rightarrow$ Engineering Constraints $\rightarrow$ Railway Rules

marks the transition from finding improved solutions toward ensuring that those solutions remain admissible within real railway systems.

Engineering therefore acts as the bridge between mathematical optimization and practical railway implementation.

Principle 59.1 (Engineering Constraint Principle). Within AIM, railway standards, regulations, and operational rules should be represented as explicit engineering constraints that can participate directly in modelling, analysis, and optimization.

Summary 59.2. Optimization explains how railway systems can be improved.

Engineering explains which improvements are admissible.

The present part therefore establishes the constraint layer that connects mathematical optimization with practical railway design and operation.

This constraint layer prepares the Applications part, where admissible concepts are exercised in interoperability and workflow contexts.

60. Standards and Data Models

Motivation

Railway alignment data does not exist in isolation. It is embedded in a landscape of established standards and engineering data models.

Relevant formats include:

- LandXML,
- IFC (Industry Foundation Classes),
- railML,
- and various proprietary or legacy formats.

The AIM framework does not replace these standards. Instead, it provides a consistent internal representation that can interpret and integrate them.

60.1. General Perspective

Most existing standards describe railway geometry in a layered or fragmented way.

- horizontal alignment,
- vertical profile,
- cant (superelevation),
- additional metadata.

In contrast, the AIM framework proposes a unified state-based representation:

$$(\gamma(s), \theta(s), \kappa(s), \phi(s)).$$

This representation serves as a canonical internal model.

60.2. LandXML

60.2.1. Overview

LandXML is one of the most widely used formats for alignment data exchange.

It typically represents:

- horizontal geometry (CoordGeom),
- vertical profiles,
- cant definitions.

60.2.2. Characteristics

- structured XML format,
- hierarchical organization,
- multiple alignments per file,
- explicit naming of geometric elements.

60.2.3. Limitations

Despite its structure, LandXML exhibits several limitations:

- separation of horizontal and vertical geometry,
- lack of a unified dynamic interpretation,
- ambiguity in coordinate reference systems,
- inconsistent usage across software systems.

60.2.4. AIM Interpretation

Within the AIM framework, LandXML is treated as a *fat container*:

- all available data is parsed,
- relevant components are extracted,
- the result is transformed into a unified state model.

The conversion step

LandXML $\rightarrow$ AIM (sparse)

is essential for further processing.

60.3. IFC

60.3.1. Overview

IFC is a general BIM standard designed for interdisciplinary data exchange.

In the context of infrastructure, IFC is extended by infrastructure schemas (IFC Rail, IFC Alignment).

60.3.2. Characteristics

- object-oriented data model,
- rich semantic structure,
- integration of geometry and metadata,
- support for relationships and hierarchies.

60.3.3. Challenges

For alignment modelling, IFC presents several challenges:

- complex schema structure,
- multiple representation alternatives,
- inconsistent implementation across tools,
- limited support for advanced alignment concepts.

60.3.4. AIM Interpretation

The AIM framework interprets IFC data by extracting:

- alignment geometry,
- reference systems,
- semantic relations.

AIM acts as a normalization layer that reduces IFC complexity to a computationally usable form.

60.4. railML

60.4.1. Overview

railML is a domain-specific XML standard for railway systems.

It focuses on:

- infrastructure,
- timetable data,
- rolling stock.

60.4.2. Relevance for Alignment

railML provides a more explicit representation of railway topology than most other formats.

However, detailed geometric modelling is often secondary to network and operational aspects.

60.4.3. AIM Interpretation

Within AIM, railML is particularly relevant for:

- network topology,
- connections between tracks,
- system-level structure.

60.5. Legacy and Proprietary Formats

In practice, a large amount of alignment data exists in proprietary or legacy formats.

Examples include:

- TRA / GRA (Technet Verm.Esn),
- spreadsheet-based formats,
- CAD exports.

60.5.1. Characteristics

- incomplete or implicit structure,
- missing type information,
- reliance on conventions rather than formal schema.

60.5.2. AIM Strategy

These formats are interpreted using:

- sniffing and classification,
- format-specific parsers,
- reconstruction of implicit structure.

60.6. Canonical Internal Model

Definition 60.1 (AIM core model). The AIM framework uses a canonical internal representation based on:

$$(\gamma(s), \theta(s), \kappa(s), \phi(s)).$$

All external formats are mapped to this representation.

60.6.1. Advantages

- unified handling of geometry and dynamics,
- compatibility with numerical optimization,
- independence from specific file formats,
- extensibility toward new standards.

60.7. Transformation Pipeline

The transformation from external data to the AIM model follows a pipeline structure.

```
external file
→ parser
→ fat representation
→ validation
→ sparse conversion
→ AIM state model
```

This pipeline ensures robustness and traceability of data processing.

60.8. Relation to BIM

60.8.1. Conceptual Position

BIM (Building Information Modelling) aims to integrate geometry, semantics, and lifecycle information.

However, alignment modelling is often underrepresented or treated as a secondary aspect.

60.8.2. AIM Contribution

The AIM framework contributes to BIM by:

- providing a rigorous alignment model,
- enabling dynamic interpretation,

- supporting optimization-driven design.

In this sense, AIM can be interpreted as a specialization of BIM for alignment-centric infrastructure modelling.

60.9. Interoperability

A key requirement for practical application is interoperability with existing tools and workflows.

The AIM framework supports interoperability through:

- import adapters,
- export converters,
- intermediate canonical representation.

60.10. Discussion

Existing standards focus primarily on data exchange and documentation.

The AIM framework complements these standards by introducing:

- a computationally consistent model,
- a dynamic interpretation,
- integration with optimization methods.

Thus, AIM bridges the gap between static data representation and active engineering design.

60.11. Summary

Summary 60.2. Railway alignment data is governed by a variety of standards, each with its own strengths and limitations.

The AIM framework provides a unified internal representation that allows these heterogeneous sources to be interpreted consistently.

By combining standard-based data exchange with a state-based, optimization-ready model, AIM enables a transition from static data handling to dynamic and computational alignment design.

61. Constraint Code Architecture

Motivation

Railway alignment rules must be represented in a form that is both mathematically meaningful and computationally tractable.

Within the AIM framework, this is achieved by expressing all rules as constraint residuals that can be evaluated uniformly inside an optimization loop.

The implementation goal is therefore not a collection of isolated rule checks, but a modular constraint library.

61.1. General Principle

Definition 61.1 (Constraint residual). A constraint is represented as a function

$$c(\mathbf{x}) \in \mathbb{R}^m$$

whose value vanishes in the ideal case and whose magnitude measures violation.

This unified representation allows the same mathematical object to be used for:

- feasibility checks,
- penalty terms,
- direct inclusion into SQP solvers.

```
function evaluateConstraint(ctx) {  
    return [residual1, residual2, ...]  
}
```

61.2. Constraint Registry

A practical implementation should organize constraints in a registry.

```
const ConstraintRegistry = {  
    geometric: {},  
    dynamic: {},  
    topologic: {},  
}
```

```

    regulatory: {}
  }

```

This keeps domain logic extensible and avoids hard-coded monolithic solver structures.

61.3. Constraint Object Model

```

const constraint = {
  id: "min_radius",
  group: "regulatory",
  appliesTo: "edge",
  weight: 1.0,
  evaluate: (ctx) => []
}

```

The fields have the following meaning:

- `id`: unique name,
- `group`: semantic class,
- `appliesTo`: edge, node, pair, network,
- `weight`: optional scaling,
- `evaluate`: residual callback.

61.4. Context Object

Constraints should not operate directly on raw global arrays. Instead, they receive a normalized context object.

```

const ctx = {
  edge,
  node,
  pair,
  i,
  kappa,
  phi,
  geometry,
  speed,
  params,
  metadata
}

```

This provides a stable interface between model, solver, and rule system.

61.5. Geometric Constraints

61.5.1. Position Continuity

```
function continuityConstraint(p1, p2) {
  return [
    p1.x - p2.x,
    p1.y - p2.y
  ]
}
```

This enforces C^0 -continuity between adjacent geometric elements or connected alignments.

61.5.2. Direction Continuity

```
function directionConstraint(t1, t2) {
  return [
    t1.dx - t2.dx,
    t1.dy - t2.dy
  ]
}
```

This corresponds to tangent continuity and therefore to C^1 -style behaviour in geometric terms.

61.5.3. Curvature Continuity

```
function curvatureConstraint(k1, k2) {
  return [k1 - k2]
}
```

This enforces continuity of curvature across boundaries.

61.6. Dynamic Constraints

61.6.1. Equilibrium Constraint

```
function equilibriumConstraint(kappa, phi, v, g) {
  return [v * v * kappa - g * Math.sin(phi)]
}
```

This residual measures cant deficiency or excess in analytic form.

61.6.2. Jerk Constraint

```
function jerkConstraint(kappa0, kappa1, phi0, phi1, ds, v, g) {
  const dk = (kappa1 - kappa0) / ds
  const dphi = (phi1 - phi0) / ds
  const j = v * v * v * dk - g * v * Math.cos(phi0) * dphi
  return [j]
}
```

This constraint reflects dynamic smoothness and comfort-related limits.

61.7. Regulatory Constraints**61.7.1. Minimum Radius**

```
function minRadiusConstraint(kappa, Rmin) {
  return [Math.abs(kappa) - 1 / Rmin]
}
```

A non-positive value indicates compliance with the radius limit.

61.7.2. Maximum Cant

```
function maxCantConstraint(phi, phiMax) {
  return [Math.abs(phi) - phiMax]
}
```

61.7.3. Cant Gradient

```
function cantGradientConstraint(phi0, phi1, ds, limit) {
  return [Math.abs((phi1 - phi0) / ds) - limit]
}
```

61.8. Topological Constraints**61.8.1. Switch Node Constraint**

```
function switchConstraint(main, branch) {
  return [
    main.x - branch.x,
    main.y - branch.y,
    main.dx - branch.dx,
    main.dy - branch.dy
  ]
}
```

This expresses the basic coupling at branching nodes.

61.8.2. Parallel Track Spacing

```
function parallelConstraint(p1, p2, d) {
  const dx = p1.x - p2.x
  const dy = p1.y - p2.y
  return [Math.sqrt(dx * dx + dy * dy) - d]
}
```

61.9. Constraint Assembly

The full residual vector is assembled from all active constraints.

```
function buildConstraintResiduals(ctx, constraints) {
  const r = []

  for (const c of constraints) {
    const ri = c.evaluate(ctx)
    const w = Math.sqrt(c.weight ?? 1)
    for (const v of ri) r.push(w * v)
  }

  return r
}
```

This produces a single solver-compatible vector independent of the semantic source of the constraints.

61.10. Factory Functions

Constraints should be created through factories rather than by writing anonymous closures throughout the code base.

```
function makeMinRadiusConstraint(Rmin, weight = 1) {
  return {
    id: "min_radius",
    group: "regulatory",
    appliesTo: "edge",
    weight,
    evaluate: ({ kappa, i }) => [
      Math.abs(kappa[i]) - 1 / Rmin
    ]
  }
}
```

```

    }
  }

function makeJerkConstraint(v, g, ds, weight = 1) {
  return {
    id: "jerk",
    group: "dynamic",
    appliesTo: "edge",
    weight,
    evaluate: ({ kappa, phi, i }) => {
      const dk = (kappa[i + 1] - kappa[i]) / ds
      const dphi = (phi[i + 1] - phi[i]) / ds
      return [v * v * v * dk - g * v * Math.cos(phi[i]) * dphi]
    }
  }
}

```

61.11. Constraint Presets

In practical railway design, different projects require different constraint configurations.

```

const presetHighSpeed = [
  makeMinRadiusConstraint(4000, 10),
  makeJerkConstraint(v, g, ds, 8),
  makeMaxCantConstraint(phiMax, 6)
]

const presetExistingLine = [
  makeMinRadiusConstraint(800, 3),
  makeJerkConstraint(v, g, ds, 2),
  makeParallelConstraint(trackDistance, 5)
]

```

61.12. Integration into the Solver

```

function buildResidualVector(ctx, constraints, objectives) {
  return [
    ...buildConstraintResiduals(ctx, constraints),
    ...buildObjectiveResiduals(ctx, objectives)
  ]
}

```

This shows the central design principle of the architecture: constraints and objectives share the same residual-based interface.

61.13. Discussion

The proposed architecture has several advantages:

- modularity,
- extensibility,
- compatibility with least-squares solvers,
- transparent mapping between engineering rules and code.

Instead of scattering rule checks throughout the application, the constraint library concentrates domain logic in a single mathematical layer.

61.14. Summary

Summary 61.2. The constraint library translates railway rules, topology, and dynamic requirements into a unified residual-based architecture.

This provides the technical bridge between theoretical modelling, engineering regulations, and computational optimization.

As a result, the AIM framework becomes not only mathematically coherent, but also directly implementable in software.

62. Railway Design Rules

Motivation

Railway alignment design is not solely a geometric or mathematical task. It is governed by engineering rules, safety requirements, and operational constraints.

These rules originate from:

- national regulations,
- international standards (e.g. UIC),
- operator-specific guidelines (e.g. Deutsche Bahn),
- empirical engineering practice.

Within the AIM framework, such rules are not treated as external constraints only, but are integrated into the modelling and optimization process.

62.1. Categories of Rules

62.1.1. Geometric Rules

- minimum radius,
- continuity of alignment,
- transition length requirements,
- curvature monotonicity in transitions.

62.1.2. Kinematic and Dynamic Rules

- limits on lateral acceleration,
- limits on jerk,
- compatibility of curvature and cant,
- speed-dependent requirements.

62.1.3. Operational Rules

- allowable speeds,
- safety margins,
- maintenance considerations,
- interoperability constraints.

62.1.4. Topological Rules

- constraints at switches and crossings,
- parallel track spacing,
- network connectivity conditions.

62.2. Geometric Constraints

62.2.1. Minimum Radius

Definition 62.1 (Minimum radius constraint). The curvature must satisfy

$$|\kappa(s)| \leq \kappa_{\max}, \quad \kappa_{\max} = \frac{1}{R_{\min}}.$$

The value of $R_{\min}$ depends on:

- design speed,
- vehicle characteristics,
- regulatory standards.

62.2.2. Continuity Conditions

Railway alignments must satisfy at least:

- C^0 continuity (position),
- C^1 continuity (direction),
- typically C^2 continuity (curvature).

Higher-order smoothness may be required for high-speed lines.

62.2.3. Transition Length

Definition 62.2 (Minimum transition length). A transition between curvatures must satisfy

$$L \geq L_{\min}(v, \kappa_1, \kappa_2).$$

Typical formulations relate transition length to:

- allowable rate of change of lateral acceleration,
- cant development limits.

62.3. Dynamic Constraints

62.3.1. Lateral Acceleration

Definition 62.3 (Lateral acceleration). The lateral acceleration is given by

$$a_y(s) = v^2 \kappa(s).$$

Constraint 62.4.

$$|a_y(s)| \leq a_{\max}.$$

62.3.2. Cant and Cant Deficiency

Definition 62.5 (Cant angle). Let $\phi(s)$ denote the cant angle.

Definition 62.6 (Effective acceleration). The effective lateral acceleration is

$$a_{\text{eff}}(s) = v^2 \kappa(s) - g \tan(\phi(s)).$$

Constraint 62.7.

$$|a_{\text{eff}}(s)| \leq a_{\text{eff},\max}.$$

62.3.3. Jerk

Definition 62.8 (Jerk). The lateral jerk is defined as

$$j(s) = \frac{d}{dt} a_y(s).$$

Using s as parameter, one obtains

$$j(s) = v^3 \kappa'(s).$$

Constraint 62.9.

$$|j(s)| \leq j_{\max}.$$

62.4. Coupling of Curvature and Cant

Curvature and cant must not be treated independently.

Constraint 62.10.

$$|\kappa'(s)| \leq f_1(v, \phi), \quad |\phi'(s)| \leq f_2(v, \kappa).$$

These relations express that:

- curvature changes require corresponding cant development,
- cant changes must respect dynamic limits.

62.5. Speed-Dependent Constraints

All relevant constraints depend on speed.

Definition 62.11 (Speed profile). A speed profile is a function

$$v : [0, L] \rightarrow \mathbb{R}_+.$$

Constraints must be evaluated pointwise with respect to $v(s)$.

62.6. Constraint Representation in AIM

62.6.1. Abstract Form

Definition 62.12 (Constraint functional). A constraint is represented as a functional

$$C_i[\kappa, \phi, v] \leq 0.$$

This allows constraints to be:

- evaluated continuously,
- discretized for numerical optimization,
- combined into a unified constraint system.

62.6.2. Discrete Representation

In numerical implementation, constraints are evaluated on a grid

$$s_0, \dots, s_N.$$

Definition 62.13 (Discrete constraint).

$$C_i(\mathbf{x}) = \max_k C_i(s_k).$$

62.7. Integration into Optimization

Constraints enter the optimization problem as:

$$g_i(\mathbf{x}) \leq 0.$$

They may be handled via:

- hard constraints (feasibility),
- penalty terms,
- barrier methods.

62.8. Relation to Standards

The presented constraints correspond to rules found in:

- UIC codes,
- national railway regulations,
- operator guidelines.

The AIM framework provides a mathematical representation that allows these rules to be applied consistently across different data sources.

62.9. Discussion

Traditional workflows treat rules as external checks applied after design.

In contrast, AIM integrates rules directly into:

- modelling,
- reconstruction,
- optimization.

This leads to a shift from validation to constraint-driven design.

62.10. Summary

Summary 62.14. Railway design rules define the feasible set of alignment solutions.

Within the AIM framework, these rules are formalized as mathematical constraints and integrated into the optimization process.

This allows alignment design to be treated as a constrained optimization problem that respects both geometric and dynamic requirements.

Part XIII.

Applications

Applications

This part establishes how the approved AIM architecture delivers engineering consequence in real workflows.

The preceding parts established approved engineering concepts for alignment semantics, state, runtime evaluation, and reality coupling. The Applications part demonstrates what this architecture enables in concrete engineering contexts.

Applications therefore follow the progression

approved engineering concepts $\rightarrow$ application context
 $\rightarrow$ engineering use case
 $\rightarrow$ required runtime/realization services $\rightarrow$ derived representation or decision support $\rightarrow$ engineering consequence.

Application chapters consume canonical concepts and do not become new conceptual homes for alignment identity, pose2/pose3, realization, or representation layers.

Relation to the AIM Roadmap

Within the roadmap, this part corresponds to deployment of the established architecture in practical engineering workflows.

The icon addresses the guiding application question: how can downstream systems consume canonical semantics without relocating conceptual ownership?

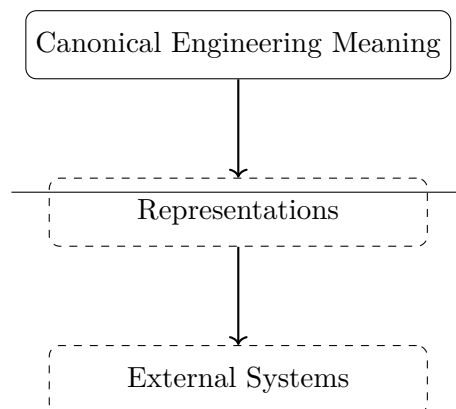


Figure 62.1.: Applications icon: canonical meaning remains upstream of representation and external systems.

Principle 62.15 (Application Principle). Within AIM, applications demonstrate architectural enablement and engineering consequence while preserving conceptual ownership in Foundations, State, Runtime, and Reality.

Summary 62.16. Applications in AIM are consequence-oriented demonstrations of established engineering semantics.

They show how canonical concepts support decision support, interoperability, and operational engineering outcomes without redefining the conceptual kernel.

With these applications established, the Discussion part can interpret their implications relative to historical and methodological context.

63. Railway Alignment Applications

Railway engineering is the primary application domain used in this thesis to demonstrate AIM capabilities. The chapter applies established alignment semantics and state/runtime concepts to practical railway questions.

63.1. Application Context

Railway alignment use involves coupled horizontal geometry, vertical profile, cant, velocity, admissibility constraints, and maintenance context. In AIM these are handled as coordinated layers consumed from prior parts.

63.2. Use of Canonical State and Runtime Concepts

Railway applications consume canonical pose2/pose3 and runtime services rather than redefining them. Operational quantities are evaluated as derived runtime quantities from established operators.

This supports engineering tasks such as alignment comparison, transition-quality assessment, and realization-aware evaluation in existing infrastructure.

63.3. Transition and Coupled Behaviour in Practice

Transition behaviour is interpreted through curvature evolution together with cant and operating speed context. The relevant application-level focus is not only geometric boundary matching, but engineering consequence in comfort, admissibility, and lifecycle behaviour.

63.4. Operational and Maintenance Consequences

In application workflows, alignment decisions interact with constraints on cant, transition quality, and maintainability. AIM supports this by keeping persistent semantics separate from derived operational quantities and by enabling runtime/realization-aware evaluation.

63.5. Interoperability Context

Railway applications often require exchange with BIM/GIS and format workflows. In this part, such standards are treated as interoperability mechanisms downstream of canonical

engineering semantics.

63.6. Connection to AIM

Summary 63.1. Railway applications in AIM demonstrate how alignment semantics, state, runtime evaluation, and realization-aware interpretation jointly support practical engineering decisions.

The chapter remains application-focused and does not redefine canonical concepts owned by upstream parts.

64. Vehicle Space and Platform Interaction

This chapter applies canonical pose3 and runtime interpretation to vehicle-space and platform-edge use cases.

64.1. Application Context

Platform interaction and clearance assessment depend on vehicle-space behaviour relative to railway runtime state. The relevant engineering object is therefore not centerline geometry alone, but the derived vehicle-relative state evaluated from canonical runtime services.

64.2. Derived Vehicle-Space Interpretation

Vehicle-space behaviour is interpreted downstream of canonical pose3 and frame/cant evaluation. It is application interpretation, not a new state-model layer.

This enables context-specific evaluation of platform gaps, clearance, and envelope behaviour under operational conditions.

64.3. Engineering Use Cases

Typical use cases include platform-edge verification, envelope checks in curves, and scenario-based assessment for differing vehicle classes. These use cases consume canonical runtime and realization layers.

64.4. Representation Boundary

Rendered or exported geometry is treated as downstream representation of runtime-evaluated state, not as engineering truth by itself.

64.5. Connection to AIM

Summary 64.1. Vehicle-space and platform applications in AIM are downstream interpretations built on canonical pose3 and runtime services.

They demonstrate practical decision support while preserving the separation between engineering objects and representations.

65. Railway Dynamics versus Aircraft Dynamics

This chapter provides a comparative application perspective that clarifies railway-specific consequences of constrained motion.

65.1. Comparative Context

Aircraft and railway domains share dynamic principles such as inertial response and comfort relevance. The key difference for applications is kinematic freedom: aircraft trajectories are actively steered in free space, whereas railway vehicles are constrained by infrastructure geometry.

65.2. Railway Consequence

Because railway motion is infrastructure-constrained, alignment, cant, and runtime interpretation become primary control levers for operational behaviour.

In application terms, the engineering question is often how existing geometry can support admissible operational envelopes through runtime and realization-aware interpretation.

65.3. AIM Interpretation

Within AIM, this comparison motivates use of canonical pose3, vehicle-space interpretation, and operational runtime evaluation for railway decision support.

The comparison is explanatory only; it does not introduce a parallel architectural model.

65.4. Connection to AIM

Summary 65.1. The railway-versus-aircraft comparison clarifies why railway applications require strong coupling between infrastructure semantics, runtime services, and operational interpretation.

It reinforces application consequences of canonical AIM concepts without redefining them.

66. Application Examples

This chapter illustrates representative application patterns that use established AIM concepts.

66.1. Example Pattern A: Transition Comparison

Two transition laws may satisfy identical boundary conditions while producing different internal curvature evolution and therefore different runtime-derived dynamic quantities.

A canonical indicator remains

$$a_{\text{lat}}(s) = v^2 \kappa(s),$$

showing that operational consequence depends on both geometry and speed context.

66.2. Example Pattern B: Cant-Coupled Interpretation

Application interpretation uses coupled curvature/cant quantities and runtime operators to evaluate admissibility and comfort consequences. Derived quantities remain downstream runtime results.

66.3. Example Pattern C: Realized-State Comparison

Target and realized states are compared in realization-aware workflows. The example focus is engineering consequence (maintenance or operational adjustment), not redefinition of upstream semantics.

66.4. Connection to AIM

Summary 66.1. The examples demonstrate how canonical AIM concepts are used for comparison, evaluation, and decision support in engineering contexts.

They intentionally illuminate established architecture rather than introducing new conceptual layers.

67. Use Cases

Use cases in this chapter are structured as application consequences of approved concepts.

67.1. Use-Case Structure

Each use case follows

$$\text{concept context} \rightarrow \text{engineering task} \rightarrow \begin{matrix} \text{runtime/realization} \\ \text{services} \end{matrix} \rightarrow \begin{matrix} \text{decision support} \\ \text{output} \end{matrix}.$$

67.2. Representative Use Cases

Representative use cases include reconstruction from heterogeneous inputs, smoothing and transition refinement, coupled curvature-cant evaluation, realized-state comparison, and multi-track context interpretation.

All use cases consume canonical state/runtime/reality concepts rather than redefining them.

67.3. Workflow Consequences

Application workflows combine data interpretation, runtime evaluation, realization-aware comparison, and interoperability output. They support engineering decisions such as maintenance prioritization, operational envelope tuning, and design-alternative comparison.

67.4. Connection to AIM

Summary 67.1. The use cases show that AIM provides a reusable engineering core for practical workflows.

Application behaviour is derived through canonical operators and service compositions, not through ad-hoc application-specific state models.

68. BIM Alignment Modelling

This chapter positions BIMalignment as an interoperability and workflow application of AIM, not as a competing canonical architecture.

68.1. Application Context

BIM and exchange standards (for example IFC, LandXML, GIS-linked containers) are essential project interoperability mechanisms. In AIM, they are treated as representation and exchange layers downstream of canonical engineering semantics.

The first figure answers the boundary question for this chapter: which semantic layer owns engineering meaning, and which layer only represents that meaning for exchange?

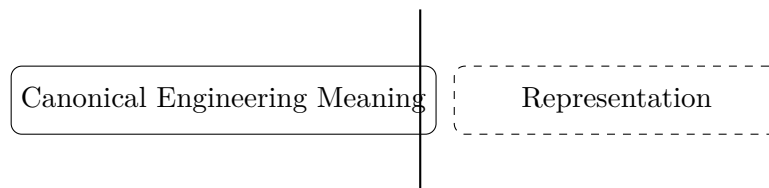


Figure 68.1.: Supporting comparison: canonical engineering meaning and representation are distinct layers.

The engineering consequence is that BIM workflows must consume canonical AIM state and operators instead of redefining them in exchange schemas.

68.2. AIM-to-BIM Integration Role

BIMalignment in this thesis denotes alignment-centred integration where AIM provides canonical semantics and runtime/realization evaluation, while BIM-oriented structures provide object-oriented representation, workflow integration, and lifecycle exchange.

Principle 68.1 (BIM integration principle). BIM integration consumes canonical AIM concepts and preserves their ownership outside application chapters.

68.3. Pipeline Interpretation

A typical interpretation pipeline is

exchange/import $\rightarrow$ $\xrightarrow[\text{and evaluation}]{\text{AIM normalization}}$ application consequence $\rightarrow$ representation/export.

This keeps engineering objects and representations conceptually separate.

The next figure addresses the interoperability question: how can native and imported alignments be interpreted consistently under one canonical state semantics?

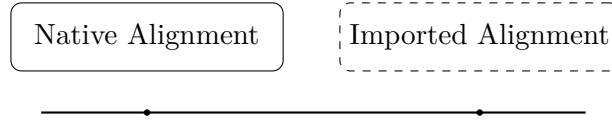


Figure 68.2.: Supporting comparison: native and imported alignments require shared canonical interpretation.

Its consequence is operational: import quality is assessed by canonical semantic compatibility, not by format-level similarity alone.

68.4. Boundary Clarification

BIMalignment is not presented as a parallel state model. It is an application-oriented integration strategy that depends on canonical alignment semantics, pose2/pose3 interpretation, metric realization, physical realization, and runtime services.

68.5. Connection to AIM

Summary 68.2. BIMalignment in this thesis is a downstream interoperability application of AIM.

It demonstrates how canonical alignment-centred engineering semantics can be connected to BIM/GIS exchange ecosystems without transferring conceptual ownership away from the approved kernel architecture.

69. Contribution

This chapter summarizes contribution consequences of the thesis from the application perspective.

69.1. Contribution Scope

The central contribution is not a new isolated application method, but a coherent alignment-centred engineering framework that supports application-level decision support through canonical concepts.

69.2. Application-Level Contributions

From the application perspective, the work contributes:

- a consistent concept-to-application progression,
- operator-based runtime and realization-aware evaluation support,
- explicit separation between engineering objects and representation/exchange forms,
- and practical interoperability pathways for BIM/GIS-oriented workflows.

69.3. Consequence for Engineering Practice

Applications become reproducible and comparable because they are derived from a stable conceptual core rather than project-specific ad-hoc semantics.

69.4. Connection to AIM

Summary 69.1. The contribution of the Applications part is to show how approved AIM concepts translate into concrete engineering consequences, including runtime-supported evaluation, realization-aware comparison, and interoperable representation for decision support.

70. Discussion

This discussion evaluates scope, strengths, and limits of the Applications part as a consequence layer of approved AIM concepts.

70.1. Positioning

Applications are not conceptual replacements for Foundations, State, Runtime, or Reality. They are validation-by-use layers that demonstrate engineering consequence in concrete contexts.

70.2. Strengths Observed in Applications

The chapter set shows consistent applicability across reconstruction, evaluation, realization-aware interpretation, vehicle-space use, and interoperability scenarios while preserving conceptual boundaries.

70.3. Limits and Practical Conditions

Application quality depends on data quality, context availability, operator configuration, and workflow discipline. These are deployment conditions, not contradictions of the conceptual architecture.

Unresolved process-centric modelling questions (including PIM-related questions) remain external research matters and are not elevated to canonical architecture in this migration.

70.4. Interoperability Consequence

BIM/GIS/exchange ecosystems are best treated as downstream representation and interoperability layers. This preserves canonical ownership of engineering semantics while enabling integration with project workflows.

70.5. Connection to AIM

Summary 70.1. The Applications part confirms that alignment-centred AIM concepts are practically usable across diverse engineering contexts without requiring new competing conceptual homes.

70. *Discussion*

Its discussion therefore emphasizes consequence and boundary discipline rather than reopening foundational research.

Part XIV.

Discussion

Discussion

This part establishes how the completed AIM framework should be interpreted within the broader engineering landscape.

What do we already know?

The previous parts developed AIM as a complete alignment-based information framework.

The framework now encompasses geometry, states, runtime evaluation, reality, modelling, optimization, engineering constraints, and practical applications.

Together, these components provide a coherent theoretical and practical foundation for railway alignment modelling.

The framework itself is therefore largely complete.

What is still missing?

A complete framework still requires interpretation.

New concepts must be related to existing engineering traditions.

Historical approaches must be re-evaluated.

Underlying assumptions must be examined.

Consequences beyond the immediate scope of the framework must be considered.

The missing element is therefore reflection.

The question is no longer how AIM works, but what AIM means within the broader context of railway engineering and information modelling.

What comes next?

This part provides an interpretive perspective on the AIM framework.

The focus shifts from framework construction toward framework understanding.

The chapters that follow relate AIM to:

- historical railway engineering approaches,
- established modelling traditions,
- previous optimization systems,
- and broader implications for alignment-based information modelling.

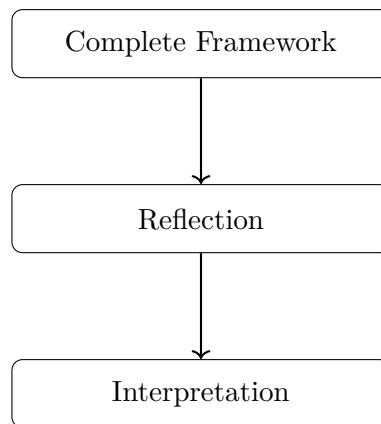
Particular attention is given to reinterpreting earlier concepts through the perspective developed throughout this thesis.

The objective is not merely comparison, but understanding.

Relation to the AIM Roadmap

Within the AIM roadmap, the present part corresponds to the transition from framework construction toward framework interpretation.

The roadmap sketch addresses the central question: how does the thesis move from technical completion to conceptual interpretation?



The progression

Complete Framework → Reflection → Interpretation

marks the transition from developing AIM toward understanding its position within the broader landscape of railway engineering and information modelling.

Discussion therefore serves as the conceptual mirror of the framework, revealing its assumptions, implications, and relationships to established practice.

Principle 70.2 (Interpretation Principle). Within AIM, engineering frameworks should not only be constructed and applied, but also interpreted in relation to historical practice, alternative approaches, and their broader conceptual consequences.

Summary 70.3. Applications demonstrate usefulness.

Discussion provides understanding.

The present part therefore reflects upon the AIM framework, its origins, its consequences, and its place within the broader evolution of railway engineering and information modelling.

This interpretive consolidation prepares the Outlook part, where the same framework is projected toward future research and deployment trajectories.

71. axtranNew: From Transition Grammar to Alignment Calculation

Motivation

Berlinish becomes engineering machinery only when its function identities, component lengths, boundary conditions, and degrees of freedom can be solved together. axtranNew originated as that calculation response. It is more than a convenient optimizer: its intended role is the alignment calculation core that turns a transition grammar into connected, editable alignment structure.

The name belongs to the implementation lineage of ufAIM. It does not denote an approved Knowledge Kernel concept, and its original intent must not be confused with the current implementation state.

71.1. Original Intent

The intended axtranNew scope comprises:

- satisfying connection, pose, curvature, and boundary conditions;
- keeping fixed, derived, constrained, coupled, and free quantities explicit;
- composing half-wave, clothoid, and zero-length transition components;
- determining admissible parameter combinations;
- coupling or separating curvature and cant functions;
- comparing alternatives and optimizing against engineering objectives;
- supporting elementary edits such as coupling alignment elements;
- realizing calculated candidates as editable alignment structures.

This list states engineering intent. It is not a claim that each item is production-ready.

71.2. Mathematical Capability

Mathematically, an axtranNew problem receives function identities, parameterizations, boundary conditions, engineering constraints, and declared degrees of freedom. It produces a

candidate parameter set and the alignment state derived from it. The Berlinish composition is one admissible structural model; other sparse transition families can use the same calculation principles.

The calculation can establish numerical consistency, residuals, applicability under stated constraints, and objective values. It cannot decide that the candidate is accepted, transfer Engineering Object Identity, or make its own result authoritative.

71.3. Observed Current Implementation

The current reference implementation provides evidence for a bounded but substantial calculation path:

- a validated registry resolves named transition families, normalized prototype functions, half-waves, and three-part length partitions;
- sparse fixed and transition elements are composed with sequential pose propagation and curvature integration;
- zero-length curvature holders and selected zero-length transition cases are represented;
- position, tangent, pose, curvature, and World–Track operations are evaluated;
- native straight, arc, and transition edits trigger deterministic sparse recalculation and preserve unaffected element identity;
- selected change-of-transition problems use equality-QP and experimental SQP machinery.

This observed implementation supports transition comparison, reconstruction, and selected edits. It does not yet demonstrate a general, reliable, production-ready optimizer for every connection, cant coupling, network, or engineering objective.

71.4. Current Boundary and Future Development

The broader SQP service remains experimental, non-authoritative, and proposal-only. Reliable convergence, globalization, full constraint and objective libraries, validated curvature–cant co-optimization, and network-level calculation remain development or research work. Results must be returned as reviewable candidates, diagnostics, or deltas and applied by the responsible external workflow.

71.5. From Geometry Optimization to Runtime-State Optimization

AIM generalizes the originating alignment calculation toward realization-aware runtime-state trajectory design.

This is not a rejection of *axtranNew* or classical systems. It explains which additional distinctions are needed when a calculated transition must retain identity, provenance, realization context, observation links, applicable knowledge, and accountable decision status.

71.6. Connection to AIM

Summary 71.1. Berlinish supplies the generalized functional grammar of railway transitions. *axtranNew* makes that grammar computationally applicable. The broader *ufAIM* environment emerged to preserve, operate, examine, explain, and extend the resulting knowledge.

The current implementation realizes important parts of this lineage, while a general production-ready optimizer and validated curvature–cant co-optimization remain future work. Computation produces candidates; it does not create authority or constitute an engineering decision.

Part XV.

Outlook

Outlook

This part establishes the forward trajectory of AIM after the completed scientific argument of this manuscript.

What do we already know?

The previous parts developed and analyzed the AIM framework in its present form.

The foundations, geometry, states, runtime services, realization concepts, modelling structures, optimization methods, engineering constraints, applications, and broader interpretations have now been established.

Together, these elements form a coherent framework for Alignment-Based Information Modelling.

The present work therefore provides a complete description of AIM as it currently exists.

What is still missing?

No framework is ever truly complete.

New technologies emerge.

Engineering workflows evolve.

Information systems become increasingly integrated.

New requirements arise from automation, digital twins, infrastructure management, and future transportation systems.

The missing element is therefore the future perspective.

The question is no longer what AIM is, but what AIM may become.

What comes next?

This part places AIM into a broader context and explores possible directions for future development.

The chapters that follow investigate:

- BIM-related perspectives,
- alignment-centric information models,
- future engineering workflows,

- extensions toward PIM concepts,
- and open research questions.

The objective is not to finalize the framework, but to identify opportunities for growth and integration.

Particular attention is given to the possibility that alignment may evolve into a first-class information entity comparable to the role currently occupied by building-centric BIM concepts.

Relation to the AIM Roadmap

Within the AIM roadmap, the present part corresponds to the transition from the present framework toward future developments.

The icon below addresses the closing orientation question of the thesis: which continuity line connects established results to the research frontier?



Figure 71.1.: Outlook icon: continuity from established knowledge to research frontier.

The progression

Present AIM → Future Development → BIM, PIM, and Beyond

marks the transition from describing the current framework toward exploring its future evolution.

The outlook therefore serves not as a conclusion, but as an invitation to continue the development of Alignment-Based Information Modelling.

Principle 71.2 (Open Framework Principle). Within AIM, Alignment-Based Information Modelling should be understood as an evolving framework whose future development is expected to extend beyond the concepts presented in this thesis.

Return to the Point of Departure

The thesis began from a railway transition problem. Berlinish provided a generalized functional grammar: entering half-wave, optional clothoid core, and exiting half-wave, including zero-length and asymmetric cases. axtranNew provided the calculation intention: solve the connection, expose its degrees of freedom, compare admissible alternatives, and return editable alignment structure.

The broader work added what those two contributions could not provide alone. transi- tionDB and TransEd preserve and examine reusable function knowledge. SPOT preserves

the identity and state of concrete engineering objects. alignmentOS makes alignment knowledge operational. The Knowledge Kernel stabilizes terminology and responsibility boundaries. Research tests claims and opens extensions. The Thesis turns the lineage into an argument that can be inspected and transferred.

What is established is the curvature-based composition and state machinery developed in this thesis. What is implementation-backed is the current registry, reconstruction, evaluation, and selected edit and optimization path. What remains open is reliable general optimization, validated independent or coupled curvature–cant design, broader network calculation, and any later Kernel treatment of the originating Berlinish terminology. The future begins from this explicit boundary, not from an assertion of completeness.

Summary 71.3. Discussion explains what AIM means.

Outlook explores what AIM may become.

The present part therefore identifies future directions, broader applications, and open questions that extend beyond the scope of the current work while building upon its foundations.

Taken together, this closing perspective frames the thesis as a continuous argument from foundational motivation to future engineering development.

72. BIM and Alignment Modelling

Motivation

Building Information Modelling (BIM) has become a central paradigm in modern infrastructure projects.

Its goal is the integration of geometry, semantics, and lifecycle information into a unified digital representation.

However, despite its broad scope, the representation of railway alignments within BIM remains conceptually and technically limited.

72.1. BIM as a Data Integration Paradigm

BIM is fundamentally concerned with:

- object-based modelling,
- semantic enrichment of geometry,
- interoperability across disciplines,
- lifecycle management of infrastructure assets.

In this context, geometry is typically associated with objects such as:

- buildings,
- bridges,
- track elements,
- terrain models.

Alignment geometry is often embedded implicitly within these objects, rather than treated as a primary modelling entity.

72.2. Alignment in BIM Standards

72.2.1. IFC Alignment

Recent extensions of IFC introduce explicit alignment entities.

These aim to represent:

- horizontal alignment,
- vertical alignment,
- cant (superelevation),
- stationing.

While this is a significant step forward, several issues remain:

- separation of components,
- lack of unified state representation,
- limited support for dynamic interpretation,
- complexity of schema usage.

72.2.2. Layered Representation

In most BIM-related formats, alignment is decomposed into layers:

- horizontal geometry,
- vertical profile,
- cant definition.

This decomposition is practical for data exchange, but does not directly support computation or optimization.

72.3. Limitations of Current BIM Alignment Modelling

72.3.1. Static representation

BIM primarily represents geometry as static data.

Dynamic properties such as acceleration, jerk, or vehicle response are not part of the core model.

72.3.2. Fragmentation

The separation of horizontal, vertical, and cant components leads to a fragmented representation.

This fragmentation complicates:

- consistency checks,
- dynamic evaluation,
- optimization-based design.

72.3.3. Lack of computational core

BIM standards focus on data exchange and documentation.

They do not provide a unified computational model for alignment analysis and optimization.

72.4. AIM as a Computational Layer for BIM

72.4.1. Conceptual Role

The AIM framework can be understood as a computational core that complements BIM data models.

Instead of replacing BIM, AIM operates between:

- external data formats,
- and internal engineering reasoning.

72.4.2. Canonical Representation

AIM introduces a unified representation:

$$(\gamma(s), \theta(s), \kappa(s), \phi(s)).$$

This representation:

- integrates horizontal geometry,
- integrates vertical and cant behaviour,
- supports dynamic evaluation,
- is directly usable in optimization.

72.4.3. Transformation Pipeline

BIM / IFC / LandXML

→ parsing
→ fat representation
→ normalization
→ AIM state model
→ analysis / optimization
→ export

This pipeline separates:

- data exchange concerns,

- computational modelling,
- engineering reasoning.

72.5. Alignment-Centred BIM

72.5.1. Shift of perspective

The AIM framework suggests a shift toward alignment-centred modelling.

Instead of treating alignment as one component among many, it becomes a primary organizing structure.

72.5.2. Implications

- infrastructure elements are referenced to alignment,
- geometry is derived from alignment state,
- dynamic behaviour becomes part of the model,
- optimization becomes a native operation.

72.6. Integration with BIM Workflows

72.6.1. Import

Existing BIM data is imported via:

- IFC parsing,
- LandXML parsing,
- legacy format interpretation.

72.6.2. Processing

Within AIM, the imported data is:

- normalized,
- reconstructed,
- enriched with dynamic interpretation,
- potentially optimized.

72.6.3. Export

Results may be exported back into:

- IFC alignment structures,
- LandXML files,
- other engineering formats.

This ensures compatibility with existing BIM ecosystems.

72.7. Relation to BIMinfra

Infrastructure BIM (often referred to as BIMinfra) extends BIM concepts to linear infrastructure such as railways and roads.

AIM can be interpreted as a specialization within BIMinfra focusing on:

- alignment-centric modelling,
- dynamic interpretation,
- optimization-driven design.

72.8. Discussion

BIM provides a powerful framework for data integration, but lacks a strong computational alignment model.

AIM addresses this gap by:

- introducing a unified state representation,
- enabling dynamic evaluation,
- supporting optimization,
- and integrating heterogeneous data sources.

Together, BIM and AIM can be seen as complementary:

- BIM for data integration and lifecycle,
- AIM for computation and alignment reasoning.

72.9. Summary

Summary 72.1. Current BIM standards provide important structures for data exchange and semantic modelling, but remain limited in their treatment of railway alignment.

The AIM framework complements BIM by introducing a unified, state-based, and optimization-ready representation of alignment.

This enables a transition from static data handling toward dynamic, computational, and alignment-centred infrastructure modelling.

73. Future Work

73.1. Purpose of this Chapter

The AIM framework presented in this manuscript establishes a mathematical and conceptual foundation for alignment-centred modelling.

However, many aspects remain open and provide opportunities for further research, development, and practical implementation.

This chapter outlines potential directions for future work, ranging from mathematical extensions to software architecture and engineering applications.

73.2. Extension of the Dynamic Model

73.2.1. From simplified dynamics to vehicle models

The current formulation relies on simplified dynamic quantities such as effective lateral acceleration and jerk.

Future work should extend this toward more realistic vehicle models, including:

- multi-body vehicle dynamics,
- wheel–rail interaction,
- suspension behaviour,
- track elasticity.

73.2.2. Speed-dependent modelling

The integration of variable speed profiles

$$v(s)$$

opens the possibility for more realistic evaluation and optimization of alignments under operational conditions.

73.3. Schwerpunkt-Trassierung

73.3.1. Conceptual development

The idea of designing the trajectory of a dynamically relevant point, such as the center of mass, represents a fundamental extension of classical alignment design.

Further work is required to:

- formalize the mapping from track geometry to vehicle trajectory,
- incorporate full dynamic models,
- define appropriate objective functions.

73.3.2. Integration into optimization

A key research direction is the coupling of Schwerpunkt-Trassierung with optimization methods, enabling:

- comfort-driven design,
- dynamic optimization,
- direct evaluation of vehicle response.

73.4. Advanced Optimization Methods

73.4.1. SQP extensions

While Sequential Quadratic Programming provides a powerful baseline, future work may include:

- tailored SQP formulations for sparse alignment structures,
- improved constraint handling,
- adaptive weighting strategies.

73.4.2. Global optimization

Alignment optimization problems are often non-convex.

Future research may explore:

- global optimization techniques,
- multi-start strategies,
- hybrid deterministic–stochastic methods.

73.5. Network-Level Modelling

73.5.1. From single alignment to network

Real railway systems consist of interconnected alignments forming complex networks.

Future work should extend the AIM framework to:

- branching structures,
- switches and crossings,
- corridor-wide optimization.

73.5.2. Topological constraints

A systematic treatment of topological constraints remains a major challenge.

This includes:

- node consistency,
- multi-track coupling,
- network-wide feasibility conditions.

73.6. Constraint and Objective Libraries

73.6.1. Constraint formalization

Although many railway rules can be expressed as constraints, a complete formalization of regulatory frameworks remains an open task.

Future work should aim at:

- systematic encoding of standards,
- validation against real projects,
- parameterization for different railway systems.

73.6.2. Objective design

The design of objective functions is crucial for optimization-based alignment design.

Potential extensions include:

- cost models,
- maintenance optimization,
- energy efficiency,
- lifecycle considerations.

73.7. Integration with BIM and Data Standards

73.7.1. Deeper BIM integration

The integration of AIM with BIM workflows can be further developed by:

- tighter coupling with IFC alignment models,
- bidirectional synchronization,
- embedding AIM states within BIM objects.

73.7.2. Standard extensions

The AIM framework may also inspire extensions to existing standards, for example:

- inclusion of dynamic properties,
- support for optimization metadata,
- representation of control-based alignment models.

73.8. Interactive and Real-Time Systems

73.8.1. Live optimization

One of the most promising directions is the development of systems that allow real-time feedback during alignment design.

This includes:

- live preview during optimization,
- interactive constraint adjustment,
- immediate visualization of dynamic effects.

73.8.2. User interaction

The grabbeltisch concept may be extended to:

- interactive constraint definition,
- visual debugging of alignment states,
- intuitive editing of transition parameters.

73.9. AI and Assisted Design

73.9.1. Data-driven approaches

Machine learning techniques may complement the AIM framework by:

- learning from existing alignments,
- suggesting initial solutions,
- identifying patterns in design choices.

73.9.2. Hybrid systems

A promising direction is the combination of:

- physics-based modelling (AIM),
- data-driven methods (AI),
- interactive user control.

73.10. AXTRAN Revival and Beyond

Classical systems such as AXTRAN demonstrated the feasibility of optimization-based alignment design.

Future work may aim to:

- reconstruct and generalize such systems,
- extend them to network-level problems,
- integrate modern numerical and computational techniques.

73.11. Software Architecture and Deployment

73.11.1. Scalability

Practical applications require scalable implementations capable of handling large datasets and complex networks.

73.11.2. Multi-view systems

Future systems may include:

- synchronized multi-window views,
- combined 2D/3D representations,
- integration of GIS and CAD functionality.

73.12. Long-Term Vision

In the long term, the AIM framework may contribute to a new paradigm of infrastructure modelling in which:

- alignment becomes the central organizing structure,
- geometry, dynamics, and optimization are unified,
- engineering design becomes an interactive computational process.

73.13. Summary

Summary 73.1. The AIM framework opens a wide range of research and development directions.

These include extensions of the dynamic model, integration with BIM, network-level optimization, real-time systems, and AI-assisted design.

Together, these directions point toward a future in which railway alignment is no longer treated as static geometry, but as a dynamic, computational, and optimizable system at the core of infrastructure engineering.

Bibliography

- [1] E. Andersson, N. G. Nilstam, and L. Ohlsson. Lateral track forces at high speed curving: Comparison of practical and theoretical results of swedish high speed train x2000. *Vehicle System Dynamics*, 25, 1995. Supplement; exact pages to be verified from source.
- [2] AREMA. Railway track design, chapter 6, 2003. Manual chapter.
- [3] Walter Bloss. *Gleis- und Weichengeometrie*. Transpress, 1978.
- [4] Tanita Fossli Brustad and Rune Dalmo. Railway transition curves: A review of the state-of-the-art and future research. *Infrastructures*, 5(43), 2020.
- [5] buildingSMART International. IFC 4.3.2 documentation: Scope. <https://ifc43-docs.standards.buildingsmart.org/IFC/RELEASE/IFC4x3/HTML/content/scope.htm>. Accessed 20 July 2026.
- [6] buildingSMART Railway Room. Rwr-ifc rail conceptual model report, 2019. Conceptual model report.
- [7] CEN. Bs en 13803:2017 railway applications – track – track alignment design parameters, 2017. Standard.
- [8] Constantin Ciobanu. Bloss transition: A short design guide. *PWI Journal*, 133:14–18, 2015.
- [9] Manfredo P. do Carmo. *Differential Geometry of Curves and Surfaces*. Prentice-Hall, 1976.
- [10] J. Glover. Transition curves for railways. *Proceedings of the Institution of Civil Engineers*, 140:161–179, 1900.
- [11] Zulfiqar Habib and Manabu Sakai. Spiral transition curves and their applications. *Scientiae Mathematicae Japonicae*, 59(2):251–262, 2004.
- [12] Ernst Hairer, Syvert P. Nørsett, and Gerhard Wanner. *Solving Ordinary Differential Equations I*. Springer, 1993.
- [13] Franz Hasslinger. Verfahren zur trassierung eines fahrwegs unter berücksichtigung der schwerpunktbewegung eines fahrzeugs, 2002. Austrian patent AT412975B on Schwerpunkt-based railway alignment.

Bibliography

- [14] A. L. Higgins. *The Transition Spiral and Its Introduction to Railway Curves*. Van Nostrand, New York, 1922.
- [15] Hofmann-Wellenhof. *GPS: Theory and Practice*. needs to be investigated, 2001.
- [16] E. M. Horsburgh. The railway transition curve. *Proceedings of the Royal Society of Edinburgh*, 32:333–347, 1913.
- [17] Simon Iwnicki, editor. *Handbook of Railway Vehicle Dynamics*. CRC Press, Boca Raton, 2006.
- [18] J. J. Kalker. A fast algorithm for the simplified theory of rolling contact. *Vehicle System Dynamics*, 11(1):1–13, 1982.
- [19] J. J. Kalker. *Three-Dimensional Elastic Bodies in Rolling Contact*. Kluwer Academic Publishers, Dordrecht, 1990.
- [20] Nam-Ho Kim, Ashok V. Kumar, and Harold F. Snider. *Geometry of Design: A Workbook*. Woodhead Publishing, 2015.
- [21] Louis T. Jr. Klauder. A better way to design railroad transition spirals. *ASCE Journal of Transportation Engineering*, 2001. Preprint submitted to ASCE Journal of Transportation Engineering.
- [22] Louis T. Jr. Klauder. Railroad curve transition spiral design method based on control of vehicle banking motion, 2001. Patent WO2001098938A1.
- [23] Louis T. Jr. Klauder. Railroad curve transition spiral design method based on control of vehicle banking motion, 2007. US application publication US20070027661A9.
- [24] Louis T. Jr. Klauder. Railroad curve transition spiral design method based on control of vehicle banking motion, 2009. Patent US7542830.
- [25] Louis T. Jr. Klauder. Railroad spiral design and performance, 2012. Conference/publication details to be completed.
- [26] Günter Klein. *Trassierung von Verkehrswegen*. Springer, 1998.
- [27] Klaus Knothe and Stephen L. Grassie. Modelling of railway track and vehicle/track interaction at high frequencies. *Vehicle System Dynamics*, 22(3–4):209–262, 1993.
- [28] W. Koc. Transition curves in railway engineering. *Journal of Transportation Engineering*, 2008. Further metadata to be completed.
- [29] W. Koc. Transition curve with smoothed curvature at its ends for railway roads. *Current Journal of Applied Science and Technology*, 22:1–10, 2017.
- [30] W. Koc. New transition curve adapted to railway operational requirements. *Journal of Surveying Engineering*, 145, 2019. Issue/pages to be completed.

Bibliography

- [31] W. Koc. Smoothed transition curve for railways. *Przegląd Komunikacyjny*, pages 19–32, 2019.
- [32] Bengt Kufver. *Optimization of Horizontal Alignment of Railway Tracks*. PhD thesis, Royal Institute of Technology (KTH), 2000.
- [33] Björn Kufver. Mathematical description of railway alignments and some preliminary comparative studies. Technical Report 420A, Swedish National Road and Transport Research Institute (VTI), 1997.
- [34] Hugo Magalhães, Jorge Ambrósio, José Pombo, and Marco Pereira. Railway vehicle modelling for the vehicle-track interaction compatibility analysis. *Proceedings of the Institution of Mechanical Engineers, Part K: Journal of Multi-body Dynamics*, 230(3):251–267, 2016.
- [35] José Martínez-Casas, Egidio Di Gialleonardo, Stefano Bruni, and Luis Baeza. A comprehensive model of the railway wheelset-track interaction in curves. *Journal of Sound and Vibration*, 333:4152–4169, 2014.
- [36] E. Mieloszyk. Modern transition curve design. *Transport Problems*, 2012. Further metadata to be completed.
- [37] A. W. Miller. The transition spiral. *Australian Surveyor*, 7:518–526, 1939.
- [38] Jorge Nocedal and Stephen J. Wright. *Numerical Optimization*. Springer, 2006.
- [39] Open Geospatial Consortium. How the OGC standards program works. <https://www.ogc.org/standards/how-our-standards-program-works/>. Accessed 20 July 2026.
- [40] A. Pirti, M. Yücel, and T. Ocalan. Transrapid and the transition curve as sinusoid. *Tehnicki Vjesnik*, 23:315–320, 2016.
- [41] Oskar Polach. A fast wheel-rail forces calculation computer code. *Vehicle System Dynamics*, 33, 1999. Supplement; exact issue/pages to be verified from source.
- [42] José Pombo and Jorge Ambrósio. Application of a wheel–rail contact model to railway dynamics in small radius curved tracks. *Multibody System Dynamics*, 19(1–2):91–114, 2008.
- [43] José Pombo, Jorge Ambrósio, and Manuel Silva. A new wheel–rail contact model for railway dynamics. *Vehicle System Dynamics*, 45(2):165–189, 2007.
- [44] William H. Press, Saul A. Teukolsky, William T. Vetterling, and Brian P. Flannery. *Numerical Recipes*. Cambridge University Press, 2007.
- [45] Andrew Pressley. *Elementary Differential Geometry*. needs to be investigated, 2010.

Bibliography

- [46] Ahmed A. Shabana, Karim E. Zaazaa, and Hiroyuki Sugiyama. *Railroad Vehicle Dynamics: A Computational Approach*. CRC Press, Boca Raton, 2008.
- [47] S. Shebl. Geometrical analysis of non-linear curvature transition curves of high speed railways. *Asian Journal of Current Engineering and Maths*, 5:52–58, 2016.
- [48] Dirk J. Struik. *Lectures on Classical Differential Geometry*. needs to be investigated, 1961.
- [49] E. Tari. The new generation transition curves. *ARI Bulletin, Istanbul Technical University*, 54:35–41, 2004.
- [50] TODO. Ifc railway, 2015. Placeholder entry; replace with exact source metadata.
- [51] Žiga Turk. Interoperability in construction—mission impossible? *Developments in the Built Environment*, 4:100018, 2020.
- [52] UIC. Uic code 519: Method for determining the running safety of railway vehicles, 2004. International Union of Railways.
- [53] A. H. Wickens. *Fundamentals of Rail Vehicle Dynamics*. Swets & Zeitlinger, Lisse, 2003.
- [54] R. Wojtczak. *Charakterystyka Krzywej Przejściowej Wiener Bogen*. Publishing House of Poznan University of Technology, Poznan, 2017.
- [55] K. Zboinski and P. Woznica. Optimization of polynomial transition curves from the viewpoint of jerk value. *Archives of Civil Engineering*, 63, 2017. Issue/pages to be completed.
- [56] Krzysztof Zboinski and Monika Golofit-Stawinska. Dynamics of 2 and 4-axle railway vehicles in transition curves above critical velocity. In *Proceedings of the Second International Conference on Railway Technology: Research, Development and Maintenance*, 2014. Paper 265.

Index

- Acceleration
 - effective lateral, 160
- Arc-length
 - vs stationing, 119
- Architecture
 - runtime, 157
- Conceptual structure, 22
- Coordinate reference system (CRS), 169
- Coordinate system, 169
 - track, 119
- CRS context principle, 169
- Curvature
 - discrete estimate, 173
- Data quality
 - imperfect data, 173
- Data source, 167
 - definition, 167
- Dynamics operator, 132
- Error
 - systematic, 173
- Figures
 - runtime architecture, 163
- Interpretation
 - observation data, 167
- Intrinsic-space primacy principle, 118
- Jerk
 - runtime, 160
- Level of detail (LOD)
 - runtime evaluation, 160
- Observation
 - data source context, 167
 - measurement data, 173
- Observation context principle, 173
- Observation principle, 167
- Observed data, 173
- Operational state
 - evaluation operator, 133
- Operator
 - runtime, 132
- Operator composition, 133
- Optimization
 - realization-aware, 264
 - runtime, 267
- Pipeline
 - realization-aware runtime, 161
 - runtime operator, 161
- Projection
 - ambiguity, 121
 - distinction from coordinate transformation, 121
 - track-to-world, 119
 - track-to-world runtime service, 159
 - world-to-track, 121
 - world-to-track runtime service, 159
 - world-track coordinate transformation, 118
- Projection ambiguity principle, 121
- Projection distinction principle, 121
- Provenance
 - observation reconstruction, 168
 - tracking, 174

- Realization
 - metric
 - coordinate handling, 170
 - operator, 132
- Reconstruction
 - from observation data, 168
- Reference frame
 - local, 170
 - runtime service, 160
- Reference system
 - kilometre line, 171
 - outer stationing, 171
 - stationing, 171
 - stationing layer, 171
 - track space, 118
 - world embedding, 169
- Runtime cache, 160
- Runtime evaluation, 157
- Runtime operators, 132
- Runtime reconstruction principle, 158
- Runtime service
 - definition, 157
- Runtime services, 157
- Special elements
 - railway network, 175
- State
 - pose3
 - runtime evaluation, 158
 - qualified use, 130
- Stationing, *see* Reference system, stationing
- Topology, 175
 - connection point, 175
 - graph representation, 175
 - operational continuity, 172
 - railway network context, 175
- Track space, 119
- Transformation
 - coordinate, 170
- Uncertainty
 - measurement, 167, 173
- Vehicle state
 - interpretation operator, 133